MDN JavaScript Documentation

Volume 2 of 9 — Reference: Expressions & operators · Statements & declarations

Unofficial print compilation. The JavaScript reference landing page, every expression and operator page, and every statement and declaration page. This volume contains 114 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

JavaScript reference 3

Expressions and operators 11
- Addition (+) 23
- Addition assignment (+=) 25
- Assignment (=) 27
- async function expression 30
- async function* expression 33
- await 36
- Bitwise AND (&) 43
- Bitwise AND assignment (&=) 45
- Bitwise NOT (~) 46
- Bitwise OR (|) 48
- Bitwise OR assignment (|=) 50
- Bitwise XOR (^) 52
- Bitwise XOR assignment (^=) 54
- class expression 55
- Comma operator (,) 57
- Conditional (ternary) operator 61
- Decrement (--) 63
- delete 66
- Destructuring 71
- Division (/) 83
- Division assignment (/=) 86
- Equality (==) 87
- Exponentiation (**) 91
- Exponentiation assignment (**=) 94
- function expression 95
- function* expression 102
- Greater than (>) 104
- Greater than or equal (>=) 106
- Grouping operator ( ) 109
- import.meta 112
  - import.meta.resolve() 115
- import() 118
  - import.defer() 124
  - import.source() 125
- in 127
- Increment (++) 133
- Inequality (!=) 135
- instanceof 137
- Left shift (<<) 143
- Left shift assignment (<<=) 145
- Less than (<) 147
- Less than or equal (<=) 150
- Logical AND (&&) 153
- Logical AND assignment (&&=) 156
- Logical NOT (!) 158
- Logical OR (||) 160
- Logical OR assignment (||=) 163
- Multiplication (*) 166
- Multiplication assignment (*=) 168
- new 169
- new.target 174
- null 178
- Nullish coalescing assignment (??=) 180
- Nullish coalescing operator (??) 182
- Object initializer 185
- Operator precedence 192
- Optional chaining (?.) 201
- Property accessors 206
- Remainder (%) 210
- Remainder assignment (%=) 212
- Right shift (>>) 214
- Right shift assignment (>>=) 216
- Spread syntax (...) 218
- Strict equality (===) 225
- Strict inequality (!==) 227
- Subtraction (-) 229
- Subtraction assignment (-=) 231
- super 233
- this 240
- typeof 252
- Unary negation (-) 257
- Unary plus (+) 259
- Unsigned right shift (>>>) 261
- Unsigned right shift assignment (>>>=) 264
- void operator 265
- yield 268
- yield* 272

Statements and declarations 278
- async function 284
- async function* 292
- await using 295
- Block statement 300
- break 304
- class 308
- const 310
- continue 314
- debugger 317
- do...while 319
- Empty statement 322
- export 323
- Expression statement 331
- for 333
- for await...of 339
- for...in 344
- for...of 350
- function 358
- function* 363
- if...else 369
- import 373
  - Import attributes 381
  - import defer 386
  - import source 393
- Labeled statement 397
- let 402
- return 409
- switch 412
- throw 418
- try...catch 421
- using 428
- var 434
- while 441
- with 444

JavaScript

JavaScript reference

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference · MDN page last modified 2026-09-15

The JavaScript reference serves as a repository of facts about the JavaScript language. The entire language is described here in detail. As you write JavaScript code, you'll refer to these pages often (thus the title "JavaScript reference").

The JavaScript language is intended to be used within some larger environment, be it a browser, server-side scripts, or similar. For the most part, this reference attempts to be environment-agnostic and does not target a web browser environment.

If you are new to JavaScript, start with the guide. Once you have a firm grasp of the fundamentals, you can use the reference to get more details on individual objects and language constructs.

Built-ins

JavaScript standard built-in objects, along with their methods and properties.

Value properties

- `globalThis`
- `Infinity`
- `NaN`
- `undefined`

Function properties

- `eval()`
- `isFinite()`
- `isNaN()`
- `parseFloat()`
- `parseInt()`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`
- `escape()` Deprecated
- `unescape()` Deprecated

Fundamental objects

- `Object`
- `Function`
- `Boolean`
- `Symbol`

Error objects

- `Error`
- `AggregateError`

- EvalError
- RangeError
- ReferenceError
- SuppressedError
- SyntaxError
- TypeError
- URIError
- InternalError Non-standard

Numbers and dates

- Number
- BigInt
- Math
- Date
- Temporal

Text processing

- String
- RegExp

Indexed collections

- Array
- Int8Array
- Uint8Array
- Uint8ClampedArray
- Int16Array
- Uint16Array
- Int32Array
- Uint32Array
- BigInt64Array
- BigUint64Array
- Float16Array
- Float32Array
- Float64Array

Keyed collections

- Map
- Set
- WeakMap
- WeakSet

Structured data

- ArrayBuffer
- SharedArrayBuffer

- DataView
- Atomics
- JSON

Managing memory

- WeakRef
- FinalizationRegistry

Control abstraction objects

- Iterator
- AsyncIterator
- Promise
- GeneratorFunction
- AsyncGeneratorFunction
- Generator
- AsyncGenerator
- AsyncFunction
- DisposableStack
- AsyncDisposableStack

Reflection

- AbstractModuleSource
- Reflect
- Proxy

Internationalization

- Intl
- Intl.Collator
- Intl.DateTimeFormat
- Intl.DisplayNames
- Intl.DurationFormat
- Intl.ListFormat
- Intl.Locale
- Intl.NumberFormat
- Intl.PluralRules
- Intl.RelativeTimeFormat
- Intl.Segmenter

Statements

JavaScript statements and declarations

Control flow

- `return`
- `break`
- `continue`
- `throw`
- `if...else`
- `switch`
- `try...catch`

Declaring variables

- `var`
- `let`
- `const`
- `using`
- `await using`

Functions and classes

- `function`
- `function*`
- `async function`
- `async function*`
- `class`

Iterations

- `do...while`
- `for`
- `for...in`
- `for...of`
- `for await...of`
- `while`

Others

- Empty
- Block
- Expression statement
- `debugger`
- `export`
- `import`
- `import defer`
- `import source`
- label
- `with` Deprecated

Expressions and operators

JavaScript expressions and operators.

Primary expressions

- `this`
- Literals
- `[]`
- `{}`
- `function`
- `class`
- `function*`
- `async function`
- `async function*`
- `/ab+c/i`
- `` `string` ``
- `( )`

Left-hand-side expressions

- Property accessors
- `?.`
- `new`
- `new.target`
- `import.meta`
- `super`
- `import()`
- `import.defer()`
- `import.source()`

Increment and decrement

- `A++`
- `A--`
- `++A`
- `--A`

Unary operators

- `delete`
- `void`
- `typeof`
- `+`
- `-`
- `~`
- `!`

- `await`

Arithmetic operators

- `**`
- `*`
- `/`
- `%`
- `+` (Plus)
- `-`

Relational operators

- `<` (Less than)
- `>` (Greater than)
- `<=`
- `>=`
- `instanceof`
- `in`

Equality operators

- `==`
- `!=`
- `===`
- `!==`

Bitwise shift operators

- `<<`
- `>>`
- `>>>`

Binary bitwise operators

- `&`
- `|`
- `^`

Binary logical operators

- `&&`
- `||`
- `??`

Conditional (ternary) operator

- `(condition ? ifTrue : ifFalse)`

Assignment operators

- `=`
- `*=`

- `/=`
- `%=`
- `+=`
- `-=`
- `<<=`
- `>>=`
- `>>>=`
- `&=`
- `^=`
- `|=`
- `**=`
- `&&=`
- `||=`
- `??=`
- `[a, b] = arr`, `{ a, b } = obj`

Yield operators

- `yield`
- `yield*`

Spread syntax

- `...obj`

Comma operator

- `,`

Functions

JavaScript functions.

- Arrow Functions
- Default parameters
- Rest parameters
- `arguments`
- Method definitions
- getter
- setter

Classes

JavaScript classes.

- `constructor`
- `extends`
- Private elements
- Public class fields

- `static`
- Static initialization blocks

Regular expressions

JavaScript regular expressions.

- Backreference: `\1`, `\2`
- Capturing group: `(...)`
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`
- Input boundary assertion: `^`, `$`
- Literal character: `a`, `b`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Modifier: `(?ims-ims:...)`
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`
- Non-capturing group: `(?:...)`
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`
- Unicode character class escape: `\p{...}`, `\P{...}`
- Wildcard: `.`
- Word boundary assertion: `\b`, `\B`

Additional reference pages

- JavaScript technologies overview
- Execution model
- Lexical grammar
- Data types and data structures
- Iteration protocols
- Trailing commas
- Errors
- Strict mode
- Deprecated features

JavaScript › Reference

Expressions and operators

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators · MDN page last modified 2026-09-15

This chapter documents all the JavaScript language operators, expressions and keywords.

Expressions and operators by category

For an alphabetical listing see the sidebar on the left.

Primary expressions

Basic keywords and general expressions in JavaScript. These expressions have the highest precedence (higher than operators).

`this`
: The `this` keyword refers to a special property of an execution context.

Literals
: Basic `null`, boolean, number, and string literals.

`[]`
: Array initializer/literal syntax.

`{}`
: Object initializer/literal syntax.

`function`
: The `function` keyword defines a function expression.

`class`
: The `class` keyword defines a class expression.

`function*`
: The `function*` keyword defines a generator function expression.

`async function`
: The `async function` defines an async function expression.

`async function*`
: The `async function*` keywords define an async generator function expression.

`/ab+c/i`
: Regular expression literal syntax.

`` `string` ``
: Template literal syntax.

`( )`
: Grouping operator.

Left-hand-side expressions

Left values are the destination of an assignment.

Property accessors
: Member operators provide access to a property or method of an object (`object.property` and `object["property"]`).

`?.`

The optional chaining operator returns `undefined` instead of causing an error if a reference is nullish (`null` or `undefined`).

`new`

The `new` operator creates an instance of a constructor.

`new.target`

In constructors, `new.target` refers to the constructor that was invoked by `new`.

`import.meta`

An object exposing context-specific metadata to a JavaScript module.

`super`

The `super` keyword calls the parent constructor or allows accessing properties of the parent object.

`import()`

The `import()` syntax allows loading a module asynchronously and dynamically into a potentially non-module environment.

`import.defer()`

The `import.defer()` syntax loads a module dynamically, deferring synchronous evaluation until the returned namespace's properties are accessed.

`import.source()`

The `import.source()` syntax results in an object representing the module's compiled source, without loading its dependencies, linking it, or evaluating it.

Increment and decrement

Postfix/prefix increment and postfix/prefix decrement operators.

`A++`

Postfix increment operator.

`A--`

Postfix decrement operator.

`++A`

Prefix increment operator.

`--A`

Prefix decrement operator.

Unary operators

A unary operation is an operation with only one operand.

`delete`

The `delete` operator deletes a property from an object.

`void`

The `void` operator evaluates an expression and discards its return value.

`typeof`

The `typeof` operator determines the type of a given object.

`+`

The unary plus operator converts its operand to Number type.

`-`
: The unary negation operator converts its operand to Number type and then negates it.

`~`
: Bitwise NOT operator.

`!`
: Logical NOT operator.

`await`
: Pause and resume an async function and wait for the promise's fulfillment/rejection.

Arithmetic operators

Arithmetic operators take numerical values (either literals or variables) as their operands and return a single numerical value.

`**`
: Exponentiation operator.

`*`
: Multiplication operator.

`/`
: Division operator.

`%`
: Remainder operator.

`+` **(Plus)**
: Addition operator.

`-`
: Subtraction operator.

Relational operators

A comparison operator compares its operands and returns a boolean value based on whether the comparison is true.

`<` **(Less than)**
: Less than operator.

`>` **(Greater than)**
: Greater than operator.

`<=`
: Less than or equal operator.

`>=`
: Greater than or equal operator.

`instanceof`
: The `instanceof` operator determines whether an object is an instance of another object.

`in`
: The `in` operator determines whether an object has a given property.

> **Note:** => is not an operator, but the notation for Arrow functions.

Equality operators

The result of evaluating an equality operator is always of type boolean based on whether the comparison is true.

==
: Equality operator.

!=
: Inequality operator.

===
: Strict equality operator.

!==
: Strict inequality operator.

Bitwise shift operators

Operations to shift all bits of the operand.

<<
: Bitwise left shift operator.

>>
: Bitwise right shift operator.

>>>
: Bitwise unsigned right shift operator.

Binary bitwise operators

Bitwise operators treat their operands as a set of 32 bits (zeros and ones) and return standard JavaScript numerical values.

&
: Bitwise AND.

|
: Bitwise OR.

^
: Bitwise XOR.

Binary logical operators

Logical operators implement boolean (logical) values and have short-circuiting behavior.

&&
: Logical AND.

||
: Logical OR.

??
: Nullish Coalescing Operator.

Conditional (ternary) operator

`(condition ? ifTrue : ifFalse)`

The conditional operator returns one of two values based on the logical value of the condition.

Assignment operators

An assignment operator assigns a value to its left operand based on the value of its right operand.

`=`

Assignment operator.

`*=`

Multiplication assignment.

`/=`

Division assignment.

`%=`

Remainder assignment.

`+=`

Addition assignment.

`-=`

Subtraction assignment

`<<=`

Left shift assignment.

`>>=`

Right shift assignment.

`>>>=`

Unsigned right shift assignment.

`&=`

Bitwise AND assignment.

`^=`

Bitwise XOR assignment.

`|=`

Bitwise OR assignment.

`**=`

Exponentiation assignment.

`&&=`

Logical AND assignment.

`||=`

Logical OR assignment.

`??=`

Nullish coalescing assignment.

`[a, b] = arr,{ a, b } = obj`

Destructuring allows you to assign the properties of an array or object to variables using syntax that looks similar to array or object literals.

Yield operators

`yield`
: Pause and resume a generator function.

`yield*`
: Delegate to another generator function or iterable object.

Spread syntax

`...obj`
: Spread syntax allows an iterable, such as an array or string, to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected. In an object literal, the spread syntax enumerates the properties of an object and adds the key-value pairs to the object being created.

Comma operator

`,`
: The comma operator allows multiple expressions to be evaluated in a single statement and returns the result of the last expression.

What are operators?

As the What are statements, declarations, and expressions? section explains, an expression is a fundamental building block that evaluates to a value. Statements, declarations, and expressions can all define specific slots where expressions are accepted. Where an expression contains slots for further nested expressions, the part(s) that are not slots are known as operators.

For example, the syntax for an addition expression is `expression + expression` (if you read the spec, the operands are called *AdditiveExpression* and *MultiplicativeExpression*, which are both subsets of *Expression*, but that's the spec's mechanism for defining precedence and associativity and is irrelevant for our purposes). Apart from the two expression slots, the code entity it introduces is just `+`: the *addition operator*. Similarly, the syntax for a yield expression is `yield expression`, so `yield` is known as the operator. In other words, each operator corresponds to an expression.

MDN also regards expressions without slots such as `null` as operators per the definition above, although we nearly always just refer to them as "syntax" or "expression".

An expression does not need to take a fixed number of slots. For example, the array literal expression, `[expression, expression, expression]`, can take an arbitrary number of expression slots. The `[,,]` part might be called an "operator". MDN avoids this usage, but you may see it in functional programming languages like Haskell.

The definition of operators gets fuzzier with certain other code entities: what if an expression has a slot that's not an expression, or a code entity combined with an expression does not make an expression? Do we still refer to that code entity as an operator?

- In the optional chaining expression `foo?.bar`, `foo` is an expression, but `bar` must be an identifier and is not evaluated to a value. Do we still regard `?.` as an operator?
- In the arrow function expression `arg => body`, `body` might be an expression (although it can also be a block body), and `arg` is just an argument list. Do we still regard `=>` as an operator?

- In the spread syntax `...foo`, `foo` is an expression, but the whole thing is not an expression because it does not evaluate to a value—it only makes sense in certain other expressions like function calls, array literals, and object literals. Do we still regard `...` as an operator?

The term "operator" is not precisely defined in JavaScript, so MDN does not give a definitive answer. Our approach is to group all these constructs under "Operators" but avoid formally referring to them as operators. Many useful concepts about operators, such as precedence, still apply to them regardless of their exact nature.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-subtraction-operator-minus
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-FormalParameters
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-generator-function-definitions-runtime-semantics-evaluation
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-generator-function-definitions
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseANDExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-equality-operators
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-typeof-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-import-calls
Deferred Imports Evaluation https://tc39.es/proposal-defer-import-eval/#sec-left-hand-side-expressions
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ArgumentList
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-PropertyDefinition
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseXORExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-SpreadElement
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-multiplicative-operators

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-left-shift-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-LogicalANDExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-MethodDefinition
HTML https://html.spec.whatwg.org/multipage/webappapis.html#import-meta-resolve
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-exp-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-generator-function-definitions
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unary-minus-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-signed-right-shift-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-this-keyword
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseORExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-YieldExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-grouping-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-void-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-addition-operator-plus
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-built-in-function-objects
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions
Source Phase Imports https://tc39.es/proposal-source-phase-imports/#sec-left-hand-side-expressions
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ComputedPropertyName
HTML https://html.spec.whatwg.org/multipage/webappapis.html#fetch-a-single-module-script
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-destructuring-assignment
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-object-initializer
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-postfix-increment-operator

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unary-plus-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-OptionalExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-CoalesceExpression
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unsigned-right-shift-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-conditional-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions
HTML https://html.spec.whatwg.org/multipage/webappapis.html#integration-with-the-javascript-module-system
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/overview.html#sec-null-value
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-logical-not-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-property-accessors
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-AssignmentRestProperty
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-destructuring-binding-patterns
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-super-keyword
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-bitwise-not-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-delete-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-comma-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ImportMeta
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-new-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-AssignmentRestElement
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-postfix-decrement-operator
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-LogicalORExpression

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Addition (+)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Addition assignment (x += y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Assignment (x = y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
async function expression	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
async function* expression	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0
await	55	14	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Use at module top level	89	89	89	75	27[1,2]	89	89	63	27[1,2]	15.0	89	27[1,2]	1.0.0	1.0	14.8.0[3]
Bitwise AND (a & b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise AND assignment (x &= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise NOT (~a)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise OR (a | b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise OR assignment (x |= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise XOR (a ^ b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise XOR assignment (x ^= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
class	42	13	45	29	7	42	45	29	7	4.0	42	7	1.0.0	1.0	6.0.0
Comma operator	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Conditional operator (c ? t : f)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Decrement (--)	2	12	1	3	4	18	4	10.1	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
delete	1	12	1	9	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Destructuring assignment	49	14	41[4]	36	8	49	41[5]	36	8	5.0	49	8	1.0.0	1.0	6.0.0
Computed property names	49	14	41	36	10	49	41	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Rest in arrays	49	16	41	36	9.1	49	41	36	9.3	5.0	49	9.3	1.0.0	1.0	6.0.0
Rest in objects	60	79	55	47	11.1	60	55	44	11.3	8.0	60	11.3	1.0.0	1.0	8.3.0
Division (/)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Division assignment (x /= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Equality (a == b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Exponentiation (**)	52	14	52	39	10.1	52	52	41	10.3	6.0	51	10.3	1.0.0	1.0	7.0.0
Exponentiation assignment (x **= y)	52	14	52	39	10.1	52	52	41	10.3	6.0	51	10.3	1.0.0	1.0	7.0.0
function expression	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
function* expression	49	12	26	36	10	49	26	36	10	5.0	49	10	1.0.0	1.0	4.0.0
Trailing comma in parameters	58	79	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
Greater than (a > b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Greater than or equal (a >= b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Grouping operator ()	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
import	63	79	67	50	11.1	63	67	46	11.3	8.0	63	11.3	1.0.0	1.0[6]	13.2.0[7,8]

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import.defer()` Experimental	**No**	**No**	Prev.[9]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?
Loading is retriable (HTTP errors are not cached)	156	156	155	140	Prev.	156	155	**No**	**No**	**No**	156	**No**	?	?	?
`import.source()` Experimental	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	24.0.0
The `options` parameter	91	91	138	77	15	91	138	64	15	16.0	91	15	1.1.5	1.17	17.5.0[10,11,12]
Available in workers	80	80	114	67	15	80	114	57	15	13.0	80	15	1.0.0	1.0	13.2.0[13]
`import.meta`	64	79	62	51	11.1	64	62	47	12	9.0	64	12	1.0.0	1.0	10.4.0
`import.meta.resolve`	105	105	106	91	16.4	105	106	72	16.4	20.0	105	16.4	1.0.0	1.24	20.8.0[14,15,16]
`in`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Increment (`++`)	2	12	1	3	4	18	4	10.1	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
Inequality (`a != b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`instanceof`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise left shift (`a << b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Left shift assignment (`x <<= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Less than (`a < b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Less than or equal (`a <= b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Logical AND (`&&`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Logical AND assignment (`x &&= y`)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
Logical NOT (`!`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Logical OR (`||`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Logical OR assignment (`x ||= y`)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
Multiplication (`*`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Multiplication assignment (`x *= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`new`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`new.target`	46	13	41	33	11	46	41	33	11	5.0	46	11	1.0.0	1.0	5.0.0
`null`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Nullish coalescing operator (`??`)	80	80	72	67	13.1	80	79	57	13.4	13.0	80	13.4	1.0.0	1.0	14.0.0
Nullish coalescing assignment (`x ??= y`)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
Object initializer	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	47	12	34	34	8	47	34	34	8	5.0	47	8	1.0.0	1.0	4.0.0
Shorthand method names	47	12	34	34	9	47	34	34	9	5.0	47	9	1.0.0	1.0	4.0.0
Shorthand property names	47	12	33	34	9	47	33	34	9	5.0	47	9	1.0.0	1.0	4.0.0
Spread properties	60	79	55	47	11.1	60	55	44	11.3	8.0	60	11.3	1.0.0	1.0	8.3.0
Optional chaining operator (`?.`)	80[17]	80[18]	74	67[19]	13.1	80[20]	79	57[21]	13.4	13.0[22]	80[23]	13.4	1.0.0	1.0	14.0.0
Property accessors	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Remainder (`%`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Remainder assignment (`x %= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise right shift (`a >> b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Right shift assignment (`x >>= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Spread syntax (...)	46	12	16	37	8	46	16	37	8	5.0	46	8	1.0.0	1.0	5.0.0
Spread in array literals	46	12	16	37	8	46	16	37	8	5.0	46	8	1.0.0	1.0	5.0.0
Spread in function calls	46	12	27	37	8	46	27	37	8	5.0	46	8	1.0.0	1.0	5.0.0
Spread in object literals	60	79	55	47	11.1	60	55	44	11.3	8.2	60	11.3	1.0.0	1.0	8.3.0
Strict equality (`a === b`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Strict inequality (`a !== b`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Subtraction (-)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Subtraction assignment (`x -= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`super`	42	13	45	29	7	42	45	29	7	4.0	42	7	1.0.0	1.0	6.0.0
`this`	1	12	1	9.5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`typeof`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unary negation (-)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unary plus (+)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Bitwise unsigned right shift (`a >>> b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Unsigned right shift assignment (`x >>= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`void`	1	12	1	4	3.1	18	4	10.1	3	1.0	4.4	3	1.0.0	1.0	0.10.0
`yield`	39	12	26[24,25]	26	10	39	26[26,27]	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`yield*`	39	12	27[24]	26	10	39	27[26]	26	10	4.0	39	10	1.0.0	1.0	4.0.0

1. Also from version 15–27: partial support.
2. Also from version 15–27: Doesn't support multiple modules simultaneously importing a module containing a top-level await (see bug 242740).
3. Not supported in CommonJS modules.
4. Firefox provided a non-standard destructuring implementation from Firefox 2 to 40.
5. Firefox for Android provided a non-standard destructuring implementation from Firefox for Android 4 to 40.
6. Bundled Deno applications (using `deno compile`) do not support dynamic imports
7. Dynamic `import` can be used in either CommonJS or ES module files, to import either CommonJS or ES module files. See Node's ECMAScript Modules documentation for more details.
8. Also from version 12.17.0–13.0.0: Dynamic `import` can be used in either CommonJS or ES module files, to import either CommonJS or ES module files. See Node's ECMAScript Modules documentation for more details.
9. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).
10. Also from version 16.15.0–17.0.0: supported.
11. Also from version 16.14.0–16.15.0: partial support.
12. Also from version 16.14.0–16.15.0: The second parameter no longer throws a parser error, but the `--experimental-json-modules` flag is still needed to load JSON modules.
13. Also from version 12.17.0–13.0.0: supported.
14. Also from version 20.6.0–20.8.0: partial support.
15. Also from version 20.6.0–20.8.0: Returns a `URL` object instead of a string.
16. Also from version 18.19.0–19.0.0: supported.
17. A call with a non-final spread argument in an optional chain fails in Chrome 80 through 90. See bug 42201552.
18. A call with a non-final spread argument in an optional chain fails in Edge 80 through 90. See bug 42201552.
19. A call with a non-final spread argument in an optional chain fails in Opera 67 through 90. See bug 42201552.
20. A call with a non-final spread argument in an optional chain fails in Chrome Android 80 through 90. See bug 42201552.
21. A call with a non-final spread argument in an optional chain fails in Opera Android 57 through 90. See bug 42201552.

22. A call with a non-final spread argument in an optional chain fails in Samsung Browser 13.0 through 90. See bug 42201552.
23. A call with a non-final spread argument in an optional chain fails in WebView Android 80 through 90. See bug 42201552.
24. Starting with Firefox 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.
25. Starting with Firefox 29, an `IteratorResult` object returned for completed generator function.
26. Starting with Firefox for Android 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.
27. Starting with Firefox for Android 29, an `IteratorResult` object returned for completed generator function.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Operator precedence

JavaScript › Reference › Expressions and operators

Addition (+)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition · MDN page last modified 2026-05-11

Baseline: Widely available — available across major browsers since July 2015.

The **addition (+)** operator produces the sum of numeric operands or string concatenation.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Addition (+) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(2 + 2);
// Expected output: 4

console.log(2 + true);
// Expected output: 3

console.log("hello " + "everyone");
// Expected output: "hello everyone"

console.log(2001 + ": A Space Odyssey");
// Expected output: "2001: A Space Odyssey"
```

Syntax

js

```
x + y
```

Description

The + operator is overloaded for two distinct operations: numeric addition and string concatenation. When evaluating, it first coerces both operands to primitives. Then, the two operands' types are tested:

- If one side is a string, the other operand is also converted to a string and they are concatenated.
- If they are both BigInts, BigInt addition is performed. If one side is a BigInt but the other is not, a `TypeError` is thrown.
- Otherwise, both sides are converted to numbers, and numeric addition is performed.

String concatenation is often thought to be equivalent with template literals or `String.prototype.concat()`, but they are not. Addition coerces the expression to a *primitive*, which

calls `valueOf()` in priority; on the other hand, template literals and `concat()` coerce the expression to a *string*, which calls `toString()` in priority. If the expression has a `[Symbol.toPrimitive]()` method, string concatenation calls it with `"default"` as hint, while template literals use `"string"`. This is important for objects that have different string and primitive representations — such as `Temporal`, whose objects' `valueOf()` methods all throw.

```js
const t = Temporal.Now.instant();
"" + t; // Throws TypeError
`${t}`; // '2022-07-31T04:48:56.113918308Z'
"".concat(t); // '2022-07-31T04:48:56.113918308Z'
```

You are advised to not use `"" + x` to perform string coercion.

Examples

Addition using numbers

```js
1 + 2; // 3
```

Other non-string, non-BigInt values are coerced to numbers:

```js
true + 1; // 2
false + false; // 0
```

Addition using BigInts

```js
1n + 2n; // 3n
```

You cannot mix BigInt and number operands in addition. `null`, `undefined`, and boolean values are coerced to numbers and are forbidden as well.

js · ✗ bad example

```
1n + 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 + 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Strings have priority over other types, so adding a string to a BigInt produces string concatenation rather than a `TypeError`.

```js
"1" + 2n; // "12"
```

To do addition with a BigInt and a non-BigInt, convert either operand:

```js
1n + BigInt(2); // 3n
Number(1n) + 2; // 3
```

Addition using strings

If one of the operands is a string, the other is converted to a string and they are concatenated:

```js
"foo" + "bar"; // "foobar"
5 + "foo"; // "5foo"
"foo" + false; // "foofalse"
"2" + 2; // "22"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-addition-operator-plus

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Addition (+)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.addition`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Addition assignment (+=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **addition assignment (+=)** operator performs addition (which is either numeric addition or string concatenation) on the two operands and assigns the result to the left operand.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Addition assignment (+=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a = 2;
let b = "hello";

console.log((a += 3)); // Addition
// Expected output: 5

console.log((b += " world")); // Concatenation
// Expected output: "hello world"
```

Syntax

js

```
x += y
```

Description

`x += y` is equivalent to `x = x + y`, except that the expression `x` is only evaluated once.

Examples

Addition assignment using numbers

js

```
let bar = 5;
bar += 2; // 7
```

Other non-string, non-BigInt values are coerced to numbers:

js

```
let baz = true;
baz += 1; // 2
baz += false; // 2
```

Addition assignment using BigInts

js

```
let x = 1n;
x += 2n; // 3n

x += 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Addition assignment using strings

js

```
let foo = "foo";
foo += false; // "foofalse"
foo += "bar"; // "foofalsebar"

let bar = 5;
bar += "foo"; // "5foo"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Addition assignment (x += y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.addition_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Addition (+)

JavaScript › Reference › Expressions and operators

Assignment (=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **assignment (=)** operator is used to assign a value to a variable or property. The assignment expression itself has a value, which is the assigned value. This allows multiple assignments to be chained in order to assign a single value to multiple variables.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Assignment (=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let x = 2;
const y = 3;

console.log(x);
// Expected output: 2

console.log((x = y + 1)); // 3 + 1
// Expected output: 4

console.log((x = x * y)); // 4 * 3
// Expected output: 12
```

Syntax

```js
x = y
```

Parameters

x

A valid assignment target, including an identifier or a property accessor. It can also be a destructuring pattern.

y

An expression specifying the value to be assigned to `x`.

Return value

The value of `y`.

Exceptions

`ReferenceError`

Thrown in strict mode if assigning to an identifier that is not declared in the scope.

`TypeError`

Thrown in strict mode if assigning to a property that is not modifiable.

Description

The assignment operator is completely different from the equals (`=`) sign used as syntactic separators in other locations, which include:

- Initializers of `var`, `let`, and `const` declarations
- Default values of destructuring
- Default parameters
- Initializers of class fields

All these places accept an assignment expression on the right-hand side of the `=`, so if you have multiple equals signs chained together:

```js
const x = y = 5;
```

This is equivalent to:

```js
const x = (y = 5);
```

Which means `y` must be a pre-existing variable, and `x` is a newly declared `const` variable. `y` is assigned the value `5`, and `x` is initialized with the value of the `y = 5` expression, which is also `5`. If `y` is not a pre-existing variable, a global variable `y` is implicitly created in non-strict mode, or a `ReferenceError` is thrown in strict mode. To declare two variables within the same declaration, use:

```js
const x = 5,
  y = 5;
```

Examples

Basic assignment and chaining

```js
let x = 5;
let y = 10;
let z = 25;

x = y; // x is 10
x = y = z; // x, y and z are all 25
```

Value of assignment expressions

The assignment expression itself evaluates to the value of the right-hand side, so you can log the value and assign to a variable at the same time.

```js
let x;
console.log(x); // undefined
console.log(x = 2); // 2
console.log(x); // 2
```

Unqualified identifier assignment

The global object sits at the top of the scope chain. When attempting to resolve a name to a value, the scope chain is searched. This means that properties on the global object are conveniently visible from every scope, without having to qualify the names with `globalThis.` or `window.` or `global.`.

Because the global object has a `String` property (`Object.hasOwn(globalThis, "String")`), you can use the following code:

```js
function foo() {
  String("s"); // The function `String` is globally available
}
```

So the global object will ultimately be searched for unqualified identifiers. You don't have to type `globalThis.String`; you can just type the unqualified `String`. To make this feature more conceptually consistent, assignment to unqualified identifiers will assume you want to create a property with that name on the global object (with `globalThis.` omitted), if there is no variable of the same name declared in the scope chain.

```js
foo = "f"; // In non-strict mode, assumes you want to create a property named `foo` on the
global object
Object.hasOwn(globalThis, "foo"); // true
```

In strict mode, assignment to an unqualified identifier in strict mode will result in a `ReferenceError`, to avoid the accidental creation of properties on the global object.

Note that the implication of the above is that, contrary to popular misinformation, JavaScript does not have implicit or undeclared variables. It just conflates the global object with the global scope and allows omitting the global object qualifier during property creation.

Assignment with destructuring

The left-hand side can also be an assignment pattern. This allows assigning to multiple variables at once.

```js
const result = /(a+)(b+)(c+)/.exec("aaabcc");
let a = "",
  b = "",
  c = "";
[, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Assignment (`x = y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.assignment`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assignment operators in the JS guide
- Destructuring

JavaScript › Reference › Expressions and operators

async function expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/async_function · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since April 2017.

The **`async function`** keywords can be used to define an async function inside an expression.

You can also define async functions using the `async function` declaration or the arrow syntax.

Syntax

```js
async function (param0) {
  statements
}
async function (param0, param1) {
  statements
}
async function (param0, param1, /* …, */ paramN) {
  statements
}

async function name(param0) {
  statements
}
async function name(param0, param1) {
  statements
}
async function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** An expression statement cannot begin with the keywords `async function` to avoid ambiguity with an `async function` declaration. The `async function` keywords only begin an expression when they appear in a context that cannot accept statements.

Parameters

`name` Optional

The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

`paramN` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

An `async function` expression is very similar to, and has almost the same syntax as, an `async function` declaration. The main difference between an `async function` expression and an `async function` declaration is the *function name*, which can be omitted in `async function` expressions to create *anonymous* functions. An `async function` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined, allowing you to mimic top-level await. See also the chapter about functions for more information.

Examples

Using async function expression

```js
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

// async function expression assigned to a variable
const add = async function (x) {
  const a = await resolveAfter2Seconds(20);
  const b = await resolveAfter2Seconds(30);
  return x + a + b;
};

add(10).then((v) => {
  console.log(v); // prints 60 after 4 seconds.
});

// async function expression used as an IIFE
(async function (x) {
  const p1 = resolveAfter2Seconds(20);
  const p2 = resolveAfter2Seconds(30);
  return x + (await p1) + (await p2);
})(10).then((v) => {
  console.log(v); // prints 60 after 2 seconds.
});
```

Async IIFE

An `async` IIFE allows you to use `await` and `for...await` in contexts where top-level await is not available. Here we use an arrow function to define the IIFE, but `async function` expressions can also be used.

```js
const getFileStream = async (url) => {
  // implementation
};

(async () => {
  const stream = await getFileStream("https://domain.name/path/file.ext");
  for await (const chunk of stream) {
    console.log({ chunk });
  }
})();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`async function` expression	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.async_function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `async function`
- `AsyncFunction`
- `await`

JavaScript › Reference › Expressions and operators

async function* expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/async_function* · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since January 2020.

The **`async function*`** keywords can be used to define an async generator function inside an expression.

You can also define async generator functions using the `async function*` declaration.

Try it

▷ Interactive example on MDN: "JavaScript Demo: async function expression". It runs live in the browser there; its source code follows.*

js · interactive example source

```
async function joinAll(generator) {
  let str = "";
  for await (const val of generator()) {
    str += val;
  }
  return str;
}
joinAll(async function* () {
  yield await Promise.resolve("a");
  yield await Promise.resolve("b");
  yield await Promise.resolve("c");
}).then((str) => console.log(str));
// Expected output: "abc"
```

Syntax

```js
async function* (param0) {
  statements
}
async function* (param0, param1) {
  statements
}
async function* (param0, param1, /* …, */ paramN) {
  statements
}

async function* name(param0) {
  statements
}
async function* name(param0, param1) {
  statements
}
async function* name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** An expression statement cannot begin with the keywords `async function` to avoid ambiguity with an `async function*` declaration. The `async function` keywords only begin an expression when they appear in a context that cannot accept statements.

Parameters

`name` Optional

The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

`paramN` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

An `async function*` expression is very similar to, and has almost the same syntax as, an `async function*` declaration. The main difference between an `async function*` expression and an `async function*` declaration is the *function name*, which can be omitted in `async function*` expressions to create *anonymous* functions. An `async function*` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined, allowing you to create an ad-hoc async iterable object. See also the chapter about functions for more information.

Examples

Using async function* expression

The following example defines an unnamed asynchronous generator function and assigns it to x. The function yields the square of its argument:

```js
const x = async function* (y) {
  yield Promise.resolve(y * y);
};
x(6)
  .next()
  .then((res) => console.log(res.value)); // 36
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-generator-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
async function* expression	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.async_generator_function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `async function*`
- `AsyncGeneratorFunction`
- Iteration protocols
- `yield`
- `yield*`

JavaScript › Reference › Expressions and operators

await

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since March 2017.

The **`await`** operator is used to wait for a `Promise` and get its fulfillment value. It can only be used inside an async function or at the top level of a module.

Syntax

```js
await expression
```

Parameters

`expression`

A `Promise`, a thenable object, or any value to wait for.

Return value

The fulfillment value of the promise or thenable object, or, if the expression is not thenable, the expression's own value.

Exceptions

Throws the rejection reason if the promise or thenable object is rejected.

Description

`await` is usually used to unwrap promises by passing a `Promise` as the `expression`. Using `await` pauses the execution of its surrounding `async` function until the promise is settled (that is, fulfilled or rejected). When execution resumes, the value of the `await` expression becomes that of the fulfilled promise.

If the promise is rejected, the `await` expression throws the rejected value. The function containing the `await` expression will appear in the stack trace of the error. Otherwise, if the rejected promise is not awaited or is immediately returned, the caller function will not appear in the stack trace.

The `expression` is resolved in the same way as `Promise.resolve()`: it's always converted to a native `Promise` and then awaited. If the `expression` is a:

- Native `Promise` (which means `expression` belongs to `Promise` or a subclass, and `expression.constructor === Promise`): The promise is directly used and awaited natively, without calling `then()`.
- Thenable object (including non-native promises, polyfill, proxy, child class, etc.): A new promise is constructed with the native `Promise()` constructor by calling the object's `then()` method and passing in a handler that calls the `resolve` callback.
- Non-thenable value: An already-fulfilled `Promise` is constructed and used.

Even when the used promise is already fulfilled, the async function's execution still pauses until the next tick. In the meantime, the caller of the async function resumes execution. See example below.

Because `await` is only valid inside async functions and modules, which themselves are asynchronous and return promises, the `await` expression never blocks the main thread and only defers execution of

code that actually depends on the result, i.e., anything after the `await` expression.

Examples

Awaiting a promise to be fulfilled

If a `Promise` is passed to an `await` expression, it waits for the `Promise` to be fulfilled and returns the fulfilled value.

```js
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

async function f1() {
  const x = await resolveAfter2Seconds(10);
  console.log(x); // 10
}

f1();
```

Thenable objects

Thenable objects are resolved just the same as actual `Promise` objects.

```js
async function f2() {
  const thenable = {
    then(resolve) {
      resolve("resolved!");
    },
  };
  console.log(await thenable); // "resolved!"
}

f2();
```

They can also be rejected:

```js
async function f2() {
  const thenable = {
    then(_, reject) {
      reject(new Error("rejected!"));
    },
  };
  await thenable; // Throws Error: rejected!
}

f2();
```

Conversion to promise

If the value is not a `Promise`, `await` converts the value to a resolved `Promise`, and waits for it. The awaited value's identity doesn't change as long as it doesn't have a `then` property that's callable.

```js
async function f3() {
  const y = await 20;
  console.log(y); // 20

  const obj = {};
  console.log((await obj) === obj); // true
}

f3();
```

Handling rejected promises

If the `Promise` is rejected, the rejected value is thrown.

```js
async function f4() {
  try {
    const z = await Promise.reject(new Error("rejected!"));
  } catch (e) {
    console.error(e); // Error: rejected!
  }
}

f4();
```

You can handle rejected promises without a `try` block by chaining a `catch()` handler before awaiting the promise.

```js
const response = await promisedFunction().catch((err) => {
  console.error(err);
  return "default response";
});
// response will be "default response" if the promise is rejected
```

This is built on the assumption that `promisedFunction()` never synchronously throws an error, but always returns a rejected promise. This is the case for most properly-designed promise-based functions, which usually look like:

```js
function promisedFunction() {
  // Immediately return a promise to minimize chance of an error being thrown
  return new Promise((resolve, reject) => {
    // do something async
  });
}
```

However, if `promisedFunction()` does throw an error synchronously, the error won't be caught by the `catch()` handler. In this case, the `try...catch` statement is necessary.

Top-level await

You can use the `await` keyword on its own (outside of an async function) at the top level of a module. This means that modules with child modules that use `await` will wait for the child modules to execute before they themselves run, all while not blocking sibling modules that don't depend on those child modules from executing.

Here is an example of a module using the Fetch API and specifying await within the `export` statement. Any modules that include this will wait for the fetch to resolve before running any code.

```js
// fetch request
const colors = fetch(
  "https://mdn.github.io/js-examples/module-examples/top-level-await/data/colors.json",
).then((response) => response.json());

export default await colors;
```

Top-level `await` is not free; it has deep implications because it means a part of the module graph can only be evaluated asynchronously. See the module graph guide for more information.

Control flow effects of await

When an `await` is encountered in code (either in an async function or in a module), the awaited expression is executed, while all code that depends on the expression's value is paused. Control exits the function and returns to the caller. When the awaited expression's value is resolved, another microtask that continues the paused code gets scheduled. This happens even if the awaited value is an already-resolved promise or not a promise: execution doesn't return to the current function until all other already-scheduled microtasks are processed. For example, consider the following code:

```js
async function foo(name) {
  console.log(name, "start");
  console.log(name, "middle");
  console.log(name, "end");
}

foo("First");
foo("Second");

// First start
// First middle
// First end
// Second start
// Second middle
// Second end
```

In this case, the function `foo` is synchronous in effect, because it doesn't contain any `await` expression. The three statements happen in the same tick. Therefore, the two function calls execute all statements in sequence. In promise terms, the function corresponds to:

```js
function foo(name) {
  return new Promise((resolve) => {
    console.log(name, "start");
    console.log(name, "middle");
    console.log(name, "end");
    resolve();
  });
}
```

However, as soon as there's one `await`, the function becomes asynchronous, and execution of following statements is deferred to the next tick.

```js
async function foo(name) {
  console.log(name, "start");
  await console.log(name, "middle");
  console.log(name, "end");
}

foo("First");
foo("Second");

// First start
// First middle
// Second start
// Second middle
// First end
// Second end
```

This corresponds to:

```js
function foo(name) {
  return new Promise((resolve) => {
    console.log(name, "start");
    resolve(console.log(name, "middle"));
  }).then(() => {
    console.log(name, "end");
  });
}
```

The extra `then()` handler can be merged with the executor passed to the constructor because it's not waiting on any asynchronous operation. However, its existence splits the code into one additional microtask for each call to `foo`. These microtasks are scheduled and executed in an intertwined manner, which can both make your code slower and introduce unnecessary race conditions. Therefore, make sure to use `await` only when necessary (to unwrap promises into their values).

Microtasks are scheduled not only by promise resolution but by other web APIs as well, and they execute with the same priority. This example uses `queueMicrotask()` to demonstrate how the microtask queue is processed when each `await` expression is encountered.

```js
let i = 0;
```

(code continues on the next page)

(code continued from the previous page)

```
queueMicrotask(function test() {
  i++;
  console.log("microtask", i);
  if (i < 3) {
    queueMicrotask(test);
  }
});

(async () => {
  console.log("async function start");
  for (let i = 1; i < 3; i++) {
    await null;
    console.log("async function resume", i);
  }
  await null;
  console.log("async function end");
})();

queueMicrotask(() => {
  console.log("queueMicrotask() after calling async function");
});

console.log("script sync part end");

// Logs:
// async function start
// script sync part end
// microtask 1
// async function resume 1
// queueMicrotask() after calling async function
// microtask 2
// async function resume 2
// microtask 3
// async function end
```

In this example, the `test()` function is always called before the async function resumes, so the microtasks they each schedule are always executed in an intertwined fashion. On the other hand, because both `await` and `queueMicrotask()` schedule microtasks, the order of execution is always based on the order of scheduling. This is why the "queueMicrotask() after calling async function" log happens after the async function resumes for the first time.

Improving stack trace

Sometimes, the `await` is omitted when a promise is directly returned from an async function.

js

```
async function noAwait() {
  // Some actions...

  return /* await */ lastAsyncTask();
}
```

However, consider the case where `lastAsyncTask` asynchronously throws an error.

```js
async function lastAsyncTask() {
  await null;
  throw new Error("failed");
}

async function noAwait() {
  return lastAsyncTask();
}

noAwait();

// Error: failed
//    at lastAsyncTask
```

Only `lastAsyncTask` appears in the stack trace, because the promise is rejected after it has already been returned from `noAwait` — in some sense, the promise is unrelated to `noAwait`. To improve the stack trace, you can use `await` to unwrap the promise, so that the exception gets thrown into the current function. The exception will then be immediately wrapped into a new rejected promise, but during error creation, the caller will appear in the stack trace.

```js
async function lastAsyncTask() {
  await null;
  throw new Error("failed");
}

async function withAwait() {
  return await lastAsyncTask();
}

withAwait();

// Error: failed
//    at lastAsyncTask
//    at async withAwait
```

Contrary to some popular belief, `return await promise` is at least as fast as `return promise`, due to how the spec and engines optimize the resolution of native promises. There's a proposal to make `return promise` faster and you can also read about V8's optimization on async functions. Therefore, except for stylistic reasons, `return await` is almost always preferable.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`await`	55	14	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Use at module top level	89	89	89	75	27[1,2]	89	89	63	27[1,2]	15.0	89	27[1,2]	1.0.0	1.0	14.8.0[3]

1. Also from version 15–27: partial support.
2. Also from version 15–27: Doesn't support multiple modules simultaneously importing a module containing a top-level await (see bug 242740).
3. Not supported in CommonJS modules.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.await`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `async function`
- `async function` expression
- `AsyncFunction`
- Top-level await on v8.dev (2019)
- typescript-eslint rule: `return-await`

JavaScript › Reference › Expressions and operators

Bitwise AND (&)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_AND · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise AND (`&`)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of both operands are `1`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Bitwise AND (&) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a & b); // 00000000000000000000000000000001
// Expected output: 1
```

Syntax

js

```
x & y
```

Description

The & operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt AND if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise AND. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the AND operation is:

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 & 9 (base 10) = 00000000000000000000000000001000 (base 2) = 8 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `& -1` to truncate numbers to integers. Bitwise ANDing any number x with `-1` returns x converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise AND

js

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 & 9;
// 8  (00000000000000000000000000001000)

14n & 9n; // 8n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseANDExpression

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise AND (a & b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_and`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide
- Bitwise AND assignment (`&=`)

JavaScript › Reference › Expressions and operators

Bitwise AND assignment (&=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_AND_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise AND assignment (`&=`)** operator performs bitwise AND on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Bitwise AND assignment (&=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 5; // 00000000000000000000000000000101
a &= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000001
// Expected output: 1
```

Syntax

js

```
x &= y
```

Description

`x &= y` is equivalent to `x = x & y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise AND assignment

```js
let a = 5;
// 5:     00000000000000000000000000000101
// 2:     00000000000000000000000000000010
a &= 2; // 0

let b = 5n;
b &= 2n; // 0n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise AND assignment (x &= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_and_assignment`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Bitwise AND (&)

JavaScript › Reference › Expressions and operators

Bitwise NOT (~)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_NOT · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise NOT (~)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bit of the operand is `0`, and a `0` otherwise.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Bitwise NOT (~) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 5; // 00000000000000000000000000000101
const b = -3; // 11111111111111111111111111111101

console.log(~a); // 11111111111111111111111111111010
// Expected output: -6

console.log(~b); // 00000000000000000000000000000010
// Expected output: 2
```

Syntax

js

```
~x
```

Description

The ~ operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt NOT if the operand becomes a BigInt; otherwise, it converts the operand to a 32-bit integer and performs number bitwise NOT.

The operator operates on the operands' bit representations in two's complement. The operator is applied to each bit, and the result is constructed bitwise.

The truth table for the NOT operation is:

x	NOT x
0	1
1	0

```
 9 (base 10) = 00000000000000000000000000001001 (base 2)
               --------------------------------
~9 (base 10) = 11111111111111111111111111110110 (base 2) = -10 (base 10)
```

Bitwise NOTing any 32-bit integer x yields `-(x + 1)`. For example, `~-5` yields `4`.

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using ~~ to truncate numbers to integers. Bitwise NOTing any number x twice returns x converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise NOT

```js
~0; // -1
~-1; // 0
~1; // -2

~0n; // -1n
~4294967295n; // -4294967296n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-bitwise-not-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise NOT (~a)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_not`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide

JavaScript › Reference › Expressions and operators

Bitwise OR (|)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_OR · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise OR (|)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of either or both operands are `1`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Bitwise OR (|) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a | b); // 00000000000000000000000000000111
// Expected output: 7
```

Syntax

js

```
x | y
```

Description

The | operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt OR if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise OR. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the OR operation is:

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 | 9 (base 10) = 00000000000000000000000000001111 (base 2) = 15 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `| 0` to truncate numbers to integers. Bitwise ORing any number `x` with `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise OR

```js
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 | 9;
// 15 (00000000000000000000000000001111)

14n | 9n; // 15n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseORExpression`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise OR (a | b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_or`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Bitwise operators in the JS guide
- Bitwise OR assignment (|=)

JavaScript › Reference › Expressions and operators

Bitwise OR assignment (|=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_OR_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise OR assignment (|=)** operator performs bitwise OR on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Bitwise OR assignment (|=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 5; // 00000000000000000000000000000101
a |= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000111
// Expected output: 7
```

Syntax

js

```
x |= y
```

Description

`x |= y` is equivalent to `x = x | y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise OR assignment

js

```
let a = 5;
a |= 2; // 7
// 5: 00000000000000000000000000000101
// 2: 00000000000000000000000000000010
// ----------------------------------
// 7: 00000000000000000000000000000111

let b = 5n;
b |= 2n; // 7n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise OR assignment (`x |= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_or_assignment`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assignment operators in the JS guide
- Bitwise OR (|)
- Logical OR assignment (||=)

JavaScript › Reference › Expressions and operators

Bitwise XOR (^)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_XOR · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise XOR (^)** operator returns a number or BigInt whose binary representation has a `1` in each bit position for which the corresponding bits of either but not both operands are `1`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Bitwise XOR (^) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 5; // 00000000000000000000000000000101
const b = 3; // 00000000000000000000000000000011

console.log(a ^ b); // 00000000000000000000000000000110
// Expected output: 6
```

Syntax

js

```
x ^ y
```

Description

The ^ operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt XOR if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number bitwise XOR. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the operands' bit representations in two's complement. Each bit in the first operand is paired with the corresponding bit in the second operand: *first bit* to *first bit*, *second bit* to *second bit*, and so on. The operator is applied to each pair of bits, and the result is constructed bitwise.

The truth table for the XOR operation is:

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

```
     9 (base 10) = 00000000000000000000000000001001 (base 2)
    14 (base 10) = 00000000000000000000000000001110 (base 2)
                   --------------------------------
14 ^ 9 (base 10) = 00000000000000000000000000000111 (base 2) = 7 (base 10)
```

Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

> **Warning:** You may see people using `^ 0` to truncate numbers to integers. Bitwise XORing any number `x` with `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using bitwise XOR

js

```
// 9  (00000000000000000000000000001001)
// 14 (00000000000000000000000000001110)

14 ^ 9;
// 7  (00000000000000000000000000000111)

14n ^ 9n; // 7n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-BitwiseXORExpression`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise XOR (a ^ b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_xor`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide
- Bitwise XOR assignment (^=)

JavaScript › Reference › Expressions and operators

Bitwise XOR assignment (^=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_XOR_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **bitwise XOR assignment (^=)** operator performs bitwise XOR on the two operands and assigns the result to the left operand.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Bitwise XOR assignment (^=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a = 5; // 00000000000000000000000000000101
a ^= 3; // 00000000000000000000000000000011

console.log(a); // 00000000000000000000000000000110
// Expected output: 6
```

Syntax

js

```
x ^= y
```

Description

`x ^= y` is equivalent to `x = x ^ y`, except that the expression `x` is only evaluated once.

Examples

Using bitwise XOR assignment

js

```
let a = 5; // (00000000000000000000000000000101)
a ^= 3; // (00000000000000000000000000000011)

console.log(a); // 6 (00000000000000000000000000000110)

let b = 5; // (00000000000000000000000000000101)
b ^= 0; // (00000000000000000000000000000000)

console.log(b); // 5 (00000000000000000000000000000101)

let c = 5n;
c ^= 3n;
console.log(c); // 6n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise XOR assignment (`x ^= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.bitwise_xor_assignment`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assignment operators in the JS guide
- Bitwise XOR (^)

JavaScript › Reference › Expressions and operators

class expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/class · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2016.

The **`class`** keyword can be used to define a class inside an expression.

You can also define classes using the `class` declaration.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: class expression". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const Rectangle = class {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
  area() {
    return this.height * this.width;
  }
};

console.log(new Rectangle(5, 8).area());
// Expected output: 40
```

Syntax

```
class {
  // class body
}
class name {
  // class body
}
```

Note: An expression statement cannot begin with the keyword `class` to avoid ambiguity with a `class` declaration. The `class` keyword only begins an expression when it appears in a context that cannot accept statements.

Description

A `class` expression is very similar to, and has almost the same syntax as, a `class` declaration. As with `class` declarations, the body of a `class` expression is executed in strict mode. The main difference between a `class` expression and a `class` declaration is the *class name*, which can be omitted in `class` expressions to create *anonymous* classes. Class expressions allow you to redefine classes, while redeclaring a class using `class` declarations throws a `SyntaxError`. See also the chapter about classes for more information.

Examples

A basic class expression

This is just an anonymous class expression which you can refer to using the variable `Foo`.

```
const Foo = class {
  constructor() {}
  bar() {
    return "Hello World!";
  }
};

const instance = new Foo();
instance.bar(); // "Hello World!"
Foo.name; // "Foo"
```

Named class expressions

If you want to refer to the current class inside the class body, you can create a *named class expression*. The name is only visible within the scope of the class expression itself.

```js
const Foo = class NamedFoo {
  constructor() {}
  whoIsThere() {
    return NamedFoo.name;
  }
};
const bar = new Foo();
bar.whoIsThere(); // "NamedFoo"
NamedFoo.name; // ReferenceError: NamedFoo is not defined
Foo.name; // "NamedFoo"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
class	42	13	45	29	7	42	45	29	7	4.0	42	7	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.class`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `class`
- Classes

JavaScript › Reference › Expressions and operators

Comma operator (,)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Comma_operator · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **comma (,)** operator evaluates each of its operands (from left to right) and returns the value of the last operand. This is commonly used to provide multiple updaters to a `for` loop's afterthought.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Comma (,) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let x = 1;

x = (x++, x);

console.log(x);
// Expected output: 2

x = (2, 3);

console.log(x);
// Expected output: 3
```

Syntax

js

```
expr1, expr2, expr3/* , … */
```

Parameters

expr1, expr2, expr3, ...

One or more expressions, the last of which is returned as the value of the compound expression.

Description

You can use the comma operator when you want to include multiple expressions in a location that requires a single expression. The most common usage of this operator is to supply multiple updaters in a `for` loop. For an idiom allowing multiple *statements* in a location that requires a single expression, you may use an IIFE.

Because all expressions except the last are evaluated and then discarded, these expressions must have side effects to be useful. Common expressions that have side effects are assignments, function calls, and `++` and `--` operators. Others may also have side effects if they invoke getters or trigger type coercions.

The comma operator has the lowest precedence of all operators. If you want to incorporate a comma-joined expression into a bigger expression, you must parenthesize it.

The comma operator is completely different from commas used as syntactic separators in other locations, which include:

- Elements in array initializers (`[1, 2, 3]`)
- Properties in object initializers (`{ a: 1, b: 2 }`)
- Parameters in function declarations/expressions (`function f(a, b) { … }`)
- Arguments in function calls (`f(1, 2)`)
- Binding lists in `let`, `const`, or `var` declarations (`const a = 1, b = 2;`)
- Import lists in `import` declarations (`import { a, b } from "c";`)
- Export lists in `export` declarations (`export { a, b };`)

In fact, although some of these places accept almost all expressions, they don't accept comma-joined expressions because that would be ambiguous with the syntactic comma separators. In this case, you

must parenthesize the comma-joined expression. For example, the following is a `const` declaration that declares two variables, where the comma is not the comma operator:

```js
const a = 1, b = 2;
```

It is different from the following, where `b = 2` is an assignment expression, not a declaration. The value of `a` is `2`, the return value of the assignment, while the value of `1` is discarded:

```js
const a = (1, b = 2);
```

Comma operators cannot appear as trailing commas.

Examples

Using the comma operator in a for loop

If `a` is a 2-dimensional array with 10 elements on each side, the following code uses the comma operator to increment `i` and decrement `j` at once, thus printing the values of the diagonal elements in the array:

```js
const a = Array.from({ length: 10 }, () =>
  Array.from({ length: 10 }, Math.random),
); // A 10×10 array of random numbers

for (let i = 0, j = 9; i <= 9; i++, j--) {
  console.log(`a[${i}][${j}] = ${a[i][j]}`);
}
```

Using the comma operator to join assignments

Because commas have the lowest precedence — even lower than assignment — commas can be used to join multiple assignment expressions. In the following example, `a` is set to the value of `b = 3` (which is 3). Then, the `c = 4` expression evaluates and its result becomes the return value of the entire comma expression.

```js
let a, b, c;

a = b = 3, c = 4; // Returns 4
console.log(a); // 3 (left-most)

let x, y, z;

x = (y = 5, z = 6); // Returns 6
console.log(x); // 6 (right-most)
```

Processing and then returning

Another example that one could make with the comma operator is processing before returning. As stated, only the last element will be returned but all others are going to be evaluated as well. So, one could do:

```js
function myFunc() {
  let x = 0;

  return (x += 1, x); // the same as return ++x;
}
```

This is especially useful for one-line arrow functions. The following example uses a single `map()` to get both the sum of an array and the squares of its elements, which would otherwise require two iterations, one with `reduce()` and one with `map()`:

```js
let sum = 0;
const squares = [1, 2, 3, 4, 5].map((x) => ((sum += x), x * x));
console.log(squares); // [1, 4, 9, 16, 25]
console.log(sum); // 15
```

Discarding reference binding

The comma operator always returns the last expression as a *value* instead of a *reference*. This causes some contextual information such as the `this` binding to be lost. For example, a property access returns a reference to the function, which also remembers the object that it's accessed on, so that calling the property works properly. If the method is returned from a comma expression, then the function is called as if it's a new function value, and `this` is `undefined`.

```js
const obj = {
  value: "obj",
  method() {
    console.log(this.value);
  },
};

obj.method(); // "obj"
(obj.method)(); // "obj" (the grouping operator still returns the reference)
(0, obj.method)(); // undefined (the comma operator returns a new value)
```

You can enter indirect eval with this technique, because direct eval requires the function call to happen on the reference to the `eval()` function.

```js
globalThis.isDirectEval = false;

{
  const isDirectEval = true;
  console.log(eval("isDirectEval")); // true
  console.log((eval)("isDirectEval")); // true (the grouping operator still returns a
reference to `eval`)
  console.log((0, eval)("isDirectEval")); // false (the comma operator returns a new value)
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-comma-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Comma operator	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.comma`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `for`
- IIFE

JavaScript › Reference › Expressions and operators

Conditional (ternary) operator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Conditional_operator · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **conditional (ternary) operator** is the only JavaScript operator that takes three operands: a condition followed by a question mark (`?`), then an expression to execute if the condition is truthy followed by a colon (`:`), and finally the expression to execute if the condition is falsy. This operator is frequently used as an alternative to an `if...else` statement.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Conditional operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
function getFee(isMember) {
  return isMember ? "$2.00" : "$10.00";
}

console.log(getFee(true));
// Expected output: "$2.00"

console.log(getFee(false));
// Expected output: "$10.00"

console.log(getFee(null));
// Expected output: "$10.00"
```

Syntax

js

```
condition ? exprIfTrue : exprIfFalse
```

Parameters

`condition`

An expression whose value is used as a condition.

`exprIfTrue`

An expression which is executed if the `condition` evaluates to a truthy value (one which equals or can be converted to `true`).

`exprIfFalse`

An expression which is executed if the `condition` is falsy (that is, has a value which can be converted to `false`).

Description

Besides `false`, possible falsy expressions are: `null`, `NaN`, `0`, the empty string (`""`), and `undefined`. If `condition` is any of these, the result of the conditional expression will be the result of executing the expression `exprIfFalse`.

Examples

A basic example

```js
const age = 26;
const beverage = age >= 21 ? "Beer" : "Juice";
console.log(beverage); // "Beer"
```

Handling null values

One common usage is to handle a value that may be `null`:

```js
const greeting = (person) => {
  const name = person ? person.name : "stranger";
  return `Howdy, ${name}`;
};

console.log(greeting({ name: "Alice" })); // "Howdy, Alice"
console.log(greeting(null)); // "Howdy, stranger"
```

Conditional chains

The ternary operator is right-associative, which means it can be "chained" in the following way, similar to an `if … else if … else if … else` chain:

```js
function example() {
  return condition1 ? value1
    : condition2 ? value2
    : condition3 ? value3
    : value4;
}
```

This is equivalent to the following `if...else` chain.

```js
function example() {
  if (condition1) {
    return value1;
  } else if (condition2) {
    return value2;
  } else if (condition3) {
    return value3;
  } else {
    return value4;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-conditional-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Conditional operator (`c ? t : f`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.conditional`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `if...else`
- Nullish coalescing operator (`??`)
- Optional chaining (`?.`)
- Learn: Making decisions in your code — conditionals
- Expressions and operators guide

JavaScript › Reference › Expressions and operators

Decrement (--)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Decrement · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **decrement (`--`)** operator decrements (subtracts one from) its operand and returns the value before or after the decrement, depending on where the operator is placed.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Decrement (--) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let x = 3;
const y = x--;

console.log(`x:${x}, y:${y}`);
// Expected output: "x:2, y:3"

let a = 3;
const b = --a;

console.log(`a:${a}, b:${b}`);
// Expected output: "a:2, b:2"
```

Syntax

js

```
x--
--x
```

Description

The `--` operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt decrement if the operand becomes a BigInt; otherwise, it performs number decrement.

If used postfix, with operator after operand (for example, `x--`), the decrement operator decrements and returns the value before decrementing.

If used prefix, with operator before operand (for example, `--x`), the decrement operator decrements and returns the value after decrementing.

The decrement operator can only be applied on operands that are references (variables and object properties; i.e., valid assignment targets). `--x` itself evaluates to a value, not a reference, so you cannot chain multiple decrement operators together.

js · ✗ bad example

```
--(--x); // SyntaxError: Invalid left-hand side expression in prefix operation
```

Examples

Postfix decrement

js

```
let x = 3;
const y = x--;
// x is 2; y is 3

let x2 = 3n;
const y2 = x2--;
// x2 is 2n; y2 is 3n
```

Prefix decrement

```js
let x = 3;
const y = --x;
// x is 2; y = 2

let x2 = 3n;
const y2 = --x2;
// x2 is 2n; y2 is 2n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-postfix-decrement-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Decrement (--)	2	12	1	3	4	18	4	10.1	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.decrement`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

delete

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/delete · MDN page last modified 2025-07-17

Baseline: Widely available — available across major browsers since July 2015.

The **`delete`** operator removes a property from an object. If the property's value is an object and there are no more references to the object, the object held by that property is eventually released automatically.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: delete operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const employee = {
  firstName: "Maria",
  lastName: "Sanchez",
};

console.log(employee.firstName);
// Expected output: "Maria"

delete employee.firstName;

console.log(employee.firstName);
// Expected output: undefined
```

Syntax

js

```
delete object.property
delete object[property]
```

> **Note:** The syntax allows a wider range of expressions following the `delete` operator, but only the above forms lead to meaningful behaviors.

Parameters

`object`

The name of an object, or an expression evaluating to an object.

`property`

The property to delete.

Return value

`true` for all cases except when the property is an own non-configurable property, in which case `false` is returned in non-strict mode.

Exceptions

`TypeError`

Thrown in strict mode if the property is an own non-configurable property.

`ReferenceError`

Thrown if `object` is `super`.

Description

The `delete` operator has the same precedence as other unary operators like `typeof`. Therefore, it accepts any expression formed by higher-precedence operators. However, the following forms lead to early syntax errors in strict mode:

js · ✗ bad example

```
delete identifier;
delete object.#privateProperty;
```

Because classes are automatically in strict mode, and private elements can only be legally referenced in class bodies, this means private elements can never be deleted. While `delete identifier` may work if `identifier` refers to a configurable property of the global object, you should avoid this form and prefix it with `globalThis` instead.

While other expressions are accepted, they don't lead to meaningful behaviors:

js · ✗ bad example

```
delete console.log(1);
// Logs 1, returns true, but nothing deleted
```

The `delete` operator removes a given property from an object. On successful deletion, it will return `true`, else `false` will be returned. Unlike what common belief suggests (perhaps due to other programming languages like delete in C++), the `delete` operator has **nothing** to do with directly freeing memory. Memory management is done indirectly via breaking references. See the memory management page for more details.

It is important to consider the following scenarios:

- If the property which you are trying to delete does not exist, `delete` will not have any effect and will return `true`.
- `delete` only has an effect on own properties. If a property with the same name exists on the object's prototype chain, then after deletion, the object will use the property from the prototype chain.
- Non-configurable properties cannot be removed. This includes properties of built-in objects like `Math`, `Array`, `Object` and properties that are created as non-configurable with methods like `Object.defineProperty()`.
- Deleting variables, including function parameters, never works. `delete variable` will throw a `SyntaxError` in strict mode, and will have no effect in non-strict mode.
  - Any variable declared with `var` cannot be deleted from the global scope or from a function's scope, because while they may be attached to the global object, they are not configurable.
  - Any variable declared with `let` or `const` cannot be deleted from the scope within which they were defined, because they are not attached to an object.

Examples

Using delete

> **Note:** The following example uses non-strict-mode only features, like implicitly creating global variables and deleting identifiers, which are forbidden in strict mode.

js

```
// Creates the property empCount on the global scope.
// Since we are using var, this is marked as non-configurable.
var empCount = 43;

// Creates the property EmployeeDetails on the global scope.
// Since it was defined without "var", it is marked configurable.
EmployeeDetails = {
  name: "xyz",
  age: 5,
  designation: "Developer",
};

// delete can be used to remove properties from objects.
delete EmployeeDetails.name; // returns true

// Even when the property does not exist, delete returns "true".
delete EmployeeDetails.salary; // returns true

// EmployeeDetails is a property of the global scope.
delete EmployeeDetails; // returns true

// On the contrary, empCount is not configurable
// since var was used.
delete empCount; // returns false

// delete also does not affect built-in static properties
// that are non-configurable.
delete Math.PI; // returns false

function f() {
  var z = 44;

  // delete doesn't affect local variable names
  delete z; // returns false
}
```

delete and the prototype chain

In the following example, we delete an own property of an object while a property with the same name is available on the prototype chain:

js

```
function Foo() {
  this.bar = 10;
}

Foo.prototype.bar = 42;

const foo = new Foo();
```

(code continues on the next page)

(code continued from the previous page)

```
// foo.bar is associated with the
// own property.
console.log(foo.bar); // 10

// Delete the own property within the
// foo object.
delete foo.bar; // returns true

// foo.bar is still available in the
// prototype chain.
console.log(foo.bar); // 42

// Delete the property on the prototype.
delete Foo.prototype.bar; // returns true

// The "bar" property can no longer be
// inherited from Foo since it has been
// deleted.
console.log(foo.bar); // undefined
```

Deleting array elements

When you delete an array element, the array `length` is not affected. This holds even if you delete the last element of the array.

When the `delete` operator removes an array element, that element is no longer in the array. In the following example, `trees[3]` is removed with `delete`.

js

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
delete trees[3];
console.log(3 in trees); // false
```

This creates a sparse array with an empty slot. If you want an array element to exist but have an undefined value, use the `undefined` value instead of the `delete` operator. In the following example, `trees[3]` is assigned the value `undefined`, but the array element still exists:

js

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees[3] = undefined;
console.log(3 in trees); // true
```

If instead, you want to remove an array element by changing the contents of the array, use the `splice()` method. In the following example, `trees[3]` is removed from the array completely using `splice()`:

js

```
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees.splice(3, 1);
console.log(trees); // ["redwood", "bay", "cedar", "maple"]
```

Deleting non-configurable properties

When a property is marked as non-configurable, `delete` won't have any effect, and will return `false`. In strict mode, this will raise a `TypeError`.

```js
const Employee = {};
Object.defineProperty(Employee, "name", { configurable: false });

console.log(delete Employee.name); // returns false
```

`var` creates non-configurable properties that cannot be deleted with the `delete` operator:

```js
// Since "nameOther" is added using with the
// var keyword, it is marked as non-configurable
var nameOther = "XYZ";

// We can access this global property using:
Object.getOwnPropertyDescriptor(globalThis, "nameOther");
// {
//   value: "XYZ",
//   writable: true,
//   enumerable: true,
//   configurable: false
// }

delete globalThis.nameOther; // return false
```

In strict mode, this would raise an exception.

Deleting global properties

If a global property is configurable (for example, via direct property assignment), it can be deleted, and subsequent references to them as global variables will produce a `ReferenceError`.

```js
globalThis.globalVar = 1;
console.log(globalVar); // 1
// In non-strict mode, you can use `delete globalVar` as well
delete globalThis.globalVar;
console.log(globalVar); // ReferenceError: globalVar is not defined
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-delete-operator`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`delete`	1	12	1	9	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.delete`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Reflect.deleteProperty()`
- `Map.prototype.delete()`

JavaScript › Reference › Expressions and operators

Destructuring

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring · MDN page last modified 2026-08-10

Baseline: Widely available — available across major browsers since August 2016.

The **destructuring** syntax is a JavaScript syntax that makes it possible to unpack values from arrays, or properties from objects, into distinct variables. It can be used in locations that receive data (such as the left-hand side of an assignment or anywhere that creates new identifier bindings).

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Destructuring". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a, b, rest;
[a, b] = [10, 20];

console.log(a);
// Expected output: 10

console.log(b);
// Expected output: 20

[a, b, ...rest] = [10, 20, 30, 40, 50];

console.log(rest);
// Expected output: Array [30, 40, 50]
```

Syntax

js

```
const [a, b] = array;
const [a, , b] = array;
const [a = aDefault, b] = array;
const [a, b, ...rest] = array;
const [a, , b, ...rest] = array;
const [a, b, ...{ pop, push }] = array;
const [a, b, ...[c, d]] = array;

const { a, b } = obj;
const { a: a1, b: b1 } = obj;
const { a: a1 = aDefault, b = bDefault } = obj;
const { a, b, ...rest } = obj;
const { a: a1, b: b1, ...rest } = obj;
const { [key]: a } = obj;
```

(code continues on the next page)

(code continued from the previous page)

```
let a, b, a1, b1, c, d, rest, pop, push;
[a, b] = array;
[a, , b] = array;
[a = aDefault, b] = array;
[a, b, ...rest] = array;
[a, , b, ...rest] = array;
[a, b, ...{ pop, push }] = array;
[a, b, ...[c, d]] = array;

({ a, b } = obj); // parentheses are required
({ a: a1, b: b1 } = obj);
({ a: a1 = aDefault, b = bDefault } = obj);
({ a, b, ...rest } = obj);
({ a: a1, b: b1, ...rest } = obj);
```

Description

The object and array literal expressions provide an easy way to create *ad hoc* packages of data.

js

```
const arr = [a, b, c];
```

The destructuring uses similar syntax but uses it on the left-hand side of the assignment instead. It performs the reverse operation of an array declaration, by declaring each element in the collection as a separate variable.

js

```
const arr = [1, 2, 3];
const [a, b, c] = arr;
// a = 1, b = 2, c = 3
```

As for objects, compare the two pairs of lines below, and see how there's a direct correspondence within each pair.

js

```
const obj = { a, b, c };
const { a, b, c } = obj;
// Equivalent to:
// const a = obj.a, b = obj.b, c = obj.c;

const obj = { prop1: x, prop2: y, prop3: z };
const { prop1: x, prop2: y, prop3: z } = obj;
// Equivalent to:
// const x = obj.prop1, y = obj.prop2, z = obj.prop3;
```

This capability is similar to features present in languages such as Perl and Python.

For features specific to array or object destructuring, refer to the individual examples below.

Binding and assignment

For both object and array destructuring, there are two kinds of destructuring patterns: *binding pattern* and *assignment pattern*, with slightly different syntaxes.

In binding patterns, the pattern starts with a declaration keyword (`var`, `let`, or `const`). Then, each individual property must either be bound to a variable or further destructured.

```js
const obj = { a: 1, b: { c: 2 } };
const {
  a,
  b: { c: d },
} = obj;
// Two variables are bound: `a` and `d`
```

All variables share the same declaration, so if you want some variables to be re-assignable but others to be read-only, you may have to destructure twice — once with `let`, once with `const`.

```js
const obj = { a: 1, b: { c: 2 } };
const { a } = obj; // a is constant
let {
  b: { c: d },
} = obj; // d is re-assignable
```

In many other syntaxes where the language binds a variable for you, you can use a binding destructuring pattern. These include:

- The looping variable of `for...in` `for...of`, and `for await...of` loops;
- Function parameters;
- The `catch` binding variable.

In assignment patterns, the pattern does not start with a keyword. Each destructured property is assigned to a target of assignment — which may either be declared beforehand with `var` or `let`, or is a property of another object — in general, anything that can appear on the left-hand side of an assignment expression.

```js
const numbers = [];
const obj = { a: 1, b: 2 };
({ a: numbers[0], b: numbers[1] } = obj);
// The properties `a` and `b` are assigned to properties of `numbers`
```

> **Note:** The parentheses `( ... )` around the assignment statement are required when using object literal destructuring without a declaration.
>
> `{ a, b } = { a: 1, b: 2 }` is not valid stand-alone syntax, as the `{ a, b }` on the left-hand side is considered a block and not an object literal according to the rules of expression statements. However, `({ a, b } = { a: 1, b: 2 })` is valid, as is `const { a, b } = { a: 1, b: 2 }`.
>
> If your coding style does not include trailing semicolons, the `( ... )` expression needs to be preceded by a semicolon, or it may be used to execute a function on the previous line.

Note that the equivalent *binding pattern* of the code above is not valid syntax:

js · ✗ bad example

```
const numbers = [];
const obj = { a: 1, b: 2 };
const { a: numbers[0], b: numbers[1] } = obj;

// This is equivalent to:
//   const numbers[0] = obj.a;
//   const numbers[1] = obj.b;
// Which definitely is not valid.
```

You can only use assignment patterns as the left-hand side of the assignment operator. You cannot use them with compound assignment operators such as `+=` or `*=`.

Default value

Each destructured property can have a *default value*. The default value is used when the property is not present, or has value `undefined`. It is not used if the property has value `null`.

js

```
const [a = 1] = []; // a is 1
const { b = 2 } = { b: undefined }; // b is 2
const { c = 2 } = { c: null }; // c is null
```

The default value can be any expression. It will only be evaluated when necessary.

js

```
const { b = console.log("hey") } = { b: 2 };
// Does not log anything, because `b` is defined and there's no need
// to evaluate the default value.
```

Rest properties and rest elements

You can end a destructuring pattern with a rest property `...rest`. For array destructuring, it collects remaining elements of the iterable into a new array called `rest` (or any name you give it). For object destructuring, it copies all enumerable own properties of the object that are not already picked off by the destructuring pattern into a new object called `rest`.

More formally, the `...rest` syntax is called "rest elements" in array destructuring and "rest properties" in object destructuring, but we often just collectively call them "rest property".

js

```
const { a, ...others } = { a: 1, b: 2, c: 3 };
console.log(others); // { b: 2, c: 3 }

const [first, ...others2] = [1, 2, 3];
console.log(others2); // [2, 3]
```

The rest property must be the last in the pattern, and must not have a trailing comma.

js · ✗ bad example

```
const [a, ...b,] = [1, 2, 3];

// SyntaxError: rest element may not have a trailing comma
// Always consider using rest operator as the last element
```

Like any other JavaScript language feature, there is no implicit deep-copying. The rest property creates a new object (or array), but all its member values, if they are object references, remain the same.

Examples

Array destructuring

Basic variable assignment

```js
const foo = ["one", "two", "three"];

const [red, yellow, green] = foo;
console.log(red); // "one"
console.log(yellow); // "two"
console.log(green); // "three"
```

Destructuring with more elements than the source

In an array destructuring from an array of length *N* specified on the right-hand side of the assignment, if the number of variables specified on the left-hand side of the assignment is greater than *N*, only the first *N* variables are assigned values. The values of the remaining variables will be undefined.

```js
const foo = ["one", "two"];

const [red, yellow, green, blue] = foo;
console.log(red); // "one"
console.log(yellow); // "two"
console.log(green); // undefined
console.log(blue); // undefined
```

Swapping variables

Two variables values can be swapped in one destructuring expression.

Without destructuring, swapping two values requires a temporary variable (or, in some low-level languages, the XOR-swap trick).

```js
let a = 1;
let b = 3;

[a, b] = [b, a];
console.log(a); // 3
console.log(b); // 1

const arr = [1, 2, 3];
[arr[2], arr[1]] = [arr[1], arr[2]];
console.log(arr); // [1, 3, 2]
```

Parsing an array returned from a function

It's always been possible to return an array from a function. Destructuring can make working with an array return value more concise.

In this example, `f()` returns the values `[1, 2]` as its output, which can be parsed in a single line with destructuring.

```js
function f() {
  return [1, 2];
}

const [a, b] = f();
console.log(a); // 1
console.log(b); // 2
```

Ignoring some returned values

You can ignore return values that you're not interested in:

```js
function f() {
  return [1, 2, 3];
}

const [a, , b] = f();
console.log(a); // 1
console.log(b); // 3

const [c] = f();
console.log(c); // 1
```

You can also ignore all returned values:

```js
[, ,] = f();
```

Though in this case, it's probably clearer to just call the function and not use destructuring at all. You don't have to use the return value.

Using a binding pattern as the rest property

The rest property of array destructuring can be another array or object binding pattern. The inner destructuring destructures from the array created after collecting the rest elements, so you cannot access any properties present on the original iterable in this way.

```js
const [a, b, ...{ length }] = [1, 2, 3];
console.log(a, b, length); // 1 2 1
```

```js
const [a, b, ...[c, d]] = [1, 2, 3, 4];
console.log(a, b, c, d); // 1 2 3 4
```

These binding patterns can even be nested, as long as each rest property is the last in the list.

```js
const [a, b, ...[c, d, ...[e, f]]] = [1, 2, 3, 4, 5, 6];
console.log(a, b, c, d, e, f); // 1 2 3 4 5 6
```

On the other hand, object destructuring can only have an identifier as the rest property.

js · ✗ bad example

```
const { a, ...{ b } } = { a: 1, b: 2 };
// SyntaxError: `...` must be followed by an identifier in declaration contexts

let a, b;
({ a, ...{ b } } = { a: 1, b: 2 });
// SyntaxError: `...` must be followed by an assignable reference in assignment contexts
```

Unpacking values from a regular expression match

When the regular expression `exec()` method finds a match, it returns an array containing first the entire matched portion of the string and then the portions of the string that matched each parenthesized group in the regular expression. Destructuring allows you to unpack the parts out of this array easily, ignoring the full match if it is not needed.

js

```
function parseProtocol(url) {
  const parsedURL = /^(\w+):\/\/([^/]+)\/(.*)$/.exec(url);
  if (!parsedURL) {
    return false;
  }
  console.log(parsedURL);
  // ["https://developer.mozilla.org/en-US/docs/Web/JavaScript",
  // "https", "developer.mozilla.org", "en-US/docs/Web/JavaScript"]

  const [, protocol, fullHost, fullPath] = parsedURL;
  return protocol;
}

console.log(
  parseProtocol("https://developer.mozilla.org/en-US/docs/Web/JavaScript"),
);
// "https"
```

Using array destructuring on any iterable

Array destructuring calls the iterable protocol of the right-hand side. Therefore, any iterable, not necessarily arrays, can be destructured.

js

```
const [a, b] = new Map([
  [1, 2],
  [3, 4],
]);
console.log(a, b); // [1, 2] [3, 4]
```

Non-iterables cannot be destructured as arrays.

js · ✗ bad example

```
const obj = { 0: "a", 1: "b", length: 2 };
const [a, b] = obj;
// TypeError: obj is not iterable
```

Iterables are only iterated until all bindings are assigned.

```js
const obj = {
  *[Symbol.iterator]() {
    for (const v of [0, 1, 2, 3]) {
      console.log(v);
      yield v;
    }
  },
};
const [a, b] = obj; // Only logs 0 and 1
```

The rest binding is eagerly evaluated and creates a new array, instead of using the old iterable.

```js
const obj = {
  *[Symbol.iterator]() {
    for (const v of [0, 1, 2, 3]) {
      console.log(v);
      yield v;
    }
  },
};
const [a, b, ...rest] = obj; // Logs 0 1 2 3
console.log(rest); // [2, 3] (an array)
```

Object destructuring

Basic assignment

```js
const user = {
  id: 42,
  isVerified: true,
};

const { id, isVerified } = user;

console.log(id); // 42
console.log(isVerified); // true
```

Assigning to new variable names

A property can be unpacked from an object and assigned to a variable with a different name than the object property.

```js
const o = { p: 42, q: true };
const { p: foo, q: bar } = o;

console.log(foo); // 42
console.log(bar); // true
```

Here, for example, `const { p: foo } = o` takes from the object `o` the property named `p` and assigns it to a local variable named `foo`.

Assigning to new variable names and providing default values

A property can be both

- Unpacked from an object and assigned to a variable with a different name.

- Assigned a default value in case the unpacked value is `undefined`.

```js
const { a: aa = 10, b: bb = 5 } = { a: 3 };

console.log(aa); // 3
console.log(bb); // 5
```

Unpacking properties from objects passed as a function parameter

Objects passed into function parameters can also be unpacked into variables, which may then be accessed within the function body. As for object assignment, the destructuring syntax allows for the new variable to have the same name or a different name than the original property, and to assign default values for the case when the original object does not define the property.

Consider this object, which contains information about a user.

```js
const user = {
  id: 42,
  displayName: "jdoe",
  fullName: {
    firstName: "Jane",
    lastName: "Doe",
  },
};
```

Here we show how to unpack a property of the passed object into a variable with the same name. The parameter value `{ id }` indicates that the `id` property of the object passed to the function should be unpacked into a variable with the same name, which can then be used within the function.

```js
function userId({ id }) {
  return id;
}

console.log(userId(user)); // 42
```

You can define the name of the unpacked variable. Here we unpack the property named `displayName`, and rename it to `dname` for use within the function body.

```js
function userDisplayName({ displayName: dname }) {
  return dname;
}

console.log(userDisplayName(user)); // "jdoe"
```

Nested objects can also be unpacked. The example below shows the property `fullname.firstName` being unpacked into a variable called `name`.

```js
function whois({ displayName, fullName: { firstName: name } }) {
  return `${displayName} is ${name}`;
}

console.log(whois(user)); // "jdoe is Jane"
```

Setting a function parameter's default value

Default values can be specified using =, and will be used as variable values if a specified property does not exist in the passed object.

Below we show a function where the default size is `'big'`, default co-ordinates are `x: 0, y: 0` and default radius is 25.

js

```
function drawChart({
  size = "big",
  coords = { x: 0, y: 0 },
  radius = 25,
} = {}) {
  console.log(size, coords, radius);
  // do some chart drawing
}

drawChart({
  coords: { x: 18, y: 30 },
  radius: 30,
});
```

In the function signature for `drawChart` above, the destructured left-hand side has a default value of an empty object `= {}`.

You could have also written the function without that default. However, if you leave out that default value, the function will look for at least one argument to be supplied when invoked, whereas in its current form, you can call `drawChart()` without supplying any parameters. Otherwise, you need to at least supply an empty object literal.

For more information, see Default parameters > Destructured parameter with default value assignment.

Nested object and array destructuring

js

```
const metadata = {
  title: "Scratchpad",
  translations: [
    {
      locale: "de",
      localizationTags: [],
      lastEdit: "2014-04-14T08:43:37",
      url: "/de/docs/Tools/Scratchpad",
      title: "JavaScript-Umgebung",
    },
  ],
  url: "/en-US/docs/Tools/Scratchpad",
};

const {
  title: englishTitle, // rename
  translations: [
    {
      title: localeTitle, // rename
    },
  ],
} = metadata;
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(englishTitle); // "Scratchpad"
console.log(localeTitle); // "JavaScript-Umgebung"
```

For of iteration and destructuring

js

```
const people = [
  {
    name: "Mike Smith",
    family: {
      mother: "Jane Smith",
      father: "Harry Smith",
      sister: "Samantha Smith",
    },
    age: 35,
  },
  {
    name: "Tom Jones",
    family: {
      mother: "Norah Jones",
      father: "Richard Jones",
      brother: "Howard Jones",
    },
    age: 25,
  },
];

for (const {
  name: n,
  family: { father: f },
} of people) {
  console.log(`Name: ${n}, Father: ${f}`);
}

// "Name: Mike Smith, Father: Harry Smith"
// "Name: Tom Jones, Father: Richard Jones"
```

Computed object property names and destructuring

Computed property names, like on object literals, can be used with destructuring.

js

```
const key = "z";
const { [key]: foo } = { z: "bar" };

console.log(foo); // "bar"
```

Invalid JavaScript identifier as a property name

Destructuring can be used with property names that are not valid JavaScript identifiers by providing an alternative identifier that is valid.

js

```
const foo = { "fizz-buzz": true };
const { "fizz-buzz": fizzBuzz } = foo;

console.log(fizzBuzz); // true
```

Destructuring primitive values

Object destructuring is almost equivalent to property accessing. This means if you try to destruct a primitive value, the value will get wrapped into the corresponding wrapper object and the property is accessed on the wrapper object.

js

```
const { a, toFixed } = 1;
console.log(a, toFixed); // undefined ƒ toFixed() { [native code] }
```

Same as accessing properties, destructuring `null` or `undefined` throws a `TypeError`.

js · ✗ bad example

```
const { a } = undefined; // TypeError: Cannot destructure property 'a' of 'undefined' as it
is undefined.
const { b } = null; // TypeError: Cannot destructure property 'b' of 'null' as it is null.
```

This happens even when the pattern is empty.

js · ✗ bad example

```
const {} = null; // TypeError: Cannot destructure 'null' as it is null.
```

Combined array and object destructuring

Array and object destructuring can be combined. Say you want the third element in the array `props` below, and then you want the `name` property in the object, you can do the following:

js

```
const props = [
  { id: 1, name: "Fizz" },
  { id: 2, name: "Buzz" },
  { id: 3, name: "FizzBuzz" },
];

const [, , { name }] = props;

console.log(name); // "FizzBuzz"
```

The prototype chain is looked up when the object is deconstructed

When deconstructing an object, if a property is not accessed in itself, it will continue to look up along the prototype chain.

js

```
const obj = {
  self: "123",
  __proto__: {
    prot: "456",
  },
};
const { self, prot } = obj;

console.log(self); // "123"
console.log(prot); // "456"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-destructuring-assignment`
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-destructuring-binding-patterns`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Destructuring assignment	49	14	41[1]	36	8	49	41[2]	36	8	5.0	49	8	1.0.0	1.0	6.0.0
Computed property names	49	14	41	36	10	49	41	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Rest in arrays	49	16	41	36	9.1	49	41	36	9.3	5.0	49	9.3	1.0.0	1.0	6.0.0
Rest in objects	60	79	55	47	11.1	60	55	44	11.3	8.0	60	11.3	1.0.0	1.0	8.3.0

1. Firefox provided a non-standard destructuring implementation from Firefox 2 to 40.
2. Firefox for Android provided a non-standard destructuring implementation from Firefox for Android 4 to 40.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.destructuring`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators
- ES6 in Depth: Destructuring on hacks.mozilla.org (2015)

JavaScript › Reference › Expressions and operators

Division (/)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Division · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **division (/)** operator produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Division (/) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(12 / 2);
// Expected output: 6

console.log(3 / 2);
// Expected output: 1.5

console.log(6 / "3");
// Expected output: 2

console.log(2 / 0);
// Expected output: Infinity
```

Syntax

js

```
x / y
```

Description

The `/` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt division if both operands become BigInts; otherwise, it performs number division. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For BigInt division, the result is the quotient of the two operands truncated towards zero, and the remainder is discarded. A `RangeError` is thrown if the divisor `y` is `0n`. This is because number division by zero returns `Infinity` or `-Infinity`, but BigInt has no concept of infinity.

Examples

Division using numbers

js

```
1 / 2; // 0.5
Math.floor(3 / 2); // 1
1.0 / 2.0; // 0.5

2 / 0; // Infinity
2.0 / 0.0; // Infinity, because 0.0 === 0
2.0 / -0.0; // -Infinity
```

Other non-BigInt values are coerced to numbers:

js

```
5 / "2"; // 2.5
5 / "foo"; // NaN
```

Division using BigInts

```js
1n / 2n; // 0n
5n / 3n; // 1n
-1n / 3n; // 0n
1n / -3n; // 0n

2n / 0n; // RangeError: BigInt division by zero
```

You cannot mix BigInt and number operands in division.

js · ✗ bad example

```
2n / 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 / 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do division with a BigInt and a non-BigInt, convert either operand:

```js
2n / BigInt(2); // 1n
Number(2n) / 2; // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-multiplicative-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Division (/)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.division`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Division assignment (/=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Division_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **division assignment (`/=`)** operator performs division on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Division assignment (/=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 3;

a /= 2;
console.log(a);
// Expected output: 1.5

a /= 0;
console.log(a);
// Expected output: Infinity

a /= "hello";
console.log(a);
// Expected output: NaN
```

Syntax

js

```
x /= y
```

Description

`x /= y` is equivalent to `x = x / y`, except that the expression `x` is only evaluated once.

Examples

Division assignment using numbers

js

```
let bar = 5;

bar /= 2; // 2.5
bar /= 2; // 1.25
bar /= 0; // Infinity
```

Other non-BigInt values are coerced to numbers:

js

```
let bar = 5;
bar /= "2"; // 2.5
bar /= "foo"; // NaN
```

Division assignment using BigInts

```js
let foo = 3n;
foo /= 2n; // 1n
foo /= 2n; // 0n

foo /= 0n; // RangeError: BigInt division by zero
foo /= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Division assignment (x /= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.division_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Division (/)

JavaScript › Reference › Expressions and operators

Equality (==)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Equality · MDN page last modified 2026-05-11

Baseline: Widely available — available across major browsers since July 2015.

The **equality (==)** operator checks whether its two operands are equal, returning a Boolean result. Unlike the strict equality operator, it attempts to convert and compare operands that are of different types.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Expressions - Equality operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(1 == 1);
// Expected output: true

console.log("hello" == "hello");
// Expected output: true

console.log("1" == 1);
// Expected output: true

console.log(0 == false);
// Expected output: true
```

Syntax

js

```
x == y
```

Description

The equality operators (`==` and `!=`) provide the IsLooselyEqual semantic. This can be roughly summarized as follows:

1. If the operands have the same type, they are compared as follows:
   - Object: return `true` only if both operands reference the same object.
   - String: return `true` only if both operands have the same characters in the same order.
   - Number: return `true` only if both operands have the same value. `+0` and `-0` are treated as the same value. If either operand is `NaN`, return `false`; so, `NaN` is never equal to `NaN`.
   - Boolean: return `true` only if operands are both `true` or both `false`.
   - BigInt: return `true` only if both operands have the same value.
   - Symbol: return `true` only if both operands reference the same symbol.
2. If one of the operands is `null` or `undefined`, the other must also be `null` or `undefined` to return `true`. Otherwise return `false`.
3. If one of the operands is an object and the other is a primitive, convert the object to a primitive.
4. At this step, both operands are converted to primitives (one of String, Number, Boolean, Symbol, and BigInt). The rest of the conversion is done case-by-case.
   - If they are of the same type, compare them using step 1.
   - If one of the operands is a Symbol but the other is not, return `false`.
   - If one of the operands is a Boolean but the other is not, convert the boolean to a number: `true` is converted to 1, and `false` is converted to 0. Then compare the two operands loosely again.
   - Number to String: convert the string to a number. Conversion failure results in `NaN`, which will guarantee the equality to be `false`.
   - Number to BigInt: compare by their mathematical value. If the number is ±Infinity or `NaN`, return `false`.

- String to BigInt: convert the string to a BigInt using the same algorithm as the `BigInt()` constructor. If conversion fails, return `false`.

Loose equality is *symmetric*: `A == B` always has identical semantics to `B == A` for any values of `A` and `B` (except for the order of applied conversions).

The most notable difference between this operator and the strict equality (`===`) operator is that the strict equality operator does not attempt type conversion. Instead, the strict equality operator always considers operands of different types to be different. The strict equality operator essentially carries out only step 1, and then returns `false` for all other cases.

There's a "willful violation" of the above algorithm: if one of the operands is `document.all`, it is treated as if it's `undefined`. This means that `document.all == null` is `true`, but `document.all === undefined || document.all === null` is `false`.

Examples

Comparison with no type conversion

```js
1 == 1; // true
"hello" == "hello"; // true
```

Comparison with type conversion

```js
"1" == 1; // true
1 == "1"; // true
0 == false; // true
0 == null; // false
0 == undefined; // false
0 == !!null; // true, look at Logical NOT operator
0 == !!undefined; // true, look at Logical NOT operator
null == undefined; // true

const number1 = new Number(3);
const number2 = new Number(3);
number1 == 3; // true
number1 == number2; // false
```

Comparison of objects

```js
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 == object2); // false
console.log(object1 == object1); // true
```

Comparing strings and String objects

Note that strings constructed using `new String()` are objects. If you compare one of these with a string literal, the `String` object will be converted to a string literal and the contents will be compared. However, if both operands are `String` objects, then they are compared as objects and must reference the same object for comparison to succeed:

```js
const string1 = "hello";
const string2 = String("hello");
const string3 = new String("hello");
const string4 = new String("hello");

console.log(string1 == string2); // true
console.log(string1 == string3); // true
console.log(string2 == string3); // true
console.log(string3 == string4); // false
console.log(string4 == string4); // true
```

Comparing Dates and strings

```js
const d = new Date("1995-12-17T03:24:00");
const s = d.toString(); // for example: "Sun Dec 17 1995 03:24:00 GMT-0800 (Pacific Standard Time)"
console.log(d == s); // true
```

Comparing arrays and strings

```js
const a = [1, 2, 3];
const b = "1,2,3";
a == b; // true, `a` converts to string

const c = [true, 0.5, "hey"];
const d = c.toString(); // "true,0.5,hey"
c == d; // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-equality-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Equality (`a == b`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.equality`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Inequality (!=)
- Strict equality (===)
- Strict inequality (!==)

JavaScript › Reference › Expressions and operators

Exponentiation (**)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Exponentiation · MDN page last modified 2025-07-08
Baseline: Widely available — available across major browsers since March 2017.

The **exponentiation (**)** operator returns the result of raising the first operand to the power of the second operand. It is equivalent to `Math.pow()`, except it also accepts BigInts as operands.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Exponentiation (**) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(3 ** 4);
// Expected output: 81

console.log(10 ** -2);
// Expected output: 0.01

console.log(2 ** (3 ** 2));
// Expected output: 512

console.log((2 ** 3) ** 2);
// Expected output: 64
```

Syntax

js

```
x ** y
```

Description

The ** operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt exponentiation if both operands become BigInts; otherwise, it performs number exponentiation. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For both numbers and BigInts, `0` raised to a positive power returns `0`, and `0` raised to a power of `0` returns `1`. For numbers, `0` raised to a negative number returns `Infinity`, while `-0` raised to a negative number returns `-Infinity`.

`NaN ** 0` (and the equivalent `Math.pow(NaN, 0)`) is the only case where `NaN` doesn't propagate through mathematical operations — it returns `1` despite the operand being `NaN`. In addition, the behavior where `base` is 1 and `exponent` is non-finite (±Infinity or `NaN`) is different from IEEE 754, which specifies that

the result should be 1, whereas JavaScript returns `NaN` to preserve backward compatibility with its original behavior.

For BigInt exponentiation, a `RangeError` is thrown if the exponent `y` is negative. This is because any negative exponent would likely result in a value between 0 and 1 (unless the base is `1`, `-1`, or `0`), which is rounded to zero, and is likely a developer mistake.

The exponentiation operator is right-associative: `a ** b ** c` is equal to `a ** (b ** c)`.

In most languages, such as PHP, Python, and others that have an exponentiation operator (`**`), the exponentiation operator is defined to have a higher precedence than unary operators, such as unary `+` and unary `-`, but there are a few exceptions. For example, in Bash, the `**` operator is defined to have a lower precedence than unary operators.

In JavaScript, it is impossible to write an ambiguous exponentiation expression. That is, you cannot put a unary operator (with precedence 14, including `+/-/~/!/++/--/delete/void/typeof/await`) immediately before the base number; doing so will cause a SyntaxError.

For example, `-2 ** 2` is 4 in Bash, but is -4 in other languages (such as Python). This is invalid in JavaScript, as the operation is ambiguous. You have to parenthesize either side — for example, as `-(2 ** 2)` — to make the intention unambiguous.

Note that some programming languages use the caret symbol `^` for exponentiation, but JavaScript uses that symbol for the bitwise XOR operator.

Examples

Exponentiation using numbers

```js
2 ** 3; // 8
3 ** 2; // 9
3 ** 2.5; // 15.588457268119896
10 ** -1; // 0.1
2 ** 1024; // Infinity
NaN ** 2; // NaN
NaN ** 0; // 1
1 ** Infinity; // NaN
```

Other non-BigInt values are coerced to numbers:

```js
2 ** "3"; // 8
2 ** "hello"; // NaN
```

Exponentiation using BigInts

```js
2n ** 3n; // 8n
2n ** 1024n; // A very large number, but not Infinity
```

You cannot mix BigInt and number operands in exponentiation.

js · ✗ bad example

```
2n ** 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 ** 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do exponentiation with a BigInt and a non-BigInt, convert either operand:

```js
2n ** BigInt(2); // 4n
Number(2n) ** 2; // 4
```

Associativity

```js
2 ** 3 ** 2; // 512
2 ** (3 ** 2); // 512
(2 ** 3) ** 2; // 64
```

Usage with unary operators

To invert the sign of the result of an exponentiation expression:

```js
-(2 ** 2); // -4
```

To force the base of an exponentiation expression to be a negative number:

```js
(-2) ** 2; // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-exp-operator`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Exponentiation (**)	52	14	52	39	10.1	52	52	41	10.3	6.0	51	10.3	1.0.0	1.0	7.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.exponentiation`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Increment (++)
- Decrement (--)

- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Exponentiation assignment (**=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Exponentiation_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2017.

The **exponentiation assignment (**=)** operator performs exponentiation on the two operands and assigns the result to the left operand.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Exponentiation assignment (**=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a = 3;

console.log((a **= 2));
// Expected output: 9

console.log((a **= 0));
// Expected output: 1

console.log((a **= 'hello'));
// Expected output: NaN
```

Syntax

js

```
x **= y
```

Description

`x **= y` is equivalent to `x = x ** y`, except that the expression `x` is only evaluated once.

Examples

Exponentiation assignment using numbers

js

```
let bar = 5;
bar **= 2; // 25
```

Other non-BigInt values are coerced to numbers:

js

```
let baz = 5;
baz **= "foo"; // NaN
```

Exponentiation assignment using BigInts

```js
let foo = 3n;
foo **= 2n; // 9n
foo **= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Exponentiation assignment (x **= y)	52	14	52	39	10.1	52	52	41	10.3	6.0	51	10.3	1.0.0	1.0	7.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.exponentiation_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Exponentiation (**)

JavaScript › Reference › Expressions and operators

function expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/function · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since July 2015.

The **`function`** keyword can be used to define a function inside an expression.

You can also define functions using the `function` declaration or the arrow syntax.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: function expression". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const getRectArea = function (width, height) {
  return width * height;
};

console.log(getRectArea(3, 4));
// Expected output: 12
```

Syntax

```js
function (param0) {
  statements
}
function (param0, param1) {
  statements
}
function (param0, param1, /* …, */ paramN) {
  statements
}

function name(param0) {
  statements
}
function name(param0, param1) {
  statements
}
function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** An expression statement cannot begin with the keyword `function` to avoid ambiguity with a `function` declaration. The `function` keyword only begins an expression when it appears in a context that cannot accept statements.

Parameters

`name` Optional

The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

`paramN` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

A `function` expression is very similar to, and has almost the same syntax as, a `function` declaration. The main difference between a `function` expression and a `function` declaration is the *function name*, which can be omitted in `function` expressions to create *anonymous* functions. A `function` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined. See also the chapter about functions for more information.

Function expression hoisting

Function expressions in JavaScript are not hoisted, unlike function declarations. You can't use function expressions before you create them:

```js
console.log(notHoisted); // undefined
// Even though the variable name is hoisted,
// the definition isn't. so it's undefined.
notHoisted(); // TypeError: notHoisted is not a function

var notHoisted = function () {
  console.log("bar");
};
```

Named function expression

If you want to refer to the current function inside the function body, you need to create a named function expression. This name is then local only to the function body (scope). This avoids using the deprecated `arguments.callee` property to call the function recursively.

```js
const math = {
  factorial: function factorial(n) {
    console.log(n);
    if (n <= 1) {
      return 1;
    }
    return n * factorial(n - 1);
  },
};

math.factorial(3); // 3;2;1;
```

If a function expression is named, the `name` property of the function is set to that name, instead of the implicit name inferred from syntax (such as the variable the function is assigned to).

Unlike declarations, the name of the function expressions is read-only.

```js
"use strict";

function foo() {
  foo = 1;
}
foo();
console.log(foo); // 1
(function foo() {
  foo = 1; // TypeError: Assignment to constant variable.
})();
```

Examples

Using function expression

The following example defines an unnamed function and assigns it to x. The function returns the square of its argument:

```js
const x = function (y) {
  return y * y;
};
```

Using a function as a callback

More commonly it is used as a callback:

```js
button.addEventListener("click", function (event) {
  console.log("button is clicked!");
});
```

Using an Immediately Invoked Function Expression (IIFE)

IIFEs are a common pattern used to execute arbitrarily many statements in their own scope (and possibly return a value), in a location that requires a single expression. Many traditional use cases of IIFEs have been obsoleted by new syntax features such as modules and block-scoped declarations. IIFEs themselves are more commonly written with arrow functions now, but the idea remains the same. In general, IIFEs look like this:

```js
// standard IIFE
(function () {
  // statements…
})();

// IIFE with arguments
(function (a, b) {
  console.log(a + b);
})(1, 2); // logs 3

// IIFE being used to initialize a variable
const value = (() => {
  const randomValue = Math.random();
  if (randomValue > 0.5) {
    return "heads";
  }
  return "tails";
})();
```

Here, we introduce several use cases with examples.

Avoid polluting the global namespace in script code

The top-level scope of all scripts is shared, which could include many functions and global variables from different files, so to avoid name conflicts, it's important to limit the number of globally declared names (this is greatly mitigated in modules, but sometimes limiting the scope of temporary variables is still useful, especially when the file is very long). If we have some initialization code that we don't need to use again, we could use the IIFE pattern, which is better than using a function declaration or a function expression because it ensures that the code is only run here and once.

```js
// top-level of a script (not a module)

var globalVariable = (() => {
  // some initialization code
  let firstVariable = something();
  let secondVariable = somethingElse();
  return firstVariable + secondVariable;
})();

// firstVariable and secondVariable cannot be accessed outside of the function body.
```

The module pattern

We would also use IIFE to create private and public variables and methods. For a more sophisticated use of the module pattern and other use of IIFE, you could see the book Learning JavaScript Design Patterns by Addy Osmani.

```js
const makeWithdraw = (balance) =>
  ((copyBalance) => {
    let balance = copyBalance; // This variable is private
    const doBadThings = () => {
      console.log("I will do bad things with your money");
    };
    doBadThings();
    return {
      withdraw(amount) {
        if (balance >= amount) {
          balance -= amount;
          return balance;
        }
        return "Insufficient money";
      },
    };
  })(balance);

const firstAccount = makeWithdraw(100); // "I will do bad things with your money"
console.log(firstAccount.balance); // undefined
console.log(firstAccount.withdraw(20)); // 80
console.log(firstAccount.withdraw(30)); // 50
console.log(firstAccount.doBadThings); // undefined; this method is private
const secondAccount = makeWithdraw(20); // "I will do bad things with your money"
console.log(secondAccount.withdraw(30)); // "Insufficient money"
console.log(secondAccount.withdraw(20)); // 0
```

For loop with var before ES6

We could see the following use of IIFE in some old code, before the introduction of the block-scoped `let` and `const` declarations. With the statement `var`, we have only function scopes and the global scope. Suppose we want to create 2 buttons with the texts Button 0 and Button 1 and when we click them, we would like them to alert 0 and 1. The following code doesn't work:

```js
for (var i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i);
  };
  document.body.appendChild(button);
}
console.log(i); // 2
```

When clicked, both Button 0 and Button 1 alert 2 because `i` is global, with the last value 2. To fix this problem before ES6, we could use the IIFE pattern:

```js
for (var i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = (function (copyOfI) {
    return function () {
      console.log(copyOfI);
    };
  })(i);
  document.body.appendChild(button);
}
console.log(i); // 2
```

When clicked, Buttons 0 and 1 alert 0 and 1. The variable `i` is globally defined. Using the statement `let`, we could simply do:

```js
for (let i = 0; i < 2; i++) {
  const button = document.createElement("button");
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i);
  };
  document.body.appendChild(button);
}
console.log(i); // Uncaught ReferenceError: i is not defined.
```

When clicked, these buttons alert 0 and 1.

Control flow statements in expression positions

IIFEs enable us to use language constructs such as `switch` in an expression.

```js
someObject.property = (() => {
  switch (someVariable) {
    case 0:
      return "zero";
    case 1:
      return "one";
    default:
      return "unknown";
  }
})();
```

This approach can be especially useful in scenarios where you want to make a variable `const`, but are forced to use `let` or `var` during initialization:

```js
let onlyAssignedOnce;
try {
  onlyAssignedOnce = someFunctionThatMightThrow();
} catch (e) {
  onlyAssignedOnce = null;
}
```

Using IIFEs, we can make the variable `const`:

```js
const onlyAssignedOnce = (() => {
  try {
    return someFunctionThatMightThrow();
  } catch (e) {
    return null;
  }
})();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`function` expression	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.function`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Functions guide
- Functions
- `function`
- `Function`
- Arrow functions

JavaScript › Reference › Expressions and operators

function* expression

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/function* · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2016.

The **`function*`** keyword can be used to define a generator function inside an expression.

You can also define generator functions using the `function*` declaration.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: function* expression". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const foo = function* () {
  yield "a";
  yield "b";
  yield "c";
};

let str = "";
for (const val of foo()) {
  str += val;
}

console.log(str);
// Expected output: "abc"
```

Syntax

js

```
function* (param0) {
  statements
}
function* (param0, param1) {
  statements
}
function* (param0, param1, /* …, */ paramN) {
  statements
}

function* name(param0) {
  statements
}
function* name(param0, param1) {
  statements
}
function* name(param0, param1, /* …, */ paramN) {
  statements
}
```

Note: An expression statement cannot begin with the keyword `function` to avoid ambiguity with a `function*` declaration. The `function` keyword only begins an expression when it appears in a context that cannot accept statements.

Parameters

name Optional

The function name. Can be omitted, in which case the function is *anonymous*. The name is only local to the function body.

paramN Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

statements Optional

The statements which comprise the body of the function.

Description

A `function*` expression is very similar to, and has almost the same syntax as, a `function*` declaration. The main difference between a `function*` expression and a `function*` declaration is the *function name*, which can be omitted in `function*` expressions to create *anonymous* functions. A `function*` expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined, allowing you to create an ad-hoc iterable iterator object. See also the chapter about functions for more information.

Examples

Using function* expression

The following example defines an unnamed generator function and assigns it to x. The function yields the square of its argument:

```js
const x = function* (y) {
  yield y * y;
};
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-generator-function-definitions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`function*` expression	49	12	26	36	10	49	26	36	10	5.0	49	10	1.0.0	1.0	4.0.0
Trailing comma in parameters	58	79	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.generator_function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `function*`
- `GeneratorFunction`
- Iteration protocols
- `yield`
- `yield*`

JavaScript › Reference › Expressions and operators

Greater than (>)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Greater_than · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **greater than (>)** operator returns `true` if the left operand is greater than the right operand, and `false` otherwise.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Greater than (>) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(5 > 3);
// Expected output: true

console.log(3 > 3);
// Expected output: false

// Compare bigint to number
console.log(3n > 5);
// Expected output: false

console.log("ab" > "aa");
// Expected output: true
```

Syntax

js

```
x > y
```

Description

The operands are compared using the same algorithm as the Less than operator, except the two operands are swapped. `x > y` is generally equivalent to `y < x`, except that `x > y` coerces `x` to a primitive before `y`, while `y < x` coerces `y` to a primitive before `x`. Because coercion may have side effects, the order of the operands may matter.

Examples

String to string comparison

```js
"a" > "b"; // false
"a" > "a"; // false
"a" > "3"; // true
```

String to number comparison

```js
"5" > 3; // true
"3" > 3; // false
"3" > 5; // false

"hello" > 5; // false
5 > "hello"; // false

"5" > 3n; // true
"3" > 5n; // false
```

Number to Number comparison

```js
5 > 3; // true
3 > 3; // false
3 > 5; // false
```

Number to BigInt comparison

```js
5n > 3; // true
3 > 5n; // false
```

Comparing Boolean, null, undefined, NaN

```js
true > false; // true
false > true; // false

true > 0; // true
true > 1; // false

null > 0; // false
1 > null; // true

undefined > 3; // false
3 > undefined; // false

3 > NaN; // false
NaN > 3; // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Greater than (a > b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.greater_than`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Greater than or equal (>=)
- Less than (<)
- Less than or equal (<=)

JavaScript › Reference › Expressions and operators

Greater than or equal (>=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Greater_than_or_equal · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **greater than or equal (>=)** operator returns `true` if the left operand is greater than or equal to the right operand, and `false` otherwise.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Greater than or equal (>=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(5 >= 3);
// Expected output: true

console.log(3 >= 3);
// Expected output: true

// Compare bigint to number
console.log(3n >= 5);
// Expected output: false

console.log("ab" >= "aa");
// Expected output: true
```

Syntax

```js
x >= y
```

Description

The operands are compared using the same algorithm as the Less than operator, with the result negated. `x >= y` is generally equivalent to `!(x < y)`, except for two cases where `x >= y` and `x < y` are both `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

`x >= y` is generally equivalent to `x > y || x == y`, except for a few cases:

- When one of `x` or `y` is `null`, and the other is something that's not `null` and becomes 0 when coerced to numeric (including `0`, `0n`, `false`, `""`, `"0"`, `new Date(0)`, etc.): `x >= y` is `true`, while `x > y || x == y` is `false`.
- When one of `x` or `y` is `undefined`, and the other is one of `null` or `undefined`: `x >= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are the same object that becomes `NaN` after the first step of Less than (such as `new Date(NaN)`): `x >= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are different objects that become the same value after the first step of Less than: `x >= y` is `true`, while `x > y || x == y` is `false`.

Examples

String to string comparison

```js
"a" >= "b"; // false
"a" >= "a"; // true
"a" >= "3"; // true
```

String to number comparison

```js
"5" >= 3; // true
"3" >= 3; // true
"3" >= 5; // false

"hello" >= 5; // false
5 >= "hello"; // false
```

Number to Number comparison

```js
5 >= 3; // true
3 >= 3; // true
3 >= 5; // false
```

Number to BigInt comparison

```js
5n >= 3; // true
3 >= 3n; // true
3 >= 5n; // false
```

Comparing Boolean, null, undefined, NaN

```js
true >= false; // true
true >= true; // true
false >= true; // false

true >= 0; // true
true >= 1; // true

null >= 0; // true
1 >= null; // true

undefined >= 3; // false
3 >= undefined; // false

3 >= NaN; // false
NaN >= 3; // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Greater than or equal (a >= b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.greater_than_or_equal`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Greater than (>)
- Less than (<)
- Less than or equal (<=)

JavaScript › Reference › Expressions and operators

Grouping operator ( )

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Grouping · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since July 2015.

The **grouping ( )** operator controls the precedence of evaluation in expressions. It also acts as a container for arbitrary expressions in certain syntactic constructs, where ambiguity or syntax errors would otherwise occur.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Grouping operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(1 + 2 * 3); // 1 + 6
// Expected output: 7

console.log(1 + (2 * 3)); // 1 + 6
// Expected output: 7

console.log((1 + 2) * 3); // 3 * 3
// Expected output: 9

console.log(1 * 3 + 2 * 3); // 3 + 6
// Expected output: 9
```

Syntax

js

```
(expression)
```

Parameters

`expression`

Any expression to be evaluated, including comma-joined expressions.

Description

The grouping operator consists of a pair of parentheses around an expression that groups the contents. The operator overrides the normal operator precedence, so that operators with lower precedence (as low as the comma operator) can be evaluated before an operator with higher precedence.

Examples

Using the grouping operator

Evaluating addition and subtraction before multiplication and division.

js

```
const a = 1;
const b = 2;
const c = 3;
```

(code continues on the next page)

(code continued from the previous page)

```
// default precedence
a + b * c; // 7
// evaluated by default like this
a + (b * c); // 7

// now overriding precedence
// addition before multiplication
(a + b) * c; // 9

// which is equivalent to
a * c + b * c; // 9
```

Notice in these examples that the order in which the *operators* evaluate has changed, but the order in which the *operands* evaluate has not. For example, in this code, the function invocations `a()`, `b()`, and `c()` are evaluated left-to-right (the normal order of evaluation) before the operator order is considered.

js

```
a() * (b() + c());
```

The function `a` will be called before the function `b`, which will be called before the function `c`. For more on operator precedence, see its reference page.

Using the grouping operator to eliminate parsing ambiguity

An expression statement cannot start with the keyword `function`, because the parser would see it as the start of a function declaration. This means the following IIFE syntax is invalid:

js · ✗ bad example

```
function () {
  // code
}();
```

The grouping operator can be used to eliminate this ambiguity, since when the parser sees the left parenthesis, it knows that what follows must be an expression instead of a declaration.

js

```
(function () {
  // code
})();
```

You may also use the `void` operator to eliminate ambiguity.

In an arrow function expression body (one that directly returns an expression without the keyword `return`), the grouping operator can be used to return an object literal expression, because otherwise the left curly brace would be interpreted as the start of the function body.

js

```
const f = () => ({ a: 1 });
```

If a property is accessed on a number literal, the property accessor dot `.` may be ambiguous with a decimal point, unless the number already has a decimal point. You can wrap integer literals in parentheses to eliminate this ambiguity.

js

```
(1).toString(); // "1"
```

Grouping operator and automatic semicolon insertion

The grouping operator can mitigate automatic semicolon insertion (ASI) pitfalls. For example, the `return` keyword and the returned expression cannot have a line break in between:

js · ✗ bad example

```
function sum(a, b) {
  return
    a + b;
}
```

This code will return `undefined`, because a semicolon is inserted directly after the `return` keyword, which causes the function to return immediately without evaluating `a + b`. In case the returned expression is long and you want to keep it well-formatted, you may use the grouping operator to signify that the `return` keyword is followed by an expression and prevent semicolon insertion:

js · ✓ good example

```
function sum(a, b) {
  return (
    a + b
  );
}
```

However, grouping may also *introduce* ASI hazards. When a line starts with a left parenthesis and the previous line ends with an expression, the parser will not insert a semicolon before the line break, because it could be the middle of a function call. For example:

js · ✗ bad example

```
const a = 1
(1).toString()
```

This code would be parsed as:

js

```
const a = 1(1).toString();
```

Which throws "TypeError: 1 is not a function". If your coding style does not use semicolons, remember that when a line starts with a left parenthesis, *prefix* it with a semicolon. This practice is recommended by several formatters and/or style guides, including Prettier and standard.

js · ✓ good example

```
const a = 1
;(1).toString()
```

For more advice on working with ASI, see its reference section.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-grouping-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Grouping operator `()`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.grouping`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Operator precedence
- `delete`
- `typeof`

JavaScript › Reference › Expressions and operators

import.meta

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import.meta · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since January 2020.

The **`import.meta`** meta-property exposes context-specific metadata to a JavaScript module. It contains information about the module, such as the module's URL.

Syntax

```js
import.meta
```

Value

The `import.meta` object is created by the host environment, as an extensible `null`-prototype object where all properties are writable, configurable, and enumerable. The spec doesn't specify any properties to be defined on it, but hosts usually implement the following properties:

`url`

The full URL to the module, includes query parameters and/or hash (following the `?` or `#`). In browsers, this is either the URL from which the script was obtained (for external scripts), or the document's base URL when the module script was created (for inline scripts). In Node.js, file-based modules use a `file:` URL, while modules loaded from other supported URL schemes use their corresponding URLs.

`resolve`

Resolves a module specifier to a URL using the current module's URL as base.

Description

The `import.meta` syntax consists of the keyword `import`, a dot, and the identifier `meta`. Because `import` is a reserved word, not an identifier, this is not a property accessor, but a special expression syntax.

The `import.meta` meta-property is available in JavaScript modules; using `import.meta` outside of a module (including direct `eval()` within a module) is a syntax error.

Examples

Passing query parameters

Using query parameters in the `import` specifier allows module-specific argument passing, which may be complementary to reading parameters from the application-wide `window.location` (or on Node.js, through `process.argv`). For example, with the following HTML:

```html
<script type="module">
  import "./index.mjs?someURLInfo=5";
</script>
```

The `index.mjs` module is able to retrieve the `someURLInfo` parameter through `import.meta`:

```js
// -- index.mjs --
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

The same applies when a module imports another:

```js
// -- index.mjs --
import "./index2.mjs?someURLInfo=5";

// -- index2.mjs --
new URL(import.meta.url).searchParams.get("someURLInfo"); // 5
```

The ES module implementation in Node.js supports resolving module specifiers containing query parameters (or the hash), as in the latter example. However, you cannot use queries or hashes when the module is specified through the CLI command (like `node index.mjs?someURLInfo=5`), because the CLI entrypoint uses a more CommonJS-like resolution mode, treating the path as a file path rather than a URL. To pass parameters to the entrypoint module, use CLI arguments and read them through `process.argv` instead (like `node index.mjs --someURLInfo=5`).

Resolving a file relative to the current one

In Node.js CommonJS modules, there's a `__dirname` variable that contains the absolute path to the folder containing current module, which is useful for resolving relative paths. However, Node.js does not provide this variable in ES modules. To resolve a relative file you can use `import.meta.url`. Note that this uses URLs rather than filesystem paths.

Before (CommonJS):

```js
const fs = require("fs/promises");
const path = require("path");

const filePath = path.join(__dirname, "someFile.txt");
fs.readFile(filePath, "utf8").then(console.log);
```

After (ES modules):

```js
import fs from "node:fs/promises";

const fileURL = new URL("./someFile.txt", import.meta.url);
fs.readFile(fileURL, "utf8").then(console.log);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ImportMeta`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import.meta`	64	79	62	51	11.1	64	62	47	12	9.0	64	12	1.0.0	1.0	10.4.0
`import.meta.resolve`	105	105	106	91	16.4	105	106	72	16.4	20.0	105	16.4	1.0.0	1.24	20.8.0 [1,2,3]

1. Also from version 20.6.0–20.8.0: partial support.
2. Also from version 20.6.0–20.8.0: Returns a `URL` object instead of a string.
3. Also from version 18.19.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.import_meta`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `import`
- `export`

JavaScript › Reference › Expressions and operators › import.meta

import.meta.resolve()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import.meta/resolve · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since March 2023.

`import.meta.resolve()` is a built-in function defined on the `import.meta` object of a JavaScript module that resolves a module specifier to a URL using the current module's URL as base.

Syntax

```js
import.meta.resolve(moduleName)
```

Parameters

`moduleName`

A string that specifies a potentially importable module. This may be a relative path (such as `"./lib/helper.js"`), a bare name (such as `"my-module"`), or an absolute URL (such as `"https://example.com/lib/helper.js"`).

Return value

Returns a string corresponding to the path that would be imported if the argument were passed to `import()`.

Description

`import.meta.resolve()` allows a script to access the *module specifier resolution* algorithm for a name, like this:

```js
// -- https://example.com/main.js --

const helperPath = import.meta.resolve("./lib/helper.js");
console.log(helperPath); // "https://example.com/lib/helper.js"
```

Note that `import.meta.resolve()` only performs resolution and does not attempt to load or import the resulting module. In browsers, its return value is the same *regardless of whether the returned URL corresponds to a file that exists, and regardless of whether that file contains valid code for a module*. The operation is *synchronous*. Non-browser hosts may inspect the file system during resolution; for example, Node.js can read package metadata and check paths.

It is different from dynamic import, because although both accept a module specifier as the first argument, `import.meta.resolve()` returns the path that *would be imported* without making any attempt to access that path. Therefore, the following two are effectively the same code:

```js
// Approach 1
console.log(await import("./lib/helper.js"));

// Approach 2
const helperPath = import.meta.resolve("./lib/helper.js");
console.log(await import(helperPath));
```

However, even if `"./lib/helper.js"` cannot be successfully imported, the second snippet will not encounter an error until it attempts to perform the import on line 2.

Bare module names

You can pass a bare module name (also known as a bare module specifier) to `import.meta.resolve()`, as long as module resolution is defined for the name. For example, you can define this using an import map inside a browser:

```html
<!-- index.html -->
<script type="importmap">
  {
    "imports": {
      "my-module": "./modules/my-module/index.js"
    }
  }
</script>

<script type="module">
  const moduleEntryPath = import.meta.resolve("my-module");
  console.log(moduleEntryPath);
</script>
```

Again, since this snippet does not try to import `moduleEntryPath` — neither does the import map — it prints the resolved URL regardless of whether `./modules/my-module/index.js` actually exists.

Comparison with new URL()

The `URL()` constructor accepts a second *base URL* argument. When the first argument is a relative path and the base URL is `import.meta.url`, the effect is similar to `import.meta.resolve()`.

```js
const helperPath = new URL("./lib/helper.js", import.meta.url).href;
console.log(helperPath);
```

This is also a useful replacement syntax when targeting older browsers. However, there are some differences:

- `import.meta.resolve()` returns a string, while `new URL()` returns a `URL` object. It is possible to use `href` or `toString()` on the constructed `URL`, but this may still not produce the exact same result in some JavaScript environments or when using tools like bundlers to statically analyze the code.
- `import.meta.resolve()` is aware of additional resolution configurations, such as resolving bare module names using import maps, as described above. `new URL()` is not aware of import maps and treats bare module names as relative paths (i.e., `new URL("my-module", import.meta.url)` means `new URL("./my-module", import.meta.url)`).

Some tools recognize `new URL("./lib/helper.js", import.meta.url).href` as a dependency on `"./lib/helper.js"` (similar to an import), and take this into account for features like bundling, rewriting imports for moved files, "go to source" functionality, etc. However, since `import.meta.resolve()` is less ambiguous and specifically designed to indicate a module path resolution dependency, you should use `import.meta.resolve(moduleName)` instead of `new URL(moduleName, import.meta.url)` for these use cases wherever possible.

Not an ECMAScript feature

`import.meta.resolve()` is not specified or documented as part of the ECMAScript specification for JavaScript modules. Instead, the specification defines the `import.meta` object but leaves all its properties as "host-defined". The WHATWG HTML standard picks up where the ECMAScript standard leaves off, and defines `import.meta.resolve` using its module specifier resolution.

This means that `import.meta.resolve()` is not required to be implemented by all conformant JavaScript implementations. However, `import.meta.resolve()` may also be available in non-browser environments:

- Deno implements compatibility with browser behavior.
- Node.js also implements the `import.meta.resolve()` function, but adds an additional `parent` parameter if you use the `--experimental-import-meta-resolve` flag.

Examples

Resolve a path for the Worker() constructor

`import.meta.resolve()` is particularly valuable for APIs that take a path to a script file as an argument, such as the `Worker()` constructor:

```js
// -- main.js --
const workerPath = import.meta.resolve("./worker.js");
const worker = new Worker(workerPath, { type: "module" });
worker.addEventListener("message", console.log);
```

```js
// -- worker.js --
self.postMessage("hello!");
```

This is also useful to calculate paths for other workers, such as service workers and shared workers. However, if you are using a relative path to calculate the URL of a service worker, keep in mind that the directory of the resolved path determines its registration scope by default (although a different scope can be specified during registration).

Specifications

Specification
HTML https://html.spec.whatwg.org/multipage/webappapis.html#import-meta-resolve

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import.meta.resolve`	105	105	106	91	16.4	105	106	72	16.4	20.0	105	16.4	1.0.0	1.24	20.8.0 [1,2,3]

1. Also from version 20.6.0–20.8.0: partial support.
2. Also from version 20.6.0–20.8.0: Returns a `URL` object instead of a string.
3. Also from version 18.19.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.import_meta.resolve`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `import`
- `import()`
- `import.meta`

JavaScript › Reference › Expressions and operators

import()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since January 2020.

The **`import()`** syntax, commonly called *dynamic import*, is a function-like expression that allows loading an ECMAScript module asynchronously and dynamically into a potentially non-module environment.

Unlike the declaration-style counterpart, dynamic imports are only evaluated when needed, and permit greater syntactic flexibility.

Syntax

```js
import(moduleName)
import(moduleName, options)
```

The `import()` call is a syntax that closely resembles a function call, but `import` itself is a keyword, not a function. You cannot alias it like `const myImport = import`, which will throw a `SyntaxError`.

Trailing commas are only allowed if the runtime also supports `options`. Check browser compatibility.

Parameters

`moduleName`

The module to import from. The resolution of the specifier is host-specified, but always follows the same algorithm as static import declarations.

`options`

An object containing import options. The following key is recognized:

`with`

The import attributes.

Return value

Returns a promise which:

- If the referenced module is loaded and evaluated successfully, fulfills to a module namespace object: an object containing all exports from `moduleName`.
- If the coercion to string of `moduleName` throws, rejects with the thrown error.

- If module fetching fails, rejects with a host-defined error (Node typically uses `Error`, while browsers use `TypeError`). Common causes may include:
  - In a file-system-based module system (Node.js, for example), if accessing the file system fails (permission denied, file not found, etc.).
  - In a web-based module system (browsers, for example), if the network request fails (not connected to the Internet, CORS issue, etc.) or an HTTP error occurs (404, 500, etc.).
- If parsing or linking fails, rejects with the corresponding error, typically a `SyntaxError`.
- If evaluation of the referenced module or one of its dependencies throws, rejects with the thrown error.

> **Note:** `import()` never synchronously throws an error.

Description

The import declaration syntax (`import something from "somewhere"`) is static and makes the imported module an eager dependency of the importing module. Dynamic imports allow one to circumvent the syntactic rigidity of import declarations and load a module conditionally or on demand. The following are some reasons why you might need to use dynamic import:

- When importing statically significantly slows the loading of your code or increases your program's memory usage, and there is a low likelihood that you will need the code you are importing, or you will not need it until a later time.
- When the module you are importing does not exist at load time.
- When the import specifier string needs to be constructed dynamically. (Static import only supports static specifiers.)
- When the module being imported has side effects, and you do not want those side effects unless some condition is true. (It is recommended not to have any side effects in a module, but you sometimes cannot control this in your module dependencies.)
- When you are in a non-module environment (for example, `eval` or a script file).

Use dynamic import only when necessary. The static form is preferable for loading initial dependencies, and can benefit more readily from static analysis tools and tree shaking. If your only goal is to defer the execution of the loaded code until when it's used, consider using `import defer`.

If your file is not run as a module (if it's referenced in an HTML file, the script tag must have `type="module"`), you will not be able to use static import declarations. On the other hand, the asynchronous dynamic import syntax is available in both scripts and modules, allowing you to import modules into non-module environments, subject to the execution-context restrictions below.

The `options` parameter allows different kinds of import options. For example, import attributes:

```js
import("./data.json", { with: { type: "json" } });
```

Dynamic module import is not permitted in all execution contexts. For example, `import()` can be used in the main thread, a shared worker, or a dedicated worker, but its promise rejects if called within a service worker or a worklet.

"Phase modifiers" for `import` statements can also be used with dynamic imports:

- `import.defer()`

- `import.source()`

Each of these syntaxes is considered a distinct type of expression.

Module namespace object

A *module namespace object* is an object that describes all unambiguous exports from a module. Its exported names are fixed, but their values reflect live bindings. It is created when needed, which can be during linking, before the module is evaluated. There are two ways to access the module namespace object of a module: through a namespace import (`import * as name from moduleName`), or through the fulfillment value of a dynamic import.

The module namespace object is a sealed object with `null` prototype. This means all string keys of the object correspond to the exports of the module and there are never extra keys. All keys are enumerable in lexicographic order (i.e., the default behavior of `Array.prototype.sort()`), with the default export available as a key called `default`. In addition, the module namespace object has a `[Symbol.toStringTag]` property with the value `"Module"`, used in `Object.prototype.toString()`.

The string properties are non-configurable and writable when you use `Object.getOwnPropertyDescriptors()` to get their descriptors. However, they are effectively read-only, because you cannot re-assign a property to a new value. This behavior mirrors the fact that static imports create "live bindings" — the values can be re-assigned by the module exporting them, but not by the module importing them. The writability of the properties reflects the possibility of the values changing, because non-configurable and non-writable properties must be constant. For example, you can re-assign the exported value of a variable, and the new value can be observed in the module namespace object.

Each (normalized) module specifier corresponds to a unique module namespace object, so the following is generally true:

```js
import * as mod from "/my-module.js";

import("/my-module.js").then((mod2) => {
  console.log(mod === mod2); // true
});
```

Except in one curious case: because a promise never fulfills to a thenable, if the `my-module.js` module exports a function called `then()`, that function will automatically get called when the dynamic import's promise is fulfilled, as part of the promise resolution process.

```js
// -- my-module.js --
export function then(resolve) {
  console.log("then() called");
  resolve(1);
}
```

```js
// -- main.js --
import * as mod from "/my-module.js";

import("/my-module.js").then((mod2) => {
  // Logs "then() called"
  console.log(mod === mod2); // false
});
```

> **Warning:** Do not export a function called `then()` from a module. This will cause the module to behave differently when imported dynamically than when imported statically.

This aggressive caching ensures that a piece of JavaScript code is never executed more than once, even if it is imported multiple times. Future imports (assuming previous success in the same environment) don't even result in HTTP requests or disk access. If you do need to re-import and re-evaluate a module without restarting the entire JavaScript environment, one possible trick is to use a unique query parameter in the module specifier. This works in non-browser runtimes that support URL specifiers too.

```js
import(`/my-module.js?t=${Date.now()}`);
```

Note that this can lead to memory leaks in a long-running application, because the engine cannot safely garbage-collect any module namespace objects. Currently, there is no way to manually clear the cache of module namespace objects.

You can also use the Fetch API to fetch module source code as text, and then evaluate the module manually depending on the module type:

- For JavaScript modules, you can dynamically import the source code as a `blob:` URL in browsers, or use `vm.Module` to evaluate it in Node.js.
- For JSON modules, you can parse the source code using `JSON.parse()`.
- For CSS modules, you can create a new `CSSStyleSheet` object and use its `replace()` method to populate it with the source code.

However, this is semantically not the same as dynamic import, because user-agent settings like fetch destination, CSP, or module resolution may not be applied correctly.

A module is imported in three steps: loading (fetching and parsing the module and its dependencies), linking (resolving imports and preparing bindings), and evaluating (executing the parsed code). Evaluation failures are cached, so subsequent imports of the same module propagate the error without executing it again. Linking can be retried, but the same graph of JavaScript module records will encounter the same linking error.

Caching of loading failures depends on the host and the kind of failure. In browsers, the module map is separate from the HTTP cache. The HTML specification requires failed fetches, including HTTP error responses, to be removed from the module map so a later import can retry. The HTTP cache may still supply a cached error response. Parse errors, however, are retained in the module map, so retrying the same module does not fetch corrected source.

Examples

Import a module for its side effects only

```js
(async () => {
  if (somethingIsTrue) {
    // import module for side effects
    await import("/modules/my-module.js");
  }
})();
```

If your project uses packages that export ESM, you can also import them for side effects only. This will run the code in the package entry point file (and any files it imports) only.

Importing defaults

If you are destructuring the imported module namespace object, then you must rename the `default` key because `default` is a reserved word.

```js
(async () => {
  if (somethingIsTrue) {
    const {
      default: myDefault,
      foo,
      bar,
    } = await import("/modules/my-module.js");
  }
})();
```

Importing on-demand in response to user action

This example shows how to load functionality on to a page based on a user action, in this case a button click, and then call a function within that module. This is not the only way to implement this functionality. The `import()` function also supports `await`.

```js
const main = document.querySelector("main");
for (const link of document.querySelectorAll("nav > a")) {
  link.addEventListener("click", (e) => {
    e.preventDefault();

    import("/modules/my-module.js")
      .then((module) => {
        module.loadPageInto(main);
      })
      .catch((err) => {
        main.textContent = err.message;
      });
  });
}
```

Importing different modules based on environment

In processes such as server-side rendering, you may need to load different logic on server or in browser because they interact with different globals or modules (for example, browser code has access to web APIs like `document` and `navigator`, while server code has access to the server file system). You can do so through a conditional dynamic import.

```js
let myModule;

if (typeof window === "undefined") {
  myModule = await import("module-used-on-server");
} else {
  myModule = await import("module-used-in-browser");
}
```

Importing modules with a non-literal specifier

Dynamic imports allow any expression as the module specifier, not necessarily string literals.

Here, we load 10 modules, `/modules/module-0.js`, `/modules/module-1.js`, etc., concurrently, and call the `load` functions that each one exports.

```js
Promise.all(
  Array.from({ length: 10 }).map(
    (_, index) => import(`/modules/module-${index}.js`),
  ),
).then((modules) => modules.forEach((module) => module.load()));
```

Using import attributes with dynamic import

Import attributes are accepted as the second parameter of the `import()` syntax.

```js
const data = await import("./data.json", {
  with: { type: "json" },
});
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-import-calls`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import`	63	79	67	50	11.1	63	67	46	11.3	8.0	63	11.3	1.0.0	1.0[1]	13.2.0[2,3]
`import.defer()` Experimental	**No**	**No**	Prev.[4]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?
Loading is retriable (HTTP errors are not cached)	156	156	155	140	Prev.	156	155	**No**	**No**	**No**	156	**No**	?	?	?
`import.source()` Experimental	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	24.0.0
The `options` parameter	91	91	138	77	15	91	138	64	15	16.0	91	15	1.1.5	1.17	17.5.0[5,6,7]
Available in workers	80	80	114	67	15	80	114	57	15	13.0	80	15	1.0.0	1.0	13.2.0[8]

1. Bundled Deno applications (using `deno compile`) do not support dynamic imports
2. Dynamic `import` can be used in either CommonJS or ES module files, to import either CommonJS or ES module files. See Node's ECMAScript Modules documentation for more details.
3. Also from version 12.17.0–13.0.0: Dynamic `import` can be used in either CommonJS or ES module files, to import either CommonJS or ES module files. See Node's ECMAScript Modules documentation for more details.
4. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).
5. Also from version 16.15.0–17.0.0: supported.
6. Also from version 16.14.0–16.15.0: partial support.

7. Also from version 16.14.0–16.15.0: The second parameter no longer throws a parser error, but the `--experimental-json-modules` flag is still needed to load JSON modules.
8. Also from version 12.17.0–13.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.import`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `import`

---

import.defer()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import/defer · MDN page last modified 2026-11-05

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import.defer()`** syntax behaves like regular `import()` syntax, except that it results in a deferred module namespace object. The module and its dependencies are fetched and linked up front, but their synchronous evaluation is deferred until the namespace's properties are accessed.

For more information about deferred evaluation, including its interaction with top-level `await`, see the `import defer` declaration form.

Syntax

```js
import.defer(moduleName)
import.defer(moduleName, options)
```

`import.defer()` is special syntax (a "meta property"), not a method on an `import` object.

Parameters

See `import()`.

Return value

Returns a promise that fulfills with a deferred module namespace object after the module graph is loaded and linked, and any eagerly evaluated top-level `await` dependencies have finished evaluating.

Like regular `import()`, the promise rejects if the module or its dependencies cannot be loaded, parsed, or linked. It also rejects if an eagerly evaluated module throws. Errors from evaluation that remains deferred are instead thrown synchronously by the namespace operation that triggers evaluation.

Examples

Using import.defer()

> **Note:** It's guaranteed that awaiting the resulting promise never accidentally calls an exported `then` method—a gotcha associated with the regular module namespace object—because the deferred module namespace object never exposes a property called `then`.

```js
const ts = await import.defer("typescript");

function compilePath(path) {
  // Evaluation of the typescript module subgraph starts here
  const program = ts.createProgram([path], {});
}
```

Never immediately destructure the returned namespace, because it triggers evaluation:

js · ✗ bad example

```js
const { createProgram } = await import.defer("typescript");
// The typescript module has now been evaluated.
```

Specifications

Specification
Deferred Imports Evaluation `https://tc39.es/proposal-defer-import-eval/#sec-left-hand-side-expressions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import.defer()` Experimental	**No**	**No**	Prev.[1]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?

1. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.import.defer`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript modules guide
- `import()`
- `import defer`

JavaScript › Reference › Expressions and operators › import()

import.source()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import/source · MDN page last modified 2026-11-05

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import.source()`** syntax behaves like regular `import()` syntax, except that it results in an object that represents the module's compiled source code. The module is fetched and compiled, but its dependencies are not loaded and it is not linked or evaluated. It can be imperatively evaluated later, such as by using dynamic import or `WebAssembly.instantiate()`.

To use `import.source()`, the target module must be of a kind that supports source phase imports. Currently, only WebAssembly modules support source phase imports, and result in `WebAssembly.Module` objects. JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

For more information about the semantics of source phase imports, see the `import source` declaration form.

Syntax

```
import.source(moduleName)
import.source(moduleName, options)
```

`import.source()` is special syntax (a "meta property"), not a method on an `import` object.

Parameters

See `import()`.

Return value

Returns a promise that fulfills with an `AbstractModuleSource` object representing the module's compiled source after the module is loaded and compiled successfully.

Like regular `import()`, the promise rejects if the module cannot be loaded or parsed. It also rejects with a `SyntaxError` if the module type does not support source phase imports. The import does not load dependencies, link, or evaluate the module, so errors from those later steps are not reported.

Examples

Using import.source()

```
const myModuleSource = await import.source("./my-module.wasm");

const instance = await WebAssembly.instantiate(myModuleSource, {
  env: { log: console.log },
});
const { exports } = instance;
```

Specifications

Specification
Source Phase Imports https://tc39.es/proposal-source-phase-imports/#sec-left-hand-side-expressions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import.source()` Experimental	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.import.import_source`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript modules guide
- `import()`
- `import source`
- `AbstractModuleSource`

JavaScript › Reference › Expressions and operators

in

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/in · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`in`** operator returns `true` if the specified property is in the specified object or its prototype chain.

The `in` operator cannot be used to search for values in other collections. To test if a certain value exists in an array, use `Array.prototype.includes()`. For sets, use `Set.prototype.has()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: in operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const car = { make: "Honda", model: "Accord", year: 1998 };

console.log("make" in car);
// Expected output: true

delete car.make;
if ("make" in car === false) {
  car.make = "Suzuki";
}

console.log(car.make);
// Expected output: "Suzuki"
```

Syntax

js

```
prop in object
#prop in object
```

Parameters

`prop`

A string or symbol representing a property name (non-symbols will be coerced to strings). Can also be a private element identifier.

`object`

Object to check if it (or its prototype chain) contains the property with specified name (`prop`).

Exceptions

`TypeError`

Thrown if `object` is not an object (i.e., a primitive).

Description

The `in` operator tests if a string or symbol property is present in an object or its prototype chain. If you want to check for only *non-inherited* properties, use `Object.hasOwn()` instead.

A property may be present in an object but have value `undefined`. Therefore, `"x" in obj` is not the same as `obj.x !== undefined`. To make `in` return `false` after a property is added, use the `delete` operator instead of setting that property's value to `undefined`.

You can also use the `in` operator to check whether a particular private class field or method has been defined in an object. The operator returns `true` if the property is defined, and `false` otherwise. This is known as a *branded check*, because it returns `true` if and only if the object was created with that class constructor, after which you can safely access other private elements as well.

This is a special syntax — the left-hand side of the `in` operator is a property identifier instead of an expression, but unquoted (because otherwise it's a string property, not a private element).

Because accessing private elements on objects unrelated to the current class throws a `TypeError` instead of returning `undefined`, this syntax allows you to shorten:

```js
class C {
  #x;
  static isC(obj) {
    try {
      obj.#x;
      return true;
    } catch {
      return false;
    }
  }
}
```

To:

```js
class C {
  #x;
  static isC(obj) {
    return #x in obj;
  }
}
```

It also generally avoids the need for dealing with error handling just to access a private element that may be nonexistent.

However, the `in` operator still requires the private element to be declared beforehand in the enclosing class — otherwise, it would throw a `SyntaxError` ("Private field '#x' must be declared in an enclosing class"), the same one as when you try to access an undeclared private element.

js · ✗ bad example

```
class C {
  foo() {
    #x in this;
  }
}

new C().foo(); // SyntaxError: Private field '#x' must be declared in an enclosing class
```

Examples

Basic usage

The following examples show some uses of the `in` operator.

js

```
// Arrays
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
0 in trees; // returns true
3 in trees; // returns true
6 in trees; // returns false
"bay" in trees; // returns false (you must specify the index number, not the value at that
index)
"length" in trees; // returns true (length is an Array property)
Symbol.iterator in trees; // returns true

// Predefined objects
"PI" in Math; // returns true

// Custom objects
const myCar = { make: "Honda", model: "Accord", year: 1998 };
"make" in myCar; // returns true
"model" in myCar; // returns true
```

You must specify an object on the right side of the `in` operator. For example, you can specify a string created with the `String` constructor, but you cannot specify a string literal.

js

```
const color1 = new String("green");
"length" in color1; // returns true

const color2 = "coral";
// generates an error (color2 is not a String object)
"length" in color2;
```

Using the in operator with deleted or undefined properties

If you delete a property with the `delete` operator, the `in` operator returns `false` for that property.

```js
const myCar = { make: "Honda", model: "Accord", year: 1998 };
delete myCar.make;
"make" in myCar; // returns false

const trees = ["redwood", "bay", "cedar", "oak", "maple"];
delete trees[3];
3 in trees; // returns false
```

If you set a property to `undefined` but do not delete it, the `in` operator returns true for that property.

```js
const myCar = { make: "Honda", model: "Accord", year: 1998 };
myCar.make = undefined;
"make" in myCar; // returns true
```

```js
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
trees[3] = undefined;
3 in trees; // returns true
```

The `in` operator will return `false` for empty array slots, even if accessing it directly returns `undefined`.

```js
const empties = new Array(3);
empties[2]; // returns undefined
2 in empties; // returns false
```

To avoid this, make sure a new array is always filled with non-empty values or not write to indexes past the end of array.

```js
const empties = new Array(3).fill(undefined);
2 in empties; // returns true
```

Inherited properties

The `in` operator returns `true` for properties in the prototype chain. This may be undesirable if you are using objects to store arbitrary key-value pairs.

js · ✗ bad example

```
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return name in ages;
}

hasPerson("hasOwnProperty"); // true
```

You can use `Object.hasOwn()` to check if the object has the key.

```js
const ages = { alice: 18, bob: 27 };

function hasPerson(name) {
  return Object.hasOwn(ages, name);
}

hasPerson("hasOwnProperty"); // false
```

Alternatively, you should consider using a null prototype object or a `Map` for storing `ages`, to avoid other bugs.

js · ✓ good example

```js
const ages = new Map([
  ["alice", 18],
  ["bob", 27],
]);

function hasPerson(name) {
  return ages.has(name);
}

hasPerson("hasOwnProperty"); // false
```

Using the in operator to implement branded checks

The code fragment below demonstrates a static function that tells if an object was created with the `Person` constructor and therefore can perform other methods safely.

```js
class Person {
  #age;
  constructor(age) {
    this.#age = age;
  }
  static isPerson(o) {
    return #age in o;
  }
  ageDifference(other) {
    return this.#age - other.#age;
  }
}

const p1 = new Person(20);
const p2 = new Person(30);
console.log(p1.ageDifference(p2)); // -10
console.log(Person.isPerson(p1)); // true

if (Person.isPerson(p1) && Person.isPerson(p2)) {
  console.log(p1.ageDifference(p2)); // -10
}
```

It helps to prevent the following case:

```js
const p2 = {};

p1.ageDifference(p2); // TypeError: Cannot read private member #age from an object whose
class did not declare it
```

Without the `in` operator, you would have to use a `try...catch` block to check if the object has the private element.

You can also implement this as a `[Symbol.hasInstance]()` method of the class, so that you can use the `instanceof` operator to perform the same check (which, by default, only checks for the existence of `Person.prototype` in the object's prototype chain).

```js
class Person {
  #age;
  constructor(age) {
    this.#age = age;
  }
  static [Symbol.hasInstance](o) {
    // Testing `this` to prevent false-positives when
    // calling `instanceof SubclassOfPerson`
    return this === Person && #age in o;
  }
  ageDifference(other) {
    return this.#age - other.#age;
  }
}

const p1 = new Person(20);
const p2 = new Person(30);

if (p1 instanceof Person && p2 instanceof Person) {
  console.log(p1.ageDifference(p2)); // -10
}
```

For more examples, see Private elements and the class guide.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`in`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.in`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `for...in`
- `delete`

- `Object.hasOwn()`
- `Reflect.has()`
- Enumerability and ownership of properties

JavaScript › Reference › Expressions and operators

Increment (++)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Increment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **increment (++)** operator increments (adds one to) its operand and returns the value before or after the increment, depending on where the operator is placed.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Increment (++) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let x = 3;
const y = x++;

console.log(`x:${x}, y:${y}`);
// Expected output: "x:4, y:3"

let a = 3;
const b = ++a;

console.log(`a:${a}, b:${b}`);
// Expected output: "a:4, b:4"
```

Syntax

js

```
x++
++x
```

Description

The ++ operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt increment if the operand becomes a BigInt; otherwise, it performs number increment.

If used postfix, with operator after operand (for example, x++), the increment operator increments and returns the value before incrementing.

If used prefix, with operator before operand (for example, ++x), the increment operator increments and returns the value after incrementing.

The increment operator can only be applied on operands that are references (variables and object properties; i.e., valid assignment targets). ++x itself evaluates to a value, not a reference, so you cannot chain multiple increment operators together.

js · ✗ bad example

```
++(++x); // SyntaxError: Invalid left-hand side expression in prefix operation
```

Examples

Postfix increment

js

```
let x = 3;
const y = x++;
// x is 4; y is 3

let x2 = 3n;
const y2 = x2++;
// x2 is 4n; y2 is 3n
```

Prefix increment

js

```
let x = 3;
const y = ++x;
// x is 4; y is 4

let x2 = 3n;
const y2 = ++x2;
// x2 is 4n; y2 is 4n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-postfix-increment-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Increment (++)	2	12	1	3	4	18	4	10.1	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.increment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)

- Exponentiation (**)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Inequality (!=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Inequality · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **inequality (!=)** operator checks whether its two operands are not equal, returning a Boolean result. Unlike the strict inequality operator, it attempts to convert and compare operands that are of different types.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Inequality (!=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(1 != 1);
// Expected output: false

console.log("hello" != "hello");
// Expected output: false

console.log("1" != 1);
// Expected output: false

console.log(0 != false);
// Expected output: false
```

Syntax

js

```
x != y
```

Description

The inequality operator checks whether its operands are not equal. It is the negation of the equality operator so the following two lines will always give the same result:

js

```
x != y;

!(x == y);
```

For details of the comparison algorithm, see the page for the equality operator.

Like the equality operator, the inequality operator will attempt to convert and compare operands of different types:

```js
3 != "3"; // false
```

To prevent this, and require that different types are considered to be different, use the strict inequality operator instead:

```js
3 !== "3"; // true
```

Examples

Comparison with no type conversion

```js
1 != 2; // true
"hello" != "hola"; // true

1 != 1; // false
"hello" != "hello"; // false
```

Comparison with type conversion

```js
"1" != 1; // false
1 != "1"; // false
0 != false; // false
0 != null; // true
0 != undefined; // true
0 != !!null; // false, look at Logical NOT operator
0 != !!undefined; // false, look at Logical NOT operator
null != undefined; // false

const number1 = new Number(3);
const number2 = new Number(3);
number1 != 3; // false
number1 != number2; // true
```

Comparison of objects

```js
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 != object2); // true
console.log(object1 != object1); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-equality-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Inequality (a != b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.inequality`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Equality (==)
- Strict equality (===)
- Strict inequality (!==)

JavaScript › Reference › Expressions and operators

instanceof

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/instanceof · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`instanceof`** operator tests to see if the `prototype` property of a constructor appears anywhere in the prototype chain of an object. The return value is a boolean value. Its behavior can be customized with `Symbol.hasInstance`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: instanceof operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
const auto = new Car("Honda", "Accord", 1998);

console.log(auto instanceof Car);
// Expected output: true

console.log(auto instanceof Object);
// Expected output: true
```

Syntax

js

```
object instanceof constructor
```

Parameters

`object`

The object to test.

`constructor`

Constructor to test against.

Exceptions

`TypeError`

Thrown if `constructor` is not an object. If `constructor` doesn't have a `[Symbol.hasInstance]()` method, it must also be a function.

Description

The `instanceof` operator tests the presence of `constructor.prototype` in `object`'s prototype chain. This usually (though not always) means `object` was constructed with `constructor`.

```js
// defining constructors
function C() {}
function D() {}

const o = new C();

// true, because: Object.getPrototypeOf(o) === C.prototype
o instanceof C;

// false, because D.prototype is nowhere in o's prototype chain
o instanceof D;

o instanceof Object; // true, because:
C.prototype instanceof Object; // true

// Re-assign `constructor.prototype`: you should
// rarely do this in practice.
C.prototype = {};
const o2 = new C();

o2 instanceof C; // true

// false, because C.prototype is nowhere in
// o's prototype chain anymore
o instanceof C;

D.prototype = new C(); // add C to [[Prototype]] linkage of D
const o3 = new D();
o3 instanceof D; // true
o3 instanceof C; // true since C.prototype is now in o3's prototype chain
```

Note that the value of an `instanceof` test can change if `constructor.prototype` is re-assigned after creating the object (which is usually discouraged). It can also be changed by changing `object`'s prototype using `Object.setPrototypeOf`.

Classes behave in the same way, because classes also have the `prototype` property.

```js
class A {}
class B extends A {}

const o1 = new A();
// true, because Object.getPrototypeOf(o1) === A.prototype
o1 instanceof A;
// false, because B.prototype is nowhere in o1's prototype chain
o1 instanceof B;

const o2 = new B();
// true, because Object.getPrototypeOf(Object.getPrototypeOf(o2)) === A.prototype
o2 instanceof A;
// true, because Object.getPrototypeOf(o2) === B.prototype
o2 instanceof B;
```

For bound functions, `instanceof` looks up for the `prototype` property on the target function, since bound functions don't have `prototype`.

```js
class Base {}
const BoundBase = Base.bind(null, 1, 2);
console.log(new Base() instanceof BoundBase); // true
```

instanceof and Symbol.hasInstance

If `constructor` has a `Symbol.hasInstance` method, the method will be called in priority, with `object` as its only argument and `constructor` as `this`.

```js
// This class allows plain objects to be disguised as this class's instance,
// as long as the object has a particular flag as its property.
class Forgeable {
  static isInstanceFlag = Symbol("isInstanceFlag");

  static [Symbol.hasInstance](obj) {
    return Forgeable.isInstanceFlag in obj;
  }
}

const obj = { [Forgeable.isInstanceFlag]: true };
console.log(obj instanceof Forgeable); // true
```

Because all functions inherit from `Function.prototype` by default, most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function. See the `Symbol.hasInstance` page for the exact algorithm of `instanceof`.

instanceof and multiple realms

JavaScript execution environments (windows, frames, etc.) are each in their own *realm*. This means that they have different built-ins (different global object, different constructors, etc.). This may result in unexpected results. For instance, `[] instanceof window.frames[0].Array` will return `false`, because `Array.prototype !== window.frames[0].Array.prototype` and arrays in the current realm inherit from the former.

This may not make sense at first, but for scripts dealing with multiple frames or windows, and passing objects from one context to another via functions, this will be a valid and strong issue. For instance, you

can securely check if a given object is in fact an Array using `Array.isArray()`, neglecting which realm it comes from.

For example, to check if a `Node` is an `SVGElement` in a different context, you can use `myNode instanceof myNode.ownerDocument.defaultView.SVGElement`.

Examples

Using instanceof with String

The following example shows the behavior of `instanceof` with `String` objects.

js

```
const literalString = "This is a literal string";
const stringObject = new String("String created with constructor");

literalString instanceof String; // false, string primitive is not a String
stringObject instanceof String; // true

literalString instanceof Object; // false, string primitive is not an Object
stringObject instanceof Object; // true

stringObject instanceof Date; // false
```

Using instanceof with Map

The following example shows the behavior of `instanceof` with `Map` objects.

js

```
const myMap = new Map();

myMap instanceof Map; // true
myMap instanceof Object; // true
myMap instanceof String; // false
```

Objects created using Object.create()

The following example shows the behavior of `instanceof` with objects created using `Object.create()`.

js

```
function Shape() {}

function Rectangle() {
  Shape.call(this); // call super constructor.
}

Rectangle.prototype = Object.create(Shape.prototype);

Rectangle.prototype.constructor = Rectangle;

const rect = new Rectangle();

rect instanceof Object; // true
rect instanceof Shape; // true
rect instanceof Rectangle; // true
rect instanceof String; // false
```

(code continues on the next page)

(code continued from the previous page)

```
const literalObject = {};
const nullObject = Object.create(null);
nullObject.name = "My object";

literalObject instanceof Object; // true, every object literal has Object.prototype as
prototype
({}) instanceof Object; // true, same case as above
nullObject instanceof Object; // false, prototype is end of prototype chain (null)
```

Demonstrating that myCar is of type Car and type Object

The following code creates an object type `Car` and an instance of that object type, `myCar`. The `instanceof` operator demonstrates that the `myCar` object is of type `Car` and of type `Object`.

js

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
const myCar = new Car("Honda", "Accord", 1998);
const a = myCar instanceof Car; // returns true
const b = myCar instanceof Object; // returns true
```

Not an instanceof

To test if an object is not an `instanceof` a specific constructor, you can do:

js

```
if (!(myCar instanceof Car)) {
  // Do something, like:
  // myCar = new Car(myCar)
}
```

This is really different from:

js · ✗ bad example

```
if (!myCar instanceof Car) {
  // unreachable code
}
```

This will always be `false`. (`!myCar` will be evaluated before `instanceof`, so you always try to know if a boolean is an instance of `Car`).

Overriding the behavior of instanceof

A common pitfall of using `instanceof` is believing that, if `x instanceof C`, then `x` was created using `C` as constructor. This is not true, because `x` could be directly assigned with `C.prototype` as its prototype. In this case, if your code reads private fields of `C` from `x`, it would still fail:

```js
class C {
  #value = "foo";
  static getValue(x) {
    return x.#value;
  }
}

const x = { __proto__: C.prototype };

if (x instanceof C) {
  console.log(C.getValue(x)); // TypeError: Cannot read private member #value from an object
whose class did not declare it
}
```

To avoid this, you can override the behavior of `instanceof` by adding a `Symbol.hasInstance` method to `C`, so that it does a branded check with `in`:

```js
class C {
  #value = "foo";

  static [Symbol.hasInstance](x) {
    return #value in x;
  }

  static getValue(x) {
    return x.#value;
  }
}

const x = { __proto__: C.prototype };

if (x instanceof C) {
  // Doesn't run, because x is not a C
  console.log(C.getValue(x));
}
```

Note that you may want to limit this behavior to the current class; otherwise, it could lead to false positives for subclasses:

```js
class D extends C {}
console.log(new C() instanceof D); // true; because D inherits [Symbol.hasInstance] from C
```

You could do this by checking that `this` is the current constructor:

```js
class C {
  #value = "foo";

  static [Symbol.hasInstance](x) {
    return this === C && #value in x;
  }
}

class D extends C {}
console.log(new C() instanceof D); // false
console.log(new C() instanceof C); // true
console.log({ __proto__: C.prototype } instanceof C); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
instanceof	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.instanceof`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `typeof`
- `Symbol.hasInstance`
- `Object.prototype.isPrototypeOf`

JavaScript › Reference › Expressions and operators

Left shift (<<)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Left_shift · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **left shift (<<)** operator returns a number or BigInt whose binary representation is the first operand shifted by the specified number of bits to the left. Excess bits shifted off to the left are discarded, and zero bits are shifted in from the right.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Left shift (<<) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 5; // 00000000000000000000000000000101
const b = 2; // 00000000000000000000000000000010

console.log(a << b); // 00000000000000000000000000010100
// Expected output: 20
```

Syntax

js

```
x << y
```

Description

The << operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt left shift if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number left shift. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

The operator operates on the left operand's bit representation in two's complement. For example, `9 << 2` yields 36:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
                  --------------------------------
9 << 2 (base 10): 00000000000000000000000000100100 (base 2) = 36 (base 10)
```

Bitwise a 32-bit integer x to the left by y bits yields `x * 2 ** y`. So for example, `9 << 3` is equivalent to `9 * (2 ** 3) = 9 * (8) = 72`.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 << 32` is the same as `100 << 0` (and produces `100`) because 32 modulo 32 is 0.

> **Warning:** You may see people using `<< 0` to truncate numbers to integers. Left shifting any number x by `0` returns x converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using left shift

```js
9 << 3; // 72

// 9 * (2 ** 3) = 9 * (8) = 72

9n << 3n; // 72n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-left-shift-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise left shift (a << b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.left_shift`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide
- Left shift assignment (<<=)

JavaScript › Reference › Expressions and operators

Left shift assignment (<<=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Left_shift_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **left shift assignment (<<=)** operator performs left shift on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Left shift assignment (<<=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 5; // 00000000000000000000000000000101

a <<= 2; // 00000000000000000000000000010100

console.log(a);
// Expected output: 20
```

Syntax

js

```
x <<= y
```

Description

`x <<= y` is equivalent to `x = x << y`, except that the expression `x` is only evaluated once.

Examples

Using left shift assignment

js

```
let a = 5;
// 00000000000000000000000000000101

a <<= 2; // 20
// 00000000000000000000000000010100

let b = 5n;
b <<= 2n; // 20n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Left shift assignment (`x <<= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.left_shift_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Left shift (<<)

JavaScript › Reference › Expressions and operators

Less than (<)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Less_than · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **less than (<)** operator returns `true` if the left operand is less than the right operand, and `false` otherwise.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Less than (<) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(5 < 3);
// Expected output: false

console.log(3 < 3);
// Expected output: false

// Compare bigint to number
console.log(3n < 5);
// Expected output: true

console.log("aa" < "ab");
// Expected output: true
```

Syntax

js

```
x < y
```

Description

The operands are compared with multiple rounds of coercion, which can be summarized as follows:

- First, objects are converted to primitives by calling its `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The left operand is always coerced before the right one. Note that although `[Symbol.toPrimitive]()` is called with the `"number"` hint (meaning there's a slight preference for the object to become a number), the return value is not converted to a number, since strings are still specially handled.
- If both values are strings, they are compared as strings, based on the values of the UTF-16 code units (not Unicode code points) they contain.
- Otherwise JavaScript attempts to convert non-numeric types to numeric values:
  - Boolean values `true` and `false` are converted to 1 and 0 respectively.
  - `null` is converted to 0.
  - `undefined` is converted to `NaN`.

    - Strings are converted based on the values they contain, and are converted as `NaN` if they do not contain numeric values.
- If either value is `NaN`, the operator returns `false`.
- Otherwise the values are compared as numeric values. BigInt and number values can be compared together.

Other operators, including `>`, `>=`, and `<=`, use the same algorithm as `<`. There are two cases where all four operators return `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

For all other cases, the four operators have the following relationships:

```js
x < y === !(x >= y);
x <= y === !(x > y);
x > y === y < x;
x >= y === y <= x;
```

> **Note:** One observable difference between `<` and `>` is the order of coercion, especially if the coercion to primitive has side effects. All comparison operators coerce the left operand before the right operand.

Examples

String to string comparison

```js
"a" < "b"; // true
"a" < "a"; // false
"a" < "3"; // false

"\uD855\uDE51" < "\uFF3A"; // true
```

String to number comparison

```js
"5" < 3; // false
"3" < 3; // false
"3" < 5; // true

"hello" < 5; // false
5 < "hello"; // false

"5" < 3n; // false
"3" < 5n; // true
```

Number to Number comparison

```js
5 < 3; // false
3 < 3; // false
3 < 5; // true
```

Number to BigInt comparison

```js
5n < 3; // false
3 < 5n; // true
```

Comparing Boolean, null, undefined, NaN

```js
true < false; // false
false < true; // true

0 < true; // true
true < 1; // false

null < 0; // false
null < 1; // true

undefined < 3; // false
3 < undefined; // false

3 < NaN; // false
NaN < 3; // false
```

Comparison with side effects

Comparisons always coerce their operands to primitives. This means the same object may end up having different values within one comparison expression. For example, you may have two values that are both greater than and less than the other.

```js
class Mystery {
  static #coercionCount = -1;
  valueOf() {
    Mystery.#coercionCount++;
    // The left operand is coerced first, so this will return 0
    // Then it returns 1 for the right operand
    return Mystery.#coercionCount % 2;
  }
}

const l = new Mystery();
const r = new Mystery();
console.log(l < r && r < l);
// true
```

> **Warning:** This can be a source of confusion. If your objects provide custom primitive conversion logic, make sure it is *idempotent*: multiple coercions should return the same value.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Less than (a < b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.less_than`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Greater than (>)
- Greater than or equal (>=)
- Less than or equal (<=)

JavaScript › Reference › Expressions and operators

Less than or equal (<=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Less_than_or_equal · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **less than or equal (<=)** operator returns `true` if the left operand is less than or equal to the right operand, and `false` otherwise.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Less than or equal (<=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(5 <= 3);
// Expected output: false

console.log(3 <= 3);
// Expected output: true

// Compare bigint to number
console.log(3n <= 5);
// Expected output: true

console.log("aa" <= "ab");
// Expected output: true
```

Syntax

js

```
x <= y
```

Description

The operands are compared using the same algorithm as the Less than operator, with the operands swapped and the result negated. `x <= y` is generally equivalent to `!(y < x)`, except for two cases where `x <= y` and `x > y` are both `false`:

- If one of the operands gets converted to a BigInt, while the other gets converted to a string that cannot be converted to a BigInt value (it throws a syntax error when passed to `BigInt()`).
- If one of the operands gets converted to `NaN`. (For example, strings that cannot be converted to numbers, or `undefined`.)

In addition, `x <= y` coerces `x` to a primitive before `y`, while `y < x` coerces `y` to a primitive before `x`. Because coercion may have side effects, the order of the operands may matter.

`x <= y` is generally equivalent to `x < y || x == y`, except for a few cases:

- When one of `x` or `y` is `null`, and the other is something that's not `null` and becomes 0 when coerced to numeric (including `0`, `0n`, `false`, `""`, `"0"`, `new Date(0)`, etc.): `x <= y` is `true`, while `x < y || x == y` is `false`.
- When one of `x` or `y` is `undefined`, and the other is one of `null` or `undefined`: `x <= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are the same object that becomes `NaN` after the first step of Less than (such as `new Date(NaN)`): `x <= y` is `false`, while `x == y` is `true`.
- When `x` and `y` are different objects that become the same value after the first step of Less than: `x <= y` is `true`, while `x < y || x == y` is `false`.

Examples

String to string comparison

```js
"a" <= "b"; // true
"a" <= "a"; // true
"a" <= "3"; // false
```

String to number comparison

```js
"5" <= 3; // false
"3" <= 3; // true
"3" <= 5; // true

"hello" <= 5; // false
5 <= "hello"; // false
```

Number to Number comparison

```js
5 <= 3; // false
3 <= 3; // true
3 <= 5; // true
```

Number to BigInt comparison

```js
5n <= 3; // false
3 <= 3n; // true
3 <= 5n; // true
```

Comparing Boolean, null, undefined, NaN

```js
true <= false; // false
true <= true; // true
false <= true; // true

true <= 0; // false
true <= 1; // true

null <= 0; // true
1 <= null; // false

undefined <= 3; // false
3 <= undefined; // false

3 <= NaN; // false
NaN <= 3; // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-relational-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Less than or equal (a <= b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.less_than_or_equal`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Greater than (>)
- Greater than or equal (>=)
- Less than (<)

JavaScript › Reference › Expressions and operators

Logical AND (&&)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_AND · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **logical AND (&&)** (logical conjunction) operator for a set of boolean operands will be `true` if and only if all the operands are `true`. Otherwise it will be `false`.

More generally, the operator returns the value of the first falsy operand encountered when evaluating from left to right, or the value of the last operand if they are all truthy.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Logical AND (&&) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 3;
const b = -2;

console.log(a > 0 && b > 0);
// Expected output: false
```

Syntax

js

```
x && y
```

Description

Logical AND (`&&`) evaluates operands from left to right, returning immediately with the value of the first falsy operand it encounters; if all values are truthy, the value of the last operand is returned.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `false`;
- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or ``` ``);
- `undefined`.

The AND operator preserves non-Boolean values and returns them as they are:

js

```
result = "" && "foo"; // result is assigned "" (empty string)
result = 2 && 0; // result is assigned 0
result = "foo" && 4; // result is assigned 4
```

Even though the && operator can be used with non-Boolean operands, it is still considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator or the `Boolean` constructor.

Short-circuit evaluation

The logical AND expression is a short-circuit operator. As each operand is converted to a boolean, if the result of one conversion is found to be `false`, the AND operator stops and returns the original value of that falsy operand; it does **not** evaluate any of the remaining operands.

Consider the pseudocode below.

```
(some falsy expression) && expr
```

The `expr` part is **never evaluated** because the first operand `(some falsy expression)` is evaluated as falsy. If `expr` is a function, the function is never called. See the example below:

```js
function A() {
  console.log("called A");
  return false;
}
function B() {
  console.log("called B");
  return true;
}

console.log(A() && B());
// Logs "called A" to the console due to the call for function A,
// && evaluates to false (function A returns false), then false is logged to the console;
// the AND operator short-circuits here and ignores function B
```

Operator precedence

The AND operator has a higher precedence than the OR operator, meaning the && operator is executed before the || operator (see operator precedence).

```js
true || false && false; // true
true && (false || false); // false
(2 === 3) || (4 < 0) && (1 === 1); // false
```

Examples

Using AND

The following code shows examples of the && (logical AND) operator.

```js
a1 = true && true; // t && t returns true
a2 = true && false; // t && f returns false
a3 = false && true; // f && t returns false
a4 = false && 3 === 4; // f && f returns false
a5 = "Cat" && "Dog"; // t && t returns "Dog"
a6 = false && "Cat"; // f && t returns false
a7 = "Cat" && false; // t && f returns false
a8 = "" && false; // f && f returns ""
a9 = false && ""; // f && f returns false
```

Conversion rules for booleans

Converting AND to OR

The following operation involving **booleans**:

```js
bCondition1 && bCondition2
```

is always equal to:

```js
!(!bCondition1 || !bCondition2)
```

Converting OR to AND

The following operation involving **booleans**:

```js
bCondition1 || bCondition2
```

is always equal to:

```js
!(!bCondition1 && !bCondition2)
```

Removing nested parentheses

As logical expressions are evaluated left to right, it is always possible to remove parentheses from a complex expression provided that certain rules are followed.

The following composite operation involving **booleans**:

```js
bCondition1 || (bCondition2 && bCondition3)
```

is always equal to:

```js
bCondition1 || bCondition2 && bCondition3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-LogicalANDExpression

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Logical AND (&&)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.logical_and`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Boolean`
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Logical AND assignment (&&=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_AND_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2020.

The **logical AND assignment (`&&=`)** operator only evaluates the right operand and assigns to the left if the left operand is truthy.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Logical AND assignment (&&=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 1;
let b = 0;

a &&= 2;
console.log(a);
// Expected output: 2

b &&= 2;
console.log(b);
// Expected output: 0
```

Syntax

js

```
x &&= y
```

Description

Logical AND assignment *short-circuits*, meaning that `x &&= y` is equivalent to `x && (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not truthy, due to short-circuiting of the logical AND operator. For example, the following does not throw an error, despite `x` being `const`:

```js
const x = 0;
x &&= 2;
```

Neither would the following trigger the setter:

```js
const x = {
  get value() {
    return 0;
  },
  set value(v) {
    console.log("Setter called");
  },
};

x.value &&= 2;
```

In fact, if `x` is not truthy, `y` is not evaluated at all.

```js
const x = 0;
x &&= console.log("y evaluated");
// Logs nothing
```

Examples

Using logical AND assignment

```js
let x = 0;
let y = 1;

x &&= 0; // 0
x &&= 1; // 0
y &&= 1; // 1
y &&= 0; // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Logical AND assignment (x &&= y)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.logical_and_assignment`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Logical AND (&&)
- Nullish coalescing operator (??)
- Bitwise AND assignment (&=)
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Logical NOT (!)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_NOT · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **logical NOT (`!`)** (logical complement, negation) operator takes truth to falsity and vice versa. It is typically used with boolean (logical) values. When used with non-Boolean values, it returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Logical NOT (!) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = 3;
const b = -2;

console.log(!(a > 0 || b > 0));
// Expected output: false
```

Syntax

js

```
!x
```

Description

Returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or ``` ``);
- `undefined`.

Even though the `!` operator can be used with operands that are not Boolean values, it can still be considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator (`!!`) or the `Boolean` constructor.

Examples

Using NOT

The following code shows examples of the `!` (logical NOT) operator.

```js
!true; // !t returns false
!false; // !f returns true
!""; // !f returns true
!"Cat"; // !t returns false
```

Double NOT (!!)

It is possible to use a couple of NOT operators in series to explicitly force the conversion of any value to the corresponding boolean primitive. The conversion is based on the "truthiness" or "falsiness" of the value (see truthy and falsy).

The same conversion can be done through the `Boolean()` function.

```js
!!true; // !!truthy returns true
!!{}; // !!truthy returns true: any object is truthy…
!!new Boolean(false); // … even Boolean objects with a false .valueOf()!
!!false; // !!falsy returns false
!!""; // !!falsy returns false
!!Boolean(false); // !!falsy returns false
```

Converting between NOTs

The following operation involving **booleans**:

```js
!!bCondition
```

is always equal to:

```
bCondition
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-logical-not-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Logical NOT (!)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.logical_not`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Boolean`
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Logical OR (||)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_OR · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **logical OR (||)** (logical disjunction) operator for a set of operands is true if and only if one or more of its operands is true. It is typically used with boolean (logical) values. When it is, it returns a Boolean value. However, the || operator actually returns the value of one of the specified operands, so if this operator is used with non-Boolean values, it will return a non-Boolean value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Logical OR (||) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = 3;
const b = -2;

console.log(a > 0 || b > 0);
// Expected output: true
```

Syntax

```js
x || y
```

Description

If `x` can be converted to `true`, returns `x`; else, returns `y`.

If a value can be converted to `true`, the value is so-called truthy. If a value can be converted to `false`, the value is so-called falsy.

Examples of expressions that can be converted to false are:

- `null`;
- `NaN`;
- `0`;
- empty string (`""` or `''` or ``` ``);
- `undefined`.

Even though the `||` operator can be used with operands that are not Boolean values, it can still be considered a boolean operator since its return value can always be converted to a boolean primitive. To explicitly convert its return value (or any expression in general) to the corresponding boolean value, use a double NOT operator or the `Boolean()` constructor.

Short-circuit evaluation

The logical OR expression is evaluated left to right, it is tested for possible "short-circuit" evaluation using the following rule:

`(some truthy expression) || expr` is short-circuit evaluated to the truthy expression.

Short circuit means that the `expr` part above is **not evaluated**, hence any side effects of doing so do not take effect (e.g., if `expr` is a function call, the calling never takes place). This happens because the value of the operator is already determined after the evaluation of the first operand. See example:

```js
function A() {
  console.log("called A");
  return false;
}
function B() {
  console.log("called B");
  return true;
}

console.log(B() || A());
// Logs "called B" due to the function call,
// then logs true (which is the resulting value of the operator)
```

Operator precedence

The following expressions might seem equivalent, but they are not, because the `&&` operator is executed before the `||` operator (see operator precedence).

```js
true || false && false; // returns true, because && is executed first
(true || false) && false; // returns false, because grouping has the highest precedence
```

Examples

Using OR

The following code shows examples of the || (logical OR) operator.

```js
true || true; // t || t returns true
false || true; // f || t returns true
true || false; // t || f returns true
false || 3 === 4; // f || f returns false
"Cat" || "Dog"; // t || t returns "Cat"
false || "Cat"; // f || t returns "Cat"
"Cat" || false; // t || f returns "Cat"
"" || false; // f || f returns false
false || ""; // f || f returns ""
false || varObject; // f || object returns varObject
```

> **Note:** If you use this operator to provide a default value to some variable, be aware that any *falsy* value will not be used. If you only need to filter out `null` or `undefined`, consider using the nullish coalescing operator.

Conversion rules for booleans

Converting AND to OR

The following operation involving **booleans**:

```js
bCondition1 && bCondition2
```

is always equal to:

```js
!(!bCondition1 || !bCondition2)
```

Converting OR to AND

The following operation involving **booleans**:

```js
bCondition1 || bCondition2
```

is always equal to:

```js
!(!bCondition1 && !bCondition2)
```

Removing nested parentheses

As logical expressions are evaluated left to right, it is always possible to remove parentheses from a complex expression following some rules.

The following composite operation involving **booleans**:

```js
bCondition1 && (bCondition2 || bCondition3)
```

is always equal to:

```js
!(!bCondition1 || !bCondition2 && !bCondition3)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-LogicalORExpression

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Logical OR (||)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.logical_or`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Nullish coalescing operator (`??`)
- `Boolean`
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Logical OR assignment (||=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Logical_OR_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2020.

The **logical OR assignment (`||=`)** operator only evaluates the right operand and assigns to the left if the left operand is falsy.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Logical OR assignment (||=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const a = { duration: 50, title: "" };

a.duration ||= 10;
console.log(a.duration);
// Expected output: 50

a.title ||= "title is empty.";
console.log(a.title);
// Expected output: "title is empty."
```

Syntax

js

```
x ||= y
```

Description

Logical OR assignment *short-circuits*, meaning that `x ||= y` is equivalent to `x || (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not falsy, due to short-circuiting of the logical OR operator. For example, the following does not throw an error, despite `x` being `const`:

js

```
const x = 1;
x ||= 2;
```

Neither would the following trigger the setter:

js

```
const x = {
  get value() {
    return 1;
  },
  set value(v) {
    console.log("Setter called");
  },
};

x.value ||= 2;
```

In fact, if `x` is not falsy, `y` is not evaluated at all.

js

```
const x = 1;
x ||= console.log("y evaluated");
// Logs nothing
```

Examples

Setting default content

If the "lyrics" element is empty, display a default value:

```js
document.getElementById("lyrics").textContent ||= "No lyrics.";
```

Here the short-circuit is especially beneficial, since the element will not be updated unnecessarily and won't cause unwanted side-effects such as additional parsing or rendering work, or loss of focus, etc.

Note: Pay attention to the value returned by the API you're checking against. If an empty string is returned (a falsy value), `||=` must be used, so that "No lyrics." is displayed instead of a blank space. However, if the API returns `null` or `undefined` in case of blank content, `??=` should be used instead.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Logical OR assignment (x ||= y)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.logical_or_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Logical OR (||)
- Nullish coalescing operator (??)
- Bitwise OR assignment (|=)
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Multiplication (*)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Multiplication · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **multiplication (*)** operator produces the product of the operands.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Multiplication (*) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(3 * 4);
// Expected output: 12

console.log(-3 * 4);
// Expected output: -12

console.log("3" * 2);
// Expected output: 6

console.log("foo" * 2);
// Expected output: NaN
```

Syntax

js

```
x * y
```

Description

The * operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt multiplication if both operands become BigInts; otherwise, it performs number multiplication. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Examples

Multiplication using numbers

js

```
2 * 2; // 4
-2 * 2; // -4

Infinity * 0; // NaN
Infinity * Infinity; // Infinity
```

Other non-BigInt values are coerced to numbers:

js

```
"foo" * 2; // NaN
"2" * 2; // 4
```

Multiplication using BigInts

```js
2n * 2n; // 4n
-2n * 2n; // -4n
```

You cannot mix BigInt and number operands in multiplication.

js · ✗ bad example

```
2n * 2; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 * 2n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do multiplication with a BigInt and a non-BigInt, convert either operand:

```js
2n * BigInt(2); // 4n
Number(2n) * 2; // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-multiplicative-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Multiplication (*)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.multiplication`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Multiplication assignment (*=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Multiplication_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **multiplication assignment (*=)** operator performs multiplication on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Multiplication assignment (=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a = 2;

console.log((a *= 3));
// Expected output: 6

console.log((a *= "hello"));
// Expected output: NaN
```

Syntax

js

```
x *= y
```

Description

`x *= y` is equivalent to `x = x * y`, except that the expression `x` is only evaluated once.

Examples

Multiplication assignment using numbers

js

```
let bar = 5;
bar *= 2; // 10
```

Other non-BigInt values are coerced to numbers:

js

```
let bar = 5;
bar *= "foo"; // NaN
```

Multiplication assignment using BigInts

js

```
let foo = 3n;
foo *= 2n; // 6n
foo *= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Multiplication assignment (`x *= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.multiplication_assignment`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assignment operators in the JS guide
- Multiplication (*)

JavaScript › Reference › Expressions and operators

new

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`new`** operator lets developers create an instance of a user-defined object type or of one of the built-in object types that has a constructor function.

Try it

▷ Interactive example on MDN: "JavaScript Demo: new operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}

const car1 = new Car("Eagle", "Talon TSi", 1993);

console.log(car1.make);
// Expected output: "Eagle"
```

Syntax

```js
new constructor
new constructor()
new constructor(arg1)
new constructor(arg1, arg2)
new constructor(arg1, arg2, /* …, */ argN)
```

Parameters

`constructor`

A class or function that specifies the type of the object instance. The expression can be anything with sufficient precedence, including an identifier, a property access, or another `new` expression, but optional chaining is not allowed.

`arg1, arg2, ..., argN`

A list of values that the `constructor` will be called with. `new Foo` is equivalent to `new Foo()`, i.e., if no argument list is specified, `Foo` is called without arguments.

Description

When a function is called with the **`new`** keyword, the function will be used as a constructor. `new` will do the following things:

1. Creates a blank, plain JavaScript object. For convenience, let's call it `newInstance`.
2. Points `newInstance`'s [[Prototype]] to the constructor function's `prototype` property, if the `prototype` is an `Object`. Otherwise, `newInstance` stays as a plain object with `Object.prototype` as its [[Prototype]].

   > **Note:** Properties/objects added to the constructor function's `prototype` property are therefore accessible to all instances created from the constructor function.

3. Executes the constructor function with the given arguments, binding `newInstance` as the `this` context (i.e., all references to `this` in the constructor function now refer to `newInstance`).
4. If the constructor function returns a non-primitive, this return value becomes the result of the whole `new` expression. Otherwise, if the constructor function doesn't return anything or returns a primitive, `newInstance` is returned instead. (Normally constructors don't return a value, but they can choose to do so to override the normal object creation process.)

Classes can only be instantiated with the `new` operator — attempting to call a class without `new` will throw a `TypeError`.

Creating an object with a user-defined constructor function requires two steps:

1. Define the object type by writing a function that specifies its name and properties. For example, a constructor function to create an object `Foo` might look like this:

   ```js
   function Foo(bar1, bar2) {
     this.bar1 = bar1;
     this.bar2 = bar2;
   }
   ```

2. Create an instance of the object with `new`.

```js
const myFoo = new Foo("Bar 1", 2021);
```

> **Note:** An object can have a property that is itself another object. See the examples below.

You can always add a property to a previously defined object instance. For example, the statement `car1.color = "black"` adds a property `color` to `car1`, and assigns it a value of `"black"`.

However, this does not affect any other objects. To add the new property to all objects of the same type, you must add the property to the constructor's `prototype` property. This defines a property that is shared by all objects created with that function, rather than by just one instance of the object type. The following code adds a `color` property with value `"original color"` to all objects of type `Car`, and then overwrites that value with the string `"black"` only in the instance object `car1`. For more information, see prototype.

```js
function Car() {}
const car1 = new Car();
const car2 = new Car();

console.log(car1.color); // undefined

Car.prototype.color = "original color";
console.log(car1.color); // 'original color'

car1.color = "black";
console.log(car1.color); // 'black'

console.log(Object.getPrototypeOf(car1).color); // 'original color'
console.log(Object.getPrototypeOf(car2).color); // 'original color'
console.log(car1.color); // 'black'
console.log(car2.color); // 'original color'
```

> **Note:** While the constructor function can be invoked like any regular function (i.e., without the `new` operator), in this case a new object is not created and the value of `this` is also different.

A function can know whether it is invoked with `new` by checking `new.target`. `new.target` is only `undefined` when the function is invoked without `new`. For example, you can have a function that behaves differently when it's called versus when it's constructed:

```js
function Car(color) {
  if (!new.target) {
    // Called as function.
    return `${color} car`;
  }
  // Called with new.
  this.color = color;
}

const a = Car("red"); // a is "red car"
const b = new Car("red"); // b is `Car { color: "red" }`
```

Prior to ES6, which introduced classes, most JavaScript built-ins are both callable and constructible, although many of them exhibit different behaviors. To name a few:

- `Array()`, `Error()`, and `Function()` behave the same when called as a function or a constructor.
- `Boolean()`, `Number()`, and `String()` coerce their argument to the respective primitive type when called, and return wrapper objects when constructed.
- `Date()` returns a string representing the current date when called, equivalent to `new Date().toString()`.

After ES6, the language is stricter about which are constructors and which are functions. For example:

- `Symbol()` and `BigInt()` can only be called without `new`. Attempting to construct them will throw a `TypeError`.
- `Proxy` and `Map` can only be constructed with `new`. Attempting to call them will throw a `TypeError`.

Examples

Object type and object instance

Suppose you want to create an object type for cars. You want this type of object to be called `Car`, and you want it to have properties for make, model, and year. To do this, you would write the following function:

```js
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
```

Now you can create an object called `myCar` as follows:

```js
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

This statement creates `myCar` and assigns it the specified values for its properties. Then the value of `myCar.make` is the string "Eagle", `myCar.year` is the integer 1993, and so on.

You can create any number of `car` objects by calls to `new`. For example:

```js
const kensCar = new Car("Nissan", "300ZX", 1992);
```

Object property that is itself another object

Suppose you define an object called `Person` as follows:

```js
function Person(name, age, sex) {
  this.name = name;
  this.age = age;
  this.sex = sex;
}
```

And then instantiate two new `Person` objects as follows:

```js
const rand = new Person("Rand McNally", 33, "M");
const ken = new Person("Ken Jones", 39, "M");
```

Then you can rewrite the definition of `Car` to include an `owner` property that takes a `Person` object, as follows:

```js
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```js
const car1 = new Car("Eagle", "Talon TSi", 1993, rand);
const car2 = new Car("Nissan", "300ZX", 1992, ken);
```

Instead of passing a literal string or integer value when creating the new objects, the above statements pass the objects `rand` and `ken` as the parameters for the owners. To find out the name of the owner of `car2`, you can access the following property:

```js
car2.owner.name;
```

Using `new` with classes

```js
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, my name is ${this.name}`);
  }
}

const p = new Person("Caroline");
p.greet(); // Hello, my name is Caroline
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-new-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
new	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.new`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Function`
- `Reflect.construct()`
- `Object`

JavaScript › Reference › Expressions and operators

new.target

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new.target · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2017.

The **`new.target`** meta-property lets you detect whether a function or constructor was called using the `new` operator. In constructors and functions invoked using the `new` operator, `new.target` returns a reference to the constructor or function that `new` was called upon. In normal function calls, `new.target` is `undefined`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: new.target". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function Foo() {
  if (!new.target) {
    throw new TypeError("calling Foo constructor without new is invalid");
  }
}

try {
  Foo();
} catch (e) {
  console.log(e);
  // Expected output: TypeError: calling Foo constructor without new is invalid
}
```

Syntax

js

```
new.target
```

Value

`new.target` is guaranteed to be a constructable function value or `undefined`.

- In class constructors, it refers to the class that `new` was called upon, which may be a subclass of the current constructor, because subclasses transitively call the superclass's constructor through `super()`.
- In ordinary functions, if the function is constructed directly with `new`, `new.target` refers to the function itself. If the function is called without `new`, `new.target` is `undefined`. Functions can be used as the base class for `extends`, in which case `new.target` may refer to the subclass.
- If a constructor (class or function) is called via `Reflect.construct()`, then `new.target` refers to the value passed as `newTarget` (which defaults to `target`).
- In arrow functions, `new.target` is inherited from the surrounding scope. If the arrow function is not defined within another class or function which has a `new.target` binding, then a syntax error is thrown.
- In static initialization blocks, `new.target` is `undefined`.

Description

The `new.target` syntax consists of the keyword `new`, a dot, and the identifier `target`. Because `new` is a reserved word, not an identifier, this is not a property accessor, but a special expression syntax.

The `new.target` meta-property is available in all function/class bodies; using `new.target` outside of functions or classes is a syntax error.

Examples

new.target in function calls

In normal function calls (as opposed to constructor function calls), `new.target` is `undefined`. This lets you detect whether a function was called with `new` as a constructor.

```js
function Foo() {
  if (!new.target) {
    throw new Error("Foo() must be called with new");
  }
  console.log("Foo instantiated with new");
}

new Foo(); // Logs "Foo instantiated with new"
Foo(); // Throws "Foo() must be called with new"
```

new.target in constructors

In class constructors, `new.target` refers to the constructor that was directly invoked by `new`. This is also the case if the constructor is in a parent class and was delegated from a child constructor. `new.target` points to the class that `new` was called upon. For example, when `b` was initialized using `new B()`, the name of `B` was printed; and similarly, in case of `a`, the name of class `A` was printed.

```js
class A {
  constructor() {
    console.log(new.target.name);
  }
}

class B extends A {
  constructor() {
    super();
  }
}

const a = new A(); // Logs "A"
const b = new B(); // Logs "B"
```

new.target using Reflect.construct()

Before `Reflect.construct()` or classes, it was common to implement inheritance by passing the value of `this`, and letting the base constructor mutate it.

js · ✗ bad example

```
function Base() {
  this.name = "Base";
}

function Extended() {
  // Only way to make the Base() constructor work on the existing
  // `this` value instead of a new object that `new` creates.
  Base.call(this);
  this.otherProperty = "Extended";
}

Object.setPrototypeOf(Extended.prototype, Base.prototype);
Object.setPrototypeOf(Extended, Base);

console.log(new Extended()); // Extended { name: 'Base', otherProperty: 'Extended' }
```

However, `call()` and `apply()` actually *call* the function instead of *constructing* it, so `new.target` has value `undefined`. This means that if `Base()` checks whether it's constructed with `new`, an error will be thrown, or it may behave in other unexpected ways. For example, you can't extend `Map` this way, because the `Map()` constructor cannot be called without `new`.

All built-in constructors directly construct the entire prototype chain of the new instance by reading `new.target.prototype`. So to make sure that (1) `Base` is constructed with `new`, and (2) `new.target` points to the subclass instead of `Base` itself, we need to use `Reflect.construct()`.

```js
function BetterMap(entries) {
  // Call the base class constructor, but setting `new.target` to the subclass,
  // so that the instance created has the correct prototype chain.
  return Reflect.construct(Map, [entries], BetterMap);
}
```

(code continues on the next page)

(code continued from the previous page)

```
BetterMap.prototype.upsert = function (key, actions) {
  if (this.has(key)) {
    this.set(key, actions.update(this.get(key)));
  } else {
    this.set(key, actions.insert());
  }
};

Object.setPrototypeOf(BetterMap.prototype, Map.prototype);
Object.setPrototypeOf(BetterMap, Map);

const map = new BetterMap([["a", 1]]);
map.upsert("a", {
  update: (value) => value + 1,
  insert: () => 1,
});
console.log(map.get("a")); // 2
```

> **Note:** In fact, due to the lack of `Reflect.construct()`, it is not possible to properly subclass built-ins (like `Error` subclassing) when transpiling to pre-ES6 code.

However, if you are writing ES6 code, prefer using classes and `extends` instead, as it's more readable and less error-prone.

js

```
class BetterMap extends Map {
  // The constructor is omitted because it's just the default one

  upsert(key, actions) {
    if (this.has(key)) {
      this.set(key, actions.update(this.get(key)));
    } else {
      this.set(key, actions.insert());
    }
  }
}

const map = new BetterMap([["a", 1]]);
map.upsert("a", {
  update: (value) => value + 1,
  insert: () => 1,
});
console.log(map.get("a")); // 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ordinary-and-exotic-objects-behaviours.html#sec-built-in-function-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`new.target`	46	13	41	33	11	46	41	33	11	5.0	46	11	1.0.0	1.0	5.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.new_target`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions
- Classes
- `new`
- `this`

JavaScript › Reference › Expressions and operators

null

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/null · MDN page last modified 2025-10-28

Baseline: Widely available — available across major browsers since July 2015.

The **`null`** keyword refers to the `null` primitive value, which represents the intentional absence of any object value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: null". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getVowels(str) {
  const m = str.match(/[aeiou]/gi);
  if (m === null) {
    return 0;
  }
  return m.length;
}

console.log(getVowels("sky"));
// Expected output: 0
```

Syntax

js

```
null
```

Description

The keyword `null` is a literal for the `null` value. Unlike `undefined`, which is a global variable, `null` is not an identifier but a syntax keyword.

`null` has the following behaviors:

- Like `undefined`, accessing any property on `null` throws a `TypeError` instead of returning `undefined` or searching prototype chains.
- Like `undefined`, `null` is treated as falsy for boolean operations, and nullish for nullish coalescing and optional chaining.
- The `typeof null` result is `"object"`. This is a bug in JavaScript that cannot be fixed due to backward compatibility.
- Unlike `undefined`, `JSON.stringify()` can represent `null` faithfully.

JavaScript is unique to have two nullish values: `null` and `undefined`. Semantically, their difference is very minor: `undefined` represents the absence of a value, while `null` represents the absence of an *object*. For example, the end of the prototype chain is `null` because the prototype chain is composed of objects; `document.querySelector()` returns `null` if it doesn't match, because had it matched, the result would be an object. If you are designing an API, you should likely accept `null` and `undefined` as equivalent inputs, because many codebases have stylistic rules about when to use `null` or `undefined` by default.

Examples

Difference between `null` and `undefined`

When checking for `null` or `undefined`, beware of the differences between equality (==) and identity (===) operators, as the former performs type-conversion.

```js
typeof null; // "object" (not "null" for legacy reasons)
typeof undefined; // "undefined"
null === undefined; // false
null == undefined; // true
null === null; // true
null == null; // true
!null; // true
Number.isNaN(1 + null); // false
Number.isNaN(1 + undefined); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/overview.html#sec-null-value

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`null`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.null`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `undefined`
- `NaN`
- `void`

JavaScript › Reference › Expressions and operators

Nullish coalescing assignment (??=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2020.

The **nullish coalescing assignment (`??=`)** operator, also known as the **logical nullish assignment** operator, only evaluates the right operand and assigns to the left if the left operand is nullish (`null` or `undefined`).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Nullish coalescing assignment (??=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = { duration: 50 };

a.speed ??= 25;
console.log(a.speed);
// Expected output: 25

a.duration ??= 10;
console.log(a.duration);
// Expected output: 50
```

Syntax

js

```
x ??= y
```

Description

Nullish coalescing assignment *short-circuits*, meaning that `x ??= y` is equivalent to `x ?? (x = y)`, except that the expression `x` is only evaluated once.

No assignment is performed if the left-hand side is not nullish, due to short-circuiting of the nullish coalescing operator. For example, the following does not throw an error, despite `x` being `const`:

```js
const x = 1;
x ??= 2;
```

Neither would the following trigger the setter:

```js
const x = {
  get value() {
    return 1;
  },
  set value(v) {
    console.log("Setter called");
  },
};

x.value ??= 2;
```

In fact, if `x` is not nullish, `y` is not evaluated at all.

```js
const x = 1;
x ??= console.log("y evaluated");
// Logs nothing
```

Examples

Using nullish coalescing assignment

You can use the nullish coalescing assignment operator to apply default values to object properties. Compared to using destructuring and default values, `??=` also applies the default value if the property has value `null`.

```js
function config(options) {
  options.duration ??= 100;
  options.speed ??= 25;
  return options;
}

config({ duration: 125 }); // { duration: 125, speed: 25 }
config({}); // { duration: 100, speed: 25 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Nullish coalescing assignment (`x ??= y`)	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.nullish_coalescing_assignment`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Nullish coalescing operator (`??`)
- Nullish
- Truthy
- Falsy

JavaScript › Reference › Expressions and operators

Nullish coalescing operator (??)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing · MDN page last modified 2025-08-26

Baseline: Widely available — available across major browsers since July 2020.

The **nullish coalescing (`??`)** operator is a logical operator that returns its right-hand side operand when its left-hand side operand is `null` or `undefined`, and otherwise returns its left-hand side operand.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Nullish coalescing (??) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const foo = null ?? "default string";
console.log(foo);
// Expected output: "default string"

const baz = 0 ?? 42;
console.log(baz);
// Expected output: 0
```

Syntax

js

```
leftExpr ?? rightExpr
```

Description

The nullish coalescing operator can be seen as a special case of the logical OR (||) operator. The latter returns the right-hand side operand if the left operand is *any* falsy value, not only `null` or `undefined`. In other words, if you use `||` to provide some default value to another variable `foo`, you may encounter unexpected behaviors if you consider some falsy values as usable (e.g., `''` or `0`). See below for more examples.

The nullish coalescing operator has the fifth-lowest operator precedence, directly lower than `||` and directly higher than the conditional (ternary) operator.

It is not possible to combine either the AND (`&&`) or OR operators (`||`) directly with `??`. A syntax error will be thrown in such cases.

js · ✗ bad example

```
null || undefined ?? "foo"; // raises a SyntaxError
true && undefined ?? "foo"; // raises a SyntaxError
```

Instead, provide parenthesis to explicitly indicate precedence:

js · ✓ good example

```
(null || undefined) ?? "foo"; // returns "foo"
```

Examples

Using the nullish coalescing operator

In this example, we will provide default values but keep values other than `null` or `undefined`.

js

```
const nullValue = null;
const emptyText = ""; // falsy
const someNumber = 42;

const valA = nullValue ?? "default for A";
const valB = emptyText ?? "default for B";
const valC = someNumber ?? 0;

console.log(valA); // "default for A"
console.log(valB); // "" (as the empty string is not null or undefined)
console.log(valC); // 42
```

Assigning a default value to a variable

Earlier, when one wanted to assign a default value to a variable, a common pattern was to use the logical OR operator (||):

js

```
let foo;

// foo is never assigned any value so it is still undefined
const someDummyText = foo || "Hello!";
```

However, due to `||` being a boolean logical operator, the left-hand-side operand was coerced to a boolean for the evaluation and any *falsy* value (including `0`, `''`, `NaN`, `false`, etc.) was not returned. This behavior may cause unexpected consequences if you consider `0`, `''`, or `NaN` as valid values.

```js
const count = 0;
const text = "";

const qty = count || 42;
const message = text || "hi!";
console.log(qty); // 42 and not 0
console.log(message); // "hi!" and not ""
```

The nullish coalescing operator avoids this pitfall by only returning the second operand when the first one evaluates to either `null` or `undefined` (but no other falsy values):

```js
const myText = ""; // An empty string (which is also a falsy value)

const notFalsyText = myText || "Hello world";
console.log(notFalsyText); // Hello world

const preservingFalsy = myText ?? "Hi neighborhood";
console.log(preservingFalsy); // '' (as myText is neither undefined nor null)
```

Short-circuiting

Like the 'OR' and 'AND' logical operators, the right-hand side expression is not evaluated if the left-hand side proves to be neither `null` nor `undefined`.

```js
function a() {
  console.log("a was called");
  return undefined;
}
function b() {
  console.log("b was called");
  return false;
}
function c() {
  console.log("c was called");
  return "foo";
}

console.log(a() ?? c());
// Logs "a was called" then "c was called" and then "foo"
// as a() returned undefined so both expressions are evaluated

console.log(b() ?? c());
// Logs "b was called" then "false"
// as b() returned false (and not null or undefined), the right
// hand side expression was not evaluated
```

Relationship with the optional chaining operator (?.)

The nullish coalescing operator treats `undefined` and `null` as specific values. So does the optional chaining operator (`?.`), which is useful to access a property of an object which may be `null` or `undefined`. Combining them, you can safely access a property of an object which may be nullish and provide a default value if it is.

js

```
const foo = { someFooProp: "hi" };

console.log(foo.someFooProp?.toUpperCase() ?? "not available"); // "HI"
console.log(foo.someBarProp?.toUpperCase() ?? "not available"); // "not available"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-CoalesceExpression

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Nullish coalescing operator (??)	80	80	72	67	13.1	80	79	57	13.4	13.0	80	13.4	1.0.0	1.0	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.nullish_coalescing`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Nullish coalescing assignment (`??=`)
- Optional chaining (`?.`)
- Logical OR (`||`)
- Default parameters

JavaScript › Reference › Expressions and operators

Object initializer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Object_initializer · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

An **object initializer** is a comma-delimited list of zero or more pairs of property names and associated values of an object, enclosed in curly braces ({}). Objects can also be initialized using `Object.create()` or by invoking a constructor function with the `new` operator.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Object initializer". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object1 = { a: "foo", b: 42, c: {} };

console.log(object1.a);
// Expected output: "foo"
```

(code continues on the next page)

(code continued from the previous page)

```
const a = "foo";
const b = 42;
const c = {};
const object2 = { a: a, b: b, c: c };

console.log(object2.b);
// Expected output: 42

const object3 = { a, b, c };

console.log(object3.a);
// Expected output: "foo"
```

Syntax

js

```
o = {
  a: "foo",
  b: 42,
  c: {},
  1: "number literal property",
  "foo:bar": "string literal property",

  shorthandProperty,

  method(parameters) {
    // …
  },

  get property() {},
  set property(value) {},

  [expression]: "computed property",

  __proto__: prototype,

  ...spreadProperty,
};
```

Description

An object initializer is an expression that describes the initialization of an `Object`. Objects consist of *properties*, which are used to describe an object. The values of object properties can either contain primitive data types or other objects.

Object literal syntax vs. JSON

The object literal syntax is not the same as the **J**ava**S**cript **O**bject **N**otation (JSON). Although they look similar, there are differences between them:

- JSON *only* permits property definition using the `"property": value` syntax. The property name must be double-quoted, and the definition cannot be a shorthand. Computed property names are not allowed either.
- JSON object property values can only be strings, numbers, `true`, `false`, `null`, arrays, or another JSON object. This means JSON cannot express methods or non-plain objects like `Map` or `RegExp`.
- In JSON, `"__proto__"` is a normal property key. In an object literal, it sets the object's prototype.

JSON is a *strict subset* of the object literal syntax, meaning that every valid JSON text can be parsed as an object literal, and would likely not cause syntax errors. The only exception is that the object literal syntax prohibits duplicate `__proto__` keys, which does not apply to `JSON.parse()`. The latter treats `__proto__` like a normal property and takes the last occurrence as the property's value. The only time when the object value they represent (a.k.a. their semantic) differ is also when the source contains the `__proto__` key — for object literals, it sets the object's prototype; for JSON, it's a normal property.

```js
console.log(JSON.parse('{ "__proto__": 0, "__proto__": 1 }')); // {__proto__: 1}
console.log({ "__proto__": 0, "__proto__": 1 }); // SyntaxError: Duplicate __proto__ fields
are not allowed in object literals

console.log(JSON.parse('{ "__proto__": {} }')); // { __proto__: {} }
console.log({ "__proto__": {} }); // {} (with {} as prototype)
```

Examples

Creating objects

An empty object with no properties can be created like this:

```js
const object = {};
```

However, the advantage of the *literal* or *initializer* notation is, that you are able to quickly create objects with properties inside the curly braces. You notate a list of `key: value` pairs delimited by commas.

The following code creates an object with three properties and the keys are `"foo"`, `"age"` and `"baz"`. The values of these keys are a string `"bar"`, the number `42`, and another object.

```js
const object = {
  foo: "bar",
  age: 42,
  baz: { myProp: 12 },
};
```

Accessing properties

Once you have created an object, you might want to read or change them. Object properties can be accessed by using the dot notation or the bracket notation. (See property accessors for detailed information.)

```js
object.foo; // "bar"
object["age"]; // 42
object.baz; // {myProp: 12}
object.baz.myProp; // 12
```

Property definitions

We have already learned how to notate properties using the initializer syntax. Oftentimes, there are variables in your code that you would like to put into an object. You will see code like this:

```js
const a = "foo";
const b = 42;
const c = {};

const o = {
  a: a,
  b: b,
  c: c,
};
```

There is a shorter notation available to achieve the same:

```js
const a = "foo";
const b = 42;
const c = {};

// Shorthand property names
const o = { a, b, c };

// In other words,
console.log(o.a === { a }.a); // true
```

Duplicate property names

When using the same name for your properties, the second property will overwrite the first.

```js
const a = { x: 1, x: 2 };
console.log(a); // {x: 2}
```

After ES2015, duplicate property names are allowed everywhere, including strict mode. You can also have duplicate property names in classes. The only exception is private elements, which must be unique in the class body.

Method definitions

A property of an object can also refer to a function or a getter or setter method.

```js
const o = {
  property: function (parameters) {},
  get property() {
    return 1;
  },
  set property(value) {},
};
```

A shorthand notation is available, so that the keyword `function` is no longer necessary.

```js
// Shorthand method names
const o = {
  property(parameters) {},
};
```

There is also a way to concisely define generator methods.

```js
const o = {
  *generator() {
    // …
  },
};
```

Which is equivalent to this ES5-like notation (but note that ECMAScript 5 has no generators):

```js
const o = {
  generator: function* () {
    // …
  },
};
```

For more information and examples about methods, see method definitions.

Computed property names

The object initializer syntax also supports computed property names. That allows you to put an expression in square brackets `[]`, that will be computed and used as the property name. This is reminiscent of the bracket notation of the property accessor syntax, which you may have used to read and set properties already.

Now you can use a similar syntax in object literals, too:

```js
// Computed property names
let i = 0;
const a = {
  [`foo${++i}`]: i,
  [`foo${++i}`]: i,
  [`foo${++i}`]: i,
};

console.log(a.foo1); // 1
console.log(a.foo2); // 2
console.log(a.foo3); // 3

const items = ["A", "B", "C"];
const obj = {
  [items]: "Hello",
};
console.log(obj); // A,B,C: "Hello"
console.log(obj["A,B,C"]); // "Hello"

const param = "size";
const config = {
  [param]: 12,
  [`mobile${param.charAt(0).toUpperCase()}${param.slice(1)}`]: 4,
};

console.log(config); // {size: 12, mobileSize: 4}
```

Spread properties

Object literals support the spread syntax. It copies own enumerable properties from a provided object onto a new object.

Shallow-cloning (excluding `prototype`) or merging objects is now possible using a shorter syntax than `Object.assign()`.

```js
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };

const clonedObj = { ...obj1 };
// { foo: "bar", x: 42 }

const mergedObj = { ...obj1, ...obj2 };
// { foo: "baz", x: 42, y: 13 }
```

> **Warning:** Note that `Object.assign()` triggers setters, whereas the spread syntax doesn't!

Prototype setter

A property definition of the form `__proto__: value` or `"__proto__": value` does not create a property with the name `__proto__`. Instead, if the provided value is an object or `null`, it points the `[[Prototype]]` of the created object to that value. (If the value is not an object or `null`, the object is not changed.)

Note that the `__proto__` key is standardized syntax, in contrast to the non-standard and non-performant `Object.prototype.__proto__` accessors. It sets the `[[Prototype]]` during object creation, similar to `Object.create` — instead of mutating the prototype chain.

```js
const obj1 = {};
console.log(Object.getPrototypeOf(obj1) === Object.prototype); // true

const obj2 = { __proto__: null };
console.log(Object.getPrototypeOf(obj2)); // null

const protoObj = {};
const obj3 = { "__proto__": protoObj };
console.log(Object.getPrototypeOf(obj3) === protoObj); // true

const obj4 = { __proto__: "not an object or null" };
console.log(Object.getPrototypeOf(obj4) === Object.prototype); // true
console.log(Object.hasOwn(obj4, "__proto__")); // false
```

Only a single prototype setter is permitted in an object literal. Multiple prototype setters are a syntax error.

Property definitions that do not use "colon" notation are not prototype setters. They are property definitions that behave identically to similar definitions using any other name.

```js
const __proto__ = "variable";

const obj1 = { __proto__ };
console.log(Object.getPrototypeOf(obj1) === Object.prototype); // true
console.log(Object.hasOwn(obj1, "__proto__")); // true
console.log(obj1.__proto__); // "variable"

const obj2 = { __proto__() { return "hello"; } };
console.log(obj2.__proto__()); // "hello"

const obj3 = { ["__proto__"]: 17 };
console.log(obj3.__proto__); // 17

// Mixing prototype setter with normal own properties with "__proto__" key
const obj4 = { ["__proto__"]: 17, __proto__: {} }; // {__proto__: 17} (with {} as prototype)
const obj5 = {
  ["__proto__"]: 17,
  __proto__: {},
  __proto__: null, // SyntaxError: Duplicate __proto__ fields are not allowed in object
literals
};
const obj6 = {
  ["__proto__"]: 17,
  ["__proto__"]: "hello",
  __proto__: null,
}; // {__proto__: "hello"} (with null as prototype)
const obj7 =  {
  ["__proto__"]: 17,
  __proto__,
  __proto__: null,
}; // {__proto__: "variable"} (with null as prototype)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-object-initializer

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Object initializer	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Computed property names	47	12	34	34	8	47	34	34	8	5.0	47	8	1.0.0	1.0	4.0.0
Shorthand method names	47	12	34	34	9	47	34	34	9	5.0	47	9	1.0.0	1.0	4.0.0
Shorthand property names	47	12	33	34	9	47	33	34	9	5.0	47	9	1.0.0	1.0	4.0.0
Spread properties	60	79	55	47	11.1	60	55	44	11.3	8.0	60	11.3	1.0.0	1.0	8.3.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.object_initializer`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Property accessors
- get
- set
- Method definitions
- Lexical grammar

JavaScript › Reference › Expressions and operators

Operator precedence

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence · MDN page last modified 2026-09-04

Operator precedence determines how operators are parsed concerning each other. Operators with higher precedence become the operands of operators with lower precedence.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Expressions - Operator precedence". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(3 + 4 * 5); // 3 + 20
// Expected output: 23

console.log(4 * 3 ** 2); // 4 * 9
// Expected output: 36

let a;
let b;

console.log((a = b = 5));
// Expected output: 5
```

Precedence and associativity

Consider an expression describable by the representation below, where both `OP1` and `OP2` are fill-in-the-blanks for OPerators.

```
a OP1 b OP2 c
```

The combination above has two possible interpretations:

```
(a OP1 b) OP2 c
a OP1 (b OP2 c)
```

Which one the language decides to adopt depends on the identity of `OP1` and `OP2`.

If `OP1` and `OP2` have different precedence levels (see the table below), the operator with the higher *precedence* goes first and associativity does not matter. Observe how multiplication has higher precedence than addition and executed first, even though addition is written first in the code.

```js
console.log(3 + 10 * 2); // 23
console.log(3 + (10 * 2)); // 23, because parentheses here are superfluous
console.log((3 + 10) * 2); // 26, because the parentheses change the order
```

Within operators of the same precedence, the language groups them by *associativity*. *Left-associativity* (left-to-right) means that it is interpreted as `(a OP1 b) OP2 c`, while *right-associativity* (right-to-left) means it is interpreted as `a OP1 (b OP2 c)`. Assignment operators are right-associative, so you can write:

```js
a = b = 5; // same as writing a = (b = 5);
```

with the expected result that `a` and `b` get the value 5. This is because the assignment operator returns the value that is assigned. First, `b` is set to 5. Then the `a` is also set to 5 — the return value of `b = 5`, a.k.a. right operand of the assignment.

As another example, the unique exponentiation operator has right-associativity, whereas other arithmetic operators have left-associativity.

```js
const a = 4 ** 3 ** 2; // Same as 4 ** (3 ** 2); evaluates to 262144
const b = 4 / 3 / 2; // Same as (4 / 3) / 2; evaluates to 0.6666...
```

Operators are first grouped by precedence, and then, for adjacent operators that have the same precedence, by associativity. So, when mixing division and exponentiation, the exponentiation always comes before the division. For example, `2 ** 3 / 3 ** 2` results in 0.8888888888888888 because it is the same as `(2 ** 3) / (3 ** 2)`.

For prefix unary operators, suppose we have the following pattern:

```
OP1 a OP2 b
```

where `OP1` is a prefix unary operator and `OP2` is a binary operator. If `OP1` has higher precedence than `OP2`, then it would be grouped as `(OP1 a) OP2 b`; otherwise, it would be `OP1 (a OP2 b)`.

```js
const a = 1;
const b = 2;
typeof a + b; // Equivalent to (typeof a) + b; result is "number2"
```

If the unary operator is on the second operand:

```
a OP2 OP1 b
```

Then the binary operator `OP2` must have lower precedence than the unary operator `OP1` for it to be grouped as `a OP2 (OP1 b)`. For example, the following is invalid:

js · ✗ bad example

```js
function* foo() {
  a + yield 1;
}
```

Because + has higher precedence than `yield`, this would become `(a + yield) 1` — but because `yield` is a reserved word in generator functions, this would be a syntax error. Luckily, most unary operators

have higher precedence than binary operators and do not suffer from this pitfall.

If we have two prefix unary operators:

```
OP1 OP2 a
```

Then the unary operator closer to the operand, `OP2`, must have higher precedence than `OP1` for it to be grouped as `OP1 (OP2 a)`. It's possible to get it the other way and end up with `(OP1 OP2) a`:

js · ✗ bad example

```
async function* foo() {
  await yield 1;
}
```

Because `await` has higher precedence than `yield`, this would become `(await yield) 1`, which is awaiting an identifier called `yield`, and a syntax error. Similarly, if you have `new !A;`, because `!` has lower precedence than `new`, this would become `(new !) A`, which is obviously invalid. (This code looks nonsensical to write anyway, since `!A` always produces a boolean, not a constructor function.)

For postfix unary operators (namely, ++ and --), the same rules apply. Luckily, both operators have higher precedence than any binary operator, so the grouping is always what you would expect. Moreover, because ++ evaluates to a *value*, not a *reference*, you can't chain multiple increments together either.

js · ✗ bad example

```
let a = 1;
a++++; // SyntaxError: Invalid left-hand side in postfix operation.
```

Operator precedence will be handled *recursively*. For example, consider this expression:

js

```
1 + 2 ** 3 * 4 / 5 >> 6
```

First, we group operators with different precedence by decreasing levels of precedence.

1. The ** operator has the highest precedence, so it's grouped first.
2. Looking around the ** expression, it has * on the right and + on the left. * has higher precedence, so it's grouped first. * and / have the same precedence, so we group them together for now.
3. Looking around the */ expression grouped in 2, because + has higher precedence than >>, the former is grouped.

js

```
   (1 + ( (2 ** 3) * 4 / 5) ) >> 6
// |    | └─ 1. ─┘        | |
// |    └──────── 2. ──────┘ |
// └──────────── 3. ──────────┘
```

Within the */ group, because they are both left-associative, the left operand would be grouped.

js

```
   (1 + ( ( (2 ** 3) * 4 ) / 5) ) >> 6
// |    | | └─ 1. ─┘     |    | |
// |    └─|──────── 2. ──|────┘ |
// └──────|───── 3. ─────|──────┘
//        └───── 4. ─────┘
```

Note that operator precedence and associativity only affect the order of evaluation of *operators* (the implicit grouping), but not the order of evaluation of *operands*. The operands are always evaluated from left-to-right. The higher-precedence expressions are always evaluated first, and their results are then composed according to the order of operator precedence.

```js
function echo(name, num) {
  console.log(`Evaluating the ${name} side`);
  return num;
}
// Exponentiation operator (**) is right-associative,
// but all call expressions (echo()), which have higher precedence,
// will be evaluated before ** does
console.log(echo("left", 4) ** echo("middle", 3) ** echo("right", 2));
// Evaluating the left side
// Evaluating the middle side
// Evaluating the right side
// 262144

// Exponentiation operator (**) has higher precedence than division (/),
// but evaluation always starts with the left operand
console.log(echo("left", 4) / echo("middle", 3) ** echo("right", 2));
// Evaluating the left side
// Evaluating the middle side
// Evaluating the right side
// 0.4444444444444444
```

If you are familiar with binary trees, think about it as a post-order traversal.



After all operators have been properly grouped, the binary operators would form a binary tree. Evaluation starts from the outermost group — which is the operator with the lowest precedence (`/` in this case). The left operand of this operator is first evaluated, which may be composed of higher-precedence operators (such as a call expression `echo("left", 4)`). After the left operand has been evaluated, the right operand is evaluated in the same fashion. Therefore, all leaf nodes — the `echo()` calls — would be visited left-to-right, regardless of the precedence of operators joining them.

Short-circuiting

In the previous section, we said "the higher-precedence expressions are always evaluated first" — this is generally true, but it has to be amended with the acknowledgement of *short-circuiting*, in which case an operand may not be evaluated at all.

Short-circuiting is jargon for conditional evaluation. For example, in the expression `a && (b + c)`, if `a` is falsy, then the sub-expression `(b + c)` will not even get evaluated, even if it is grouped and therefore has higher precedence than `&&`. We could say that the logical AND operator (`&&`) is "short-circuited". Along with logical AND, other short-circuited operators include logical OR (`||`), nullish coalescing (`??`), and optional chaining (`?.`).

```js
a || (b * c); // evaluate `a` first, then produce `a` if `a` is "truthy"
a && (b < c); // evaluate `a` first, then produce `a` if `a` is "falsy"
a ?? (b || c); // evaluate `a` first, then produce `a` if `a` is not `null` and not
`undefined`
a?.b.c; // evaluate `a` first, then produce `undefined` if `a` is `null` or `undefined`
```

When evaluating a short-circuited operator, the left operand is always evaluated. The right operand will only be evaluated if the left operand cannot determine the result of the operation.

> **Note:** The behavior of short-circuiting is baked in these operators. Other operators would *always* evaluate both operands, regardless if that's actually useful — for example, `NaN * foo()` will always call `foo`, even when the result would never be something other than `NaN`.

The previous model of a post-order traversal still stands. However, after the left subtree of a short-circuiting operator has been visited, the language will decide if the right operand needs to be evaluated. If not (for example, because the left operand of `||` is already truthy), the result is directly returned without visiting the right subtree.

Consider this case:

```js
function A() { console.log('called A'); return false; }
function B() { console.log('called B'); return false; }
function C() { console.log('called C'); return true; }

console.log(C() || B() && A());

// Logs:
// called C
// true
```

Only `C()` is evaluated, despite `&&` having higher precedence. This does not mean that `||` has higher precedence in this case — it's exactly *because* `(B() && A())` has higher precedence that causes it to be neglected as a whole. If it's re-arranged as:

```js
console.log(A() && B() || C());
// Logs:
// called A
// called C
// true
```

Then the short-circuiting effect of `&&` would only prevent `B()` from being evaluated, but because `A() && B()` as a whole is `false`, `C()` would still be evaluated.

However, note that short-circuiting does not change the final evaluation outcome. It only affects the evaluation of *operands*, not how *operators* are grouped — if evaluation of operands doesn't have side effects (for example, logging to the console, assigning to variables, throwing an error), short-circuiting would not be observable at all.

The assignment counterparts of these operators (`&&=`, `||=`, `??=`) are short-circuited as well. They are short-circuited in a way that the assignment does not happen at all.

Table

The following table lists operators in order from highest precedence (18) to lowest precedence (1).

Several general notes about the table:

1. Not all syntax included here are "operators" in the strict sense. For example, spread `...` and arrow `=>` are typically not regarded as operators. However, we still included them to show how tightly they bind compared to other operators/expressions.
2. Some operators have certain operands that require expressions narrower than those produced by higher-precedence operators. For example, the right-hand side of member access `.` (precedence 17) must be an identifier instead of a grouped expression. The left-hand side of arrow `=>` (precedence 2) must be an arguments list or a single identifier instead of some random expression.
3. Some operators have certain operands that accept expressions wider than those produced by higher-precedence operators. For example, the bracket-enclosed expression of bracket notation `[ ... ]` (precedence 17) can be any expression, even comma (precedence 1) joined ones. These operators act as if that operand is "automatically grouped". In this case we will omit the associativity.

Precedence	Associativity	Individual operators	Notes
18: grouping	n/a	Grouping `(x)`	[1]
17: access and call	left-to-right	Member access `x.y`	[2]
		Optional chaining `x?.y`	
	n/a	Computed member access `x[y]`	[3]
		`new` with argument list `new x(y)`	[4]
		Function call `x(y)`	
		`import(x)`	
16: new	n/a	`new` without argument list `new x`	
15: postfix operators	n/a	Postfix increment `x++`	[5]
		Postfix decrement `x--`	
14: prefix operators	n/a	Prefix increment `++x`	[6]
		Prefix decrement `--x`	
		Logical NOT `!x`	
		Bitwise NOT `~x`	
		Unary plus `+x`	

Precedence	Associativity	Individual operators	Notes
		Unary negation `-x`	
		`typeof x`	
		`void x`	
		`delete x`	[7]
		`await x`	
13: exponentiation	right-to-left	Exponentiation `x ** y`	[8]
12: multiplicative operators	left-to-right	Multiplication `x * y`	
		Division `x / y`	
		Remainder `x % y`	
11: additive operators	left-to-right	Addition `x + y`	
		Subtraction `x - y`	
10: bitwise shift	left-to-right	Left shift `x << y`	
		Right shift `x >> y`	
		Unsigned right shift `x >>> y`	
9: relational operators	left-to-right	Less than `x < y`	
		Less than or equal `x <= y`	
		Greater than `x > y`	
		Greater than or equal `x >= y`	
		`x in y`	
		`x instanceof y`	
8: equality operators	left-to-right	Equality `x == y`	
		Inequality `x != y`	
		Strict equality `x === y`	
		Strict inequality `x !== y`	
7: bitwise AND	left-to-right	Bitwise AND `x & y`	

Precedence	Associativity	Individual operators	Notes
6: bitwise XOR	left-to-right	Bitwise XOR `x ^ y`	
5: bitwise OR	left-to-right	Bitwise OR `x | y`	
4: logical AND	left-to-right	Logical AND `x && y`	
3: logical OR, nullish coalescing	left-to-right	Logical OR `x || y`	
		Nullish coalescing operator `x ?? y`	[9]
2: assignment and miscellaneous	right-to-left	Assignment `x = y`	[10]
		Addition assignment `x += y`	
		Subtraction assignment `x -= y`	
		Exponentiation assignment `x **= y`	
		Multiplication assignment `x *= y`	
		Division assignment `x /= y`	
		Remainder assignment `x %= y`	
		Left shift assignment `x <<= y`	
		Right shift assignment `x >>= y`	
		Unsigned right shift assignment `x >>>= y`	
		Bitwise AND assignment `x &= y`	
		Bitwise XOR assignment `x ^= y`	
		Bitwise OR assignment `x |= y`	
		Logical AND assignment `x &&= y`	
		Logical OR assignment `x ||= y`	
		Nullish coalescing assignment `x ??= y`	
	right-to-left	Conditional (ternary) operator `x ? y : z`	[11]
	right-to-left	Arrow `x => y`	[12]

Precedence	Associativity	Individual operators	Notes
	n/a	`yield x`	
		`yield* x`	
		Spread `...x`	[13]
1: comma	left-to-right	Comma operator `x, y`	

Notes:

1. The operand can be any expression.
2. The "right-hand side" must be an identifier.
3. The "right-hand side" can be any expression.
4. The "right-hand side" is a comma-separated list of any expression with precedence > 1 (i.e., not comma expressions). The constructor of a `new` expression cannot be an optional chain.
5. The operand must be a valid assignment target (identifier or property access). Its precedence means `new Foo++` is `(new Foo)++` (a syntax error) and not `new (Foo++)` (a TypeError: (Foo++) is not a constructor).
6. The operand must be a valid assignment target (identifier or property access).
7. The operand cannot be an identifier or a private element access.
8. The left-hand side cannot have precedence 14.
9. The operands cannot be a logical OR `||` or logical AND `&&` operator without grouping.
10. The "left-hand side" must be a valid assignment target (identifier or property access).
11. The associativity means the two expressions after `?` are implicitly grouped.
12. The "left-hand side" is a single identifier or a parenthesized parameter list.
13. Only valid inside object literals, array literals, or argument lists.

The precedence of groups 17 and 16 may be a bit ambiguous. Here are a few examples to clarify.

- Optional chaining is always substitutable for its respective syntax without optionality (barring a few special cases where optional chaining is forbidden). For example, any place that accepts `a?.b` also accepts `a.b` and vice versa, and similarly for `a?.()`, `a()`, etc.
- Member expressions and computed member expressions are always substitutable for each other.
- Call expressions and `import()` expressions are always substitutable for each other.
- This leaves four classes of expressions: member access, `new` with arguments, function call, and `new` without arguments.
  - The "left-hand side" of a member access can be: a member access (`a.b.c`), `new` with arguments (`new a().b`), and function call (`a().b`).
  - The "left-hand side" of `new` with arguments can be: a member access (`new a.b()`) and `new` with arguments (`new new a()()`).
  - The "left-hand side" of a function call can be: a member access (`a.b()`), `new` with arguments (`new a()()`), and function call (`a()()`).
  - The operand of `new` without arguments can be: a member access (`new a.b`), `new` with arguments (`new new a()`), and `new` without arguments (`new new a`).

JavaScript › Reference › Expressions and operators

Optional chaining (?.)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since July 2020.

The **optional chaining (`?.`)** operator accesses an object's property or calls a function. If the object accessed or function called using this operator is `undefined` or `null`, the expression short circuits and evaluates to `undefined` instead of throwing an error.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Optional chaining (?.) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const adventurer = {
  name: "Alice",
  cat: {
    name: "Dinah",
  },
};

const dogName = adventurer.dog?.name;
console.log(dogName);
// Expected output: undefined

console.log(adventurer.someNonExistentMethod?.());
// Expected output: undefined
```

Syntax

js

```
obj?.prop
obj?.[expr]
func?.(args)
```

Description

The `?.` operator is like the `.` chaining operator, except that instead of causing an error if a reference is nullish (`null` or `undefined`), the expression short-circuits with a return value of `undefined`. When used with function calls, it returns `undefined` if the given function does not exist.

This results in shorter and simpler expressions when accessing chained properties when the possibility exists that a reference may be missing. It can also be helpful while exploring the content of an object when there's no known guarantee as to which properties are required.

For example, consider an object `obj` which has a nested structure. Without optional chaining, looking up a deeply-nested subproperty requires validating the references in between, such as:

js

```
const nestedProp = obj.first && obj.first.second;
```

The value of `obj.first` is confirmed to be non-`null` (and non-`undefined`) before accessing the value of `obj.first.second`. This prevents the error that would occur if you accessed `obj.first.second` directly without testing `obj.first`.

This is an idiomatic pattern in JavaScript, but it gets verbose when the chain is long, and it's not safe. For example, if `obj.first` is a Falsy value that's not `null` or `undefined`, such as `0`, it would still short-circuit and make `nestedProp` become `0`, which may not be desirable.

With the optional chaining operator (`?.`), however, you don't have to explicitly test and short-circuit based on the state of `obj.first` before trying to access `obj.first.second`:

```js
const nestedProp = obj.first?.second;
```

By using the `?.` operator instead of just `.`, JavaScript knows to implicitly check to be sure `obj.first` is not `null` or `undefined` before attempting to access `obj.first.second`. If `obj.first` is `null` or `undefined`, the expression automatically short-circuits, returning `undefined`.

This is equivalent to the following, except that the temporary variable is in fact not created:

```js
const temp = obj.first;
const nestedProp =
  temp === null || temp === undefined ? undefined : temp.second;
```

Optional chaining cannot be used on a non-declared root object, but can be used with a root object with value `undefined`.

js · ✗ bad example

```
undeclaredVar?.prop; // ReferenceError: undeclaredVar is not defined
```

Optional chaining with function calls

You can use optional chaining when attempting to call a method which may not exist. This can be helpful, for example, when using an API in which a method might be unavailable, either due to the age of the implementation or because of a feature which isn't available on the user's device.

Using optional chaining with function calls causes the expression to automatically return `undefined` instead of throwing an exception if the method isn't found:

```js
const result = someInterface.customMethod?.();
```

However, if there is a property with such a name which is not a function, using `?.` will still raise a `TypeError` exception "someInterface.customMethod is not a function".

> **Note:** If `someInterface` itself is `null` or `undefined`, a `TypeError` exception will still be raised ("someInterface is null"). If you expect that `someInterface` itself may be `null` or `undefined`, you have to use `?.` at this position as well: `someInterface?.customMethod?.()`.

`eval?.()` is the shortest way to enter *indirect eval* mode.

Optional chaining with expressions

You can also use the optional chaining operator with bracket notation, which allows passing an expression as the property name:

```js
const propName = "x";
const nestedProp = obj?.[propName];
```

This is particularly useful for arrays, since array indices must be accessed with square brackets.

```js
function printMagicIndex(arr) {
  console.log(arr?.[42]);
}

printMagicIndex([0, 1, 2, 3, 4, 5]); // undefined
printMagicIndex(); // undefined; if not using ?., this would throw an error: "Cannot read
properties of undefined (reading '42')"
```

Invalid optional chaining

It is invalid to try to assign to the result of an optional chaining expression:

js · ✗ bad example

```
const object = {};
object?.property = 1; // SyntaxError: Invalid left-hand side in assignment
```

Template literal tags cannot be an optional chain (see SyntaxError: tagged template cannot be used with optional chain):

js · ✗ bad example

```
String?.raw`Hello, world!`;
String.raw?.`Hello, world!`; // SyntaxError: Invalid tagged template on optional chain
```

The constructor of `new` expressions cannot be an optional chain (see SyntaxError: new keyword cannot be used with an optional chain):

js · ✗ bad example

```
new Intl?.DateTimeFormat(); // SyntaxError: Invalid optional chain from new expression
new Map?.();
```

Short-circuiting

When using optional chaining with expressions, if the left operand is `null` or `undefined`, the expression will not be evaluated. For instance:

```js
const potentiallyNullObj = null;
let x = 0;
const prop = potentiallyNullObj?.[x++];

console.log(x); // 0 as x was not incremented
```

Subsequent property accesses will not be evaluated either.

```js
const potentiallyNullObj = null;
const prop = potentiallyNullObj?.a.b;
// This does not throw, because evaluation has already stopped at
// the first optional chain
```

This is equivalent to:

```js
const potentiallyNullObj = null;
const prop =
  potentiallyNullObj === null || potentiallyNullObj === undefined
    ? undefined
    : potentiallyNullObj.a.b;
```

However, this short-circuiting behavior only happens along one continuous "chain" of property accesses. If you group one part of the chain, then subsequent property accesses will still be evaluated.

```js
const potentiallyNullObj = null;
const prop = (potentiallyNullObj?.a).b;
// TypeError: Cannot read properties of undefined (reading 'b')
```

This is equivalent to:

```js
const potentiallyNullObj = null;
const temp = potentiallyNullObj?.a;
const prop = temp.b;
```

Except the `temp` variable isn't created.

Examples

Basic example

This example looks for the value of the `name` property for the member `CSS` in a map when there is no such member. The result is therefore `undefined`.

```js
const myMap = new Map();
myMap.set("JS", { name: "Josh", desc: "I maintain things" });

const nameBar = myMap.get("CSS")?.name;
```

Dealing with optional callbacks or event handlers

If you use callbacks or fetch methods from an object with a destructuring pattern, you may have non-existent values that you cannot call as functions unless you have tested their existence. Using `?.`, you can avoid this extra test:

```js
// Code written without optional chaining
function doSomething(onContent, onError) {
  try {
    // Do something with the data
  } catch (err) {
    // Testing if onError really exists
    if (onError) {
      onError(err.message);
    }
  }
}
```

```js
// Using optional chaining with function calls
function doSomething(onContent, onError) {
  try {
    // Do something with the data
  } catch (err) {
    onError?.(err.message); // No exception if onError is undefined
  }
}
```

Stacking the optional chaining operator

With nested structures, it is possible to use optional chaining multiple times:

```js
const customer = {
  name: "Carl",
  details: {
    age: 82,
    location: "Paradise Falls", // Detailed address is unknown
  },
};
const customerCity = customer.details?.address?.city;

// This also works with optional chaining function call
const customerName = customer.name?.getName?.(); // Method does not exist, customerName is
undefined
```

Combining with the nullish coalescing operator

The nullish coalescing operator may be used after optional chaining in order to build a default value when none was found:

```js
function printCustomerCity(customer) {
  const customerCity = customer?.city ?? "Unknown city";
  console.log(customerCity);
}

printCustomerCity({
  name: "Nathan",
  city: "Paris",
}); // "Paris"
printCustomerCity({
  name: "Carl",
  details: { age: 82 },
}); // "Unknown city"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-OptionalExpression

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Optional chaining operator (`?.`)	80[1]	80[2]	74	67[3]	13.1	80[4]	79	57[5]	13.4	13.0[6]	80[7]	13.4	1.0.0	1.0	14.0.0

1. A call with a non-final spread argument in an optional chain fails in Chrome 80 through 90. See bug 42201552.
2. A call with a non-final spread argument in an optional chain fails in Edge 80 through 90. See bug 42201552.
3. A call with a non-final spread argument in an optional chain fails in Opera 67 through 90. See bug 42201552.
4. A call with a non-final spread argument in an optional chain fails in Chrome Android 80 through 90. See bug 42201552.
5. A call with a non-final spread argument in an optional chain fails in Opera Android 57 through 90. See bug 42201552.
6. A call with a non-final spread argument in an optional chain fails in Samsung Browser 13.0 through 90. See bug 42201552.
7. A call with a non-final spread argument in an optional chain fails in WebView Android 80 through 90. See bug 42201552.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.optional_chaining`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Nullish coalescing operator (`??`)

JavaScript › Reference › Expressions and operators

Property accessors

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Property_accessors · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

Property accessors provide access to an object's properties by using the dot notation or the bracket notation.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Property accessors". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const person1 = {};
person1["firstName"] = "Mario";
person1["lastName"] = "Rossi";

console.log(person1.firstName);
// Expected output: "Mario"

const person2 = {
  firstName: "John",
  lastName: "Doe",
};

console.log(person2["lastName"]);
// Expected output: "Doe"
```

Syntax

```js
object.propertyName
object[expression]
object.#privateProperty
```

Description

One can think of an object as an *associative array* (a.k.a. *map*, *dictionary*, *hash*, *lookup table*). The *keys* in this array are the names of the object's properties.

There are two ways to access properties: *dot notation* and *bracket notation*.

Dot notation

In the `object.propertyName` syntax, the `propertyName` must be a valid JavaScript identifier which can also be a reserved word. For example, `object.$1` is valid, while `object.1` is not.

```js
const variable = object.propertyName;
object.propertyName = value;
```

```js
const object = {};
object.$1 = "foo";
console.log(object.$1); // 'foo'
```

js · ✗ bad example

```
const object = {};
object.1 = "bar"; // SyntaxError
console.log(object.1); // SyntaxError
```

Here, the method named `createElement` is retrieved from `document` and is called.

```js
document.createElement("pre");
```

If you use a method for a numeric literal, and the numeric literal has no exponent and no decimal point, you should leave white-space(s) before the dot preceding the method call, so that the dot is not interpreted as a decimal point.

```js
77 .toExponential();
// or
77
.toExponential();
// or
(77).toExponential();
// or
77..toExponential();
// or
77.0.toExponential();
// because 77. === 77.0, no ambiguity
```

In addition, private elements can only be accessed using dot notation within the class that defines them.

Bracket notation

In the `object[expression]` syntax, the `expression` should evaluate to a string or Symbol that represents the property's name. So, it can be any string literal, for example, including `'1foo'`, `'!bar!'`, or even `' '` (a space).

```js
const variable = object[propertyName];
object[propertyName] = value;
```

This does the exact same thing as the previous example.

```js
document["createElement"]("pre");
```

A space before bracket notation is allowed.

```js
document ["createElement"]("pre");
```

Passing expressions that evaluate to property name will do the same thing as directly passing the property name.

```js
const key = "name";
const getKey = () => "name";
const Obj = { name: "Michel" };

Obj["name"]; // returns "Michel"
Obj[key]; // evaluates to Obj["name"], and returns "Michel"
Obj[getKey()]; // evaluates to Obj["name"], and returns "Michel"
```

However, beware of using square brackets to access properties whose names are given by external input. This may make your code susceptible to object injection attacks.

Property names

Each property name is a string or a Symbol. Any other value, including a number, is coerced to a string. This outputs `'value'`, since `1` is coerced into `'1'`.

```js
const object = {};
object["1"] = "value";
console.log(object[1]);
```

This also outputs `'value'`, since both `foo` and `bar` are converted to the same string (`"[object Object]"`).

```js
const foo = { uniqueProp: 1 };
const bar = { uniqueProp: 2 };
const object = {};
object[foo] = "value";
console.log(object[bar]);
```

Method binding

It's typical when speaking of an object's properties to make a distinction between properties and methods. However, the property/method distinction is little more than a convention. A method is a property that can be called (for example, if it has a reference to a `Function` instance as its value).

A method is not bound to the object that it is a property of. Specifically, `this` is not fixed in a method and does not necessarily refer to the object containing the method. Instead, `this` is "passed" by the function call. See the reference for `this`.

Examples

Bracket notation vs. eval()

JavaScript novices often make the mistake of using `eval()` where the bracket notation can be used instead.

For example, the following syntax is often seen in many scripts.

```js
const x = eval(`document.forms.form_name.elements.${strFormControl}.value`);
```

`eval()` is slow and should be avoided whenever possible. Also, `strFormControl` would have to hold an identifier, which is not required for names and `id`s of form controls. It is better to use bracket notation instead:

```js
const x = document.forms.form_name.elements[strFormControl].value;
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-property-accessors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Property accessors	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.property_accessors`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Object`
- `Object.defineProperty()`
- Optional chaining (`?.`)

JavaScript › Reference › Expressions and operators

Remainder (%)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Remainder · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **remainder (%)** operator returns the remainder left over when one operand is divided by a second operand. It always takes the sign of the dividend.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Remainder (%) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(13 % 5);
// Expected output: 3

console.log(-13 % 5);
// Expected output: -3

console.log(4 % 2);
// Expected output: 0

console.log(-4 % 2);
// Expected output: -0
```

Syntax

js

```
x % y
```

Description

The `%` operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt remainder if both operands become BigInts; otherwise, it performs number remainder. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

For the operation `n % d`, `n` is called the dividend and `d` is called the divisor. The operation returns `NaN` if one of the operands is `NaN`, `n` is ±Infinity, or if `d` is ±0. Otherwise, if `d` is ±Infinity or if `n` is ±0, the dividend `n` is returned.

When both operands are non-zero and finite, the remainder `r` is calculated as `r := n - d * q` where `q` is the integer such that `r` has the same sign as the dividend `n` while being as close to 0 as possible.

Note that while in most languages, '%' is a remainder operator, in some (e.g., Python, Perl) it is a modulo operator. Modulo is defined as `k := n - d * q` where `q` is the integer such that `k` has the same sign as the divisor `d` while being as close to 0 as possible. For two values of the same sign, the two are equivalent, but when the operands are of different signs, the modulo result always has the same sign as the *divisor*, while the remainder has the same sign as the *dividend*, which can make them differ by one unit of `d`. To obtain a modulo in JavaScript, in place of `n % d`, use `((n % d) + d) % d`. In JavaScript, the

modulo operation (which doesn't have a dedicated operator) is used to normalize the second operand of bitwise shift operators (<<, >>, etc.), making the offset always a positive value.

For BigInt division, a `RangeError` is thrown if the divisor `y` is `0n`. This is because number remainder by zero returns `NaN`, but BigInt has no concept of `NaN`.

Examples

Remainder with positive dividend

```js
13 % 5; // 3
1 % -2; // 1
1 % 2; // 1
2 % 3; // 2
5.5 % 2; // 1.5

3n % 2n; // 1n
```

Remainder with negative dividend

```js
-13 % 5; // -3
-1 % 2; // -1
-4 % 2; // -0

-3n % 2n; // -1n
```

Remainder with NaN

```js
NaN % 2; // NaN
```

Remainder with Infinity

```js
Infinity % 2; // NaN
Infinity % 0; // NaN
Infinity % Infinity; // NaN
2 % Infinity; // 2
0 % Infinity; // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-multiplicative-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Remainder (%)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.remainder`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)
- Remainder operator vs. modulo operator by Dr. Axel Rauschmayer (2019)

JavaScript › Reference › Expressions and operators

Remainder assignment (%=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Remainder_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **remainder assignment (%=)** operator performs remainder on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Remainder assignment (%=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 3;

console.log((a %= 2));
// Expected output: 1

console.log((a %= 0));
// Expected output: NaN

console.log((a %= "hello"));
// Expected output: NaN
```

Syntax

```
x %= y
```

Description

`x %= y` is equivalent to `x = x % y`, except that the expression `x` is only evaluated once.

Examples

Using remainder assignment

```
let bar = 5;

bar %= 2; // 1
bar %= "foo"; // NaN
bar %= 0; // NaN

let foo = 3n;
foo %= 2n; // 1n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Remainder assignment (`x %= y`)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.remainder_assignment`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Assignment operators in the JS guide
- Remainder (%)

JavaScript › Reference › Expressions and operators

Right shift (>>)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Right_shift · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **right shift (>>)** operator returns a number or BigInt whose binary representation is the first operand shifted by the specified number of bits to the right. Excess bits shifted off to the right are discarded, and copies of the leftmost bit are shifted in from the left. This operation is also called "sign-propagating right shift" or "arithmetic right shift", because the sign of the resulting number is the same as the sign of the first operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Right shift (>>) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = 5; //  00000000000000000000000000000101
const b = 2; //  00000000000000000000000000000010
const c = -5; //  11111111111111111111111111111011

console.log(a >> b); //  00000000000000000000000000000001
// Expected output: 1

console.log(c >> b); //  11111111111111111111111111111110
// Expected output: -2
```

Syntax

js

```
x >> y
```

Description

The >> operator is overloaded for two types of operands: number and BigInt. For numbers, the operator returns a 32-bit integer. For BigInts, the operator returns a BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt right shift if both operands become BigInts; otherwise, it converts both operands to 32-bit integers and performs number right shift. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Since the new leftmost bit has the same value as the previous leftmost bit, the sign bit (the leftmost bit) does not change. Hence the name "sign-propagating".

The operator operates on the left operand's bit representation in two's complement. Consider the 32-bit binary representations of the decimal (base 10) numbers `9` and `-9`:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
    -9 (base 10): 11111111111111111111111111110111 (base 2)
```

The binary representation under two's complement of the negative decimal (base 10) number `-9` is formed by inverting all the bits of its opposite number, which is `9` and `00000000000000000000000000001001` in binary, and adding `1`.

In both cases, the sign of the binary number is given by its leftmost bit: for the positive decimal number `9`, the leftmost bit of the binary representation is `0`, and for the negative decimal number `-9`, the leftmost bit of the binary representation is `1`.

Given those binary representations of the decimal (base 10) numbers `9`, and `-9`:

`9 >> 2` yields 2:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
                  --------------------------------
9 >> 2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
```

Notice how two rightmost bits, `01`, have been shifted off, and two copies of the leftmost bit, `0` have been shifted in from the left.

`-9 >> 2` yields `-3`:

```
     -9 (base 10): 11111111111111111111111111110111 (base 2)
                   --------------------------------
-9 >> 2 (base 10): 11111111111111111111111111111101 (base 2) = -3 (base 10)
```

Notice how two rightmost bits, `11`, have been shifted off. But as far as the leftmost bits: in this case, the leftmost bit is `1`. So two copies of that leftmost `1` bit have been shifted in from the left — which preserves the negative sign.

The binary representation `11111111111111111111111111111101` is equal to the negative decimal (base 10) number `-3`, because all negative integers are stored as two's complements, and this one can be calculated by inverting all the bits of the binary representation of the positive decimal (base 10) number `3`, which is `00000000000000000000000000000011`, and then adding one.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 >> 32` is the same as `100 >> 0` (and produces `100`) because 32 modulo 32 is 0.

> **Warning:** You may see people using `>> 0` to truncate numbers to integers. Right shifting any number `x` by `0` returns `x` converted to a 32-bit integer, which additionally removes leading bits for numbers outside the range -2147483648 to 2147483647. Use `Math.trunc()` instead.

For BigInts, there's no truncation. Conceptually, understand positive BigInts as having an infinite number of leading `0` bits, and negative BigInts having an infinite number of leading `1` bits.

Examples

Using right shift

```js
9 >> 2; // 2
-9 >> 2; // -3

9n >> 2n; // 2n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-signed-right-shift-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise right shift (a >> b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.right_shift`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide
- Right shift assignment (>>=)
- Unsigned right shift (>>>)

JavaScript › Reference › Expressions and operators

Right shift assignment (>>=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Right_shift_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **right shift assignment (>>=)** operator performs right shift on the two operands and assigns the result to the left operand.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Right shift assignment (>>=) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let a = 5; //  00000000000000000000000000000101

a >>= 2; //  00000000000000000000000000000001
console.log(a);
// Expected output: 1

let b = -5; //  11111111111111111111111111111011

b >>= 2; //  11111111111111111111111111111110
console.log(b);
// Expected output: -2
```

Syntax

js

```
x >>= y
```

Description

`x >>= y` is equivalent to `x = x >> y`, except that the expression `x` is only evaluated once.

Examples

Using right shift assignment

js

```
let a = 5; //   (00000000000000000000000000000101)
a >>= 2; //   1 (00000000000000000000000000000001)

let b = -5; //  (-00000000000000000000000000000101)
b >>= 2; //  -2 (-00000000000000000000000000000010)

let c = 5n;
c >>= 2n; // 1n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Right shift assignment (x >>= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.right_shift_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Right shift (>>)

JavaScript › Reference › Expressions and operators

Spread syntax (...)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since October 2015.

The **spread (. . .)** syntax allows an iterable, such as an array or string, to be expanded in places where zero or more arguments (for function calls) or elements (for array literals) are expected. In an object literal, the spread syntax enumerates the properties of an object and adds the key-value pairs to the object being created.

Spread syntax looks exactly like rest syntax. In a way, spread syntax is the opposite of rest syntax. Spread syntax "expands" an array into its elements, while rest syntax collects multiple elements and "condenses" them into a single element. See rest parameters and rest property.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Spread syntax (...)". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function sum(x, y, z) {
  return x + y + z;
}

const numbers = [1, 2, 3];

console.log(sum(...numbers));
// Expected output: 6

console.log(sum.apply(null, numbers));
// Expected output: 6
```

Syntax

```js
myFunction(a, ...iterableObj, b)
[1, ...iterableObj, '4', 'five', 6]
{ ...obj, key: 'value' }
```

Description

Spread syntax can be used when all elements from an object or array need to be included in a new array or object, or should be applied one-by-one in a function call's arguments list. There are three distinct places that accept the spread syntax:

- Function arguments list (`myFunction(a, ...iterableObj, b)`)
- Array literals (`[1, ...iterableObj, '4', 'five', 6]`)
- Object literals (`{ ...obj, key: 'value' }`)

Although the syntax looks the same, they come with slightly different semantics.

Only iterable values, like `Array` and `String`, can be spread in array literals and argument lists. Many objects are not iterable, including all plain objects that lack a `Symbol.iterator` method:

js · ✗ bad example

```
const obj = { key1: "value1" };
const array = [...obj]; // TypeError: obj is not iterable
```

On the other hand, spreading in object literals enumerates the own properties of the value. For typical arrays, all indices are enumerable own properties, so arrays can be spread into objects.

```js
const array = [1, 2, 3];
const obj = { ...array }; // { 0: 1, 1: 2, 2: 3 }
```

All primitives can be spread in objects. Only strings have enumerable own properties, and spreading anything else doesn't create properties on the new object.

```js
const obj = { ...true, ..."test", ...10 };
// { '0': 't', '1': 'e', '2': 's', '3': 't' }
```

When using spread syntax for function calls, be aware of the possibility of exceeding the JavaScript engine's argument length limit. See `Function.prototype.apply()` for more details.

Examples

Spread in function calls

Replace apply()

It is common to use `Function.prototype.apply()` in cases where you want to use the elements of an array as arguments to a function.

```js
function myFunction(x, y, z) {}
const args = [0, 1, 2];
myFunction.apply(null, args);
```

With spread syntax the above can be written as:

```js
function myFunction(x, y, z) {}
const args = [0, 1, 2];
myFunction(...args);
```

Any argument in the argument list can use spread syntax, and the spread syntax can be used multiple times.

```js
function myFunction(v, w, x, y, z) {}
const args = [0, 1];
myFunction(-1, ...args, 2, ...[3]);
```

Apply for new operator

When calling a constructor with `new`, it's not possible to **directly** use an array and `apply()`, because `apply()` *calls* the target function instead of *constructing* it, which means, among other things, that `new.target` will be `undefined`. However, an array can be easily used with `new` thanks to spread syntax:

```js
const dateFields = [1970, 0, 1]; // 1 Jan 1970
const d = new Date(...dateFields);
```

Spread in array literals

A more powerful array literal

Without spread syntax, the array literal syntax is no longer sufficient to create a new array using an existing array as one part of it. Instead, imperative code must be used using a combination of methods, including `push()`, `splice()`, `concat()`, etc. With spread syntax, this becomes much more succinct:

```js
const parts = ["shoulders", "knees"];
const lyrics = ["head", ...parts, "and", "toes"];
//  ["head", "shoulders", "knees", "and", "toes"]
```

Just like spread for argument lists, `...` can be used anywhere in the array literal, and may be used more than once.

Copying an array

You can use spread syntax to make a shallow copy of an array. Each array element retains its identity without getting copied.

```js
const arr = [1, 2, 3];
const arr2 = [...arr]; // like arr.slice()

arr2.push(4);
// arr2 becomes [1, 2, 3, 4]
// arr remains unaffected
```

Spread syntax effectively goes one level deep while copying an array. Therefore, it may be unsuitable for copying multidimensional arrays. The same is true with `Object.assign()` — no native operation in JavaScript does a deep clone. The web API method `structuredClone()` allows deep copying values of certain supported types. See shallow copy for more details.

js · ✗ bad example

```
const a = [[1], [2], [3]];
const b = [...a];

b.shift().shift();
// 1

// Oh no! Now array 'a' is affected as well:
console.log(a);
// [[], [2], [3]]
```

A better way to concatenate arrays

`Array.prototype.concat()` is often used to concatenate an array to the end of an existing array. Without spread syntax, this is done as:

js

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

// Append all items from arr2 onto arr1
arr1 = arr1.concat(arr2);
```

With spread syntax this becomes:

js

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

arr1 = [...arr1, ...arr2];
// arr1 is now [0, 1, 2, 3, 4, 5]
```

`Array.prototype.unshift()` is often used to insert an array of values at the start of an existing array. Without spread syntax, this is done as:

js

```
const arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

//  Prepend all items from arr2 onto arr1
Array.prototype.unshift.apply(arr1, arr2);
console.log(arr1); // [3, 4, 5, 0, 1, 2]
```

With spread syntax, this becomes:

js

```
let arr1 = [0, 1, 2];
const arr2 = [3, 4, 5];

arr1 = [...arr2, ...arr1];
console.log(arr1); // [3, 4, 5, 0, 1, 2]
```

> **Note:** Unlike `unshift()`, this creates a new `arr1`, instead of modifying the original `arr1` array in-place.

Conditionally adding values to an array

You can make an element present or absent in an array literal, depending on a condition, using a conditional operator.

```js
const isSummer = false;
const fruits = ["apple", "banana", ...(isSummer ? ["watermelon"] : [])];
// ['apple', 'banana']
```

When the condition is `false`, we spread an empty array, so that nothing gets added to the final array. Note that this is different from the following:

```js
const fruits = ["apple", "banana", isSummer ? "watermelon" : undefined];
// ['apple', 'banana', undefined]
```

In this case, an extra `undefined` element is added when `isSummer` is `false`, and this element will be visited by methods such as `Array.prototype.map()`.

Spread in object literals

Copying and merging objects

You can use spread syntax to merge multiple objects into one new object.

```js
const obj1 = { foo: "bar", x: 42 };
const obj2 = { bar: "baz", y: 13 };

const mergedObj = { ...obj1, ...obj2 };
// { foo: "bar", x: 42, bar: "baz", y: 13 }
```

A single spread creates a shallow copy of the original object (but without non-enumerable properties and without copying the prototype), similar to copying an array.

```js
const clonedObj = { ...obj1 };
// { foo: "bar", x: 42 }
```

Overriding properties

When one object is spread into another object, or when multiple objects are spread into one object, and properties with identical names are encountered, the property takes the last value assigned while remaining in the position it was originally set.

```js
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };

const mergedObj = { x: 41, ...obj1, ...obj2, y: 9 }; // { x: 42, foo: "baz", y: 9 }
```

Conditionally adding properties to an object

You can make an element present or absent in an object literal, depending on a condition, using a conditional operator.

```js
const isSummer = false;
const fruits = {
  apple: 10,
  banana: 5,
  ...(isSummer ? { watermelon: 30 } : {}),
};
// { apple: 10, banana: 5 }
```

The case where the condition is `false` is an empty object, so that nothing gets spread into the final object. Note that this is different from the following:

```js
const fruits = {
  apple: 10,
  banana: 5,
  watermelon: isSummer ? 30 : undefined,
};
// { apple: 10, banana: 5, watermelon: undefined }
```

In this case, the `watermelon` property is always present and will be visited by methods such as `Object.keys()`.

Because primitives can be spread into objects as well, and from the observation that all falsy values do not have enumerable properties, you can simply use a logical AND operator:

```js
const isSummer = false;
const fruits = {
  apple: 10,
  banana: 5,
  ...(isSummer && { watermelon: 30 }),
};
```

In this case, if `isSummer` is any falsy value, no property will be created on the `fruits` object.

Comparing with Object.assign()

Note that `Object.assign()` can be used to mutate an object, whereas spread syntax can't.

```js
const obj1 = { foo: "bar", x: 42 };
Object.assign(obj1, { x: 1337 });
console.log(obj1); // { foo: "bar", x: 1337 }
```

In addition, `Object.assign()` triggers setters on the target object, whereas spread syntax does not.

```js
const objectAssign = Object.assign(
  {
    set foo(val) {
      console.log(val);
    },
  },
  { foo: 1 },
);
// Logs "1"; objectAssign.foo is still the original setter
```

(code continues on the next page)

(code continued from the previous page)

```
const spread = {
  set foo(val) {
    console.log(val);
  },
  ...{ foo: 1 },
};
// Nothing is logged; spread.foo is 1
```

You cannot naively re-implement the `Object.assign()` function through a single spreading:

js

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };
const merge = (...objects) => ({ ...objects });

const mergedObj1 = merge(obj1, obj2);
// { 0: { foo: 'bar', x: 42 }, 1: { foo: 'baz', y: 13 } }

const mergedObj2 = merge({}, obj1, obj2);
// { 0: {}, 1: { foo: 'bar', x: 42 }, 2: { foo: 'baz', y: 13 } }
```

In the above example, the spread syntax does not work as one might expect: it spreads an *array* of arguments into the object literal, due to the rest parameter. Here is an implementation of `merge` using the spread syntax, whose behavior is similar to `Object.assign()`, except that it doesn't trigger setters, nor mutates any object:

js

```
const obj1 = { foo: "bar", x: 42 };
const obj2 = { foo: "baz", y: 13 };
const merge = (...objects) =>
  objects.reduce((acc, cur) => ({ ...acc, ...cur }));

const mergedObj = merge(obj1, obj2);
// { foo: 'baz', x: 42, y: 13 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-SpreadElement
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-ArgumentList
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#prod-PropertyDefinition

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Spread syntax (...)	46	12	16	37	8	46	16	37	8	5.0	46	8	1.0.0	1.0	5.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Spread in array literals	46	12	16	37	8	46	16	37	8	5.0	46	8	1.0.0	1.0	5.0.0
Spread in function calls	46	12	27	37	8	46	27	37	8	5.0	46	8	1.0.0	1.0	5.0.0
Spread in object literals	60	79	55	47	11.1	60	55	44	11.3	8.2	60	11.3	1.0.0	1.0	8.3.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.spread`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Rest parameters
- Rest property
- `Function.prototype.apply()`

JavaScript › Reference › Expressions and operators

Strict equality (===)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Strict_equality · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **strict equality (`===`)** operator checks whether its two operands are equal, returning a Boolean result. Unlike the equality operator, the strict equality operator always considers operands of different types to be different.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Strict equality (===) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(1 === 1);
// Expected output: true

console.log("hello" === "hello");
// Expected output: true

console.log("1" === 1);
// Expected output: false

console.log(0 === false);
// Expected output: false
```

Syntax

js

```
x === y
```

Description

The strict equality operators (`===` and `!==`) provide the IsStrictlyEqual semantic.

- If the operands are of different types, return `false`.
- If both operands are objects, return `true` only if they refer to the same object.
- If both operands are `null` or both operands are `undefined`, return `true`.
- If either operand is `NaN`, return `false`.
- Otherwise, compare the two operand's values:
  - Numbers must have the same numeric values. `+0` and `-0` are considered to be the same value.
  - Strings must have the same characters in the same order.
  - Booleans must be both `true` or both `false`.

The most notable difference between this operator and the equality (`==`) operator is that if the operands are of different types, the `==` operator attempts to convert them to the same type before comparing.

Examples

Comparing operands of the same type

```js
"hello" === "hello"; // true
"hello" === "hola"; // false

3 === 3; // true
3 === 4; // false

true === true; // true
true === false; // false

null === null; // true
```

Comparing operands of different types

```js
"3" === 3; // false
true === 1; // false
null === undefined; // false
3 === new Number(3); // false
```

Comparing objects

```js
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 === object2); // false
console.log(object1 === object1); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-equality-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Strict equality (`a === b`)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.strict_equality`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Equality (`==`)
- Inequality (`!=`)
- Strict inequality (`!==`)

JavaScript › Reference › Expressions and operators

Strict inequality (!==)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Strict_inequality · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **strict inequality (`!==`)** operator checks whether its two operands are not equal, returning a Boolean result. Unlike the inequality operator, the strict inequality operator always considers operands of different types to be different.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Strict inequality (!==) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(1 !== 1);
// Expected output: false

console.log("hello" !== "hello");
// Expected output: false

console.log("1" !== 1);
// Expected output: true

console.log(0 !== false);
// Expected output: true
```

Syntax

```js
x !== y
```

Description

The strict inequality operator checks whether its operands are not equal. It is the negation of the strict equality operator so the following two lines will always give the same result:

```js
x !== y;

!(x === y);
```

For details of the comparison algorithm, see the page for the strict equality operator.

Like the strict equality operator, the strict inequality operator will always consider operands of different types to be different:

```js
3 !== "3"; // true
```

Examples

Comparing operands of the same type

```js
"hello" !== "hello"; // false
"hello" !== "hola"; // true

3 !== 3; // false
3 !== 4; // true

true !== true; // false
true !== false; // true

null !== null; // false
```

Comparing operands of different types

```js
"3" !== 3; // true
true !== 1; // true
null !== undefined; // true
```

Comparing objects

```js
const object1 = {
  key: "value",
};

const object2 = {
  key: "value",
};

console.log(object1 !== object2); // true
console.log(object1 !== object1); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-equality-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Strict inequality (a !== b)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.strict_inequality`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Equality (==)
- Inequality (!=)
- Strict equality (===)

JavaScript › Reference › Expressions and operators

Subtraction (-)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Subtraction · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **subtraction (-)** operator subtracts the two operands, producing their difference.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Subtraction (-) operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(5 - 3);
// Expected output: 2

console.log(3.5 - 5);
// Expected output: -1.5

console.log(5 - "hello");
// Expected output: NaN

console.log(5 - true);
// Expected output: 4
```

Syntax

js

```
x - y
```

Description

The - operator is overloaded for two types of operands: number and BigInt. It first coerces both operands to numeric values and tests the types of them. It performs BigInt subtraction if both operands become BigInts; otherwise, it performs number subtraction. A `TypeError` is thrown if one operand becomes a BigInt but the other becomes a number.

Examples

Subtraction using numbers

js

```
5 - 3; // 2
3 - 5; // -2
```

Other non-BigInt values are coerced to numbers:

js

```
"foo" - 3; // NaN; "foo" is converted to the number NaN
5 - "3"; // 2; "3" is converted to the number 3
```

Subtraction using BigInts

js

```
2n - 1n; // 1n
```

You cannot mix BigInt and number operands in subtraction.

js · ✗ bad example

```
2n - 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
2 - 1n; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

To do subtraction with a BigInt and a non-BigInt, convert either operand:

```js
2n - BigInt(1); // 1n
Number(2n) - 1; // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-subtraction-operator-minus

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Subtraction (-)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.subtraction`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Subtraction assignment (-=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Subtraction_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **subtraction assignment (-=)** operator performs subtraction on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Subtraction assignment (-=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 2;

console.log((a -= 3));
// Expected output: -1

console.log((a -= "Hello"));
// Expected output: NaN
```

Syntax

js

```
x -= y
```

Description

`x -= y` is equivalent to `x = x - y`, except that the expression `x` is only evaluated once.

Examples

Subtraction assignment using numbers

js

```
let bar = 5;

bar -= 2; // 3
```

Other non-BigInt values are coerced to numbers:

js

```
bar -= "foo"; // NaN
```

Subtraction assignment using BigInts

js

```
let foo = 3n;
foo -= 2n; // 1n
foo -= 1; // TypeError: Cannot mix BigInt and other types, use explicit conversions
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Subtraction assignment (x -= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.subtraction_assignment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Subtraction (-)

JavaScript › Reference › Expressions and operators

super

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/super · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2016.

The **`super`** keyword is used to access properties on an object literal or class's [[Prototype]], or invoke a superclass's constructor.

The `super.prop` and `super[expr]` expressions are valid in any method definition in both classes and object literals. The `super(...args)` expression is valid in class constructors.

Try it

▷ Interactive example on MDN: "JavaScript Demo: super expression". It runs live in the browser there; its source code follows.

js · interactive example source

```
class Foo {
  constructor(name) {
    this.name = name;
  }

  getNameSeparator() {
    return "-";
  }
}

class FooBar extends Foo {
  constructor(name, index) {
    super(name);
    this.index = index;
  }

  // Does not get called
  getNameSeparator() {
    return "/";
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  getFullName() {
    return this.name + super.getNameSeparator() + this.index;
  }
}

const firstFooBar = new FooBar("foo", 1);

console.log(firstFooBar.name);
// Expected output: "foo"

console.log(firstFooBar.getFullName());
// Expected output: "foo-1"
```

Syntax

js

```
super()
super(arg1)
super(arg1, arg2)
super(arg1, arg2, /* …, */ argN)

super.propertyOnParent
super[expression]
```

Description

The `super` keyword can be used in two ways: as a "function call" (`super(...args)`), or as a "property lookup" (`super.prop` and `super[expr]`).

> **Note:** `super` is a keyword and these are special syntactic constructs. `super` is not a variable that points to the prototype object. Attempting to read `super` itself is a `SyntaxError`.
>
> js · ✗ bad example
>
> ```
> const child = {
>   myParent() {
>     console.log(super); // SyntaxError: 'super' keyword unexpected here
>   },
> };
> ```

In the constructor body of a derived class (with `extends`), the `super` keyword may appear as a "function call" (`super(...args)`), which must be called before the `this` keyword is used, and before the constructor returns. It calls the parent class's constructor and binds the parent class's public fields, after which the derived class's constructor can further access and modify `this`.

The "property lookup" form can be used to access methods and properties of an object literal's or class's [[Prototype]]. Within a class's body, the reference of `super` can be either the superclass's constructor itself, or the constructor's `prototype`, depending on whether the execution context is instance creation or class initialization. See the Examples section for more details.

Note that the reference of `super` is determined by the class or object literal `super` was declared in, not the object the method is called on. Therefore, unbinding or re-binding a method doesn't change the reference of `super` in it (although they do change the reference of `this`). You can see `super` as a variable in the class or object literal scope, which the methods create a closure over. (But also beware that it's not actually a variable, as explained above.)

When setting properties through `super`, the property is set on `this` instead.

Examples

Using super in classes

This code snippet is taken from the classes sample (live demo). Here `super()` is called to avoid duplicating the constructor parts' that are common between `Rectangle` and `Square`.

```js
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
  sayName() {
    console.log(`Hi, I am a ${this.name}.`);
  }
  get area() {
    return this.height * this.width;
  }
  set area(value) {
    this._area = value;
  }
}

class Square extends Rectangle {
  constructor(length) {
    // Here, it calls the parent class's constructor with lengths
    // provided for the Rectangle's width and height
    super(length, length);

    // Note: In derived classes, super() must be called before you
    // can use 'this'. Moving this to the top causes a ReferenceError.
    this.name = "Square";
  }
}
```

Super-calling static methods

You are also able to call super on static methods.

```js
class Rectangle {
  static logNbSides() {
    return "I have 4 sides";
  }
}

class Square extends Rectangle {
  static logDescription() {
    return `${super.logNbSides()} which are all equal`;
  }
}
Square.logDescription(); // 'I have 4 sides which are all equal'
```

Accessing super in class field declaration

`super` can also be accessed during class field initialization. The reference of `super` depends on whether the current field is an instance field or a static field.

js

```
class Base {
  static baseStaticField = 90;
  baseMethod() {
    return 10;
  }
}

class Extended extends Base {
  extendedField = super.baseMethod(); // 10
  static extendedStaticField = super.baseStaticField; // 90
}
```

Note that instance fields are set on the instance instead of the constructor's `prototype`, so you can't use `super` to access the instance field of a superclass.

js · ✗ bad example

```
class Base {
  baseField = 10;
}

class Extended extends Base {
  extendedField = super.baseField; // undefined
}
```

Here, `extendedField` is `undefined` instead of 10, because `baseField` is defined as an own property of the `Base` instance, instead of `Base.prototype`. `super`, in this context, only looks up properties on `Base.prototype`, because that's the [[Prototype]] of `Extended.prototype`.

Deleting super properties will throw an error

You cannot use the `delete` operator and `super.prop` or `super[expr]` to delete a parent class' property — it will throw a `ReferenceError`.

js

```
class Base {
  foo() {}
}
class Derived extends Base {
  delete() {
    delete super.foo; // this is bad
  }
}

new Derived().delete(); // ReferenceError: invalid delete involving 'super'.
```

Using super.prop in object literals

Super can also be used in the object initializer notation. In this example, two objects define a method. In the second object, `super` calls the first object's method. This works with the help of `Object.setPrototypeOf()` with which we are able to set the prototype of `obj2` to `obj1`, so that `super` is able to find `method1` on `obj1`.

```js
const obj1 = {
  method1() {
    console.log("method 1");
  },
};

const obj2 = {
  method2() {
    super.method1();
  },
};

Object.setPrototypeOf(obj2, obj1);
obj2.method2(); // Logs "method 1"
```

Methods that read super.prop do not behave differently when bound to other objects

Accessing `super.x` behaves like `Reflect.get(Object.getPrototypeOf(objectLiteral), "x", this)`, which means the property is always sought on the object literal/class declaration's prototype, and unbinding and re-binding a method won't change the reference of `super`.

```js
class Base {
  baseGetX() {
    return 1;
  }
}
class Extended extends Base {
  getX() {
    return super.baseGetX();
  }
}

const e = new Extended();
console.log(e.getX()); // 1
const { getX } = e;
console.log(getX()); // 1
```

The same happens in object literals.

```js
const parent1 = { prop: 1 };
const parent2 = { prop: 2 };

const child = {
  myParent() {
    console.log(super.prop);
  },
};

Object.setPrototypeOf(child, parent1);
child.myParent(); // Logs "1"

const myParent = child.myParent;
myParent(); // Still logs "1"

const anotherChild = { __proto__: parent2, myParent };
anotherChild.myParent(); // Still logs "1"
```

Only resetting the entire inheritance chain will change the reference of `super`.

```js
class Base {
  baseGetX() {
    return 1;
  }
  static staticBaseGetX() {
    return 3;
  }
}
class AnotherBase {
  baseGetX() {
    return 2;
  }
  static staticBaseGetX() {
    return 4;
  }
}
class Extended extends Base {
  getX() {
    return super.baseGetX();
  }
  static staticGetX() {
    return super.staticBaseGetX();
  }
}

const e = new Extended();
// Reset instance inheritance
Object.setPrototypeOf(Extended.prototype, AnotherBase.prototype);
console.log(e.getX()); // Logs "2" instead of "1", because the prototype chain has changed
console.log(Extended.staticGetX()); // Still logs "3", because we haven't modified the
static part yet
// Reset static inheritance
Object.setPrototypeOf(Extended, AnotherBase);
console.log(Extended.staticGetX()); // Now logs "4"
```

Calling methods from super

When calling `super.prop` as a function, the `this` value inside the `prop` function is the current `this`, not the object that `super` points to. For example, the `super.getName()` call logs `"Extended"`, despite the code looking like it's equivalent to `Base.getName()`.

```js
class Base {
  static getName() {
    console.log(this.name);
  }
}

class Extended extends Base {
  static getName() {
    super.getName();
  }
}

Extended.getName(); // Logs "Extended"
```

This is especially important when interacting with static private elements.

Setting super.prop sets the property on this instead

Setting properties of `super`, such as `super.x = 1`, behaves like `Reflect.set(Object.getPrototypeOf(objectLiteral), "x", 1, this)`. This is one of the cases where understanding `super` as simply "reference of the prototype object" falls short, because it actually sets the property on `this` instead.

js

```
class A {}
class B extends A {
  setX() {
    super.x = 1;
  }
}

const b = new B();
b.setX();
console.log(b); // B { x: 1 }
console.log(Object.hasOwn(b, "x")); // true
```

`super.x = 1` will look for the property descriptor of `x` on `A.prototype` (and invoke the setters defined there), but the `this` value will be set to `this`, which is `b` in this context. You can read `Reflect.set` for more details on the case when `target` and `receiver` differ.

This means that while methods that *get* `super.prop` are usually not susceptible to changes in the `this` context, those that *set* `super.prop` are.

js · ✗ bad example

```
/* Reusing same declarations as above */

const b2 = new B();
b2.setX.call(null); // TypeError: Cannot assign to read only property 'x' of object 'null'
```

However, `super.x = 1` still consults the property descriptor of the prototype object, which means you cannot rewrite non-writable properties, and setters will be invoked.

js

```
class X {
  constructor() {
    // Create a non-writable property
    Object.defineProperty(this, "prop", {
      configurable: true,
      writable: false,
      value: 1,
    });
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
class Y extends X {
  constructor() {
    super();
  }
  foo() {
    super.prop = 2; // Cannot overwrite the value.
  }
}

const y = new Y();
y.foo(); // TypeError: "prop" is read-only
console.log(y.prop); // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-super-keyword`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`super`	42	13	45	29	7	42	45	29	7	4.0	42	7	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.super`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Classes

JavaScript › Reference › Expressions and operators

this

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since July 2015.

The **`this`** keyword refers to the context where a piece of code, such as a function's body, is supposed to run. Most typically, it is used in object methods, where `this` refers to the object that the method is attached to, thus allowing the same method to be reused on different objects.

The value of `this` in JavaScript depends on how a function is invoked (runtime binding), not how it is defined. When a regular function is invoked as a method of an object (`obj.method()`), `this` points to that object. When invoked as a standalone function (not attached to an object: `func()`), `this` typically refers to the global object (in non-strict mode) or `undefined` (in strict mode). The `Function.prototype.bind()` method can create a function whose `this` binding doesn't change, and methods `Function.prototype.apply()` and `Function.prototype.call()` can also set the `this` value for a particular call.

Arrow functions differ in their handling of `this`: they inherit `this` from the parent scope at the time they are defined. This behavior makes arrow functions particularly useful for callbacks and preserving context. However, arrow functions do not have their own `this` binding. Therefore, their `this` value cannot be set by `bind()`, `apply()` or `call()` methods, nor does it point to the current object in object methods.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: this expression". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const test = {
  prop: 42,
  func() {
    return this.prop;
  },
};

console.log(test.func());
// Expected output: 42
```

Syntax

js

```
this
```

Value

In non–strict mode, `this` is always a reference to an object. In strict mode, it can be any value. For more information on how the value is determined, see the description below.

Description

The value of `this` depends on in which context it appears: function, class, or global.

Function context

Inside a function, the value of `this` depends on how the function is called. Think about `this` as a hidden parameter of a function — just like the parameters declared in the function definition, `this` is a binding that the language creates for you when the function body is evaluated.

For a regular function (not an arrow function, bound function, etc.), the value of `this` is the object that the function is accessed on. In other words, if the function call is in the form `obj.f()`, then `this` refers to `obj`. For example:

```js
function getThis() {
  return this;
}

const obj1 = { name: "obj1" };
const obj2 = { name: "obj2" };

obj1.getThis = getThis;
obj2.getThis = getThis;

console.log(obj1.getThis()); // { name: 'obj1', getThis: [Function: getThis] }
console.log(obj2.getThis()); // { name: 'obj2', getThis: [Function: getThis] }
```

Note how the function is the same, but based on how it's invoked, the value of `this` is different. This is analogous to how function parameters work.

The value of `this` is not the object that has the function as an own property, but the object that is used to call the function. You can prove this by calling a method of an object up in the prototype chain.

```js
const obj3 = {
  __proto__: obj1,
  name: "obj3",
};

console.log(obj3.getThis()); // { name: 'obj3' }
```

The value of `this` always changes based on how a function is called, even when the function was defined on an object at creation:

```js
const obj4 = {
  name: "obj4",
  getThis() {
    return this;
  },
};

const obj5 = { name: "obj5" };

obj5.getThis = obj4.getThis;
console.log(obj5.getThis()); // { name: 'obj5', getThis: [Function: getThis] }
```

If the value that the method is accessed on is a primitive, `this` will be a primitive value as well — but only if the function is in strict mode.

```js
function getThisStrict() {
  "use strict"; // Enter strict mode
  return this;
}

// Only for demonstration — you should not mutate built-in prototypes
Number.prototype.getThisStrict = getThisStrict;
console.log(typeof (1).getThisStrict()); // "number"
```

If the function is called without being accessed on anything, `this` will be `undefined` — but only if the function is in strict mode.

```js
console.log(typeof getThisStrict()); // "undefined"
```

In non-strict mode, a special process called `this` substitution ensures that the value of `this` is always an object. This means:

- If a function is called with `this` set to `undefined` or `null`, `this` gets substituted with `globalThis`.
- If the function is called with `this` set to a primitive value, `this` gets substituted with the primitive value's wrapper object.

```js
function getThis() {
  return this;
}

// Only for demonstration — you should not mutate built-in prototypes
Number.prototype.getThis = getThis;
console.log(typeof (1).getThis()); // "object"
console.log(getThis() === globalThis); // true
```

In typical function calls, `this` is implicitly passed like a parameter through the function's prefix (the part before the dot). You can also explicitly set the value of `this` using the `Function.prototype.call()`, `Function.prototype.apply()`, or `Reflect.apply()` methods. Using `Function.prototype.bind()`, you can create a new function with a specific value of `this` that doesn't change regardless of how the function is called. When using these methods, the `this` substitution rules above still apply if the function is non-strict.

Callbacks

When a function is passed as a callback, the value of `this` depends on how the callback is called, which is determined by the implementor of the API. Callbacks are *typically* called with a `this` value of `undefined` (calling it directly without attaching it to any object), which means if the function is non–strict, the value of `this` is the global object (`globalThis`). This is the case for iterative array methods, the `Promise()` constructor, etc.

```js
function logThis() {
  "use strict";
  console.log(this);
}

[1, 2, 3].forEach(logThis); // undefined, undefined, undefined
```

Some APIs allow you to set a `this` value for invocations of the callback. For example, all iterative array methods and related ones like `Set.prototype.forEach()` accept an optional `thisArg` parameter.

```js
[1, 2, 3].forEach(logThis, { name: "obj" });
// { name: 'obj' }, { name: 'obj' }, { name: 'obj' }
```

Occasionally, a callback is called with a `this` value other than `undefined`. For example, the `reviver` parameter of `JSON.parse()` and the `replacer` parameter of `JSON.stringify()` are both called with `this` set to the object that the property being parsed/serialized belongs to.

Arrow functions

In arrow functions, `this` retains the value of the enclosing lexical context's `this`. In other words, when evaluating an arrow function's body, the language does not create a new `this` binding.

For example, in global code, `this` is always `globalThis` regardless of strictness, because of the global context binding:

```js
const globalObject = this;
const foo = () => this;
console.log(foo() === globalObject); // true
```

Arrow functions create a closure over the `this` value of its surrounding scope, which means arrow functions behave as if they are "auto-bound" — no matter how it's invoked, `this` is bound to what it was when the function was created (in the example above, the global object). The same applies to arrow functions created inside other functions: their `this` remains that of the enclosing lexical context. See example below.

Furthermore, when invoking arrow functions using `call()`, `bind()`, or `apply()`, the `thisArg` parameter is ignored. You can still pass other arguments using these methods, though.

```js
const obj = { name: "obj" };

// Attempt to set this using call
console.log(foo.call(obj) === globalObject); // true

// Attempt to set this using bind
const boundFoo = foo.bind(obj);
console.log(boundFoo() === globalObject); // true
```

Constructors

When a function is used as a constructor (with the new keyword), its `this` is bound to the new object being constructed, no matter which object the constructor function is accessed on. The value of `this` becomes the value of the `new` expression unless the constructor returns another non–primitive value.

```js
function C() {
  this.a = 37;
}

let o = new C();
console.log(o.a); // 37

function C2() {
  this.a = 37;
  return { a: 38 };
}

o = new C2();
console.log(o.a); // 38
```

In the second example (`C2`), because an object was returned during construction, the new object that `this` was bound to gets discarded. (This essentially makes the statement `this.a = 37;` dead code. It's not exactly dead because it gets executed, but it can be eliminated with no outside effects.)

super

When a function is invoked in the `super.method()` form, the `this` inside the `method` function is the same value as the `this` value around the `super.method()` call, and is generally not equal to the object that `super` refers to. This is because `super.method` is not an object member access like the ones above — it's a special syntax with different binding rules. For examples, see the `super` reference.

Class context

A class can be split into two contexts: static and instance. Constructors, methods, and instance field initializers (public or private) belong to the instance context. Static methods, static field initializers, and static initialization blocks belong to the static context. The `this` value is different in each context.

Class constructors are always called with `new`, so their behavior is the same as function constructors: the `this` value is the new instance being created. Class methods behave like methods in object literals — the `this` value is the object that the method was accessed on. If the method is not transferred to another object, `this` is generally an instance of the class.

Static methods are not properties of `this`. They are properties of the class itself. Therefore, they are generally accessed on the class, and `this` is the value of the class (or a subclass). Static initialization blocks are also evaluated with `this` set to the current class.

Field initializers are also evaluated in the context of the class. Instance fields are evaluated with `this` set to the instance being constructed. Static fields are evaluated with `this` set to the current class. This is why arrow functions in field initializers are bound to the instance for instance fields and to the class for static fields.

```js
class C {
  instanceField = this;
  static staticField = this;
}

const c = new C();
console.log(c.instanceField === c); // true
console.log(C.staticField === C); // true
```

Derived class constructors

Unlike base class constructors, derived constructors have no initial `this` binding. Calling `super()` creates a `this` binding within the constructor and essentially has the effect of evaluating the following line of code, where `Base` is the base class:

```js
this = new Base();
```

> **Warning:** Referring to `this` before calling `super()` will throw an error.

Derived classes must not return before calling `super()`, unless the constructor returns an object (so the `this` value is overridden) or the class has no constructor at all.

```js
class Base {}
class Good extends Base {}
class AlsoGood extends Base {
  constructor() {
    return { a: 5 };
  }
}
class Bad extends Base {
  constructor() {}
}

new Good();
new AlsoGood();
new Bad(); // ReferenceError: Must call super constructor in derived class before accessing
'this' or returning from derived constructor
```

Global context

In the global execution context (outside of any functions or classes; may be inside blocks or arrow functions defined in the global scope), the `this` value depends on what execution context the script runs in. Like callbacks, the `this` value is determined by the runtime environment (the caller).

At the top level of a script, `this` refers to `globalThis` whether in strict mode or not. This is generally the same as the global object — for example, if the source is put inside an HTML `<script>` element and executed as a script, `this === window`.

> **Note:** `globalThis` is generally the same concept as the global object (i.e., adding properties to `globalThis` makes them global variables) — this is the case for browsers and Node — but hosts are allowed to provide a different value for `globalThis` that's unrelated to the global object.

```js
// In web browsers, the window object is also the global object:
console.log(this === window); // true

this.b = "MDN";
console.log(window.b); // "MDN"
console.log(b); // "MDN"
```

If the source is loaded as a module (for HTML, this means adding `type="module"` to the `<script>` tag), `this` is always `undefined` at the top level.

If the source is executed with `eval()`, `this` is the same as the enclosing context for direct eval, or `globalThis` (as if it's run in a separate global script) for indirect eval.

```js
function test() {
  // Direct eval
  console.log(eval("this") === this);
  // Indirect eval, non-strict
  console.log(eval?.("this") === globalThis);
  // Indirect eval, strict
  console.log(eval?.("'use strict'; this") === globalThis);
}

test.call({ name: "obj" }); // Logs 3 "true"
```

Note that some source code, while looking like the global scope, is actually wrapped in a function when executed. For example, Node.js CommonJS modules are wrapped in a function and executed with the `this` value set to `module.exports`. Event handler attributes are executed with `this` set to the element they are attached to.

Object literals don't create a `this` scope — only functions (methods) defined within the object do. Using `this` in an object literal inherits the value from the surrounding scope.

```js
const obj = {
  a: this,
};

console.log(obj.a === window); // true
```

Examples

this in function contexts

The value of the `this` parameter depends on how the function is called, not on how it's defined.

```js
// An object can be passed as the first argument to 'call'
// or 'apply' and 'this' will be bound to it.
const obj = { a: "Custom" };

// Variables declared with var become properties of 'globalThis'.
var a = "Global";

function whatsThis() {
  return this.a; // 'this' depends on how the function is called
}

whatsThis(); // 'Global'; the 'this' parameter defaults to 'globalThis' in non-strict mode
obj.whatsThis = whatsThis;
obj.whatsThis(); // 'Custom'; the 'this' parameter is bound to obj
```

Using `call()` and `apply()`, you can pass the value of `this` as if it's an explicit parameter.

```js
function add(c, d) {
  return this.a + this.b + c + d;
}

const o = { a: 1, b: 3 };

// The first argument is bound to the implicit 'this' parameter; the remaining
// arguments are bound to the named parameters.
add.call(o, 5, 7); // 16

// The first argument is bound to the implicit 'this' parameter; the second
// argument is an array whose members are bound to the named parameters.
add.apply(o, [10, 20]); // 34
```

this and object conversion

In non–strict mode, if a function is called with a `this` value that's not an object, the `this` value is substituted with an object. `null` and `undefined` become `globalThis`. Primitives like `7` or `'foo'` are converted to an object using the related constructor, so the primitive number `7` is converted to a `Number` wrapper class and the string `'foo'` to a `String` wrapper class.

```js
function bar() {
  console.log(Object.prototype.toString.call(this));
}

bar.call(7); // [object Number]
bar.call("foo"); // [object String]
bar.call(undefined); // [object Window]
```

The bind() method

Calling `f.bind(someObject)` creates a new function with the same body and scope as `f`, but the value of `this` is permanently bound to the first argument of `bind`, regardless of how the function is being called.

```js
function f() {
  return this.a;
}

const g = f.bind({ a: "azerty" });
console.log(g()); // azerty

const h = g.bind({ a: "yoo" }); // bind only works once!
console.log(h()); // azerty

const o = { a: 37, f, g, h };
console.log(o.a, o.f(), o.g(), o.h()); // 37 37 azerty azerty
```

this in arrow functions

Arrow functions create closures over the `this` value of the enclosing execution context. In the following example, we create `obj` with a method `getThisGetter` that returns a function that returns the value of `this`. The returned function is created as an arrow function, so its `this` is permanently bound to the `this` of its enclosing function. The value of `this` inside `getThisGetter` can be set in the call, which in turn sets the return value of the returned function. We will assume that `getThisGetter` is a non-strict function, which means it's contained in a non-strict script and not further nested in a class or strict function.

```js
const obj = {
  getThisGetter() {
    const getter = () => this;
    return getter;
  },
};
```

We can call `getThisGetter` as a method of `obj`, which binds `this` to `obj` inside its body. The returned function is assigned to a variable `fn`. Now, when calling `fn`, the value of `this` returned is still the one set

by the call to `getThisGetter`, which is `obj`. If the returned function was not an arrow function, such calls would cause the `this` value to be `globalThis`, because `getThisGetter` is non-strict.

```js
const fn = obj.getThisGetter();
console.log(fn() === obj); // true
```

But be careful if you unbind the method of `obj` without calling it, because `getThisGetter` is still a method that has a varying `this` value. Calling `fn2()()` in the following example returns `globalThis`, because it follows the `this` from `fn2()`, which is `globalThis` since it's called without being attached to any object.

```js
const fn2 = obj.getThisGetter;
console.log(fn2()() === globalThis); // true in non-strict mode
```

This behavior is very useful when defining callbacks. Usually, each function expression creates its own `this` binding, which shadows the `this` value of the upper scope. Now, you can define functions as arrow functions if you don't care about the `this` value, and only create `this` bindings where you do (e.g., in class methods). See example with `setTimeout()`.

this with a getter or setter

`this` in getters and setters is based on which object the property is accessed on, not which object the property is defined on. A function used as getter or setter has its `this` bound to the object from which the property is being set or gotten.

```js
function sum() {
  return this.a + this.b + this.c;
}

const o = {
  a: 1,
  b: 2,
  c: 3,
  get average() {
    return (this.a + this.b + this.c) / 3;
  },
};

Object.defineProperty(o, "sum", {
  get: sum,
  enumerable: true,
  configurable: true,
});

console.log(o.average, o.sum); // 2 6
```

this in DOM event handlers

When a function is used as an event handler, its `this` parameter is bound to the DOM element on which the listener is placed (some browsers do not follow this convention for listeners added dynamically with methods other than `addEventListener()`).

```js
// When called as a listener, turns the related element blue
function bluify(e) {
  // Always true
  console.log(this === e.currentTarget);
  // true when currentTarget and target are the same object
  console.log(this === e.target);
  this.style.backgroundColor = "#A5D9F3";
}

// Get a list of every element in the document
const elements = document.getElementsByTagName("*");

// Add bluify as a click listener so when the
// element is clicked on, it turns blue
for (const element of elements) {
  element.addEventListener("click", bluify);
}
```

this in inline event handlers

When the code is called from an inline event handler attribute, its `this` is bound to the DOM element on which the listener is placed:

```html
<button onclick="alert(this.tagName.toLowerCase());">Show this</button>
```

The above alert shows `button`. Note, however, that only the outer scope has its `this` bound this way:

```html
<button onclick="alert((function () { return this; })());">
  Show inner this
</button>
```

In this case, the `this` parameter of the inner function is bound to `globalThis` (i.e., the default object in non–strict mode where `this` isn't passed in the call).

Bound methods in classes

Just like with regular functions, the value of `this` within methods depends on how they are called. Sometimes it is useful to override this behavior so that `this` within classes always refers to the class instance. To achieve this, bind the class methods in the constructor:

```js
class Car {
  constructor() {
    // Bind sayBye but not sayHi to show the difference
    this.sayBye = this.sayBye.bind(this);
  }

  sayHi() {
    console.log(`Hello from ${this.name}`);
  }

  sayBye() {
    console.log(`Bye from ${this.name}`);
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  get name() {
    return "Ferrari";
  }
}

class Bird {
  get name() {
    return "Tweety";
  }
}

const car = new Car();
const bird = new Bird();

// The value of 'this' in methods depends on their caller
car.sayHi(); // Hello from Ferrari
bird.sayHi = car.sayHi;
bird.sayHi(); // Hello from Tweety

// For bound methods, 'this' doesn't depend on the caller
bird.sayBye = car.sayBye;
bird.sayBye(); // Bye from Ferrari
```

Note: Classes are always in strict mode. Calling methods with an undefined `this` will throw an error if the method tries to access properties on `this`.

js · ✗ bad example

```
const carSayHi = car.sayHi;
carSayHi(); // TypeError because the 'sayHi' method tries to access 'this.name', but
'this' is undefined in strict mode.
```

Note, however, that auto-bound methods suffer from the same problem as using arrow functions for class properties: each instance of the class will have its own copy of the method, which increases memory usage. Only use it where absolutely necessary. You can also mimic the implementation of `Intl.NumberFormat.prototype.format()`: define the property as a getter that returns a bound function when accessed and saves it, so that the function is only created once and only created when necessary.

this in with statements

Although `with` statements are deprecated and not available in strict mode, they still serve as an exception to the normal `this` binding rules. If a function is called within a `with` statement and that function is a property of the scope object, the `this` value is bound to the scope object, as if the `obj.` prefix exists.

js

```
const obj = {
  foo() {
    return this;
  },
};

with (obj) {
  console.log(foo() === obj); // true
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-this-keyword`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`this`	1	12	1	9.5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.this`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Strict mode
- `globalThis`

JavaScript › Reference › Expressions and operators

typeof

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/typeof · MDN page last modified 2026-05-22

Baseline: Widely available — available across major browsers since July 2015.

The **`typeof`** operator returns a string indicating the type of the operand's value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: typeof operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(typeof 42);
// Expected output: "number"

console.log(typeof "blubber");
// Expected output: "string"

console.log(typeof true);
// Expected output: "boolean"

console.log(typeof undeclaredVariable);
// Expected output: "undefined"
```

Syntax

```
typeof operand
```

Parameters

`operand`

An expression representing the object or primitive whose type is to be returned.

Description

The following table summarizes the possible return values of `typeof`. For more information about types and primitives, see also the JavaScript data structure page.

Type	Result
Undefined	`"undefined"`
Null	`"object"` (reason)
Boolean	`"boolean"`
Number	`"number"`
BigInt	`"bigint"`
String	`"string"`
Symbol	`"symbol"`
Function (implements [[Call]] in ECMA-262 terms; classes are functions as well)	`"function"`
Any other object	`"object"`

This list of values is exhaustive. No spec-compliant engines are reported to produce (or had historically produced) values other than those listed.

Examples

Basic usage

```
// Numbers
typeof 37 === "number";
typeof 3.14 === "number";
typeof 42 === "number";
typeof Math.LN2 === "number";
typeof Infinity === "number";
typeof NaN === "number"; // Despite being "Not-A-Number"
typeof Number("1") === "number"; // Number tries to parse things into numbers
typeof Number("shoe") === "number"; // including values that cannot be type coerced to a
number

typeof 42n === "bigint";
```

(code continues on the next page)

(code continued from the previous page)

```
// Strings
typeof "" === "string";
typeof "bla" === "string";
typeof `template literal` === "string";
typeof "1" === "string"; // note that a number within a string is still typeof string
typeof typeof 1 === "string"; // typeof always returns a string
typeof String(1) === "string"; // String converts anything into a string, safer than
toString

// Booleans
typeof true === "boolean";
typeof false === "boolean";
typeof Boolean(1) === "boolean"; // Boolean() will convert values based on if they're truthy
or falsy
typeof !!1 === "boolean"; // two calls of the ! (logical NOT) operator are equivalent to
Boolean()

// Symbols
typeof Symbol() === "symbol";
typeof Symbol("foo") === "symbol";
typeof Symbol.iterator === "symbol";

// Undefined
typeof undefined === "undefined";
typeof declaredButUndefinedVariable === "undefined";
typeof undeclaredVariable === "undefined";

// Objects
typeof { a: 1 } === "object";

// use Array.isArray or Object.prototype.toString.call
// to differentiate regular objects from arrays
typeof [1, 2, 4] === "object";

typeof new Date() === "object";
typeof /regex/ === "object";

// The following are confusing, dangerous, and wasteful. Avoid them.
typeof new Boolean(true) === "object";
typeof new Number(1) === "object";
typeof new String("abc") === "object";

// Functions
typeof function () {} === "function";
typeof class C {} === "function";
typeof Math.sin === "function";
```

typeof null

js

```
// This stands since the beginning of JavaScript
typeof null === "object";
```

In the first implementation of JavaScript, JavaScript values were represented as a type tag and a value. The type tag for objects was `0`. `null` was represented as the NULL pointer (`0x00` in most platforms). Consequently, `null` had `0` as type tag, hence the `typeof` return value `"object"`. (reference)

A fix was proposed for ECMAScript (via an opt-in), but was rejected. It would have resulted in `typeof null === "null"`.

Using new operator

All constructor functions called with new will return non-primitives ("object" or "function"). Most return objects, with the notable exception being Function, which returns a function.

```js
const str = new String("String");
const num = new Number(100);

typeof str; // "object"
typeof num; // "object"

const func = new Function();

typeof func; // "function"
```

Need for parentheses in syntax

The typeof operator has higher precedence than binary operators like addition (+). Therefore, parentheses are needed to evaluate the type of an addition result.

```js
// Parentheses can be used for determining the data type of expressions.
const someData = 99;

typeof someData + " foo"; // "number foo"
typeof (someData + " foo"); // "string"
```

Interaction with undeclared and uninitialized variables

typeof works with undeclared identifiers, returning "undefined" instead of throwing an error.

```js
typeof undeclaredVariable; // "undefined"
```

However, using typeof on lexical declarations (let const, using await using, and class) in the same block before the place of declaration will throw a ReferenceError. Block scoped variables are in a *temporal dead zone* from the start of the block until the initialization is processed, during which it will throw an error if accessed.

js · ✗ bad example

```js
typeof newLetVariable; // ReferenceError
typeof newConstVariable; // ReferenceError
typeof newClass; // ReferenceError

let newLetVariable;
const newConstVariable = "hello";
class newClass {}
```

See typeof operator and undefined for more details.

Exceptional behavior of document.all

All current browsers expose a non-standard host object document.all with type undefined.

```js
typeof document.all === "undefined";
```

Although `document.all` is also falsy and loosely equal to `undefined`, it is not `undefined`. The case of `document.all` having type `"undefined"` is classified in the web standards as a "willful violation" of the original ECMAScript standard for web compatibility.

Custom method that gets a more specific type

`typeof` is very useful, but it's not as versatile as might be required. For example, `typeof []` is `"object"`, as well as `typeof new Date()`, `typeof /abc/`, etc.

For greater specificity in checking types, here we present a custom `type(value)` function, which mostly mimics the behavior of `typeof`, but for non-primitives (i.e., objects and functions), it returns a more granular type name where possible.

js

```
function type(value) {
  if (value === null) {
    return "null";
  }
  const baseType = typeof value;
  // Primitive types
  if (!["object", "function"].includes(baseType)) {
    return baseType;
  }

  // Symbol.toStringTag often specifies the "display name" of the
  // object's class. It's used in Object.prototype.toString().
  const tag = value[Symbol.toStringTag];
  if (typeof tag === "string") {
    return tag;
  }

  // If it's a function whose source code starts with the "class" keyword
  if (
    baseType === "function" &&
    Function.prototype.toString.call(value).startsWith("class")
  ) {
    return "class";
  }

  // The name of the constructor; for example `Array`, `GeneratorFunction`,
  // `Number`, `String`, `Boolean` or `MyCustomClass`
  const className = value.constructor.name;
  if (typeof className === "string" && className !== "") {
    return className;
  }

  // At this point there's no robust way to get the type of value,
  // so we use the base implementation.
  return baseType;
}
```

For checking potentially non-existent variables that would otherwise throw a `ReferenceError`, use `typeof nonExistentVar === "undefined"` because this behavior cannot be mimicked with custom code.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-typeof-operator`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`typeof`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.typeof`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `instanceof`
- `document.all` willful violation of the standard

JavaScript › Reference › Expressions and operators

Unary negation (-)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Unary_negation · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **unary negation (-)** operator precedes its operand and negates it.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Unary negation (-) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const x = 4;
const y = -x;

console.log(y);
// Expected output: -4

const a = "4";
const b = -a;

console.log(b);
// Expected output: -4
```

Syntax

```js
-x
```

Description

The - operator is overloaded for two types of operands: number and BigInt. It first coerces the operand to a numeric value and tests the type of it. It performs BigInt negation if the operand becomes a BigInt; otherwise, it performs number negation.

Examples

Negating numbers

```js
const x = 3;
const y = -x;
// y is -3; x is 3
```

Negating non-numbers

The unary negation operator can convert a non-number into a number.

```js
const x = "4";
const y = -x;

// y is -4
```

BigInts can be negated using the unary negation operator.

```js
const x = 4n;
const y = -x;

// y is -4n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unary-minus-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Unary negation (-)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.unary_negation`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary plus (+)

JavaScript › Reference › Expressions and operators

Unary plus (+)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Unary_plus · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **unary plus (+)** operator precedes its operand and evaluates to its operand but attempts to convert it into a number, if it isn't already.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Unary plus (+) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const x = 1;
const y = -1;

console.log(+x);
// Expected output: 1

console.log(+y);
// Expected output: -1

console.log(+"");
// Expected output: 0

console.log(+true);
// Expected output: 1

console.log(+false);
// Expected output: 0

console.log(+"hello");
// Expected output: NaN
```

Syntax

```js
+x
```

Description

Although unary negation (`-`) also can convert non-numbers, unary plus is the fastest and preferred way of converting something into a number, because it does not perform any other operations on the number.

Unary plus does the exact same steps as normal number coercion used by most built-in methods expecting numbers. It can convert string representations of integers and floats, as well as the non-string values `true`, `false`, and `null`. Integers in both decimal and hexadecimal (`0x`-prefixed) formats are supported. Negative numbers are supported (though not for hex). If it cannot parse a particular value, it will evaluate to `NaN`. Unlike other arithmetic operators, which work with both numbers and BigInts, using the + operator on BigInt values throws a `TypeError`.

Examples

Usage with numbers

```js
const x = 1;
const y = -1;

console.log(+x);
// 1
console.log(+y);
// -1
```

Usage with non-numbers

```js
+true  // 1
+false // 0
+null  // 0
+[]    // 0
+function (val) { return val; } // NaN
+1n    // throws TypeError: Cannot convert BigInt value to number
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unary-plus-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Unary plus (+)	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.unary_plus`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Remainder (%)
- Exponentiation (**)
- Increment (++)
- Decrement (--)
- Unary negation (-)

JavaScript › Reference › Expressions and operators

Unsigned right shift (>>>)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Unsigned_right_shift · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **unsigned right shift (>>>)** operator returns a number whose binary representation is the first operand shifted by the specified number of bits to the right. Excess bits shifted off to the right are discarded, and zero bits are shifted in from the left. This operation is also called "zero-filling right shift", because the sign bit becomes `0`, so the resulting number is always positive. Unsigned right shift does not accept BigInt values.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Unsigned right shift (>>>) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
const a = 5; //  00000000000000000000000000000101
const b = 2; //  00000000000000000000000000000010
const c = -5; //  11111111111111111111111111111011

console.log(a >>> b); //  00000000000000000000000000000001
// Expected output: 1

console.log(c >>> b); //  00111111111111111111111111111110
// Expected output: 1073741822
```

Syntax

```
x >>> y
```

Description

Unlike other arithmetic and bitwise operators, the unsigned right shift operator does not accept BigInt values. This is because it fills the leftmost bits with zeroes, but conceptually, BigInts have an infinite number of leading sign bits, so there's no "leftmost bit" to fill with zeroes.

The operator operates on the left operand's bit representation in two's complement. Consider the 32-bit binary representations of the decimal (base 10) numbers `9` and `-9`:

```
     9 (base 10): 00000000000000000000000000001001 (base 2)
    -9 (base 10): 11111111111111111111111111110111 (base 2)
```

The binary representation under two's complement of the negative decimal (base 10) number `-9` is formed by inverting all the bits of its opposite number, which is `9` and `00000000000000000000000000001001` in binary, and adding `1`.

In both cases, the sign of the binary number is given by its leftmost bit: for the positive decimal number `9`, the leftmost bit of the binary representation is `0`, and for the negative decimal number `-9`, the leftmost bit of the binary representation is `1`.

Given those binary representations of the decimal (base 10) numbers `9`, and `-9`:

For the positive number `9`, zero-fill right shift and sign-propagating right shift yield the same result: `9 >>> 2` yields `2`, the same as `9 >> 2`:

```
      9 (base 10): 00000000000000000000000000001001 (base 2)
                   --------------------------------
9 >>  2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
9 >>> 2 (base 10): 00000000000000000000000000000010 (base 2) = 2 (base 10)
```

Notice how two rightmost bits, `01`, have been shifted off, and two zeroes have been shifted in from the left.

However, notice what happens for `-9`: `-9 >> 2` (sign-propagating right shift) yields `-3`, but `-9 >>> 2` (zero-fill right shift) yields 1073741821:

```
      -9 (base 10): 11111111111111111111111111110111 (base 2)
                    --------------------------------
-9 >>  2 (base 10): 11111111111111111111111111111101 (base 2) = -3 (base 10)
-9 >>> 2 (base 10): 00111111111111111111111111111101 (base 2) = 1073741821 (base 10)
```

Notice how two rightmost bits, `11`, have been shifted off. For `-9 >> 2` (sign-propagating right shift), two copies of the leftmost `1` bit have been shifted in from the left, which preserves the negative sign. On the other hand, for `-9 >>> 2` (zero-fill right shift), zeroes have instead been shifted in from the left, so the negative sign of the number is not preserved, and the result is instead a (large) positive number.

If the left operand is a number with more than 32 bits, it will get the most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 11100110111110100000000000000110000000000001
After:              10100000000000000110000000000001
```

The right operand will be converted to an unsigned 32-bit integer and then taken modulo 32, so the actual shift offset will always be a positive integer between 0 and 31, inclusive. For example, `100 >>> 32` is the same as `100 >>> 0` (and produces `100`) because 32 modulo 32 is 0.

Examples

Using unsigned right shift

js

```
9 >>> 2; // 2
-9 >>> 2; // 1073741821
```

Unsigned right shift doesn't work with BigInts.

js

```
9n >>> 2n; // TypeError: BigInts have no unsigned right shift, use >> instead
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-unsigned-right-shift-operator

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Bitwise unsigned right shift (a >>> b)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.unsigned_right_shift`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Bitwise operators in the JS guide
- Unsigned right shift assignment (>>>=)

JavaScript › Reference › Expressions and operators

Unsigned right shift assignment (>>>=)

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Unsigned_right_shift_assignment · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **unsigned right shift assignment (>>>=)** operator performs unsigned right shift on the two operands and assigns the result to the left operand.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Unsigned right shift assignment (>>>=) operator". It runs live in the browser there; its source code follows.

js · interactive example source

```
let a = 5; //  00000000000000000000000000000101

a >>>= 2; //  00000000000000000000000000000001
console.log(a);
// Expected output: 1

let b = -5; // -00000000000000000000000000000101

b >>>= 2; //  00111111111111111111111111111110
console.log(b);
// Expected output: 1073741822
```

Syntax

js

```
x >>>= y
```

Description

`x >>>= y` is equivalent to `x = x >>> y`, except that the expression `x` is only evaluated once.

Examples

Using unsigned right shift assignment

js

```
let a = 5; // (00000000000000000000000000000101)
a >>>= 2; // 1 (00000000000000000000000000000001)

let b = -5; // (-00000000000000000000000000000101)
b >>>= 2; // 1073741822 (00111111111111111111111111111110)

let c = 5n;
c >>>= 2n; // 1n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-assignment-operators`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Unsigned right shift assignment (x >>>= y)	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.unsigned_right_shift_assignment`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Assignment operators in the JS guide
- Unsigned right shift (>>>)

JavaScript › Reference › Expressions and operators

void operator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/void · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`void`** operator evaluates the given `expression` and then returns `undefined`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: void operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const output = void 1;
console.log(output);
// Expected output: undefined

void console.log("expression evaluated");
// Expected output: "expression evaluated"

void (function iife() {
  console.log("iife is executed");
})();
// Expected output: "iife is executed"
```

(code continues on the next page)

(code continued from the previous page)

```
void function test() {
  console.log("test function executed");
};
try {
  test();
} catch (e) {
  console.log("test function is not defined");
  // Expected output: "test function is not defined"
}
```

Syntax

js

```
void expression
```

Description

This operator allows evaluating expressions that produce a value into places where an expression that evaluates to `undefined` is desired.

The `void` operator is often used merely to obtain the `undefined` primitive value, usually using `void(0)` (which is equivalent to `void 0`). In these cases, the global variable `undefined` can be used.

It should be noted that the precedence of the `void` operator should be taken into account and that parentheses can help clarify the resolution of the expression following the `void` operator:

js

```
void 2 === "2"; // (void 2) === '2', returns false
void (2 === "2"); // void (2 === '2'), returns undefined
```

Examples

Immediately Invoked Function Expressions

When using an immediately-invoked function expression, the `function` keyword cannot be at the immediate start of the statement, because that would be parsed as a function declaration, and would generate a syntax error when the parentheses representing invocation is reached — if the function is unnamed, it would immediately be a syntax error if the function is parsed as a declaration.

js · ✗ bad example

```
function iife() {
  console.log("Executed!");
}(); // SyntaxError: Unexpected token ')'

function () {
  console.log("Executed!");
}(); // SyntaxError: Function statements require a function name
```

In order for the function to be parsed as an expression, the `function` keyword has to appear at a position that only accepts expressions, not statements. This can be achieved by prefixing the keyword with a unary operator, which only accepts expressions as operands. Function invocation has higher precedence than unary operators, so it will be executed first. Its return value (which is almost always `undefined`) will be passed to the unary operator and then immediately discarded.

Of all the unary operators, `void` offers the best semantic, because it clearly signals that the return value of the function invocation should be discarded.

js

```
void function () {
  console.log("Executed!");
}();

// Logs "Executed!"
```

This is a bit longer than wrapping the function expression in parentheses, which has the same effect of forcing the `function` keyword to be parsed as the start of an expression instead of a statement.

js

```
(function () {
  console.log("Executed!");
})();
```

Note that this trick only applies to IIFEs defined with the `function` keyword. Attempting to use the `void` operator to avoid parentheses for an arrow function results in a syntax error. Arrow function expressions always require parentheses around them when being called.

js · ✗ bad example

```
void () => { console.log("iife!"); }(); // SyntaxError: Malformed arrow function parameter
list
```

JavaScript URIs

When a browser follows a `javascript:` URI, it evaluates the code in the URI and then replaces the contents of the page with the returned value, unless the returned value is `undefined`. The `void` operator can be used to return `undefined`. For example:

html

```
<a href="javascript:void(0);">Click here to do nothing</a>

<a href="javascript:void(document.body.style.backgroundColor='green');">
  Click here for green background
</a>
```

> **Note:** `javascript:` pseudo protocol is discouraged over other alternatives, such as unobtrusive event handlers.

Non-leaking Arrow Functions

Arrow functions introduce a short-hand braceless syntax that returns an expression. This can cause unintended side effects if the expression is a function call where the returned value changes from `undefined` to some other value.

For example, if `doSomething()` returns `false` in the code below, the checkbox will no longer be marked as checked or unchecked when the checkbox is clicked (returning `false` from the handler disables the default action).

js · ✗ bad example

```
checkbox.onclick = () => doSomething();
```

This is unlikely to be desired behavior! To be safe, when the return value of a function is not intended to be used, it can be passed to the `void` operator to ensure that (for example) changing APIs do not cause arrow functions' behaviors to change.

js · ✓ good example

```
checkbox.onclick = () => void doSomething();
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-expressions.html#sec-void-operator`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`void`	1	12	1	4	3.1	18	4	10.1	3	1.0	4.4	3	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.void`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `undefined`

JavaScript › Reference › Expressions and operators

yield

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/yield · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2016.

The **`yield`** operator is used to pause and resume a generator function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: yield operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function* foo(index) {
  while (index < 2) {
    yield index;
    index++;
  }
}

const iterator = foo(0);

console.log(iterator.next().value);
// Expected output: 0

console.log(iterator.next().value);
// Expected output: 1
```

Syntax

js

```
yield
yield expression
```

Parameters

`expression` Optional

The value to yield from the generator function via the iterator protocol. If omitted, `undefined` is yielded.

Return value

Returns the optional value passed to the generator's `next()` method to resume its execution.

> **Note:** This means `next()` is asymmetric: it always sends a value to the currently suspended `yield`, but returns the operand of the next `yield`. The argument passed to the first `next()` call cannot be retrieved because there's no currently suspended `yield`.

Description

The `yield` keyword pauses generator function execution and the value of the expression following the `yield` keyword is returned to the generator's caller. It can be thought of as a generator-based version of the `return` keyword.

`yield` can only be used directly within the generator function that contains it. It cannot be used within nested functions.

Calling a generator function constructs a `Generator` object. Each time the generator's `next()` method is called, the generator resumes execution, and runs until it reaches one of the following:

- A `yield` expression. In this case, the generator pauses, and the `next()` method return an iterator result object with two properties: `value` and `done`. The `value` property is the value of the expression

after the `yield` operator, and `done` is `false`, indicating that the generator function has not fully completed.

- The end of the generator function. In this case, execution of the generator ends, and the `next()` method returns an iterator result object where the `value` is `undefined` and `done` is `true`.
- A `return` statement. In this case, execution of the generator ends, and the `next()` method returns an iterator result object where the `value` is the specified return value and `done` is `true`.
- A `throw` statement. In this case, execution of the generator halts entirely, and the `next()` method throws the specified exception.

Once paused on a `yield` expression, the generator's code execution remains paused until the generator's `next()` method is called again. If an optional value is passed to the generator's `next()` method, that value becomes the value returned by the generator's current `yield` operation. The first `next()` call does not have a corresponding suspended `yield` operation, so there's no way to get the argument passed to the first `next()` call.

If the generator's `return()` or `throw()` method is called, it acts as if a `return` or `throw` statement was executed at the paused `yield` expression. You can use `try...catch...finally` within the generator function body to handle these early completions. If the `return()` or `throw()` method is called but there's no suspended `yield` expression (because `next()` has not been called yet, or because the generator has already completed), then the early completions cannot be handled and always terminate the generator.

Examples

Using yield

The following code is the declaration of an example generator function.

```js
function* countAppleSales() {
  const saleList = [3, 7, 5];
  for (const sale of saleList) {
    yield sale;
  }
}
```

Once a generator function is defined, it can be used by constructing an iterator as shown.

```js
const appleStore = countAppleSales(); // Generator { }
console.log(appleStore.next()); // { value: 3, done: false }
console.log(appleStore.next()); // { value: 7, done: false }
console.log(appleStore.next()); // { value: 5, done: false }
console.log(appleStore.next()); // { value: undefined, done: true }
```

You can also send a value with `next(value)` into the generator. `step` evaluates as a return value of the `yield` expression — although the value passed to the generator's `next()` method the first time `next()` is called is ignored.

```js
function* counter(value) {
  while (true) {
    const step = yield value++;
```

(code continues on the next page)

(code continued from the previous page)

```
    if (step) {
      value += step;
    }
  }
}

const generatorFunc = counter(0);
console.log(generatorFunc.next().value); // 0
console.log(generatorFunc.next().value); // 1
console.log(generatorFunc.next().value); // 2
console.log(generatorFunc.next().value); // 3
console.log(generatorFunc.next(10).value); // 14
console.log(generatorFunc.next().value); // 15
console.log(generatorFunc.next(10).value); // 26
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#prod-YieldExpression`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`yield`	39	12	26[1,2]	26	10	39	26[3,4]	26	10	4.0	39	10	1.0.0	1.0	4.0.0

1. Starting with Firefox 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.
2. Starting with Firefox 29, an `IteratorResult` object returned for completed generator function.
3. Starting with Firefox for Android 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.
4. Starting with Firefox for Android 29, an `IteratorResult` object returned for completed generator function.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.yield`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Iteration protocols
- `function*`
- `function*` expression
- `yield*`

JavaScript › Reference › Expressions and operators

yield*

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/yield* · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since September 2016.

The **`yield*`** operator can be used within generator (sync or async) functions to delegate to another iterable object, such as a `Generator`. Inside async generator functions, it can additionally be used to delegate to another async iterable object, such as an `AsyncGenerator`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: yield* operator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function* func1() {
  yield 42;
}

function* func2() {
  yield* func1();
}

const iterator = func2();

console.log(iterator.next().value);
// Expected output: 42
```

Syntax

js

```
yield* expression
```

Parameters

`expression` Optional

An iterable object.

Return value

Returns the value returned by that iterator when it's closed (when `done` is `true`).

Description

The `yield*` expression iterates over the operand and yields each value returned by it. It delegates iteration of the current generator to an underlying iterator — which we will refer to as "generator" and "iterator", respectively. `yield*` first gets the iterator from the operand by calling the latter's `[Symbol.iterator]()` method. Then, each time the `next()` method of the generator is called, `yield*` calls the iterator's `next()` method, passing the argument received by the generator's `next()` method (always `undefined` for the first call), and yielding the same result object as what's returned from the iterator's `next()` method. If the iterator result has `done: true`, then the `yield*` expression stops executing and returns the `value` of that result.

The `yield*` operator forwards the current generator's `throw()` and `return()` methods to the underlying iterator as well. If the current generator is prematurely closed through one of these methods, the underlying iterator will be notified. If the generator's `throw()`/`return()` method is called, the `throw()`/`return()` method of the underlying iterator is called with the same argument. The return value of `throw()`/`return()` is handled like the `next()` method's result, and if the method throws, the exception is propagated from the `yield*` expression.

If the underlying iterator doesn't have a `return()` method, the `yield*` expression turns into a `return` statement, just like calling `return()` on a suspended `yield` expression.

If the underlying iterator doesn't have a `throw()` method, this causes `yield*` to throw a `TypeError` – but before throwing the error, the underlying iterator's `return()` method is called if one exists.

Examples

Delegating to another generator

In following code, values yielded by `g1()` are returned from `next()` calls just like those which are yielded by `g2()`.

js

```
function* g1() {
  yield 2;
  yield 3;
  yield 4;
}

function* g2() {
  yield 1;
  yield* g1();
  yield 5;
}

const gen = g2();

console.log(gen.next()); // {value: 1, done: false}
console.log(gen.next()); // {value: 2, done: false}
console.log(gen.next()); // {value: 3, done: false}
console.log(gen.next()); // {value: 4, done: false}
console.log(gen.next()); // {value: 5, done: false}
console.log(gen.next()); // {value: undefined, done: true}
```

Other Iterable objects

Besides generator objects, `yield*` can also `yield` other kinds of iterables (e.g., arrays, strings, or `arguments` objects).

js

```
function* g3(...args) {
  yield* [1, 2];
  yield* "34";
  yield* args;
}

const gen = g3(5, 6);
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(gen.next()); // {value: 1, done: false}
console.log(gen.next()); // {value: 2, done: false}
console.log(gen.next()); // {value: "3", done: false}
console.log(gen.next()); // {value: "4", done: false}
console.log(gen.next()); // {value: 5, done: false}
console.log(gen.next()); // {value: 6, done: false}
console.log(gen.next()); // {value: undefined, done: true}
```

The value of yield* expression itself

`yield*` is an expression, not a statement, so it evaluates to a value.

js

```
function* g4() {
  yield* [1, 2, 3];
  return "foo";
}

function* g5() {
  const g4ReturnValue = yield* g4();
  console.log(g4ReturnValue); // 'foo'
  return g4ReturnValue;
}

const gen = g5();

console.log(gen.next()); // {value: 1, done: false}
console.log(gen.next()); // {value: 2, done: false}
console.log(gen.next()); // {value: 3, done: false} done is false because g5 generator isn't
finished, only g4
console.log(gen.next()); // {value: 'foo', done: true}
```

Use with async generators

js

```
async function* g1() {
  await Promise.resolve(0);
  yield "foo";
}

function* g2() {
  yield "bar";
}

async function* g3() {
  // Can use yield* on both async and sync iterators
  yield* g1();
  yield* g2();
}

const gen = g3();

console.log(await gen.next()); // {value: "foo", done: false}
console.log(await gen.next()); // {value: "bar", done: false}
console.log(await gen.next()); // {done: true}
```

Method forwarding

The `next()`, `throw()`, and `return()` methods of the current generator are all forwarded to the underlying iterator.

```js
const iterable = {
  [Symbol.iterator]() {
    let count = 0;
    return {
      next(v) {
        console.log("next called with", v);
        count++;
        return { value: count, done: false };
      },
      return(v) {
        console.log("return called with", v);
        return { value: "iterable return value", done: true };
      },
      throw(v) {
        console.log("throw called with", v);
        return { value: "iterable thrown value", done: true };
      },
    };
  },
};

function* gf() {
  yield* iterable;
  return "gf return value";
}

const gen = gf();
console.log(gen.next(10));
// next called with undefined; the argument of the first next() call is always ignored
// { value: 1, done: false }
console.log(gen.next(20));
// next called with 20
// { value: 2, done: false }
console.log(gen.return(30));
// return called with 30
// { value: 'iterable return value', done: true }
console.log(gen.next(40));
// { value: undefined, done: true }; gen is already closed

const gen2 = gf();
console.log(gen2.next(10));
// next called with undefined
// { value: 1, done: false }
console.log(gen2.throw(50));
// throw called with 50
// { value: 'gf return value', done: true }
console.log(gen.next(60));
// { value: undefined, done: true }; gen is already closed
```

If the `return()`/`throw()` method of the underlying iterator returns `done: false`, the current generator continues executing and `yield*` continues to delegate to the underlying iterator.

```js
const iterable = {
  [Symbol.iterator]() {
    let count = 0;
    return {
      next(v) {
        console.log("next called with", v);
        count++;
        return { value: count, done: false };
      },
      return(v) {
        console.log("return called with", v);
        return { value: "iterable return value", done: false };
      },
    };
  },
};

function* gf() {
  yield* iterable;
  return "gf return value";
}

const gen = gf();
console.log(gen.next(10));
// next called with undefined
// { value: 1, done: false }
console.log(gen.return(20));
// return called with 20
// { value: 'iterable return value', done: false }
console.log(gen.next(30));
// { value: 2, done: false }; gen is not closed
```

If the underlying iterator doesn't have a `throw()` method and the generator's `throw()` is called, `yield*` throws an error.

```js
const iterable = {
  [Symbol.iterator]() {
    let count = 0;
    return {
      next(v) {
        count++;
        return { value: count, done: false };
      },
    };
  },
};

function* gf() {
  yield* iterable;
  return "gf return value";
}

const gen = gf();
gen.next(); // First next() starts the yield* expression
gen.throw(20); // TypeError: The iterator does not provide a 'throw' method.
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-generator-function-definitions-runtime-semantics-evaluation`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`yield*`	39	12	27[1]	26	10	39	27[2]	26	10	4.0	39	10	1.0.0	1.0	4.0.0

1. Starting with Firefox 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.
2. Starting with Firefox for Android 33, the parsing of the `yield` expression has been updated to conform with the ES2015 specification.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.operators.yield_star`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Iteration protocols
- `function*`
- `function* expression`
- `yield`

JavaScript › Reference

Statements and declarations

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements · MDN page last modified 2026-09-15

JavaScript code is built from statements and declarations, which can contain expressions. This reference groups the constructs used to control execution and declare bindings. A statement can span multiple lines, and a line can contain multiple statements.

Statements and declarations by category

For an alphabetical listing see the sidebar on the left.

Control flow

`return`
: Specifies the value to be returned by a function.

`break`
: Terminates the current loop, switch, or label statement and transfers program control to the statement following the terminated statement.

`continue`
: Terminates execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.

`throw`
: Throws a user-defined exception.

`if...else`
: Executes a statement if a specified condition is true. If the condition is false, another statement can be executed.

`switch`
: Evaluates an expression, matching the expression's value to a case clause, and executes statements associated with that case.

`try...catch`
: Marks a block of statements to try, and specifies a response, should an exception be thrown.

Declaring variables

`var`
: Declares a variable, optionally initializing it to a value.

`let`
: Declares a block-scoped variable, optionally initializing it to a value.

`const`
: Declares a block-scoped variable that cannot be re-assigned, which must be initialized at declaration.

`using`
: Declares a variable like `const` that is *synchronously disposed*.

`await using`
: Declares a variable like `const` that is *asynchronously disposed*.

Functions and classes

`function`

Declares a function with the specified parameters.

`function*`

Generator Functions enable writing iterators more easily.

`async function`

Declares an async function with the specified parameters.

`async function*`

Asynchronous Generator Functions enable writing async iterators more easily.

`class`

Declares a class.

Iterations

`do...while`

Creates a loop that executes a specified statement until the test condition evaluates to false. The condition is evaluated after executing the statement, resulting in the specified statement executing at least once.

`for`

Creates a loop that consists of three optional expressions, enclosed in parentheses and separated by semicolons, followed by a statement executed in the loop.

`for...in`

Iterates over the enumerable properties of an object, in arbitrary order. For each distinct property, statements can be executed.

`for...of`

Iterates over iterable objects (including arrays, array-like objects, iterators and generators), invoking a custom iteration hook with statements to be executed for the value of each distinct property.

`for await...of`

Iterates over async iterable objects, array-like objects, iterators and generators, invoking a custom iteration hook with statements to be executed for the value of each distinct property.

`while`

Creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated before executing the statement.

Others

Empty

An empty statement is used to provide no statement, although the JavaScript syntax would expect one.

Block

A block statement is used to group zero or more statements. The block is delimited by a pair of curly braces.

Expression statement

An expression statement evaluates an expression and discards its result. It allows the expression to perform side effects, such as executing a function or updating a variable.

`debugger`
Invokes any available debugging functionality. If no debugging functionality is available, this statement has no effect.

`export`
Used to export functions to make them available for imports in external modules, and other scripts.

`import`
Used to import functions exported from an external module, another script.

`import defer`
Loads a module as a namespace, deferring synchronous evaluation until the returned namespace's properties are accessed.

`import source`
Results in an object representing the module's compiled source, without loading its dependencies, linking it, or evaluating it.

label
Provides a statement with an identifier that you can refer to using a `break` or `continue` statement.

`with` Deprecated
Extends the scope chain for a statement.

What are statements, declarations, and expressions?

All JavaScript programs are composed of a sequence of top-level constructs, using one of the syntaxes listed above. These are known as *statements* and *declarations*. In MDN docs, we often colloquially refer to both as *statements*, but they are technically two disjoint sets of grammars.

The following are declarations:

- `let`
- `const`
- `using`
- `await using`
- `function`
- `function*`
- `async function`
- `async function*`
- `class`
- `export` (Note: it can only appear at the top-level of a module)
- `import` (Note: it can only appear at the top-level of a module)

Everything else in the list above is a statement.

The terms "statement" and "declaration" have a precise meaning in the formal syntax of JavaScript that affects where they may be placed in code. For example, in most control-flow structures, the body only accepts statements — such as the two arms of an `if...else`:

```js
if (condition)
  statement1;
else
  statement2;
```

If you use a declaration instead of a statement, it would be a `SyntaxError`. For example, a `let` declaration is not a statement, so you can't use it in its bare form as the body of an `if` statement.

js · ✗ bad example

```js
if (condition)
  let i = 0; // SyntaxError: Lexical declaration cannot appear in a single-statement context
```

On the other hand, `var` is a statement, so you can use it on its own as the `if` body.

js · ✓ good example

```js
if (condition)
  var i = 0;
```

You can see declarations as "binding identifiers to values", and statements as "carrying out actions". The fact that `var` is a statement instead of a declaration is a special case, because it doesn't follow normal lexical scoping rules and may create side effects — in the form of creating global variables, mutating existing `var`-defined variables, and defining variables that are visible outside of its block (because `var`-defined variables aren't block-scoped).

As another example, labels can only be attached to statements.

js · ✗ bad example

```js
label: const a = 1; // SyntaxError: Lexical declaration cannot appear in a single-statement context
```

> **Note:** There's a legacy grammar that allows function declarations to have labels, but it's only standardized for compatibility with web reality.

To get around this, you can wrap the declaration in braces — this makes it part of a block statement.

js · ✓ good example

```js
label: {
  const a = 1;
}

if (condition) {
  let i = 0;
}
```

In JavaScript, statements and declarations produce values, but these values are almost never observable (other than `eval()`). Their purpose is to manipulate the surrounding environment and produce side effects—creating variable bindings, outputting things, modifying variable values, etc. The values they use come from evaluating *expressions*.

Expressions are not top-level constructs; they can only be used in specific slots inside statements and declarations, such as `if (expression)`, `const x = expression`, etc. The expression statement syntax allows most expressions to be used as statements, but that's just a statement with a single expression slot.

You can map out JavaScript syntax structures using just statements, declarations, and expressions:

- Statements can contain statements, declarations, and expressions (like `if (expression) statement` and block statements `{ statement; declaration }`)
- Declarations can contain statements, declarations, and expressions (like `function x() { statement; declaration }` and `const x = expression`)
- Expressions can contain statements, declarations, and expressions (like `() => { statement; declaration }` and `console.log(expression)`)

In the reference documentation for each piece of syntax, we describe all its "slots" and say whether each slot is a statement, declaration, or expression.

Operators is another important concept in JavaScript grammar, but they do not work like building blocks. See What are operators? for more information.

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`async function` statement	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
`async function*` statement	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`await using` declaration	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0
`block`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`break`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`class`	49[1]	13	45	36[2]	10.1	49[1]	45	36[2]	10.3	5.0[3]	49[1]	10.3	1.0.0	1.0	6.0.0
`const`	21	12	36[4,5]	9	5.1	25	36[6,7]	10.1	5	1.5	4.4	5	1.0.0	1.0	6.0.0
`continue`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`debugger`	5	12	1	10	5	18	4	10.1	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
`do...while`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Empty statement (`;`)	3	12	1	3	5	18	4	10.1	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0
`export`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0[8]
Arbitrary module namespace identifier names	88	88	87	74	14.1	88	87	63	14.5	15.0	88	14.5	1.0.15	1.6	16.0.0
`default` keyword with `export`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0[8]
`export * as namespace`	72	79	80	60	14.1	72	80	51	14.5	11.0	72	14.5	1.0.0	1.0	13.2.0[8]
`for`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`for await...of`	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`for...in`	1	12	1	2	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`for...of`	38	12	13[9]	25	7	38	14[10]	25	7	3.0	38	7	1.0.0	1.0	0.12.0
async iterators	63	12	57	50	7	63	57	46	7	8.0	63	7	1.0.0	1.0	10.0.0
Closing iterators	51	14	53	38	7	51	53	41	7	5.0	51	7	1.0.0	1.0	6.5.0
`function` statement	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
`function*` statement	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`IteratorResult` object instead of throwing	49	13	29	36	10	49	29	36	10	5.0	49	10	1.0.0	1.0	6.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Not constructable with `new` (ES2016)	50	13	43	37	10	50	43	37	10	5.0	50	10	1.0.0	1.0	6.0.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0
`if...else`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`import`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0[8,11]
Arbitrary module namespace identifier names	88	88	87	74	14.1	88	87	63	14.5	15.0	88	14.5	1.0.15	1.6	16.0.0
`import defer` Experimental	**No**	**No**	Prev.[1][2]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?
Loading is retriable (HTTP errors are not cached)	156	156	155	140	Prev.	156	155	**No**	**No**	**No**	156	**No**	?	?	?
Import attributes with `assert` syntax (formerly import assertions) Deprecated Non-standard	**91–126**	**91–126**	**No**	**No**	**No**	**91–126**	**No**	**No**	**No**	**16.0–28.0**	**91–126**	**No**	1.0.0	1.17	**16.14.0–22.0.0**
`assert {type: 'css'}` Deprecated Non-standard	**93–126**	**93–126**	**No**	**No**	**No**	**93–126**	**No**	**No**	**No**	**17.0–28.0**	**93–126**	**No**	1.0.0	**No**	**No**
`assert {type: 'json'}` Deprecated Non-standard	**91–126**	**91–126**	**No**	**No**	**No**	**91–126**	**No**	**No**	**No**	**16.0–28.0**	**91–126**	**No**	1.0.0	1.17	**17.5.0–22.0.0**[13]
Import attributes (`with` syntax)	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[14]
`with {type: 'css'}`	123	123	147	109	**No**	123	147	82	**No**	27.0	123	**No**	1.0.0	2.9	**No**
`with {type: 'json'}`	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[14]
`with {type: 'text'}`	155	155	153	139	**No**	155	153	**No**	**No**	**No**	155	**No**	1.1.5	2.8	**No**
`import source` Experimental	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Source phase import of WASM modules Experimental	**No**	**No**	Prev.[1][5]	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Available in service workers	91	91	147	77	15	91	147	64	15	16.0	91	15	1.0.0	**No**	**No**
Available in workers	80	80	114	67	15	80	114	57	15	13.0	80	15	1.0.0	1.0	**No**
Available in worklets	**No**	**No**	114	**No**	**No**	**No**	114	**No**	**No**	**No**	**No**	**No**	1.0.0	**No**	**No**
`label`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`let`	49[16,17]	14[18,19]	44[20,21,22]	17	10	49[16,17]	44[23,24,25]	18	10	5.0[26,27]	49[16,17]	10	1.0.0	1.0	6.0.0
`return`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`switch`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`throw`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`try...catch`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Optional catch binding	66	79	58	53	11.1	66	58	47	11.3	9.0	66	11.3	1.0.0	1.0	10.0.0
`using` declaration	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	1.37	24.0.0
`var`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`while`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`with` Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.13	1.0	0.10.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.

3. Also from version 4.0–5.0: Strict mode is required.
4. Before Firefox 13, `const` is implemented, but re-assignment is not failing.
5. Before Firefox 46, a `TypeError` was thrown on redeclaration instead of a `SyntaxError`.
6. Before Firefox for Android 14, `const` is implemented, but re-assignment is not failing.
7. Before Firefox for Android 46, a `TypeError` was thrown on redeclaration instead of a `SyntaxError`.
8. Modules must either have a filename ending in `.mjs`, or the nearest parent `package.json` file must contain `"type": "module"`. See Node's ECMAScript Modules documentation for more details.
9. Before Firefox 51, using the `for...of` loop construct with the `const` keyword threw a `SyntaxError` ("missing = in const declaration").
10. Before Firefox for Android 51, using the `for...of` loop construct with the `const` keyword threw a `SyntaxError` ("missing = in const declaration").
11. Also from version 12.17.0–13.0.0: Modules must either have a filename ending in `.mjs`, or the nearest parent `package.json` file must contain `"type": "module"`. See Node's ECMAScript Modules documentation for more details.
12. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).
13. Also from version 16.15.0–17.0.0: supported.
14. Also from version 18.20.0–19.0.0: supported.
15. Disabled by default; the user must enable the preference `javascript.options.experimental.wasm_esm_integration` (set to `true`).
16. Also from version 41–49: partial support.
17. Also from version 41–49: Strict mode is required.
18. Also from version 12–14: partial support.
19. Also from version 12–14: In Edge 12 and 13, `let` within a `for` loop initializer does not create a separate variable for each loop iteration as defined by ES2015. Instead, it behaves as though the loop were wrapped in a scoping block with the `let` immediately before the loop.
20. Before Firefox 44, `let` is only available to code blocks in HTML wrapped in a `<script type="application/javascript;version=1.7">` block (or higher version) and has different semantics (e.g. no temporal dead zone).
21. Before Firefox 46, a `TypeError` is thrown on redeclaration instead of a `SyntaxError`.
22. Firefox 54 adds support of `let` in workers.
23. Before Firefox for Android 44, `let` is only available to code blocks in HTML wrapped in a `<script type="application/javascript;version=1.7">` block (or higher version) and has different semantics (e.g. no temporal dead zone).
24. Before Firefox for Android 46, a `TypeError` is thrown on redeclaration instead of a `SyntaxError`.
25. Firefox for Android 54 adds support of `let` in workers.
26. Also from version 4.0–5.0: partial support.
27. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Expressions and operators

JavaScript › Reference › Statements and declarations

async function

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since April 2017.

The **`async function`** declaration creates a binding of a new async function to a given name. The `await` keyword is permitted within the function body, enabling asynchronous, promise-based behavior to be written in a cleaner style and avoiding the need to explicitly configure promise chains.

You can also define async functions using the `async function` expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: async function declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function resolveAfter2Seconds() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("resolved");
    }, 2000);
  });
}

async function asyncCall() {
  console.log("calling");
  const result = await resolveAfter2Seconds();
  console.log(result);
  // Expected output: "resolved"
}

asyncCall();
```

Syntax

js

```
async function name(param0) {
  statements
}
async function name(param0, param1) {
  statements
}
async function name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** There cannot be a line terminator between `async` and `function`, otherwise a semicolon is automatically inserted, causing `async` to become an identifier and the rest to become a `function` declaration.

Parameters

`name`

The function's name.

`param` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements comprising the body of the function. The `await` mechanism may be used.

Description

An `async function` declaration creates an `AsyncFunction` object. Each time when an async function is called, it returns a new `Promise` which will be resolved with the value returned by the async function, or rejected with an exception uncaught within the async function.

Async functions can contain zero or more `await` expressions. Await expressions make promise-returning functions behave as though they're synchronous by suspending execution until the returned promise is fulfilled or rejected. The resolved value of the promise is treated as the return value of the await expression. Use of `async` and `await` enables the use of ordinary `try / catch` blocks around asynchronous code.

> **Note:** The `await` keyword is only valid inside async functions within regular JavaScript code. If you use it outside of an async function's body, you will get a `SyntaxError`.
>
> `await` can be used on its own with JavaScript modules.

> **Note:** The purpose of `async/await` is to simplify the syntax necessary to consume promise-based APIs. The behavior of `async/await` is similar to combining generators and promises.

Async functions always return a promise. If the return value of an async function is not explicitly a promise, it will be implicitly wrapped in a promise.

For example, consider the following code:

```js
async function foo() {
  return 1;
}
```

It is similar to:

```js
function foo() {
  return Promise.resolve(1);
}
```

Note that even though the return value of an async function behaves as if it's wrapped in a `Promise.resolve`, they are not equivalent. An async function will return a different *reference*, whereas `Promise.resolve` returns the same reference if the given value is a promise. It can be a problem when you want to check the equality of a promise and a return value of an async function.

```js
const p = new Promise((res, rej) => {
  res(1);
});

async function asyncReturn() {
  return p;
}

function basicReturn() {
  return Promise.resolve(p);
}

console.log(p === basicReturn()); // true
console.log(p === asyncReturn()); // false
```

The body of an async function can be thought of as being split by zero or more await expressions. Top-level code, up to and including the first await expression (if there is one), is run synchronously. In this way, an async function without an await expression will run synchronously. If there is an await expression inside the function body, however, the async function will always complete asynchronously.

For example:

```js
async function foo() {
  await 1;
}
```

It is also equivalent to:

```js
function foo() {
  return Promise.resolve(1).then(() => undefined);
}
```

Code after each await expression can be thought of as existing in a `.then` callback. In this way a promise chain is progressively constructed with each reentrant step through the function. The return value forms the final link in the chain.

In the following example, we successively await two promises. Progress moves through function `foo` in three stages.

1. The first line of the body of function `foo` is executed synchronously, with the await expression configured with the pending promise. Progress through `foo` is then suspended and control is yielded back to the function that called `foo`.
2. Some time later, when the first promise has either been fulfilled or rejected, control moves back into `foo`. The result of the first promise fulfillment (if it was not rejected) is returned from the await expression. Here `1` is assigned to `result1`. Progress continues, and the second await expression is evaluated. Again, progress through `foo` is suspended and control is yielded.
3. Some time later, when the second promise has either been fulfilled or rejected, control re-enters `foo`. The result of the second promise resolution is returned from the second await expression. Here `2` is assigned to `result2`. Control moves to the return expression (if any). The default return value of `undefined` is returned as the resolution value of the current promise.

```js
async function foo() {
  const result1 = await new Promise((resolve) =>
    setTimeout(() => resolve("1")),
  );
  const result2 = await new Promise((resolve) =>
    setTimeout(() => resolve("2")),
  );
}
foo();
```

Note how the promise chain is not built-up in one go. Instead, the promise chain is constructed in stages as control is successively yielded from and returned to the async function. As a result, we must be mindful of error handling behavior when dealing with concurrent asynchronous operations.

For example, in the following code an unhandled promise rejection error will be thrown, even if a `.catch` handler has been configured further along the promise chain. This is because `p2` will not be "wired into" the promise chain until control returns from `p1`.

```js
async function foo() {
  const p1 = new Promise((resolve) => setTimeout(() => resolve("1"), 1000));
  const p2 = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("failed")), 500),
  );
  const results = [await p1, await p2]; // Do not do this! Use Promise.all or
Promise.allSettled instead.
}
foo().catch(() => {}); // Attempt to swallow all errors...
```

`async function` declarations behave similar to `function` declarations — they are hoisted to the top of their scope and can be called anywhere in their scope, and they can be redeclared only in certain contexts.

Examples

Async functions and execution order

```js
function resolveAfter2Seconds() {
  console.log("starting slow promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("slow");
      console.log("slow promise is done");
    }, 2000);
  });
}
```

(code continues on the next page)

(code continued from the previous page)

```
function resolveAfter1Second() {
  console.log("starting fast promise");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("fast");
      console.log("fast promise is done");
    }, 1000);
  });
}

async function sequentialStart() {
  console.log("== sequentialStart starts ==");

  // 1. Start a timer, log after it's done
  const slow = resolveAfter2Seconds();
  console.log(await slow);

  // 2. Start the next timer after waiting for the previous one
  const fast = resolveAfter1Second();
  console.log(await fast);

  console.log("== sequentialStart done ==");
}

async function sequentialWait() {
  console.log("== sequentialWait starts ==");

  // 1. Start two timers without waiting for each other
  const slow = resolveAfter2Seconds();
  const fast = resolveAfter1Second();

  // 2. Wait for the slow timer to complete, and then log the result
  console.log(await slow);
  // 3. Wait for the fast timer to complete, and then log the result
  console.log(await fast);

  console.log("== sequentialWait done ==");
}

async function concurrent1() {
  console.log("== concurrent1 starts ==");

  // 1. Start two timers concurrently and wait for both to complete
  const results = await Promise.all([
    resolveAfter2Seconds(),
    resolveAfter1Second(),
  ]);
  // 2. Log the results together
  console.log(results[0]);
  console.log(results[1]);

  console.log("== concurrent1 done ==");
}

async function concurrent2() {
  console.log("== concurrent2 starts ==");
```

(code continues on the next page)

(code continued from the previous page)

```
  // 1. Start two timers concurrently, log immediately after each one is done
  await Promise.all([
    (async () => console.log(await resolveAfter2Seconds()))(),
    (async () => console.log(await resolveAfter1Second()))(),
  ]);
  console.log("== concurrent2 done ==");
}

sequentialStart(); // after 2 seconds, logs "slow", then after 1 more second, "fast"

// wait above to finish
setTimeout(sequentialWait, 4000); // after 2 seconds, logs "slow" and then "fast"

// wait again
setTimeout(concurrent1, 7000); // same as sequentialWait

// wait again
setTimeout(concurrent2, 10000); // after 1 second, logs "fast", then after 1 more second,
"slow"
```

await and concurrency

In `sequentialStart`, execution suspends 2 seconds for the first `await`, and then another second for the second `await`. The second timer is not created until the first has already fired, so the code finishes after 3 seconds.

In `sequentialWait`, both timers are created and then `await`ed. The timers run concurrently, which means the code finishes in 2 rather than 3 seconds, i.e., the slowest timer. However, the `await` calls still run in series, which means the second `await` will wait for the first one to finish. In this case, the result of the fastest timer is processed after the slowest.

If you wish to safely perform other jobs after two or more jobs run concurrently and are complete, you must await a call to `Promise.all()` or `Promise.allSettled()` before that job.

> **Warning:** The functions `sequentialWait` and `concurrent1` are not functionally equivalent.
>
> In `sequentialWait`, if promise `fast` rejects before promise `slow` is fulfilled, then an unhandled promise rejection error will be raised, regardless of whether the caller has configured a catch clause.
>
> In `concurrent1`, `Promise.all` wires up the promise chain in one go, meaning that the operation will fail-fast regardless of the order of rejection of the promises, and the error will always occur within the configured promise chain, enabling it to be caught in the normal way.

Rewriting a Promise chain with an async function

An API that returns a `Promise` will result in a promise chain, and it splits the function into many parts. Consider the following code:

```
function getProcessedData(url) {
  return downloadData(url) // returns a promise
    .catch((e) => downloadFallbackData(url)) // returns a promise
    .then((v) => processDataInWorker(v)); // returns a promise
}
```

js

it can be rewritten with a single async function as follows:

```js
async function getProcessedData(url) {
  let v;
  try {
    v = await downloadData(url);
  } catch (e) {
    v = await downloadFallbackData(url);
  }
  return processDataInWorker(v);
}
```

Alternatively, you can chain the promise with `catch()`:

```js
async function getProcessedData(url) {
  const v = await downloadData(url).catch((e) => downloadFallbackData(url));
  return processDataInWorker(v);
}
```

In the two rewritten versions, notice there is no `await` statement after the `return` keyword, although that would be valid too: The return value of an async function is implicitly wrapped in `Promise.resolve` - if it's not already a promise itself (as in the examples).

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`async function` statement	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.async_function`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Functions guide
- Using promises guide
- Functions
- `AsyncFunction`
- `async function` expression
- `function`
- `function*`
- `async function*`

- `await`
- `Promise`
- Decorating async JavaScript functions on innolitics.com (2016)

JavaScript › Reference › Statements and declarations

async function*

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function* · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since January 2020.

The **`async function*`** declaration creates a binding of a new async generator function to a given name.

You can also define async generator functions using the `async function*` expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: async function* declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
async function* foo() {
  yield await Promise.resolve("a");
  yield await Promise.resolve("b");
  yield await Promise.resolve("c");
}

let str = "";

async function generate() {
  for await (const val of foo()) {
    str += val;
  }
  console.log(str);
}

generate();
// Expected output: "abc"
```

Syntax

js

```
async function* name(param0) {
  statements
}
async function* name(param0, param1) {
  statements
}
async function* name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** Async generator functions do not have arrow function counterparts.

> **Note:** `function` and `*` are separate tokens, so they can be separated by whitespace or line terminators. However, there cannot be a line terminator between `async` and `function`, otherwise a semicolon is automatically inserted, causing `async` to become an identifier and the rest to become a `function*` declaration.

Parameters

`name`

The function name.

`param` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements comprising the body of the function.

Description

An `async function*` declaration creates an `AsyncGeneratorFunction` object. Each time when an async generator function is called, it returns a new `AsyncGenerator` object, which conforms to the async iterator protocol. Every call to `next()` returns a `Promise` that resolves to the iterator result object.

An async generator function combines the features of async functions and generator functions. You can use both the `await` and `yield` keywords within the function body. This empowers you to handle asynchronous tasks ergonomically with `await`, while leveraging the lazy nature of generator functions.

When a promise is yielded from an async generator, the iterator result promise's eventual state will match that of the yielded promise. For example:

```js
async function* foo() {
  yield Promise.reject(new Error("failed"));
}

foo()
  .next()
  .catch((e) => console.error(e));
```

`Error: failed` will be logged, because if the yielded promise rejects, the iterator result will reject as well. The `value` property of an async generator's resolved result will not be another promise.

`async function*` declarations behave similar to `function` declarations — they are hoisted to the top of their scope and can be called anywhere in their scope, and they can be redeclared only in certain contexts.

Examples

Declaring an async generator function

Async generator functions always produce promises of results — even when each `yield` step is synchronous.

```js
async function* myGenerator(step) {
  await new Promise((resolve) => setTimeout(resolve, 10));
  yield 0;
  yield step;
  yield step * 2;
}

const gen = myGenerator(2);
gen
  .next()
  .then((res) => {
    console.log(res); // { value: 0, done: false }
    return gen.next();
  })
  .then((res) => {
    console.log(res); // { value: 2, done: false }
    return gen.next();
  })
  .then((res) => {
    console.log(res); // { value: 4, done: false }
    return gen.next();
  })
  .then((res) => {
    console.log(res); // { value: undefined, done: true }
    return gen.next();
  });
```

Using an async generator function to read a series of files

In this example, we read a series of files and only access its content when requested, using Node's `fs/promises` module.

```js
async function* readFiles(directory) {
  const files = await fs.readdir(directory);
  for (const file of files) {
    const stats = await fs.stat(file);
    if (stats.isFile()) {
      yield {
        name: file,
        content: await fs.readFile(file, "utf8"),
      };
    }
  }
}

const files = readFiles(".");
console.log((await files.next()).value);
// Possible output: { name: 'file1.txt', content: '...' }
console.log((await files.next()).value);
// Possible output: { name: 'file2.txt', content: '...' }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-generator-function-definitions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`async function*` statement	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.async_generator_function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Iterators and generators guide
- Functions
- `AsyncGeneratorFunction`
- `async function*` expression
- `function`
- `function*`
- `async function`
- Iteration protocols
- `yield`
- `yield*`
- `AsyncGenerator`

JavaScript › Reference › Statements and declarations

await using

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/await_using · MDN page last modified 2025-11-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`await using`** declaration declares block-scoped local variables that are *asynchronously disposed*. Like `const`, variables declared with `await using` must be initialized and cannot be reassigned. The variable's value must be either `null`, `undefined`, or an object with a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method. When the variable goes out of scope, the `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method of the object is called and awaited, to ensure that resources are freed.

Syntax

```js
await using name1 = value1;
await using name1 = value1, name2 = value2;
await using name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

nameN

The name of the variable to declare. Each must be a legal JavaScript identifier and *not* a destructuring binding pattern.

valueN

Initial value of the variable. It can be any legal expression but its value must be either `null`, `undefined`, or an object with a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method.

Description

This declaration can only be used in places where both `await` and `using` can be used, which include:

- Inside a block (if the block is also in an async context)
- Inside an async function or async generator function body
- At the top level of a module
- In the initializer of a `for`, `for...of` (if the `for` loop is also in an async context), or `for await...of` loop

An `await using` declares an async disposable resource that's tied to the lifetime of the variable's scope (block, function, module, etc.). When the scope exits, the resource is disposed of asynchronously. Its syntax may be somewhat confusing, because the `await` does not have an awaiting effect when the variable is first declared, but only when the variable goes out of scope.

When a variable is first declared and its value is non-nullish, a *disposer* is retrieved from the object. The `[Symbol.asyncDispose]` property is tried first, and falls back to `[Symbol.dispose]` if `[Symbol.asyncDispose]` is `undefined`. If neither property contains a function, a `TypeError` is thrown. Notably, the `[Symbol.dispose]()` method is wrapped into a function that looks like `async () => { object[Symbol.dispose](); }`, which means if it returns a promise, that promise is *not* awaited. This disposer is saved to the scope.

When the variable goes out of scope, the disposer is called and awaited. If the scope contains multiple `using` or `await using` declarations, all disposers are run in sequence in the reverse order of declaration, regardless of the type of declaration. All disposers are guaranteed to run (much like the `finally` block in `try...catch...finally`). All errors thrown during disposal, including the initial error that caused the scope exit (if applicable), are all aggregated inside one `SuppressedError`, with each earlier exception as the `suppressed` property and the later exception as the `error` property. This `SuppressedError` is thrown after disposal is complete.

The variable is allowed to have value `null` or `undefined`, so the resource can be optionally present. As long as one `await using` variable is declared in this scope, at least one `await` is guaranteed to happen on scope exit, even if the variable actually has value `null` or `undefined`. This prevents the disposal from happening synchronously, causing timing issues (see control flow effects of `await`).

`await using` ties resource management to lexical scopes, which is both convenient and sometimes confusing. See below for some examples where it may not behave how you expect. If you want to hand-

manage resource disposal, while maintaining the same error handling guarantees, you can use `AsyncDisposableStack` instead.

Examples

You should also check `using` for more examples, especially some general caveats with respect to scope-based resource management.

Basic usage

Usually, you use `await using` on some library-provided resource that already implements the async disposable protocol. For example, the Node.js `FileHandle` is async disposable:

```js
import fs from "node:fs/promises";

async function example() {
  await using file = await fs.open("example.txt", "r");
  console.log(await file.read());
  // Before `file` goes out of scope, it is disposed by calling `file[Symbol.asyncDispose]
()` and awaited.
}
```

Note that there are two `await` operations in the declaration for `file`, which do different things and are both necessary. `await fs.open()` causes an await during *acquisition*: it waits for the file to be opened and unwraps the returned promise into a `FileHandle` object. `await using file` causes an await during *disposal*: it makes `file` disposed asynchronously when the variable goes out of scope.

`await using` with `for await...of`

It's very easy to confuse the following three syntaxes:

- `for await (using x of y) { ... }`
- `for (await using x of y) { ... }`
- `for (using x of await y) { ... }`

It may be even more confusing to know that they can be used together.

```js
for await (await using x of await y) {
  // ...
}
```

First, `await y` does what you expect: we `await` the promise `y`, which is expected to resolve to an object we iterate over. Let us set this variant aside.

The `for await...of` loop requires the `y` object to be an *async iterable*. This means that the object must have a `[Symbol.asyncIterator]` method that returns an *async iterator*, whose `next()` method returns a promise representing the result. This is for when the iterable doesn't know what the next value is, or even if it's done yet, until some async operation is complete.

On the other hand, the `await using x` syntax requires the `x` object, as yielded from the iterable, to be an *async disposable*. This means that the object must have a `[Symbol.asyncDispose]` method that returns a promise representing the disposal operation. This is a separate concern from the iteration itself, and is only called when the variable `x` goes out of scope.

In other words, all of the following four combinations are valid and do different things:

- `for (using x of y)`: `y` is synchronously iterated, yielding one result at a time, which can be disposed synchronously.
- `for await (using x of y)`: `y` is asynchronously iterated, yielding one result at a time after awaiting, but the result value can be disposed synchronously.
- `for (await using x of y)`: `y` is synchronously iterated, yielding one result at a time, but the result value can only be disposed asynchronously.
- `for await (await using x of y)`: `y` is asynchronously iterated, yielding one result at a time after awaiting, and the result value can only be disposed asynchronously.

Below, we create some fictitious values of `y` to demonstrate their use cases. For asynchronous APIs, we base our code on the Node.js `fs/promises` module.

```js
const syncIterableOfSyncDisposables = [
  stream1.getReader(),
  stream2.getReader(),
];
for (using reader of syncIterableOfSyncDisposables) {
  console.log(reader.read());
}
```

```js
async function* requestMany(urls) {
  for (const url of urls) {
    const res = await fetch(url);
    yield res.body.getReader();
  }
}
const asyncIterableOfSyncDisposables = requestMany([
  "https://example.com",
  "https://example.org",
]);
for await (using reader of asyncIterableOfSyncDisposables) {
  console.log(reader.read());
}
```

```js
const syncIterableOfAsyncDisposables = await Promise.all(
  fs.globSync("*.txt").map((path) => fs.open(path, "r")),
);
for (await using file of syncIterableOfAsyncDisposables) {
  console.log(await file.read());
}
```

```js
async function* globHandles(pattern) {
  for await (const path of fs.glob(pattern)) {
    yield await fs.open(path, "r");
  }
}
const asyncIterableOfAsyncDisposables = globHandles("*.txt");
for await (await using file of asyncIterableOfAsyncDisposables) {
  console.log(await file.read());
}
```

Implicit await on scope exit

As soon as one `await using` is declared in a scope, the scope will always have an `await` on exit, even if the variable is `null` or `undefined`. This ensures stable execution order and error handling. The Control flow effects of await examples have more details on this.

In the example below, the `example()` call below doesn't resolve until one tick after, because of an implicit `await` when the function returns.

js

```
async function example() {
  await using nothing = null;
  console.log("Example call");
}

example().then(() => console.log("Example done"));
Promise.resolve().then(() => console.log("Microtask done"));
// Output:
// Example call
// Microtask done
// Example done
```

Consider the same code but with a synchronous `using` instead. This time, the `example()` call immediately resolves, so the two `then()` handlers are called in the same tick.

js

```
async function example() {
  using nothing = null;
  console.log("Example call");
}

example().then(() => console.log("Example done"));
Promise.resolve().then(() => console.log("Microtask done"));
// Output:
// Example call
// Example done
// Microtask done
```

For a more realistic example, consider two concurrent calls to a function:

js

```
class Resource {
  #name;
  constructor(name) {
    this.#name = name;
  }
  async [Symbol.asyncDispose]() {
    console.log(`Disposing resource ${this.#name}`);
  }
}

async function example(id, createOptionalResource) {
  await using required = new Resource(`required ${id}`);
  await using optional = createOptionalResource
    ? new Resource("optional")
    : null;
  await using another = new Resource(`another ${id}`);
}
```

(code continues on the next page)

(code continued from the previous page)

```
example(1, true);
example(2, false);
// Output:
// Disposing resource another 1
// Disposing resource another 2
// Disposing resource optional
// Disposing resource required 1
// Disposing resource required 2
```

As you can see, the `required 2` resource is disposed in the same tick as `required 1`. If the `optional` resource did not cause a redundant `await`, then `required 2` would have been disposed earlier, which would be simultaneous with `optional`.

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-let-and-const-declarations`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`await using` declaration	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.await_using`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript resource management
- `const`
- `let`
- `using`
- `Symbol.asyncDispose`
- `AsyncDisposableStack`

JavaScript › Reference › Statements and declarations

Block statement

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/block · MDN page last modified 2025-07-29

Baseline: Widely available — available across major browsers since July 2015.

A **block statement** is used to group zero or more statements. The block is delimited by a pair of braces ("curly braces") and contains a list of zero or more statements and declarations.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Block statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
var x = 1;
let y = 1;

if (true) {
  var x = 2;
  let y = 2;
}

console.log(x);
// Expected output: 2

console.log(y);
// Expected output: 1
```

Syntax

js

```
{
  StatementList
}
```

`StatementList`

Statements and declarations grouped within the block statement.

Description

The block statement is often called the *compound statement* in other languages. It allows you to use multiple statements where JavaScript expects only one statement. Combining statements into blocks is a common practice in JavaScript, especially when used in association with control flow statements like `if...else` and `for`. The opposite behavior is possible using an empty statement, where you provide no statement, although one is required.

In addition, combined with block-scoped declarations like `let`, `const`, and `class`, blocks can prevent temporary variables from polluting the global namespace, just like IIFEs do.

Block scoping rules with var or function declaration in non-strict mode

Variables declared with `var` or created by function declarations in non-strict mode **do not** have block scope. Variables introduced within a block are scoped to the containing function or script, and the effects of setting them persist beyond the block itself. For example:

js

```
var x = 1;
{
  var x = 2;
}
console.log(x); // 2
```

This logs 2 because the `var x` statement within the block is in the same scope as the `var x` statement before the block.

In non-strict code, function declarations inside blocks behave strangely. Do not use them.

Block scoping rules with let, const, class, or function declaration in strict mode

By contrast, identifiers declared with `let`, `const`, and `class` do have block scope:

```js
let x = 1;
{
  let x = 2;
}
console.log(x); // 1
```

The `x = 2` is limited in scope to the block in which it was defined.

The same is true of `const`:

```js
const c = 1;
{
  const c = 2;
}
console.log(c); // 1; does not throw SyntaxError
```

Note that the block-scoped `const c = 2` *does not* throw a `SyntaxError: Identifier 'c' has already been declared` because it can be declared uniquely within the block.

In strict mode, function declarations inside blocks are scoped to that block and are hoisted to the top of the block.

```js
"use strict";

{
  foo(); // Logs "foo"
  function foo() {
    console.log("foo");
  }
}

foo(); // ReferenceError: foo is not defined
```

Examples

Using a block statement as the body of a for loop

A `for` loop accepts a single statement as its body.

```js
for (let i = 0; i < 10; i++) console.log(i);
```

If you want to use more than one statement in the loop body, you can group them into one block statement:

```js
for (let i = 0; i < 10; i++) {
  console.log(i);
  console.log(i ** 2);
}
```

Using a block statement to encapsulate data

`let` and `const` declarations are scoped to the containing block. This allows you to hide data from the global scope without wrapping it in a function.

```js
let sector;
{
  // These variables are scoped to this block and are not
  // accessible after the block
  const angle = Math.PI / 3;
  const radius = 10;
  sector = {
    radius,
    angle,
    area: (angle / 2) * radius ** 2,
    perimeter: 2 * radius + angle * radius,
  };
}
console.log(sector);
// {
//   radius: 10,
//   angle: 1.0471975511965976,
//   area: 52.35987755982988,
//   perimeter: 30.471975511965976
// }
console.log(typeof radius); // "undefined"
```

`using` declarations in a block

You can declare variables with `using` or `await using` in a block, which causes the object stored in the variable to be disposed when control exits the block. For more information, see resource management.

```js
{
  using reader1 = stream1.getReader();
  using reader2 = stream2.getReader();

  // do something with reader1 and reader2

  // Before we exit the block, reader1 and reader2 are automatically released
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-block

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`block`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.block`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `while`
- `if...else`
- `let`

JavaScript › Reference › Statements and declarations

break

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/break · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`break`** statement terminates the current loop or `switch` statement and transfers program control to the statement following the terminated statement. It can also be used to jump past a labeled statement when used within that labeled statement.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: break statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let i = 0;

while (i < 6) {
  if (i === 3) {
    break;
  }
  i += 1;
}

console.log(i);
// Expected output: 3
```

Syntax

js

```
break;
break label;
```

`label` Optional

Identifier associated with the label of the statement to break to. If the `break` statement is not nested within a loop or `switch`, then the label identifier is required.

Description

When `break;` is encountered, the program breaks out of the innermost `switch` or looping statement and continues executing the next statement after that.

When `break label;` is encountered, the program breaks out of the statement labeled with `label` and continues executing the next statement after that. The `break` statement needs to be nested within the referenced label. The labeled statement can be any statement (commonly a block statement); it does not have to be another loop statement.

A `break` statement, with or without a following label, cannot be used at the top level of a script, module, function's body, or static initialization block, even when the function or class is further contained within a loop.

Examples

break in while loop

The following function has a `break` statement that terminates the `while` loop when `i` is 3, and then returns the value `3 * x`.

```js
function testBreak(x) {
  let i = 0;

  while (i < 6) {
    if (i === 3) {
      break;
    }
    i += 1;
  }

  return i * x;
}
```

break in switch statements

The following code has a `break` statement that terminates the `switch` statement when a case is matched and the corresponding code has run.

```js
const food = "sushi";

switch (food) {
  case "sushi":
    console.log("Sushi is originally from Japan.");
    break;
  case "pizza":
    console.log("Pizza is originally from Italy.");
    break;
  default:
    console.log("I have never heard of that dish.");
    break;
}
```

break in labeled blocks

The following code uses `break` statements with labeled blocks. By using `break outerBlock`, control is transferred to the end of the block statement marked as `outerBlock`.

```js
outerBlock: {
  innerBlock: {
    console.log("1");
    break outerBlock; // breaks out of both innerBlock and outerBlock
    console.log(":-("); // skipped
  }
  console.log("2"); // skipped
}
```

Unsyntactic break statements

A `break` statement must be nested within any label it references. The following code also uses `break` statements with labeled blocks, but generates a syntax error because its `break` statement references `block2` but it's not nested within `block2`.

js · ✗ bad example

```
block1: {
  console.log("1");
  break block2; // SyntaxError: label not found
}

block2: {
  console.log("2");
}
```

Syntax errors are also generated in the following code examples which use `break` statements within functions that are nested within a loop, or labeled block that the `break` statements are intended to break out of.

js · ✗ bad example

```
function testBreak(x) {
  let i = 0;
```

(code continues on the next page)

(code continued from the previous page)

```
  while (i < 6) {
    if (i === 3) {
      (() => {
        break;
      })();
    }
    i += 1;
  }

  return i * x;
}

testBreak(1); // SyntaxError: Illegal break statement
```

js · ✗ bad example

```
block1: {
  console.log("1");
  (() => {
    break block1; // SyntaxError: Undefined label 'block1'
  })();
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-break-statement`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`break`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.break`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `continue`
- label
- `switch`

JavaScript › Reference › Statements and declarations

class

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/class · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since March 2017.

The **`class`** declaration creates a binding of a new class to a given name.

You can also define classes using the `class` expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: class declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
class Polygon {
  constructor(height, width) {
    this.area = height * width;
  }
}

console.log(new Polygon(4, 3).area);
// Expected output: 12
```

Syntax

js

```
class name {
  // class body
}
class name extends otherName {
  // class body
}
```

Description

The class body of a class declaration is executed in strict mode. The `class` declaration is very similar to `let`:

- `class` declarations are scoped to blocks as well as functions.
- `class` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `class` declarations are commonly regarded as non-hoisted (unlike function declarations).
- `class` declarations do not create properties on `globalThis` when declared at the top level of a script (unlike function declarations).
- `class` declarations cannot be redeclared by any other declaration in the same scope.

Outside the class body, `class` declarations can be re-assigned like `let`, but you should avoid doing so. Within the class body, the binding is constant like `const`.

```js
class Foo {
  static {
    Foo = 1; // TypeError: Assignment to constant variable.
  }
}

class Foo2 {
  bar = (Foo2 = 1); // TypeError: Assignment to constant variable.
}

class Foo3 {}
Foo3 = 1;
console.log(Foo3); // 1
```

Examples

A class declaration

In the following example, we first define a class named `Rectangle`, then extend it to create a class named `FilledRectangle`.

Note that `super()`, used in the `constructor`, can only be used in constructors, and *must* be called *before* the `this` keyword can be used.

```js
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
}

class FilledRectangle extends Rectangle {
  constructor(height, width, color) {
    super(height, width);
    this.name = "Filled rectangle";
    this.color = color;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-class-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`class`	49[1]	13	45	36[2]	10.1	49[1]	45	36[2]	10.3	5.0[3]	49[1]	10.3	1.0.0	1.0	6.0.0

1. Also from version 42–49: Strict mode is required.
2. Also from version 29–36: Strict mode is required.
3. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.class`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function`
- `class` expression
- Classes

JavaScript › Reference › Statements and declarations

const

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const · MDN page last modified 2026-09-04

Baseline: Widely available — available across major browsers since July 2015.

The **`const`** declaration declares block-scoped local variables. The value of a constant can't be changed through reassignment using the assignment operator, but if a constant is an object, its properties can be added, updated, or removed.

Try it

▷ Interactive example on MDN: "JavaScript Demo: const declaration". It runs live in the browser there; its source code follows.

js · interactive example source

```
const number = 42;

try {
  number = 99;
} catch (err) {
  console.log(err);
  // Expected output: TypeError: invalid assignment to const 'number'
  // (Note: the exact output may be browser-dependent)
}

console.log(number);
// Expected output: 42
```

Syntax

```
const name1 = value1;
const name1 = value1, name2 = value2;
const name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN`

Initial value of the variable. It can be any legal expression.

Description

The `const` declaration is very similar to `let`:

- `const` declarations are scoped to blocks as well as functions.
- `const` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `const` declarations are commonly regarded as non-hoisted.
- `const` declarations do not create properties on `globalThis` when declared at the top level of a script.
- `const` declarations cannot be redeclared by any other declaration in the same scope.
- `const` begins *declarations*, not *statements*. That means you cannot use a lone `const` declaration as the body of a block (which makes sense, since there's no way to access the variable).

  ```
  if (true) const a = 1; // SyntaxError: Lexical declaration cannot appear in a single-statement context
  ```

An initializer for a constant is required. You must specify its value in the same declaration. (This makes sense, given that it can't be changed later.)

```
const FOO; // SyntaxError: Missing initializer in const declaration
```

The `const` declaration creates an immutable reference to a value. It does *not* mean the value it holds is immutable — just that the variable identifier cannot be reassigned. For instance, in the case where the content is an object, this means the object's contents (e.g., its properties) can be altered. You should understand `const` declarations as "create a variable whose *identity* remains constant", not "whose *value* remains constant" — or, "create immutable bindings", not "immutable values".

Many style guides (including MDN's) recommend using `const` over `let` whenever a variable is not reassigned in its scope. This makes the intent clear that a variable's type (or value, in the case of a primitive) can never change. Others may prefer `let` for non-primitives that are mutated.

The list that follows the `const` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list.

Examples

Basic const usage

Constants can be declared with uppercase or lowercase, but a common convention is to use all-uppercase letters, especially for primitives because they are truly immutable.

js

```
// define MY_FAV as a constant and give it the value 7
const MY_FAV = 7;

console.log(`my favorite number is: ${MY_FAV}`);
```

js · ✗ bad example

```
// Re-assigning to a constant variable throws an error
MY_FAV = 20; // TypeError: Assignment to constant variable

// Redeclaring a constant throws an error
const MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
var MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
let MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
```

Block scoping

It's important to note the nature of block scoping.

js

```
const MY_FAV = 7;

if (MY_FAV === 7) {
  // This is fine because it's in a new block scope
  const MY_FAV = 20;
  console.log(MY_FAV); // 20

  // var declarations are not scoped to blocks so this throws an error
  var MY_FAV = 20; // SyntaxError: Identifier 'MY_FAV' has already been declared
}

console.log(MY_FAV); // 7
```

const in objects and arrays

`const` also works on objects and arrays. Attempting to overwrite the object throws an error "Assignment to constant variable".

js · ✗ bad example

```
const MY_OBJECT = { key: "value" };
MY_OBJECT = { OTHER_KEY: "value" };
```

However, object keys are not protected, so the following statement is executed without problem.

js

```
MY_OBJECT.key = "otherValue";
```

You would need to use `Object.freeze()` to make an object immutable.

The same applies to arrays. Assigning a new array to the variable throws an error "Assignment to constant variable".

js · ✗ bad example

```
const MY_ARRAY = [];
MY_ARRAY = ["B"];
```

Still, it's possible to push items into the array and thus mutate it.

js

```
MY_ARRAY.push("A"); // ["A"]
```

Declaration with destructuring

The left-hand side of each = can also be a binding pattern. This allows creating multiple variables at once.

js

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
const [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-let-and-const-declarations

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
const	21	12	36[1,2]	9	5.1	25	36[3,4]	10.1	5	1.5	4.4	5	1.0.0	1.0	6.0.0

1. Before Firefox 13, const is implemented, but re-assignment is not failing.
2. Before Firefox 46, a TypeError was thrown on redeclaration instead of a SyntaxError.
3. Before Firefox for Android 14, const is implemented, but re-assignment is not failing.
4. Before Firefox for Android 46, a TypeError was thrown on redeclaration instead of a SyntaxError.

Compatibility data: MDN browser-compat-data 8.1.8, javascript.statements.const. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- var
- let
- Constants in the JavaScript Guide

JavaScript › Reference › Statements and declarations

continue

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/continue · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`continue`** statement terminates execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: continue statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let text = "";

for (let i = 0; i < 10; i++) {
  if (i === 3) {
    continue;
  }
  text += i;
}

console.log(text);
// Expected output: "012456789"
```

Syntax

js

```
continue;
continue label;
```

`label` Optional

Identifier associated with the label of the statement.

Description

In contrast to the `break` statement, `continue` does not terminate the execution of the loop entirely, but instead:

- In a `while` or `do...while` loop, it jumps back to the condition.
- In a `for` loop, it jumps to the update expression.
- In a `for...in`, `for...of`, or `for await...of` loop, it jumps to the next iteration.

The `continue` statement can include an optional label that allows the program to jump to the next iteration of a labeled loop statement instead of the innermost loop. In this case, the `continue` statement needs to be nested within this labeled statement.

A `continue` statement, with or without a following label, cannot be used at the top level of a script, module, function's body, or static initialization block, even when the function or class is further contained within a loop.

Examples

Using continue with while

The following example shows a `while` loop that has a `continue` statement that executes when the value of `i` is 3. Thus, `n` takes on the values 1, 3, 7, and 12.

```js
let i = 0;
let n = 0;

while (i < 5) {
  i++;

  if (i === 3) {
    continue;
  }

  n += i;
}
```

Using continue with a label

In the following example, a statement labeled `checkIAndJ` contains a statement labeled `checkJ`. If `continue` is encountered, the program continues at the top of the `checkJ` statement. Each time `continue` is encountered, `checkJ` reiterates until its condition returns false. When false is returned, the remainder of the `checkIAndJ` statement is completed.

If `continue` had a label of `checkIAndJ`, the program would continue at the top of the `checkIAndJ` statement.

```js
let i = 0;
let j = 8;

checkIAndJ: while (i < 4) {
  console.log(`i: ${i}`);
  i += 1;

  checkJ: while (j > 4) {
    console.log(`j: ${j}`);
    j -= 1;

    if (j % 2 === 0) continue;
    console.log(`${j} is odd.`);
  }
  console.log(`i = ${i}`);
  console.log(`j = ${j}`);
}
```

Output:

```
i: 0

// start checkJ
j: 8
7 is odd.
j: 7
```

(code continues on the next page)

(code continued from the previous page)

```
j: 6
5 is odd.
j: 5
// end checkJ

i = 1
j = 4

i: 1
i = 2
j = 4

i: 2
i = 3
j = 4

i: 3
i = 4
j = 4
```

Unsyntactic continue statements

`continue` cannot be used within loops across function boundaries.

js · ✗ bad example

```
for (let i = 0; i < 10; i++) {
  (() => {
    continue; // SyntaxError: Illegal continue statement: no surrounding iteration statement
  })();
}
```

When referencing a label, the labeled statement must contain the `continue` statement.

js · ✗ bad example

```
label: for (let i = 0; i < 10; i++) {
  console.log(i);
}

for (let i = 0; i < 10; i++) {
  continue label; // SyntaxError: Undefined label 'label'
}
```

The labeled statement must be a loop.

js · ✗ bad example

```
label: {
  for (let i = 0; i < 10; i++) {
    continue label; // SyntaxError: Illegal continue statement: 'label' does not denote an
iteration statement
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-continue-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`continue`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.continue`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `break`
- `label`

JavaScript › Reference › Statements and declarations

debugger

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/debugger · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`debugger`** statement invokes any available debugging functionality, such as setting a breakpoint. If no debugging functionality is available, this statement has no effect.

Syntax

```js
debugger;
```

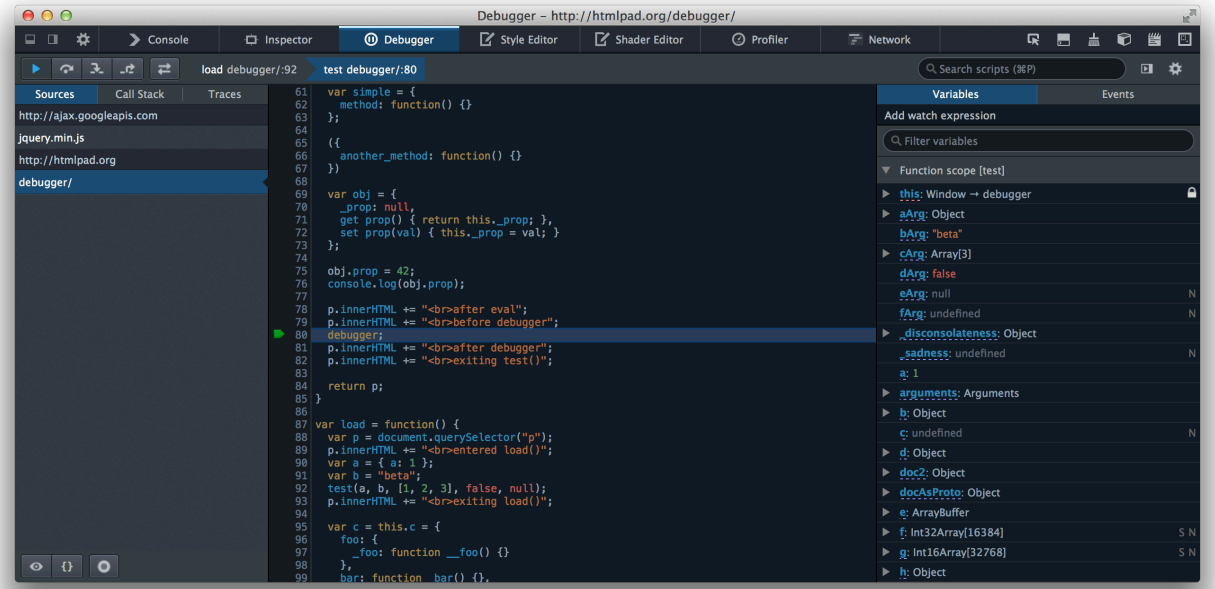
Examples

Using the debugger statement

The following example shows code where a `debugger` statement has been inserted, to invoke a debugger (if one exists) when the function is called.

```js
function potentiallyBuggyCode() {
  debugger;
  // do potentially buggy stuff to examine, step through, etc.
}
```

When the debugger is invoked, execution is paused at the `debugger` statement. It is like a breakpoint in the script source.



Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-debugger-statement`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`debugger`	5	12	1	10	5	18	4	10.1	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.debugger`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- The Firefox JavaScript Debugger¶ in the Firefox source docs

JavaScript › Reference › Statements and declarations

do...while

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/do...while · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`do...while`** statement creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated after executing the statement, resulting in the specified statement executing at least once.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: do...while statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let result = "";
let i = 0;

do {
  i += 1;
  result += i;
} while (i < 5);

console.log(result);
// Expected output: "12345"
```

Syntax

js

```
do
  statement
while (condition);
```

`statement`

A statement that is executed at least once and re-executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements.

`condition`

An expression evaluated *after* each pass through the loop. If this condition evaluates to true, `statement` is re-executed. When condition evaluates to false, execution continues with the statement after the `do...while` loop.

Description

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and re-evaluates `condition`.

The `do...while` statement syntax requires a semicolon at the end, but the automatic semicolon insertion process may insert one for you if the lack of a semicolon results in invalid syntax.

Examples

Using do...while

In the following example, the `do...while` loop iterates at least once and reiterates until `i` is no longer less than 5.

```js
let result = "";
let i = 0;
do {
  i += 1;
  result += `${i} `;
} while (i > 0 && i < 5);
// Despite i === 0 this will still loop as it starts off without the test

console.log(result);
```

Using false as do...while condition

Because the statement is always executed once, `do...while (false)` is the same as executing the statement itself. This is a common idiom in C-like languages, which allows you to use `break` to break out of branching logic early.

```js
do {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    break;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    break;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
} while (false);
// The rest of code
```

In JavaScript, there are some alternatives, such as using a labeled block statement with `break`:

```js
handleFriends: {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    break handleFriends;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    break handleFriends;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
}
```

Or using a function:

```js
function handleFriends() {
  if (!user.loggedIn) {
    console.log("You are not logged in");
    return;
  }
  const friends = user.getFriends();
  if (!friends.length) {
    console.log("No friends found");
    return;
  }
  for (const friend of friends) {
    handleFriend(friend);
  }
}
```

Using an assignment as a condition

In some cases, it can make sense to use an assignment as a condition, such as this:

```js
do {
  // …
} while ((match = regexp.exec(str)));
```

But when you do, there are readability tradeoffs. The `while` documentation has a Using an assignment as a condition section with our recommendations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-do-while-statement`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`do...while`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.do_while`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `while`
- `for`
- `break`
- `continue`

JavaScript › Reference › Statements and declarations

Empty statement

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/Empty · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

An **empty statement** is used to provide no statement, although the JavaScript syntax would expect one.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Empty statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [1, 2, 3];

// Assign all array values to 0
for (let i = 0; i < array.length; array[i++] = 0 /* empty statement */);

console.log(array);
// Expected output: Array [0, 0, 0]
```

Syntax

js

```
;
```

Description

The empty statement is a semicolon (`;`) indicating that no statement will be executed, even if JavaScript syntax requires one.

The opposite behavior, where you want multiple statements, but JavaScript only allows a single one, is possible using a block statement, which combines several statements into a single one.

Examples

Empty loop body

The empty statement is sometimes used with loop statements. See the following example with an empty loop body:

js

```
const arr = [1, 2, 3];

// Assign all array values to 0
for (let i = 0; i < arr.length; arr[i++] = 0) /* empty statement */ ;

console.log(arr);
// [0, 0, 0]
```

Unintentional usage

It is a good idea to comment *intentional* use of the empty statement, as it is not really obvious to distinguish from a normal semicolon.

In the following example, the usage is probably not intentional:

js · ✗ bad example

```
if (condition);      // Caution, this "if" does nothing!
  killTheUniverse(); // So this always gets executed!!!
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-empty-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Empty statement (;)	3	12	1	3	5	18	4	10.1	4.2	1.0	4.4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.empty`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Block statement

JavaScript › Reference › Statements and declarations

export

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/export · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since May 2018.

The **`export`** declaration is used to export values from a JavaScript module. Exported values can then be imported into other programs with the `import` declaration or dynamic import. The value of an imported binding is subject to change in the module that exports it — when a module updates the value of a binding that it exports, the update will be visible in its imported value.

In order to use the `export` declaration in a source file, the file must be interpreted by the runtime as a module. In HTML, this is done by adding `type="module"` to the `<script>` tag, or by being imported by another module. Modules are automatically interpreted in strict mode.

Syntax

```js
// Exporting declarations
export let name1, name2/*, … */; // also var
export const name1 = 1, name2 = 2/*, … */; // also var, let
export function functionName() { /* … */ }
export class ClassName { /* … */ }
export function* generatorFunctionName() { /* … */ }
export const { name1, name2: bar } = o;
export const [ name1, name2 ] = array;

// Export list
export { name1, /* …, */ nameN };
export { variable1 as name1, variable2 as name2, /* …, */ nameN };
export { variable1 as "string name" };
export { name1 as default /*, … */ };

// Default exports
export default expression;
export default function functionName() { /* … */ }
export default class ClassName { /* … */ }
export default function* generatorFunctionName() { /* … */ }
export default function () { /* … */ }
export default class { /* … */ }
export default function* () { /* … */ }

// Aggregating modules
export * from "module-name";
export * as name1 from "module-name";
export { name1, /* …, */ nameN } from "module-name";
export { import1 as name1, import2 as name2, /* …, */ nameN } from "module-name";
export { default, /* …, */ } from "module-name";
export { default as name1 } from "module-name";
```

`nameN`

Identifier to be exported (so that it can be imported via `import` in another script). If you use an alias with `as`, the actual exported name can be specified as a string literal, which may not be a valid identifier.

Description

Every module can have two different types of export, *named export* and *default export*. You can have multiple named exports per module but only one default export. Each type corresponds to one of the above syntax.

Named exports:

```js
// export features declared elsewhere
export { myFunction2, myVariable2 };

// export individual features (can export var, let,
// const, function, class)
export let myVariable = Math.sqrt(2);
export function myFunction() {
  // …
}
```

After the `export` keyword, you can use `let`, `const`, and `var` declarations, as well as function or class declarations. You can also use the `export { name1, name2 }` syntax to export a list of names declared elsewhere. Note that `export {}` does not export an empty object — it's a no-op declaration that exports nothing (an empty name list).

You cannot use `export` on a `using` or `await using` declaration. You can, however, export a variable that was declared elsewhere using `using` or `await using`. Doing so is still strongly discouraged, because the variable is disposed as soon as the module finishes executing, causing all importers to receive a value that's already disposed.

js · ✗ bad example

```
export using resource1 = getResource(); // SyntaxError

// Allowed by syntax but discouraged
using resource2 = getResource();
export { resource2 };
```

Export declarations are not subject to temporal dead zone rules. You can declare that the module exports `x` before the name `x` itself is declared.

js

```
export { x };
const x = 1;
// This works, because `export` is only a declaration, but doesn't
// utilize the value of `x`.
```

Default exports:

js

```
// export feature declared elsewhere as default
export { myFunction as default };
// This is equivalent to:
export default myFunction;

// export individual features as default
export default function () { /* … */ }
export default class { /* … */ }
```

`export { myFunction as default }` exports a live binding, whereas `export default myFunction` evaluates `myFunction` and exports its value at that point. Later reassignment of `myFunction` only updates the default export in the first form.

> **Note:** Names for export declarations must be distinct from each other. Having exports with duplicate names or using more than one `default` export will result in a `SyntaxError` and prevent the module from being evaluated.

The `export default` syntax allows any expression.

js

```
export default 1 + 1;
```

As a special case, functions and classes are exported as *declarations*, not expressions, and these declarations can be anonymous. This means functions will be hoisted.

```js
// Works because `foo` is a function declaration,
// not a function expression
foo();

export default function foo() {
  console.log("Hi");
}

// It's still technically a declaration, but it's allowed
// to be anonymous
export default function () {
  console.log("Hi");
}
```

Named exports are useful when you need to export several values. When importing this module, named exports must be referred to by the exact same name (optionally renaming it with `as`), but the default export can be imported with any name. For example:

```js
// -- test.js --
const k = 12;
export default k;
```

```js
// some other file
import m from "./test.js"; // note that we have the freedom to use import m instead of
import k, because k was default export

console.log(m); // 12
```

You can also rename named exports to avoid naming conflicts:

```js
export { myFunction as function1, myVariable as variable };
```

You can rename a name to something that's not a valid identifier by using a string literal. For example:

```js
export { myFunction as "my-function" };
```

Re-exporting / Aggregating

A module can also "relay" values exported from other modules without the hassle of writing two separate import/export statements. This is often useful when creating a single module concentrating various exports from various modules (usually called a "barrel module").

This can be achieved with the "export from" syntax:

```js
export { default as function1, function2 } from "bar.js";
```

Which is comparable to a combination of import and export, except that `function1` and `function2` do not become available inside the current module:

```js
import { default as function1, function2 } from "bar.js";

export { function1, function2 };
```

Most of the "import from" syntaxes have "export from" counterparts.

```js
export { x } from "mod";
export { x as v } from "mod";
export * as ns from "mod";
```

There is also `export * from "mod"`, although there's no `import * from "mod"`. This re-exports all **named** exports from `mod` as the named exports of the current module, but the default export of `mod` is not re-exported. If two wildcard export statements implicitly re-export the same name from different bindings, the name is ambiguous and is omitted from the module namespace. If both resolve to the same binding, there is no ambiguity. An explicit export of that name takes precedence over wildcard exports.

```js
// -- mod1.js --
export const a = 1;

// -- mod2.js --
export const a = 3;

// -- barrel.js --
export * from "./mod1.js";
export * from "./mod2.js";

// -- main.js --
import * as ns from "./barrel.js";

console.log(ns.a); // undefined
```

Attempting to import the ambiguous name directly will throw a syntax error during linking, before evaluation starts.

```js
import { a } from "./barrel.js";
// SyntaxError: The requested module './barrel.js' contains conflicting star exports for
name 'a'
```

The following is syntactically invalid despite its import equivalent:

js · ✗ bad example

```
export DefaultExport from "bar.js"; // Invalid
```

> **Note:** Due to the lack of this syntax, `import source` also has no re-exporting counterpart.

The correct way of doing this is to rename the export:

```js
export { default as DefaultExport } from "bar.js";
```

The "export from" syntax allows the `as` token to be omitted, which makes the default export still re-exported as default export.

```js
export { default, function2 } from "bar.js";
```

`export from` supports all features that `import` supports — for example, import attributes:

```js
export { default } from "./data.json" with { type: "json" };
```

> **Note:** The `import defer * as ns` declaration has no re-exporting counterpart, despite `import * as ns` having one. Deferred re-exports are being developed in a separate proposal, because they can also avoid loading unused modules, which requires additional tree-shaking semantics. You can instead import and export in two separate statements, although this immediately loads the module.

Examples

Using named exports

```js
// -- my-module.js --
function cube(x) {
  return x * x * x;
}

const foo = Math.PI + Math.SQRT2;

const graph = {
  options: {
    color: "white",
    thickness: "2px",
  },
  draw() {
    console.log("From graph draw function");
  },
};

export { cube, foo, graph };
```

Then in the top-level module included in your HTML page, we could have:

```js
import { cube, foo, graph } from "./my-module.js";

graph.options = {
  color: "blue",
  thickness: "3px",
};

graph.draw(); // Logs "From graph draw function"
console.log(cube(3)); // 27
console.log(foo); // 4.555806215962888
```

It is important to note the following:

- You need to include this script in your HTML with a `<script>` element of `type="module"`, so that it gets recognized as a module and dealt with appropriately.
- You can't run JS modules via a `file://` URL — you'll get CORS errors. You need to run it via an HTTP server.

Using the default export

If we want to export a single value representing an entire module, we could use a default export:

```js
// -- cube.js --

export default function cube(x) {
  return x * x * x;
}
```

Then, in another script, it is straightforward to import the default export:

```js
import cube from "./cube.js";

console.log(cube(3)); // 27
```

Using export from

Let's take an example where we have the following hierarchy:

- `childModule1.js`: exporting `myFunction` and `myVariable`
- `childModule2.js`: exporting `MyClass`
- `parentModule.js`: acting as an aggregator (and doing nothing else)
- top level module: consuming the exports of `parentModule.js`

This is what it would look like using code snippets:

```js
// -- childModule1.js --
function myFunction() {
  console.log("Hello!");
}
const myVariable = 1;
export { myFunction, myVariable };
```

```js
// -- childModule2.js --
class MyClass {
  constructor(x) {
    this.x = x;
  }
}

export { MyClass };
```

```js
// -- parentModule.js --
// Only aggregating the exports from childModule1 and childModule2
// to re-export them
export { myFunction, myVariable } from "./childModule1.js";
export { MyClass } from "./childModule2.js";
```

```js
// -- top-level module --
// We can consume the exports from a single module since parentModule
// "collected"/"bundled" them in a single source
import { myFunction, myVariable, MyClass } from "./parentModule.js";
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html#sec-exports

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`export`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0[1]
Arbitrary module namespace identifier names	88	88	87	74	14.1	88	87	63	14.5	15.0	88	14.5	1.0.15	1.6	16.0.0
`default` keyword with `export`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0[1]
`export * as namespace`	72	79	80	60	14.1	72	80	51	14.5	11.0	72	14.5	1.0.0	1.0	13.2.0[1]

1. Modules must either have a filename ending in `.mjs`, or the nearest parent `package.json` file must contain `"type": "module"`. See Node's ECMAScript Modules documentation for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.export`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `import`
- JavaScript modules guide
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)

JavaScript › Reference › Statements and declarations

Expression statement

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/Expression_statement · MDN page last modified 2025-07-08

An **expression statement** is an expression used in a place where a statement is expected. The expression is evaluated and its result is discarded — therefore, it makes sense only for expressions that have side effects, such as executing a function or updating a variable.

Syntax

```
expression;
```

`expression`

An arbitrary expression to be evaluated. There are certain expressions that may be ambiguous with other statements and are thus forbidden.

Description

Apart from the dedicated statement syntaxes, you can also use almost any expression as a statement on its own. The expression statement syntax requires a semicolon at the end, but the automatic semicolon insertion process may insert one for you if the lack of a semicolon results in invalid syntax.

Because the expression is evaluated and then discarded, the result of the expression is not available. Therefore, the expression must have some side effect for it to be useful. Expression statements are commonly:

- Function calls (`console.log("Hello");`, `[1, 2, 3].forEach((i) => console.log(i));`)
- Tagged template literals
- Assignment expressions, including compound assignments
- Increment and decrement operators
- `delete`
- `import()`
- `yield` and `yield*`

Others may also have side effects if they invoke getters or trigger type coercions.

Forbidden expressions

In order for an expression to be used as a statement, it must not be ambiguous with other statement syntaxes. Therefore, the expression must not start with any of the following tokens:

- `function`: which would be a `function` declaration or `function*` declaration, not a `function` expression or `function*` expression
- `async function`: which would be an `async function` declaration or `async function*` declaration, not an `async function` expression or `async function*` expression
- `class`: which would be a `class` declaration, not a `class` expression
- `let[`: which would be a `let` declaration with array destructuring, not a property accessor on a variable called `let` (`let` can only be an identifier in non-strict mode)

- {: which would be a block statement, not an object literal

Therefore, all of the following are invalid:

js · ✗ bad example

```
function foo() {
  console.log("foo");
}(); // SyntaxError: Unexpected token '('

// For some reason, you have a variable called `let`
var let = [1, 2, 3];
let[0] = 4; // SyntaxError: Invalid destructuring assignment target

{
  foo: 1,
  bar: 2, // SyntaxError: Unexpected token ':'
};
```

More dangerously, sometimes the code happens to be valid syntax, but is not what you intend.

js · ✗ bad example

```
// For some reason, you have a variable called `let`
var let = [1, 2, 3];

function setIndex(index, value) {
  if (index >= 0) {
    // Intend to assign to the array `let`, but instead creates an extra variable!
    let[index] = value;
  }
}

setIndex(0, [1, 2]);
console.log(let); // [1, 2, 3]

// This is not an object literal, but a block statement,
// where `foo` is a label and `1` is an expression statement.
// This often happens in the console
{ foo: 1 };
```

To avoid these problems, you can use parentheses, so that the statement is unambiguously an expression statement.

js · ✓ good example

```
(function foo() {
  console.log("foo");
})();
```

Examples

Avoiding control flow statements

You can avoid almost all use of control flow statements using expression statements. For example, `if...else` can be replaced with ternary operators and logical operators. Iterative statements like `for` or `for...of` can be replaced with array methods.

```js
// Using control flow statements
function range(start, end) {
  if (start > end) {
    [start, end] = [end, start];
  }
  const result = [];
  for (let i = start; i < end; i++) {
    result.push(i);
  }
  return result;
}

// Using expression statements
function range2(start, end) {
  start > end && ([start, end] = [end, start]);
  return Array.from({ length: end - start }, (_, i) => start + i);
}
```

> **Warning:** This only demonstrates a capability of the language. Excessive use of expression statements as a substitute for control-flow statements can make code much less readable.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-expression-statement`

See also

- Statements and declarations
- Expressions and operators

JavaScript › Reference › Statements and declarations

for

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for · MDN page last modified 2026-08-27

Baseline: Widely available — available across major browsers since July 2015.

The **`for`** statement creates a loop that consists of three optional expressions, enclosed in parentheses and separated by semicolons, followed by a statement (usually a block statement) to be executed in the loop.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: for statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let str = "";

for (let i = 0; i < 9; i++) {
  str += i;
}

console.log(str);
// Expected output: "012345678"
```

Syntax

js

```
for (initialization; condition; afterthought)
  statement
```

`initialization` Optional

An expression (including assignment expressions) or variable declaration evaluated once before the loop begins. Typically used to initialize a counter variable. This expression may optionally declare new variables with `var` or `let` keywords. Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for` loop is in. Variables declared with `let` are local to the statement.

The result of this expression is discarded.

`condition` Optional

An expression to be evaluated before each loop iteration. If this expression evaluates to true, `statement` is executed. If the expression evaluates to false, execution exits the loop and goes to the first statement after the `for` construct.

This conditional test is optional. If omitted, the condition always evaluates to true.

`afterthought` Optional

An expression to be evaluated at the end of each loop iteration. This occurs before the next evaluation of `condition`. Generally used to update or increment the counter variable.

`statement`

A statement that is executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements. To execute no statement within the loop, use an empty statement (`;`).

Description

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and re-evaluates `afterthought` then `condition`.

Examples

Using for

The following `for` statement starts by declaring the variable `i` and initializing it to `0`. It checks that `i` is less than nine, performs the two succeeding statements, and increments `i` by 1 after each pass through the loop.

```js
for (let i = 0; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Initialization block syntax

The initialization block accepts both expressions and variable declarations. However, expressions cannot use the `in` operator unparenthesized, because that is ambiguous with a `for...in` loop.

js · ✗ bad example

```
for (let i = "start" in window ? window.start : 0; i < 9; i++) {
  console.log(i);
}
// SyntaxError: 'for-in' loop variable declaration may not have an initializer.
```

js · ✓ good example

```
// Parenthesize the whole initializer
for (let i = ("start" in window ? window.start : 0); i < 9; i++) {
  console.log(i);
}

// Parenthesize the `in` expression
for (let i = ("start" in window) ? window.start : 0; i < 9; i++) {
  console.log(i);
}
```

Optional for expressions

All three expressions in the head of the `for` loop are optional. For example, it is not required to use the `initialization` block to initialize variables:

```js
let i = 0;
for (; i < 9; i++) {
  console.log(i);
  // more statements
}
```

Like the `initialization` block, the `condition` part is also optional. If you are omitting this expression, you must make sure to break the loop in the body in order to not create an infinite loop.

```js
for (let i = 0; ; i++) {
  console.log(i);
  if (i > 3) break;
  // more statements
}
```

You can also omit all three expressions. Again, make sure to use a `break` statement to end the loop and also modify (increase) a variable, so that the condition for the break statement is true at some point.

```js
let i = 0;

for (;;) {
  if (i > 3) break;
  console.log(i);
  i++;
}
```

However, in the case where you are not fully using all three expression positions — especially if you are not declaring variables with the first expression but mutating something in the upper scope — consider using a `while` loop instead, which makes the intention clearer.

```js
let i = 0;

while (i <= 3) {
  console.log(i);
  i++;
}
```

Lexical declarations in the initialization block

Declaring a variable within the initialization block has important differences from declaring it in the upper scope, especially when creating a closure within the loop body. For example, for the code below:

```js
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

It logs `0`, `1`, and `2`, as expected. However, if the variable is defined in the upper scope:

```js
let i = 0;
for (; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

It logs `3`, `3`, and `3`. The reason is that each `setTimeout` creates a new closure that closes over the `i` variable, but if the `i` is not scoped to the loop body, all closures will reference the same variable when they eventually get called — and due to the asynchronous nature of `setTimeout()`, it will happen after the loop has already exited, causing the value of `i` in all queued callbacks' bodies to have the value of `3`.

This also happens if you use a `var` statement as the initialization, because variables declared with `var` are only function-scoped, but not lexically scoped (i.e., they can't be scoped to the loop body).

```js
for (var i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
// Logs 3, 3, 3
```

The scoping effect of the initialization block can be understood as if the declaration happens within the loop body, but just happens to be accessible within the `condition` and `afterthought` parts. More precisely, `let` declarations are special-cased by `for` loops — if `initialization` is a `let` declaration, then every time, after the loop body is evaluated, the following happens:

1. A new lexical scope is created with new `let`-declared variables.
2. The binding values from the last iteration are used to re-initialize the new variables.
3. `afterthought` is evaluated in the new scope.

So re-assigning the new variables within `afterthought` does not affect the bindings from the previous iteration.

A new lexical scope is also created after `initialization`, just before `condition` is evaluated for the first time. These details can be observed by creating closures, which allow you to get hold of a binding at any particular point. For example, in this code a closure created within the `initialization` section does not get updated by re-assignments of `i` in the `afterthought`:

```js
for (let i = 0, getI = () => i; i < 3; i++) {
  console.log(getI());
}
// Logs 0, 0, 0
```

This does not log "0, 1, 2", like what would happen if `getI` is declared in the loop body. This is because `getI` is not re-evaluated on each iteration — rather, the function is created once and closes over the `i` variable, which refers to the variable declared when the loop was first initialized. Subsequent updates to the value of `i` actually create new variables called `i`, which `getI` does not see. A way to fix this is to re-compute `getI` every time `i` updates:

```js
for (let i = 0, getI = () => i; i < 3; i++, getI = () => i) {
  console.log(getI());
}
// Logs 0, 1, 2
```

The `i` variable inside the `initialization` is distinct from the `i` variable inside every iteration, including the first. So, in this example, `getI` returns 0, even though the value of `i` inside the iteration is incremented beforehand:

```js
for (let i = 0, getI = () => i; i < 3;) {
  i++;
  console.log(getI());
}
// Logs 0, 0, 0
```

In fact, you can capture this initial binding of the `i` variable and re-assign it later, and this updated value will not be visible to the loop body, which sees the next new binding of `i`.

```js
for (
  let i = 0, getI = () => i, incrementI = () => i++;
  getI() < 3;
  incrementI()
) {
  console.log(i);
}
// Logs 0, 0, 0
```

This logs "0, 0, 0", because the `i` variable in each loop evaluation is actually a separate variable, but `getI` and `incrementI` both read and write the *initial* binding of `i`, not what was subsequently declared.

Using for without a body

The following `for` cycle calculates the offset position of a node in the `afterthought` section, and therefore it does not require the use of a `statement` section, a semicolon is used instead.

```js
function showOffsetPos(id) {
  let left = 0;
  let top = 0;
  for (
    let itNode = document.getElementById(id); // initialization
    itNode; // condition
    left += itNode.offsetLeft,
      top += itNode.offsetTop,
      itNode = itNode.offsetParent // afterthought
  ); // semicolon

  console.log(
    `Offset position of "${id}" element:
left: ${left}px;
top: ${top}px;`,
  );
}

showOffsetPos("content");

// Logs:
// Offset position of "content" element:
// left: 0px;
// top: 153px;
```

Note that the semicolon after the `for` statement is mandatory, because it stands as an empty statement. Otherwise, the `for` statement acquires the following `console.log` line as its `statement` section, which makes the `log` execute multiple times.

Using for with two iterating variables

You can create two counters that are updated simultaneously in a for loop using the comma operator. Multiple `let` and `var` declarations can also be joined with commas.

```js
const arr = [1, 2, 3, 4, 5, 6];
for (let l = 0, r = arr.length - 1; l < r; l++, r--) {
  console.log(arr[l], arr[r]);
}
// 1 6
// 2 5
// 3 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-for-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
for	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.for`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Empty statement
- `break`
- `continue`
- `while`
- `do...while`
- `for...in`
- `for...of`

JavaScript › Reference › Statements and declarations

for await...of

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for-await...of · MDN page last modified 2026-09-03

Baseline: Widely available — available across major browsers since January 2020.

The **`for await...of`** statement creates a loop iterating over async iterable objects as well as sync iterables. This statement can only be used in contexts where `await` can be used, which includes inside an async function body and in a module.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: for await...of statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
async function* foo() {
  yield 1;
  yield 2;
}

(async () => {
  for await (const num of foo()) {
    console.log(num);
    // Expected output: 1

    break; // Closes iterator, triggers return
  }
})();
```

Syntax

js

```
for await (variable of iterable)
  statement
```

`variable`

Receives a value from the sequence on each iteration. May be either a declaration with `const`, `let`, or `var`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for await...of` loop is in.

`iterable`

An async iterable or sync iterable. The source of the sequence of values on which the loop operates.

`statement`

A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

When a `for await...of` loop iterates over an iterable, it first gets the iterable's `[Symbol.asyncIterator]()` method and calls it, which returns an async iterator. If the `@asyncIterator` method does not exist, it then looks for a `[Symbol.iterator]()` method, which returns a sync iterator. The sync iterator returned is then wrapped into an async iterator by wrapping every object returned from the `next()`, `return()`, and `throw()` methods into a resolved or rejected promise, with the `value` property resolved if it's also a promise. The loop then repeatedly calls the final async iterator's `next()` method and awaits the returned promise, producing the sequence of values to be assigned to `variable`.

A `for await...of` loop exits when the iterator has completed (the awaited `next()` result is an object with `done: true`). Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

If the `for await...of` loop exited early (e.g., a `break` statement is encountered or an error is thrown), the `return()` method of the iterator is called to perform any cleanup. The returned promise is awaited before the loop exits.

`for await...of` generally functions the same as the `for...of` loop and shares many of the same syntax and semantics. There are a few differences:

- `for await...of` works on both sync and async iterables, while `for...of` only works on sync iterables.
- `for await...of` can only be used in contexts where `await` can be used, which includes inside an async function body and in a module. Even when the iterable is sync, the loop still awaits the return value for every iteration, leading to slower execution due to repeated promise unwrapping.
- If the `iterable` is a sync iterable that yields promises, `for await...of` would produce a sequence of resolved values, while `for...of` would produce a sequence of promises.
- For `for await...of`, the `variable` can be the identifier `async` (e.g., `for await (async of foo)`); `for...of` forbids this case.

Like `for...of`, if you use a `using` declaration, then the variable cannot be called `of`:

js · ✗ bad example

```
for await (using of of []); // SyntaxError
```

This is to avoid syntax ambiguity with the valid code `for await (using of [])`, before `using` was introduced.

Examples

Iterating over async iterables

You can also iterate over an object that explicitly implements async iterable protocol:

js

```
const LIMIT = 3;

const asyncIterable = {
  [Symbol.asyncIterator]() {
    let i = 0;
    return {
      next() {
        const done = i === LIMIT;
        const value = done ? undefined : i++;
        return Promise.resolve({ value, done });
      },
      return() {
        // This will be reached if the consumer called 'break' or 'return' early in the
loop.
        return { done: true };
      },
    };
  },
};
```

(code continues on the next page)

(code continued from the previous page)

```
(async () => {
  for await (const num of asyncIterable) {
    console.log(num);
  }
})();
// 0
// 1
// 2
```

Iterating over async generators

Since the return values of async generator functions conform to the async iterable protocol, they can be looped using `for await...of`.

js

```
async function* asyncGenerator() {
  let i = 0;
  while (i < 3) {
    yield i++;
  }
}

(async () => {
  for await (const num of asyncGenerator()) {
    console.log(num);
  }
})();
// 0
// 1
// 2
```

For a more concrete example of iterating over an async generator using `for await...of`, consider iterating over data from an API.

This example first creates an async iterable for a stream of data, then uses it to find the size of the response from the API.

js

```
async function* streamAsyncIterable(stream) {
  const reader = stream.getReader();
  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) return;
      yield value;
    }
  } finally {
    reader.releaseLock();
  }
}

// Fetches data from URL and calculates response size using the async generator.
async function getResponseSize(url) {
  const response = await fetch(url);
  // Will hold the size of the response, in bytes.
  let responseSize = 0;
  // The for-await-of loop. Async iterates over each portion of the response.
  for await (const chunk of streamAsyncIterable(response.body)) {
    // Incrementing the total response length.
```

(code continues on the next page)

(code continued from the previous page)

```
    responseSize += chunk.length;
  }

  console.log(`Response Size: ${responseSize} bytes`); // "Response Size: 1071472"
  return responseSize;
}
getResponseSize("https://jsonplaceholder.typicode.com/photos");
```

Iterating over sync iterables and generators

`for await...of` loop also consumes sync iterables and generators. In that case it internally awaits emitted values before assign them to the loop control variable.

js

```
function* generator() {
  yield 0;
  yield 1;
  yield Promise.resolve(2);
  yield Promise.resolve(3);
  yield 4;
}

(async () => {
  for await (const num of generator()) {
    console.log(num);
  }
})();
// 0
// 1
// 2
// 3
// 4

// compare with for-of loop:

for (const numOrPromise of generator()) {
  console.log(numOrPromise);
}
// 0
// 1
// Promise { 2 }
// Promise { 3 }
// 4
```

If a sync generator yields a rejected promise, `for await...of` calls the generator's `return()` method before throwing the rejection reason, allowing `finally` blocks within that generator to run.

js

```
function* generatorWithRejectedPromises() {
  try {
    yield 0;
    yield 1;
    yield Promise.resolve(2);
    yield Promise.reject(new Error("failed"));
    yield 4;
    throw new Error("throws");
  } finally {
    console.log("called finally");
```

(code continues on the next page)

(code continued from the previous page)

```
  }
}

(async () => {
  try {
    for await (const num of generatorWithRejectedPromises()) {
      console.log(num);
    }
  } catch (e) {
    console.log("caught", e);
  }
})();
// 0
// 1
// 2
// called finally
// caught Error: failed
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-for-in-and-for-of-statements

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`for await...of`	63	79	57	50	12	63	57	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.for_await_of`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Symbol.asyncIterator`
- `for...of`

JavaScript › Reference › Statements and declarations

for...in

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...in · MDN page last modified 2025-07-29

Baseline: Widely available — available across major browsers since July 2015.

The **`for...in`** statement iterates over all enumerable string properties of an object (ignoring properties keyed by symbols), including inherited enumerable properties.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: for...in statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const object = { a: 1, b: 2, c: 3 };

for (const property in object) {
  console.log(`${property}: ${object[property]}`);
}

// Expected output:
// "a: 1"
// "b: 2"
// "c: 3"
```

Syntax

js

```
for (variable in object)
  statement
```

Parameters

`variable`

Receives a string property name on each iteration. May be either a declaration with `const`, `let`, or `var`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for...in` loop is in.

`object`

Object whose non-symbol enumerable properties are iterated over.

`statement`

A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

The loop will iterate over all enumerable properties of the object itself and those the object inherits from its prototype chain (properties of nearer prototypes take precedence over those of prototypes further away from the object in its prototype chain).

Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

A `for...in` loop only iterates over enumerable, non-symbol properties. Objects created from built–in constructors like `Array` and `Object` have inherited non–enumerable properties from `Array.prototype` and `Object.prototype`, such as `Array`'s `indexOf()` method or `Object`'s `toString()` method, which will not be visited in the `for...in` loop.

The traversal order, as of modern ECMAScript specification, is well-defined and consistent across implementations. Within each component of the prototype chain, all non-negative integer keys (those

that can be array indices) will be traversed first in ascending order by value, then other string keys in ascending chronological order of property creation.

The `variable` part of `for...in` accepts anything that can come before the `=` operator. You can use `const` to declare the variable as long as it's not reassigned within the loop body (it can change between iterations, because those are two separate variables). Otherwise, you can use `let`. You can use destructuring to assign multiple local variables, or use a property accessor like `for (x.y in iterable)` to assign the value to an object property. However, `using` and `await using` are not allowed, because the variable is always a string or symbol while these declarations require a disposable object.

A legacy syntax allows `var` declarations of the loop variable to have an initializer. This throws a syntax error in strict mode and is ignored in non–strict mode.

Deleted, added, or modified properties

`for...in` visits property keys in the following fashion:

1. It first gets all own string keys of the current object, in a fashion very similar to `Object.getOwnPropertyNames()`.
2. For each key, if no string with the same value has ever been visited, the property descriptor is retrieved and the property is only visited if it is enumerable. However, this property string will be marked as visited even if it's not enumerable.
3. Then, the current object is replaced with its prototype, and the process is repeated.

This means:

- Any property added to the currently visited object during iteration will not be visited, because all own properties of the current object have already been saved beforehand.
- If multiple objects in the prototype chain have a property with the same name, only the first one will be considered, and it is only visited if it's enumerable. If it is non-enumerable, no other properties with the same name further up the prototype chain will be visited, even if they are enumerable.

In general, it is best not to add, modify, or remove properties from the object during iteration, other than the property currently being visited. The spec explicitly allows the implementation to not follow the algorithm above in one of the following cases:

- The object's prototype chain is modified during iteration.
- A property is deleted from the object or its prototype chain during iteration.
- A property is added to the object's prototype chain during iteration.
- A property's enumerability is changed during iteration.

In these cases, implementations may behave differently from what you may expect, or even from each other.

Array iteration and for...in

Array indexes are just enumerable properties with integer names and are otherwise identical to general object properties. The `for...in` loop will traverse all integer keys before traversing other keys, and in strictly increasing order, making the behavior of `for...in` close to normal array iteration. However, the `for...in` loop will return all enumerable properties, including those with non–integer names and those that are inherited. Unlike `for...of`, `for...in` uses property enumeration instead of the array's iterator. In sparse arrays, `for...of` will visit the empty slots, but `for...in` will not.

It is better to use a `for` loop with a numeric index, `Array.prototype.forEach()`, or the `for...of` loop, because they will return the index as a number instead of a string, and also avoid non-index properties.

Iterating over own properties only

If you only want to consider properties attached to the object itself, and not its prototypes, you can use one of the following techniques:

- `Object.keys()`
- `Object.getOwnPropertyNames()`

`Object.keys` will return a list of enumerable own string properties, while `Object.getOwnPropertyNames` will also contain non-enumerable ones.

Many JavaScript style guides and linters recommend against the use of `for...in`, because it iterates over the entire prototype chain which is rarely what one wants, and may be a confusion with the more widely-used `for...of` loop. `for...in` is most practically used for debugging purposes, being an easy way to check the properties of an object (by outputting to the console or otherwise). In situations where objects are used as ad hoc key-value pairs, `for...in` allows you check if any of those keys hold a particular value.

Examples

Using for...in

The `for...in` loop below iterates over all of the object's enumerable, non-symbol properties and logs a string of the property names and their values.

js

```
const obj = { a: 1, b: 2, c: 3 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}

// Logs:
// "obj.a = 1"
// "obj.b = 2"
// "obj.c = 3"
```

Iterating own properties

The following function illustrates the use of `Object.hasOwn()`: the inherited properties are not displayed.

js

```
const triangle = { a: 1, b: 2, c: 3 };

function ColoredTriangle() {
  this.color = "red";
}

ColoredTriangle.prototype = triangle;
```

(code continues on the next page)

(code continued from the previous page)

```
const obj = new ColoredTriangle();

for (const prop in obj) {
  if (Object.hasOwn(obj, prop)) {
    console.log(`obj.${prop} = ${obj[prop]}`);
  }
}

// Logs:
// "obj.color = red"
```

Concurrent modification

> **Warning:** You should not write code like this yourself. It is only included here to illustrate the behavior of `for...in` in some corner cases.

Properties added to the current object during iteration are never visited:

js

```
const obj = { a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  obj.c = 3;
}

// Logs:
// obj.a = 1
// obj.b = 2
```

Shadowed properties are only visited once:

js

```
const proto = { a: 1 };
const obj = { __proto__: proto, a: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}

// Logs:
// obj.a = 2

Object.defineProperty(obj, "a", { enumerable: false });

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
}
// Logs nothing, because the first "a" property visited is non-enumerable.
```

In addition, consider the following cases, where the behavior is unspecified, and implementations tend to diverge from the specified algorithm:

Changing the prototype during iteration:

```js
const obj = { a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  Object.setPrototypeOf(obj, { c: 3 });
}
```

Deleting a property during iteration:

```js
const obj = { a: 1, b: 2, c: 3 };

// Deleting a property before it is visited
for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  delete obj.c;
}

const obj2 = { a: 1, b: 2, c: 3 };

// Deleting a property after it is visited
for (const prop in obj2) {
  console.log(`obj2.${prop} = ${obj2[prop]}`);
  delete obj2.a;
}
```

Enumerable properties added to the prototype during iteration:

```js
const proto = {};
const obj = { __proto__: proto, a: 1, b: 2 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  proto.c = 3;
}
```

Changing the enumerability of a property during iteration:

```js
const obj = { a: 1, b: 2, c: 3 };

for (const prop in obj) {
  console.log(`obj.${prop} = ${obj[prop]}`);
  Object.defineProperty(obj, "c", { enumerable: false });
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-for-in-and-for-of-statements

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`for...in`	1	12	1	2	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.for_in`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `for...of`
- `for`
- Enumerability and ownership of properties
- `Object.getOwnPropertyNames()`
- `Object.hasOwn()`
- `Array.prototype.forEach()`

JavaScript › Reference › Statements and declarations

for...of

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...of · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`for...of`** statement executes a loop that operates on a sequence of values sourced from an iterable object. Iterable objects include instances of built-ins such as `Array`, `String`, `TypedArray`, `Map`, `Set`, `NodeList` (and other DOM collections), as well as the `arguments` object, generators produced by generator functions, and user-defined iterables.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: for...of statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = ["a", "b", "c"];

for (const element of array) {
  console.log(element);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

```js
for (variable of iterable)
  statement
```

`variable`

Receives a value from the sequence on each iteration. May be either a declaration with `const`, `let`, `var`, `using`, `await using`, or an assignment target (e.g., a previously declared variable, an object property, or a destructuring pattern). Variables declared with `var` are not local to the loop, i.e., they are in the same scope the `for...of` loop is in.

`iterable`

An iterable object. The source of the sequence of values on which the loop operates.

`statement`

A statement to be executed on every iteration. May reference `variable`. You can use a block statement to execute multiple statements.

Description

A `for...of` loop operates on the values sourced from an iterable one by one in sequential order. Each operation of the loop on a value is called an *iteration*, and the loop is said to *iterate over the iterable*. Each iteration executes statements that may refer to the current sequence value.

When a `for...of` loop iterates over an iterable, it first calls the iterable's `[Symbol.iterator]()` method, which returns an iterator, and then repeatedly calls the resulting iterator's `next()` method to produce the sequence of values to be assigned to `variable`.

A `for...of` loop exits when the iterator has completed (the `next()` result is an object with `done: true`). Like other looping statements, you can use control flow statements inside `statement`:

- `break` stops `statement` execution and goes to the first statement after the loop.
- `continue` stops `statement` execution and goes to the next iteration of the loop.

If the `for...of` loop exited early (e.g., a `break` statement is encountered or an error is thrown), the `return()` method of the iterator is called to perform any cleanup.

The `variable` part of `for...of` accepts anything that can come before the `=` operator. You can use `const` to declare the variable as long as it's not reassigned within the loop body (it can change between iterations, because those are two separate variables). Otherwise, you can use `let`.

```js
const iterable = [10, 20, 30];

for (let value of iterable) {
  value += 1;
  console.log(value);
}
// 11
// 21
// 31
```

> **Note:** Each iteration creates a new variable. Reassigning the variable inside the loop body does not affect the original value in the iterable (an array, in this case).

Variables declared using the `using` or `await using` declaration are disposed every time a loop iteration is done (and `await using` causes an implicit `await` at the end of the iteration). However, if the loop early-exits, any values left in the iterator that haven't been visited are not disposed (although the current value is).

js

```
const resources = [dbConnection1, dbConnection2, dbConnection3];

for (using dbConnection of resources) {
  dbConnection.query("...");
  // dbConnection is disposed here
}
```

You can use destructuring to assign multiple local variables, or use a property accessor like `for (x.y of iterable)` to assign the value to an object property.

However, a special rule forbids using `async` as the variable name. This is invalid syntax:

js · ✗ bad example

```
let async;
for (async of [1, 2, 3]); // SyntaxError: The left-hand side of a for-of loop may not be
'async'.
```

This is to avoid syntax ambiguity with the valid code `for (async of => {};;)`, which is a `for` loop.

Similarly, if you use the `using` declaration, then the variable cannot be called `of`:

js · ✗ bad example

```
for (using of of []); // SyntaxError
```

This is to avoid syntax ambiguity with the valid code `for (using of [])`, before `using` was introduced.

Examples

Iterating over an Array

js

```
const iterable = [10, 20, 30];

for (const value of iterable) {
  console.log(value);
}
// 10
// 20
// 30
```

Iterating over a string

Strings are iterated by Unicode code points.

```js
const iterable = "boo";

for (const value of iterable) {
  console.log(value);
}
// "b"
// "o"
// "o"
```

Iterating over a TypedArray

```js
const iterable = new Uint8Array([0x00, 0xff]);

for (const value of iterable) {
  console.log(value);
}
// 0
// 255
```

Iterating over a Map

```js
const iterable = new Map([
  ["a", 1],
  ["b", 2],
  ["c", 3],
]);

for (const entry of iterable) {
  console.log(entry);
}
// ['a', 1]
// ['b', 2]
// ['c', 3]

for (const [key, value] of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over a Set

```js
const iterable = new Set([1, 1, 2, 2, 3, 3]);

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over the arguments object

You can iterate over the `arguments` object to examine all parameters passed into a function.

```js
function foo() {
  for (const value of arguments) {
    console.log(value);
  }
}

foo(1, 2, 3);
// 1
// 2
// 3
```

Iterating over a NodeList

The following example adds a `read` class to paragraphs that are direct descendants of the `<article>` element by iterating over a `NodeList` DOM collection.

```js
const articleParagraphs = document.querySelectorAll("article > p");
for (const paragraph of articleParagraphs) {
  paragraph.classList.add("read");
}
```

Iterating over a user-defined iterable

Iterating over an object with a `[Symbol.iterator]()` method that returns a custom iterator:

```js
const iterable = {
  [Symbol.iterator]() {
    let i = 1;
    return {
      next() {
        if (i <= 3) {
          return { value: i++, done: false };
        }
        return { value: undefined, done: true };
      },
    };
  },
};

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over an object with a `[Symbol.iterator]()` generator method:

```js
const iterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};

for (const value of iterable) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterable iterators (iterators with a `[Symbol.iterator]()` method that returns `this`) are a fairly common technique to make iterators usable in syntaxes expecting iterables, such as `for...of`.

```js
let i = 1;

const iterator = {
  next() {
    if (i <= 3) {
      return { value: i++, done: false };
    }
    return { value: undefined, done: true };
  },
  [Symbol.iterator]() {
    return this;
  },
};

for (const value of iterator) {
  console.log(value);
}
// 1
// 2
// 3
```

Iterating over a generator

```js
function* source() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = source();

for (const value of generator) {
  console.log(value);
}
// 1
// 2
// 3
```

Early exiting

Execution of the `break` statement in the first loop causes it to exit early. The iterator is not finished yet, so the second loop will continue from where the first one stopped at.

js

```
const source = [1, 2, 3];

const iterator = source[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
  if (value === 1) {
    break;
  }
  console.log("This string will not be logged.");
}
// 1

// Another loop using the same iterator
// picks up where the last loop left off.
for (const value of iterator) {
  console.log(value);
}
// 2
// 3

// The iterator is used up.
// This loop will execute no iterations.
for (const value of iterator) {
  console.log(value);
}
// [No output]
```

Generators implement the `return()` method, which causes the generator function to early return when the loop exits. This makes generators not reusable between loops.

js · ✗ bad example

```
function* source() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = source();

for (const value of generator) {
  console.log(value);
  if (value === 1) {
    break;
  }
  console.log("This string will not be logged.");
}
// 1
```

(code continues on the next page)

(code continued from the previous page)

```
// The generator is used up.
// This loop will execute no iterations.
for (const value of generator) {
  console.log(value);
}
// [No output]
```

Difference between for...of and for...in

Both `for...in` and `for...of` statements iterate over something. The main difference between them is in what they iterate over.

The `for...in` statement iterates over the enumerable string properties of an object, while the `for...of` statement iterates over values that the iterable object defines to be iterated over.

The following example shows the difference between a `for...of` loop and a `for...in` loop when used with an `Array`.

js

```
Object.prototype.objCustom = function () {};
Array.prototype.arrCustom = function () {};

const iterable = [3, 5, 7];
iterable.foo = "hello";

for (const i in iterable) {
  console.log(i);
}
// "0", "1", "2", "foo", "arrCustom", "objCustom"

for (const i in iterable) {
  if (Object.hasOwn(iterable, i)) {
    console.log(i);
  }
}
// "0" "1" "2" "foo"

for (const i of iterable) {
  console.log(i);
}
// 3 5 7
```

The object `iterable` inherits the properties `objCustom` and `arrCustom` because it contains both `Object.prototype` and `Array.prototype` in its prototype chain.

The `for...in` loop logs only enumerable properties of the `iterable` object. It doesn't log array *elements* `3`, `5`, `7` or `"hello"` because those are not *properties* — they are *values*. It logs array *indexes* as well as `arrCustom` and `objCustom`, which are actual properties. If you're not sure why these properties are iterated over, there's a more thorough explanation of how array iteration and `for...in` work.

The second loop is similar to the first one, but it uses `Object.hasOwn()` to check if the found enumerable property is the object's own, i.e., not inherited. If it is, the property is logged. Properties `0`, `1`, `2` and `foo` are logged because they are own properties. Properties `arrCustom` and `objCustom` are not logged because they are inherited.

The `for...of` loop iterates and logs *values* that `iterable`, as an array (which is iterable), defines to be iterated over. The object's *elements* `3`, `5`, `7` are shown, but none of the object's *properties* are.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-for-in-and-for-of-statements`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`for...of`	38	12	13[1]	25	7	38	14[2]	25	7	3.0	38	7	1.0.0	1.0	0.12.0
async iterators	63	12	57	50	7	63	57	46	7	8.0	63	7	1.0.0	1.0	10.0.0
Closing iterators	51	14	53	38	7	51	53	41	7	5.0	51	7	1.0.0	1.0	6.5.0

1. Before Firefox 51, using the `for...of` loop construct with the `const` keyword threw a `SyntaxError` ("missing = in const declaration").
2. Before Firefox for Android 51, using the `for...of` loop construct with the `const` keyword threw a `SyntaxError` ("missing = in const declaration").

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.for_of`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Array.prototype.forEach()`
- `Map.prototype.forEach()`
- `Object.entries()`

JavaScript › Reference › Statements and declarations

function

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/function · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`function`** declaration creates a binding of a new function to a given name.

You can also define functions using the `function` expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: function declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function calcRectArea(width, height) {
  return width * height;
}

console.log(calcRectArea(5, 6));
// Expected output: 30
```

Syntax

```js
function name(param0) {
  statements
}
function name(param0, param1) {
  statements
}
function name(param0, param1, /* …, */ paramN) {
  statements
}
```

Parameters

`name`

The function name.

`param` Optional

The name of a formal parameter for the function. Maximum number of arguments varies in different engines. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements which comprise the body of the function.

Description

A `function` declaration creates a `Function` object. Each time when a function is called, it returns the value specified by the last executed `return` statement, or `undefined` if the end of the function body is reached. See functions for detailed information on functions.

`function` declarations behave like a mix of `var` and `let`:

- Like `let`, in strict mode, function declarations are scoped to the most closely containing block.
- Like `let`, function declarations at the top level of a module or within blocks in strict mode cannot be redeclared by any other declaration.
- Like `var`, function declarations at the top level of a script (strict or non-strict) become properties on `globalThis`. Function declarations at the top level of a script or function body (strict or non-strict) can be redeclared by another `function` or `var`.
- Like both, function declarations can be re-assigned, but you should avoid doing so.
- Unlike either, function declarations are hoisted together with its value and can be called anywhere in its scope.

Block-level function declaration

> **Warning:** In non-strict mode, function declarations inside blocks behave strangely. Only declare functions in blocks if you are in strict mode.

Functions can be conditionally declared — that is, a function statement can be nested within an `if` statement. However, in non-strict mode, the results are inconsistent across implementations.

```js
console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
if (false) {
  function foo() {
    return 1;
  }
}

// In Chrome:
// 'foo' name is global. typeof foo is undefined
//
// In Firefox:
// 'foo' name is global. typeof foo is undefined
//
// In Safari:
// 'foo' name is global. typeof foo is function
```

The scoping and hoisting effect won't change regardless of whether the `if` body is actually executed.

```js
console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
if (true) {
  function foo() {
    return 1;
  }
}

// In Chrome:
// 'foo' name is global. typeof foo is undefined
//
// In Firefox:
// 'foo' name is global. typeof foo is undefined
//
// In Safari:
// 'foo' name is global. typeof foo is function
```

In strict mode, block-level function declarations are scoped to that block and are hoisted to the top of the block.

```js
"use strict";

{
  foo(); // Logs "foo"
  function foo() {
    console.log("foo");
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(
  `'foo' name ${
    "foo" in globalThis ? "is" : "is not"
  } global. typeof foo is ${typeof foo}`,
);
// 'foo' name is not global. typeof foo is undefined
```

Hoisting

Function declarations in JavaScript are hoisted to the top of the enclosing function or global scope. You can use the function before you declared it:

js

```
hoisted(); // Logs "foo"

function hoisted() {
  console.log("foo");
}
```

Note that function expressions are not hoisted:

js

```
notHoisted(); // TypeError: notHoisted is not a function

var notHoisted = function () {
  console.log("bar");
};
```

Redeclarations

Whether `function` declarations can be redeclared in the same scope depends on what scope it's contained in.

At the top level of a script, `function` declarations behave like `var` and can be redeclared by another `function` or `var` but not by `let`, `const`, or `class`.

js · ✗ bad example

```
function a(b) {}
function a(b, c) {}
console.log(a.length); // 2
let a = 2; // SyntaxError: Identifier 'a' has already been declared
```

When `function` declarations are redeclared by `var`, the `var` declaration's initializer always overrides the function's value, regardless of their relative position. This is because function declarations are hoisted before any initializer gets evaluated, so the initializer comes later and overrides the value.

js

```
var a = 1;
function a() {}
console.log(a); // 1
```

At the top level of a function's body, `function` also behaves like `var` and can be redeclared or have the same name as a parameter.

js

```
function foo(a) {
  function a() {}
  console.log(typeof a);
}

foo(2); // Logs "function"
```

At the top level of a module or a block in strict mode, `function` declarations behave like `let` and cannot be redeclared by any other declaration.

js · ✗ bad example

```
// Assuming current source is a module
function foo() {}
function foo() {} // SyntaxError: Identifier 'foo' has already been declared
```

js · ✗ bad example

```
"use strict";
{
  function foo() {}
  function foo() {} // SyntaxError: Identifier 'foo' has already been declared
}
```

A `function` declaration within a `catch` block cannot have the same name as the `catch`-bound identifier, even in non-strict mode.

js · ✗ bad example

```
try {
} catch (e) {
  function e() {} // SyntaxError: Identifier 'e' has already been declared
}
```

Examples

Using function

The following code declares a function that returns the total amount of sales, when given the number of units sold of three products.

js

```
function calcSales(unitsA, unitsB, unitsC) {
  return unitsA * 79 + unitsB * 129 + unitsC * 699;
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-function-definitions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`function` statement	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Functions
- `Function`
- `function` expression
- `function*`
- `async function`
- `async function*`

JavaScript › Reference › Statements and declarations

function*

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/function* · MDN page last modified 2026-10-10

Baseline: Widely available — available across major browsers since September 2016.

The **`function*`** declaration creates a binding of a new generator function to a given name. A generator function can be exited and later re-entered, with its context (variable bindings) saved across re-entrances.

You can also define generator functions using the `function*` expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: function* declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function* generator(i) {
  yield i;
  yield i + 10;
}

const gen = generator(10);

console.log(gen.next().value);
// Expected output: 10

console.log(gen.next().value);
// Expected output: 20
```

Syntax

```js
function* name(param0) {
  statements
}
function* name(param0, param1) {
  statements
}
function* name(param0, param1, /* …, */ paramN) {
  statements
}
```

> **Note:** Generator functions do not have arrow function counterparts.

> **Note:** `function` and `*` are separate tokens, so they can be separated by whitespace or line terminators.

Parameters

`name`

The function name.

`param` Optional

The name of a formal parameter for the function. For the parameters' syntax, see the Functions reference.

`statements` Optional

The statements comprising the body of the function.

Description

A `function*` declaration creates a `GeneratorFunction` object. Each time a generator function is called, it returns a new `Generator` object, which conforms to the iterator protocol. The generator function's execution is *suspended* at some place, which is initially at the very beginning of the function body. The generator function can be called multiple times to create multiple generators simultaneously; every generator maintains its own execution context of the generator function and can be stepped independently.

The generator allows bidirectional control flow: control flow can transfer between the generator function (callee) and its caller as many times as both parties wish to. Control flow can go from the caller to the callee by calling the generator's methods: `next()`, `throw()`, and `return()`. Control flow can go from the callee to the caller by exiting the function as normal using `return` or `throw` or execution all statements, or by using the `yield` and `yield*` expressions.

When the generator's `next()` method is called, the generator function's body is executed until one of the following:

- A `yield` expression. In this case, the `next()` method returns an object with a `value` property containing the yielded value and a `done` property that is always `false`. The next time `next()` is called, the `yield` expression evaluates to the value passed to `next()`.
- A `yield*`, delegating to another iterator. In this case, this call and any future calls to `next()` on the generator is the same as calling `next()` on the delegated iterator, until the delegated iterator is

finished.

- A `return` statement (that is not intercepted by a `try...catch...finally`), or the end of the control flow which implicitly means `return undefined`. In this case, the generator is finished, and the `next()` method returns an object with a `value` property containing the returned value and a `done` property that is always `true`. Any further `next()` calls have no effect and always return `{ value: undefined, done: true }`.
- An error thrown inside the function, either via a `throw` statement or an unhandled exception. The `next()` method throws that error, and the generator is finished. Any further `next()` calls have no effect and always return `{ value: undefined, done: true }`.

When the generator's `throw()` method is called, it acts as if a `throw` statement is inserted in the generator's body at the current suspended position. Similarly, when the generator's `return()` method is called, it acts as if a `return` statement is inserted in the generator's body at the current suspended position. Both methods usually finish the generator, unless the generator function catches the completion via `try...catch...finally`.

Generators used to be a paradigm for asynchronous programming, avoiding Callback Hell by achieving Inversion of Control. Nowadays, this use case is solved with the simpler async functions model and the `Promise` object. However, generators are still useful for many other tasks, such as defining iterators in a straightforward way.

`function*` declarations behave similar to `function` declarations — they are hoisted to the top of their scope and can be called anywhere in their scope, and they can be redeclared only in certain contexts.

Examples

Basic example

```js
function* idMaker() {
  let index = 0;
  while (true) {
    yield index++;
  }
}

const gen = idMaker();

console.log(gen.next().value); // 0
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
// …
```

Example with yield*

```js
function* anotherGenerator(i) {
  yield i + 1;
  yield i + 2;
  yield i + 3;
}
```

(code continues on the next page)

(code continued from the previous page)

```
function* generator(i) {
  yield i;
  yield* anotherGenerator(i);
  yield i + 10;
}

const gen = generator(10);

console.log(gen.next().value); // 10
console.log(gen.next().value); // 11
console.log(gen.next().value); // 12
console.log(gen.next().value); // 13
console.log(gen.next().value); // 20
```

Passing arguments into Generators

js

```
function* logGenerator() {
  console.log(0);
  console.log(1, yield);
  console.log(2, yield);
  console.log(3, yield);
}

const gen = logGenerator();

// the first call of next executes from the start of the function
// until the first yield statement
gen.next(); // 0
gen.next("pretzel"); // 1 pretzel
gen.next("california"); // 2 california
gen.next("mayonnaise"); // 3 mayonnaise
```

Return statement in a generator

js

```
function* yieldAndReturn() {
  yield "Y";
  return "R";
  yield "unreachable";
}

const gen = yieldAndReturn();
console.log(gen.next()); // { value: "Y", done: false }
console.log(gen.next()); // { value: "R", done: true }
console.log(gen.next()); // { value: undefined, done: true }
```

Generator as an object property

```js
const someObj = {
  *generator() {
    yield "a";
    yield "b";
  },
};

const gen = someObj.generator();

console.log(gen.next()); // { value: 'a', done: false }
console.log(gen.next()); // { value: 'b', done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Generator as an object method

```js
class Foo {
  *generator() {
    yield 1;
    yield 2;
    yield 3;
  }
}

const f = new Foo();
const gen = f.generator();

console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Generator as a computed property

```js
class Foo {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
  }
}

const SomeObj = {
  *[Symbol.iterator]() {
    yield "a";
    yield "b";
  },
};

console.log(Array.from(new Foo())); // [ 1, 2 ]
console.log(Array.from(SomeObj)); // [ 'a', 'b' ]
```

Generators are not constructable

```js
function* f() {}
const obj = new f(); // throws "TypeError: f is not a constructor
```

Generator example

```js
function* powers(n) {
  // Endless loop to generate
  for (let current = n; ; current *= n) {
    yield current;
  }
}

for (const power of powers(2)) {
  // Controlling generator
  if (power > 32) {
    break;
  }
  console.log(power);
  // 2
  // 4
  // 8
  // 16
  // 32
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-generator-function-definitions

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`function*` statement	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`IteratorResult` object instead of throwing	49	13	29	36	10	49	29	36	10	5.0	49	10	1.0.0	1.0	6.0.0
Not constructable with `new` (ES2016)	50	13	43	37	10	50	43	37	10	5.0	50	10	1.0.0	1.0	6.0.0
Trailing comma in parameters	58	14	52	45	10	58	52	43	10	7.0	58	10	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.generator_function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions guide
- Iterators and generators guide
- Functions
- `GeneratorFunction`
- `function*` expression
- `function`
- `async function`
- `async function*`
- Iteration protocols
- `yield`
- `yield*`
- `Generator`
- Promises and Generators: control flow utopia presentation by Forbes Lindesay at JSConf (2013)
- Task.js on GitHub
- You Don't Know JS: Async & Performance, Ch.4: Generators by Kyle Simpson

---

JavaScript › Reference › Statements and declarations

if...else

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/if...else · MDN page last modified 2026-05-04

Baseline: Widely available — available across major browsers since July 2015.

The **`if...else`** statement executes a statement if a specified condition is truthy. If the condition is falsy, another statement in the optional `else` clause will be executed.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: if...else statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function testNum(a) {
  let result;
  if (a > 0) {
    result = "positive";
  } else {
    result = "NOT positive";
  }
  return result;
}

console.log(testNum(-5));
// Expected output: "NOT positive"
```

Syntax

```js
if (condition)
  statement1

// With an else clause
if (condition)
  statement1
else
  statement2
```

`condition`

An expression that is considered to be either truthy or falsy.

`statement1`

Statement that is executed if *condition* is truthy. Can be any statement, including further nested `if` statements. To execute multiple statements, use a block statement (`{ /* ... */ }`) to group those statements. To execute no statements, use an empty statement.

`statement2`

Statement that is executed if `condition` is falsy and the `else` clause exists. Can be any statement, including block statements and further nested `if` statements.

Description

Multiple `if...else` statements can be nested to create an `else if` clause. Note that there is no `elseif` (in one word) keyword in JavaScript.

```js
if (condition1)
  statement1
else if (condition2)
  statement2
else if (condition3)
  statement3
// …
else
  statementN
```

To see how this works, this is how it would look if the nesting were properly indented:

```js
if (condition1)
  statement1
else
  if (condition2)
    statement2
  else
    if (condition3)
      statement3
// …
```

The conditions are evaluated in order until one evaluates to `true`. At that point, the associated statement is executed and the rest of the `else if` clauses are skipped.

To execute multiple statements within a clause, use a block statement (`{ /* ... */ }`) to group those statements.

```js
if (condition) {
  statements1
} else {
  statements2
}
```

Not using blocks may lead to confusing behavior, especially if the code is hand-formatted. For example:

js · ✗ bad example

```
function checkValue(a, b) {
  if (a === 1)
    if (b === 2)
      console.log("a is 1 and b is 2");
  else
    console.log("a is not 1");
}
```

This code looks innocent — however, executing `checkValue(1, 3)` will log "a is not 1". This is because in the case of dangling else, the `else` clause will be connected to the closest `if` clause. Therefore, the code above, with proper indentation, would look like:

```js
function checkValue(a, b) {
  if (a === 1)
    if (b === 2)
      console.log("a is 1 and b is 2");
    else
      console.log("a is not 1");
}
```

In general, it is a good practice to always use block statements, especially in code involving nested `if` statements.

js · ✓ good example

```
function checkValue(a, b) {
  if (a === 1) {
    if (b === 2) {
      console.log("a is 1 and b is 2");
    }
  } else {
    console.log("a is not 1");
  }
}
```

Do not confuse the primitive Boolean values `true` and `false` with truthiness or falsiness of the `Boolean` object. Any value that is not `false`, `undefined`, `null`, `0`, `-0`, `NaN`, or the empty string (`""`), and any object, including a Boolean object whose value is `false`, is considered truthy when used as the condition. For example:

```js
const b = new Boolean(false);
if (b) {
  console.log("b is truthy"); // "b is truthy"
}
```

Examples

Using if...else

```js
if (cipherChar === fromChar) {
  result += toChar;
  x++;
} else {
  result += clearChar;
}
```

Using else if

Note that there is no `elseif` syntax in JavaScript. However, you can write it with a space between `else` and `if`:

```js
if (x > 50) {
  /* do something */
} else if (x > 5) {
  /* do something */
} else {
  /* do something */
}
```

Using an assignment as a condition

You should almost never have an `if...else` with an assignment like `x = y` as a condition:

js · ✗ bad example

```
if ((x = y)) {
  // …
}
```

Because unlike `while` loops, the condition is only evaluated once, so the assignment is only performed once. The code above is equivalent to:

js · ✓ good example

```
x = y;
if (x) {
  // …
}
```

Which is much clearer. However, in the rare case you find yourself wanting to do something like that, the `while` documentation has a Using an assignment as a condition section with our recommendations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-if-statement`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`if...else`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.if_else`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `block`
- `switch`
- Conditional (ternary) operator

JavaScript › Reference › Statements and declarations

import

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import · MDN page last modified 2026-11-05

Baseline: Widely available — available across major browsers since May 2018.

The static **`import`** declaration is used to import read-only live bindings which are exported by another module. The imported bindings are called *live bindings* because they are updated by the module that exported the binding, but cannot be re-assigned by the importing module.

In order to use the `import` declaration in a source file, the file must be interpreted by the runtime as a module. In HTML, this is done by adding `type="module"` to the `<script>` tag. Modules are automatically interpreted in strict mode.

There is also a function-like dynamic `import()`, which does not require scripts of `type="module"`.

Syntax

```js
import defaultExport from "module-name";
import * as name from "module-name";
import { export1 } from "module-name";
import { export1 as alias1 } from "module-name";
import { default as alias } from "module-name";
import { export1, export2 } from "module-name";
import { export1, export2 as alias2, /* … */ } from "module-name";
import { "string name" as alias } from "module-name";
import defaultExport, { export1, /* … */ } from "module-name";
import defaultExport, * as name from "module-name";
import "module-name";
```

`defaultExport`

Name that will refer to the default export from the module. Must be a valid JavaScript identifier.

`module-name`

The module to import from. Only single- and double-quoted string literals are allowed. The evaluation of the specifier is host-specified. Most hosts align with browsers and resolve the specifiers as URLs relative to the current module URL (see `import.meta.url`). Node, bundlers, and other non-browser environments often define their own features on top of this, so you should find documentation for them to understand the exact rules. The module specifier resolution section also has more information.

`name`

Name of the module namespace object that will be used as a kind of namespace when referring to the imports. Must be a valid JavaScript identifier.

`exportN`

Name of the exports to be imported. The name can be either an identifier or a string literal, depending on what `module-name` declares to export. If it is a string literal, it must be aliased to a valid identifier.

`aliasN`

Names that will refer to the named imports. Must be a valid JavaScript identifier.

The `"module-name"` may be followed by a set of import attributes, starting with the `with` keyword.

Description

`import` declarations can only be present in modules, and only at the top-level (i.e., not inside blocks, functions, etc.). If an `import` declaration is encountered in non-module contexts (for example, `<script>` tags without `type="module"`, `eval`, `new Function`, which all have "script" or "function body" as parsing goals), a `SyntaxError` is thrown. To load modules in non-module contexts, use the dynamic import syntax instead.

All imported bindings cannot be in the same scope as any other declaration, including `let`, `const`, `class`, `function`, `var`, and `import` declaration.

`import` declarations are designed to be syntactically rigid (for example, only string literal specifiers, only permitted at the top-level, all bindings must be identifiers), which allows modules to be statically analyzed and linked before getting evaluated. This allows dependencies to be loaded asynchronously and supports features like top-level await.

The `import` keyword may be followed by a "phase modifier" that stops the module import process at a particular phase:

- `import defer`
- `import source`

Each of these syntaxes is considered a distinct type of declaration.

Forms of import declarations

There are four forms of `import` declarations:

- Named import: `import { export1, export2 } from "module-name";`
- Default import: `import defaultExport from "module-name";`
- Namespace import: `import * as name from "module-name";`
- Side effect import: `import "module-name";`

Below are examples to clarify the syntax.

Named import

Given a value named `myExport` which has been exported from the module `my-module` either implicitly as `export * from "another.js"` or explicitly using the `export` statement, this inserts `myExport` into the current scope.

```js
import { myExport } from "/modules/my-module.js";
```

You can import multiple names from the same module.

```js
import { foo, bar } from "/modules/my-module.js";
```

You can rename an export when importing it. For example, this inserts `shortName` into the current scope.

```js
import { reallyReallyLongModuleExportName as shortName } from "/modules/my-module.js";
```

A module may also export a member as a string literal which is not a valid identifier, in which case you must alias it in order to use it in the current module.

```js
// -- /modules/my-module.js --
const a = 1;
export { a as "a-b" };
```

```js
import { "a-b" as a } from "/modules/my-module.js";
```

> **Note:** `import { x, y } from "mod"` is not equivalent to `import defaultExport from "mod"` and then destructuring `x` and `y` from `defaultExport`. Named and default imports are distinct syntaxes in JavaScript modules.

Default import

Default exports can be imported with the corresponding default import syntax. This version directly imports the default:

```js
import myDefault from "/modules/my-module.js";
```

Since the default export doesn't explicitly specify a name, you can give the identifier any name you like.

It is also possible to specify a default import with namespace imports or named imports. In such cases, the default import will have to be declared first. For instance:

```js
import myDefault, * as myModule from "/modules/my-module.js";
// myModule.default and myDefault point to the same binding
```

or

```js
import myDefault, { foo, bar } from "/modules/my-module.js";
```

Importing a name called `default` has the same effect as a default import. It is necessary to alias the name because `default` is a reserved word.

```js
import { default as myDefault } from "/modules/my-module.js";
```

Namespace import

The following code inserts `myModule` into the current scope, containing all the exports from the module located at `/modules/my-module.js`.

```js
import * as myModule from "/modules/my-module.js";
```

Here, `myModule` represents a *namespace* object which contains all exports as properties. For example, if the module imported above includes an export `doAllTheAmazingThings()`, you would call it like this:

```js
myModule.doAllTheAmazingThings();
```

`myModule` is a sealed object with `null` prototype. The default export is available as a key called `default`. For more information, see module namespace object.

> **Note:** JavaScript does not have wildcard imports like `import * from "module-name"`, because of the high possibility of name conflicts.

Import a module for its side effects only

Import an entire module for side effects only, without importing anything. This runs the module's global code, but doesn't actually import any values.

```js
import "/modules/my-module.js";
```

This is often used for polyfills, which mutate the global variables.

Hoisting

Import declarations are hoisted. In this case, that means that the bindings the imports introduce are available in the entire module scope, and the imported modules' side effects are produced before the rest of the module's code runs. With cyclic imports, an imported module may not have finished evaluating, so an imported binding may still be uninitialized.

```js
myModule.doAllTheAmazingThings(); // myModule.doAllTheAmazingThings is imported by the next line

import * as myModule from "/modules/my-module.js";
```

Module specifier resolution

The ECMAScript specification does not define how module specifiers are resolved and leaves it to the host environment (e.g., browsers, Node.js, Deno). Browser behavior is specified by the HTML spec, and this has become the *de facto* baseline for all environments.

There are three types of specifiers widely recognized, as implemented by the HTML spec, Node, and many others:

- *Relative specifiers* that start with `/`, `./`, or `../`, which are resolved relative to the current module URL.
- *Absolute specifiers* that are parsable URLs, which are resolved as-is.
- *Bare specifiers* that are not one of the above.

The most notable caveat for relative specifiers, especially for people familiar with the CommonJS conventions, is that browsers forbid one specifier to implicitly resolve to many potential candidates. In CommonJS, if you have `main.js` and `utils/index.js`, then all of the following will import the "default export" from `utils/index.js`:

```js
// -- main.js --
const utils = require("./utils"); // Omit the "index.js" file name
const utils = require("./utils/index"); // Omit only the ".js" extension
const utils = require("./utils/index.js"); // The most explicit form
```

On the web, this is costly because if you write `import x from "./utils"`, the browser needs to send requests to `utils`, `utils/index.js`, `utils.js`, and potentially many other URLs until it finds an importable module. Therefore, in the HTML spec, the specifier by default can only be a URL resolved relative to the current module URL. You cannot omit the file extension or the `index.js` file name. This behavior has been inherited by Node's ESM implementation, but it is not a part of the ECMAScript specification.

Note that this does not mean that `import x from "./utils"` never works on the web. The browser still sends a request to that URL, and if the server can respond with the correct content, the import will succeed. This requires the server to implement some custom resolution logic, because usually extension-less requests are understood as requests for HTML files.

Absolute specifiers can be any kind of URL that resolve to importable source code. Most notably:

- HTTP URLs are always supported on the web since most scripts already have HTTP URLs. It's supported natively by Deno (which initially predicated its entire module system on HTTP URLs), but it only has experimental support in Node via custom HTTPS loaders.

- `file:` URLs are supported by many non-browser runtimes such as Node, since scripts there already have `file:` URLs, but they are not supported by browsers due to security reasons.
- Data URLs are supported by many runtimes including browsers, Node, Deno, etc. They are useful for embedding small modules directly into the source code. Supported MIME types are those that designate importable source code, such as `text/javascript` for JavaScript, `application/json` for JSON modules, `application/wasm` for WebAssembly modules, etc. (They may still require import attributes.)

  ```js
  // HTTP URLs
  import x from "https://example.com/x.js";
  // Data URLs
  import x from "data:text/javascript,export default 42;";
  // Data URLs for JSON modules
  import x from 'data:application/json,{"foo":42}' with { type: "json" };
  ```

  `text/javascript` data URLs are still interpreted as modules, but they cannot use relative imports — because the `data:` URL scheme is not hierarchical. That is, `import x from "data:text/javascript,import y from './y.js';"` will throw an error because the relative specifier `'./y.js'` cannot be resolved.
- `node:` URLs resolve to built-in Node.js modules. They are supported by Node and other runtimes that claim compatibility with Node, such as Bun.

Bare specifiers, popularized by CommonJS, are resolved according to host-specific rules. In Node.js, package names generally resolve to packages within `node_modules` directories. For example, if you have `import x from "foo"`, then Node.js searches for the `foo` package in `node_modules` directories starting from the current module's directory and continuing through its ancestors. Browsers require import maps to resolve bare specifiers; these maps associate names with URLs rather than searching `node_modules` directories.

The module resolution algorithm can also be executed programmatically using the `import.meta.resolve` function defined by the HTML spec.

Examples

Standard Import

In this example, we create a re-usable module that exports a function to get all primes within a given range.

```js
// -- getPrimes.js --
/**
 * Returns a list of prime numbers that are smaller than `max`.
 */
export function getPrimes(max) {
  const isPrime = Array.from({ length: max }, () => true);
  isPrime[0] = isPrime[1] = false;
  isPrime[2] = true;
  for (let i = 2; i * i < max; i++) {
    if (isPrime[i]) {
      for (let j = i ** 2; j < max; j += i) {
        isPrime[j] = false;
      }
    }
```

(code continues on the next page)

(code continued from the previous page)

```
  }
  return [...isPrime.entries()]
    .filter(([, isPrime]) => isPrime)
    .map(([number]) => number);
}
```

js

```
import { getPrimes } from "/modules/getPrimes.js";

console.log(getPrimes(10)); // [2, 3, 5, 7]
```

Imported values can only be modified by the exporter

The identifier being imported is a *live binding*, because the module exporting it may re-assign it and the imported value would change. However, the module importing it cannot re-assign it. Still, any module holding an exported object can mutate the object, and the mutated value can be observed by all other modules importing the same value.

You can also observe the new value through the module namespace object.

js

```
// -- my-module.js --
export let myValue = 1;
setTimeout(() => {
  myValue = 2;
}, 500);
```

js

```
// -- main.js --
import { myValue } from "/modules/my-module.js";
import * as myModule from "/modules/my-module.js";

console.log(myValue); // 1
console.log(myModule.myValue); // 1
setTimeout(() => {
  console.log(myValue); // 2; my-module has updated its value
  console.log(myModule.myValue); // 2
  myValue = 3; // TypeError: Assignment to constant variable.
  // The importing module can only read the value but can't re-assign it.
}, 1000);
```

Importing non-JavaScript modules

Non-JavaScript modules can also be imported using the `import` statement. Some module types need to be explicitly declared using import attributes. For example, to import a JSON module, you need to specify the `type: "json"` attribute.

js

```
import data from "./data.json" with { type: "json" };
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html#sec-imports

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import`	61	16	60	48	10.1	61	60	45	10.3	8.0	61	10.3	1.0.0	1.0	13.2.0 [1,2]
Arbitrary module namespace identifier names	88	88	87	74	14.1	88	87	63	14.5	15.0	88	14.5	1.0.15	1.6	16.0.0
`import defer` **Experimental**	**No**	**No**	Prev.[3]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?
Loading is retriable (HTTP errors are not cached)	156	156	155	140	Prev.	156	155	**No**	**No**	**No**	156	**No**	?	?	?
Import attributes with `assert` syntax (formerly import assertions) **Deprecated** **Non-standard**	**91–126**	**91–126**	**No**	**No**	**No**	**91–126**	**No**	**No**	**No**	**16.0–28.0**	**91–126**	**No**	1.0.0	1.17	**16.14.0–22.0.0**
`assert {type: 'css'}` **Deprecated** **Non-standard**	**93–126**	**93–126**	**No**	**No**	**No**	**93–126**	**No**	**No**	**No**	**17.0–28.0**	**93–126**	**No**	1.0.0	**No**	**No**
`assert {type: 'json'}` **Deprecated** **Non-standard**	**91–126**	**91–126**	**No**	**No**	**No**	**91–126**	**No**	**No**	**No**	**16.0–28.0**	**91–126**	**No**	1.0.0	1.17	**17.5.0–22.0.0**[4]
Import attributes (`with` syntax)	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[5]
`with {type: 'css'}`	123	123	147	109	**No**	123	147	82	**No**	27.0	123	**No**	1.0.0	2.9	**No**
`with {type: 'json'}`	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[5]
`with {type: 'text'}`	155	155	153	139	**No**	155	153	**No**	**No**	**No**	155	**No**	1.1.5	2.8	**No**
`import source` **Experimental**	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Source phase import of WASM modules **Experimental**	**No**	**No**	Prev.[6]	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Available in service workers	91	91	147	77	15	91	147	64	15	16.0	91	15	1.0.0	**No**	**No**
Available in workers	80	80	114	67	15	80	114	57	15	13.0	80	15	1.0.0	1.0	**No**
Available in worklets	**No**	**No**	114	**No**	**No**	**No**	114	**No**	**No**	**No**	**No**	**No**	1.0.0	**No**	**No**

1. Modules must either have a filename ending in `.mjs`, or the nearest parent `package.json` file must contain `"type": "module"`. See Node's ECMAScript Modules documentation for more details.
2. Also from version 12.17.0–13.0.0: Modules must either have a filename ending in `.mjs`, or the nearest parent `package.json` file must contain `"type": "module"`. See Node's ECMAScript Modules documentation for more details.
3. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).
4. Also from version 16.15.0–17.0.0: supported.
5. Also from version 18.20.0–19.0.0: supported.
6. Disabled by default; the user must enable the preference `javascript.options.experimental.wasm_esm_integration` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.import`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `export`
- `import()`
- `import.meta`
- Import attributes

- Previewing ES6 Modules and more from ES2015, ES2016 and beyond on blogs.windows.com (2016)
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)
- Exploring JS, Ch.16: Modules by Dr. Axel Rauschmayer
- Export and Import on javascript.info

JavaScript › Reference › Statements and declarations › import

Import attributes

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import/with · MDN page last modified 2026-11-05

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

> **Note:** A previous version of this proposal used the `assert` keyword instead of `with`. The assertion feature is now non-standard. Check the browser compatibility table for details.

The **import attributes** feature instructs the runtime about how a module should be loaded, including the behavior of module resolution, fetching, parsing, and evaluation. It's supported in `import` declarations, `export...from` declarations, and dynamic `import()`.

Attributes can be attached to any kind of `import`/`export from` statement, including default import, namespace import, etc. They follow the module specifier string and start with the `with` keyword. When used with `import()`, the attributes are specified in the `options` parameter as the `with` property.

Syntax

```js
import { names } from "module-name" with {};
import { names } from "module-name" with { key: "data" };
import { names } from "module-name" with { key: "data", key2: "data2" };
import { names } from "module-name" with { key: "data", key2: "data2", /* …, */ keyN:
"dataN" };

export { names } from "module-name" with {};
export { names } from "module-name" with { key: "data" };
export { names } from "module-name" with { key: "data", key2: "data2" };
export { names } from "module-name" with { key: "data", key2: "data2", /* …, */ keyN:
"dataN" };
```

Parameters

`keyN`

An attribute key. Can be an identifier or a string literal. All keys must be unique and must be known to the runtime.

`"dataN"`

An attribute value. Must be a string literal.

Exceptions

`SyntaxError`

An unsupported `key` was specified in a *static import*.

`TypeError`

An unsupported `key` was specified in a *dynamic import*.

Note that specifying an unsupported value for a supported key may also result in an exception in some cases, depending on the key.

Description

Import attributes tell the runtime how a particular module should be loaded.

The primary use case is to load non-JS modules, such as JSON modules and CSS modules. Consider the following statement:

```js
import data from "https://example.com/data.json";
```

On the web, loading an uncached module from an HTTP URL involves an HTTP request. The response is then prepared into a JavaScript value and made available to the program by the runtime. For example, the response may look like this:

```http
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
...
{"name":"Maria"}
```

Modules are identified and parsed only according to their served media type (MIME type) — the file extension in the URL cannot be used to identify a file's type. In this case, the MIME type is `application/json`, which tells the browser that the file is JSON and must be parsed as JSON. If, for some reason (e.g., the server is hijacked or bogus), the media type in the server response is set to `text/javascript` (for JavaScript source), then the file would be parsed and executed as code. If the "JSON" file actually contains malicious code, the `import` declaration would unintentionally execute external code, posing a serious security threat.

Import attributes fix this problem by allowing the author to explicitly specify how a module should be validated. In particular, the `type` attribute allows you to validate that the file is served with a particular media type, and fails the import if a different media type is used.

For example, the code above can be written to specify that the expected type is `"json"` and the import would fail if it was served with `text/javascript` (or any media type other than a JSON MIME type like `application/json`):

```js
import data from "https://example.com/data.json" with { type: "json" };
```

The `type` attribute allows you to specify that modules are served as JSON, CSS, or plain text (and implicitly as JavaScript).

Other attributes may also be supported, and can affect the behavior of different parts of the loading process. A syntax error is thrown if an unknown attribute is used in a static import; a dynamic import rejects with a type error.

Standard attributes

The available attributes depend on the language and runtime environment. The ECMAScript standard defines the `type` attribute with the values `"json"` and `"text"`.

The HTML specification also defines the `type` attribute with values `"json"`, `"text"` and `"css"` — these are the attributes that are supported in browser environments.

JSON Modules (`{ type: "json" }`)

The `json` type indicates that the imported file must contain JSON. You can load JSON from a file into the `data` object using the following code:

```js
import data from "https://example.com/data.json" with { type: "json" };
```

If the file is served with any other media type than a JSON MIME type, the import will fail.

The `type` attribute changes how the module is fetched (the browser sends the request with `Accept: application/json` header), but does *not* change how the module is parsed or evaluated. The runtime already knows to parse the module as JSON given the response MIME type. It only uses the attribute to do *after-the-fact* checking that the `data.json` module is, in fact, a JSON module. For example, if the response header changes to `Content-Type: text/javascript` instead, the program will fail with a similar error as above.

The specification explicitly calls out `type: "json"` to be supported — if a module is asserted to be `type: "json"` and the runtime does not fail this import, then it must be parsed as JSON.

However, there's no behavior requirement otherwise: for imports without a `type: "json"` attribute, the runtime may still parse it as JSON if security is not an issue in this environment.

Browsers, on the other hand, implicitly assume that the module is JavaScript if the `type` is not specified, and fail if the module is not JavaScript (for example, JSON). This ensures that module types are always strictly validated and prevents any security risks. Non-browser runtimes such as Node and Deno align with browser semantics and enforce `type` for JSON modules.

In other words, if you omit the `type` and attempt to import a file as `"application/json"` you will usually get an error like the following:

```
Failed to load module script: Expected a JavaScript module script but the server responded
with a MIME type of "application/json". Strict MIME type checking is enforced for module
scripts per HTML spec.
```

CSS Modules (`{ type: "css" }`)

The HTML spec defines the `css` type, which imports a stylesheet into a script as a `CSSStyleSheet` object.

The code below shows how you might import a style and add it to your document. The import will throw an exception if `example_styles.css` is served with any media type other than `"text/css"`.

```js
import exampleStyles from "https://example.com/example_styles.css" with { type: "css" };

document.adoptedStyleSheets.push(exampleStyles);
```

Note that importing CSS modules into workers is usually not supported, because the CSSOM specification only exposes `CSSStyleSheet` in the window context.

Text Modules (`{ type: "text" }`)

The `text` type allows importing a module's source as a string value. You can load text from a file into the `text` string using the following code:

```js
import text from "https://example.com/file.txt" with { type: "text" };
```

The file will be requested with an `Accept: text/plain` header, but the value of the response's `Content-Type` header is ignored, and all files are parsed as UTF-8. It can contain any textual data, even JavaScript code (which is treated as plain text).

If possible, prefer `import source` to obtain a compiled module for later instantiation, or `import defer` to defer the synchronous evaluation of a module. Using `with { type: "text" }` means the module cannot reuse the cache if it was imported somewhere else, and userland evaluation of string sources may be incompatible with strict CSP settings.

Intended semantics for import attributes

An attribute can change the runtime's behavior at every stage of the module loading process:

- Resolution: the attributes are part of the module request, alongside the module specifier (the string in the `from` clause). Therefore, given the same string path, different attributes may lead to entirely different modules being loaded. For example, TypeScript supports the `resolution-mode` attribute.

  ```ts
  import type { TypeFromRequire } from "pkg" with {
    "resolution-mode": "require",
  };
  ```

- Fetching: for example, CSS modules are fetched with the `destination` set to `"style"`, JSON modules are fetched with `destination: "json"`, and text modules are fetched with `destination: "text"`. This means given the same destination URL, the server may still return different content.
- Parsing and evaluation: the runtime may use the attribute to determine how to parse and evaluate the module.

Examples

Importing JSON modules with the type attribute

In `data.json`:

```json
{
  "name": "Shilpa"
}
```

In `index.html`:

```html
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <script type="module">
      import data from "./data.json" with { type: "json" };

      const p = document.createElement("p");
      p.textContent = `name: ${data.name}`;
      document.body.appendChild(p);
    </script>
  </head>
  <body></body>
</html>
```

Start a local HTTP server (see using modules on the web) and go to the `index.html` page. You should see `Shilpa` on the page.

> **Note:** JSON modules only have one default export. You cannot do named imports from them (like `import { name } from "data.json"`).

Using import attributes with dynamic import

Import attributes are also accepted as the second parameter of the `import()` syntax.

```js
const data = await import("./data.json", {
  with: { type: "json" },
});
```

Note that, like static imports, dynamic imports are cached for the lifetime of the environment (e.g., a page or worker). If you expect this data to change (such as the latest news or a user's credits), use the Fetch API instead.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-scripts-and-modules.html#prod-WithClause

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Import attributes (`with` syntax)	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[1]
`with {type: 'css'}`	123	123	147	109	**No**	123	147	82	**No**	27.0	123	**No**	1.0.0	2.9	**No**
`with {type: 'json'}`	123	123	138	109	17.2	123	138	82	17.2	27.0	123	17.2	1.0.0	1.37	20.10.0[1]

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`with {type: 'text'}`	155	155	153	139	**No**	155	153	**No**	**No**	**No**	155	**No**	1.1.5	2.8	**No**

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.import.import_attributes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `import`
- `export`
- `import()`
- Import attributes proposal
- JSON modules proposal
- Import Text proposal

JavaScript › Reference › Statements and declarations › import

import defer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import/defer · MDN page last modified 2026-11-05

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import defer`** declaration behaves like regular `import` declarations, except that it results in a deferred module namespace object. The module and its dependencies are fetched and linked up front, but their synchronous evaluation is deferred until the namespace's properties are accessed. Modules that use top-level `await` are evaluated eagerly.

Syntax

```js
import defer * as name from "module-name";
```

`name`

Name that will refer to the deferred module namespace object. Must be a valid JavaScript identifier.

`module-name`

The module to import from. Handled the same way as the `module-name` in regular `import` declarations.

Import attributes are also supported, using a `with` clause after the module specifier.

`defer` is not a reserved word. For example, `import defer from "./module.js"` is still a regular default import whose local binding is named `defer`.

Description

By default, the `import` declaration performs many tasks at once: resolving the module specifier, fetching the module source code, parsing (potentially discovering transitive dependencies), linking, and evaluating it. This form of eager evaluation is not always desirable: it may cause slower startup, the environment for its evaluation may not be fully prepared, or the module may not need to be evaluated at all.

The *import phase modifier* allows the module import process to stop at a particular phase. By adding `defer` after `import`, the source code is linked but remains unevaluated, provided that it can be evaluated synchronously (i.e., does not use top-level `await`). Accessing an export through the deferred namespace synchronously evaluates the module and any dependencies that need to be evaluated before it. The access returns the export's value after evaluation finishes. This executes the module's top-level code, not just the code needed to initialize the requested export. Transitive dependencies imported with their own `import defer` declarations can remain deferred.

By ensuring that the paused subgraph can be evaluated synchronously, `import defer` can be "dropped in" with almost no code changes to the places that use the module:

```js
// Before:
import * as ts from "typescript";
// The full typescript module graph evaluates here

function compileFile(path) {
  const program = ts.createProgram([path], {});
}
```

```js
// After:
import defer * as ts from "typescript";
// No code is evaluated; potentially faster startup

function compileFile(path) {
  // The typescript module graph evaluates when `ts.createProgram`
  // is accessed, i.e., when `compileFile` is called.
  // If `compileFile` is never called, then the module graph
  // is never evaluated.
  const program = ts.createProgram([path], {});
}
```

> **Warning:** Deferring an import changes when its side effects occur. Do not defer modules whose side effects are needed before the rest of your code runs, such as modules that install polyfills.

Unlike `import source`, a deferred module's dependencies are loaded and the module is linked up front, catching missing dependencies or invalid imports before the module is used. Source-phase imports instead leave dependencies unloaded and allow you to control how the module is instantiated.

Unlike `import()`, the deferred module is still fetched, parsed, and linked up front, avoiding unnecessary async coloring (entire chain of function calls forced to become async). `import defer` also enjoys most benefits of a static declaration, such as better static analysis.

Note that only the "namespace import" syntax is supported. You cannot use `import defer { property } from "./my-module.js"`, etc., because the execution is triggered by property access on the

namespace object.

Caching semantics

The modifier applies to an import, not to the module itself. If another part of the application imports the same module without `defer`, the module is evaluated as usual. Both forms share the same module state, and the module's code executes at most once. Changing the import phase does not create a separate module in the cache:

```js
import defer * as ts from "typescript";
// No code is evaluated
import * as ts2 from "typescript";
// The full typescript module graph evaluates here

function compileFile(path) {
  // Accessing `ts.createProgram` no longer evaluates the subgraph
  // because it's already evaluated.
  const program = ts.createProgram([path], {});
}
```

In contrast, import attributes can affect module identity. For example, in a host that supports text modules, these two declarations request different module types:

```js
import * as mod from "./module.js";
import text from "./module.js" with { type: "text" };
```

The two imports are considered to be from different modules that happen to share the same string specifier (on the web, they will be requested with different HTTP headers). The supported attributes and their effects on loading and module identity are defined by the host.

Deferred module namespace object

A deferred module namespace object behaves much like a regular module namespace object: it has a `null` prototype, is non-extensible and sealed, and exposes read-only live bindings to the module's exports. Its string keys are enumerable and sorted in lexicographic order. The default export is available as the property named `default`.

There are three differences from a regular namespace:

- Operations that inspect exports can trigger evaluation and throw evaluation errors, as described below.
- Its `[Symbol.toStringTag]` property is `"Deferred Module"` instead of `"Module"`. This remains the case after evaluation.
- It does not expose an export named `then`, even after evaluation. Reading `namespace.then` always returns `undefined`. This prevents promise resolution from treating the namespace as a thenable and triggering evaluation. To access such an export, use a regular import, or introduce an intermediate module that re-exports `then` under a different name.

The deferred and regular namespaces for the same module are distinct objects, even after evaluation. Repeated deferred imports of the same module, whether static or dynamic, share the same deferred namespace object.

In order to implement the behavior of "triggering module evaluation when keys are accessed", the deferred module namespace is essentially a `Proxy` that intercepts the following actions to trigger

module evaluation:

- `defineProperty()` for any string key other than `then`: for example, `Object.defineProperty(namespace, "value", {})`.

  > **Note:** Because the module namespace object is non-extensible and sealed, you can't meaningfully add or change any property descriptor, including its value. Nevertheless, even if the operation fails, evaluation is still triggered.
  >
  > The proxy does not intercept `set()`. Setting properties like `namespace.value = 1;` always fails.

- `deleteProperty()` for any string key other than `then`: for example, `delete namespace.value`.

  > **Note:** You cannot actually delete any property that the namespace has. Nevertheless, even if the operation fails, evaluation is still triggered.

- `get()` and `getOwnPropertyDescriptor()` for any string key other than `then`: for example, `namespace.value`, `namespace["missing"]`, `const { default: value } = namespace`, `Object.getOwnPropertyDescriptor(namespace, "value")`.

  > **Note:** Destructuring an export at the top level therefore defeats the deferral of that module.

- `has()` for any string key other than `then`: for example, `"value" in namespace`, `Object.hasOwn(namespace, "missing")`.
- `ownKeys()`: for example, `Object.keys(namespace)`, `Object.getOwnPropertySymbols(namespace)`, `for (const key in namespace) {}`. Even enumerating only symbol keys triggers evaluation.

Merely referring to the namespace, assigning it to another variable, comparing its identity, or passing it to a function does not trigger evaluation. Neither does reading `then` or a symbol-keyed property, such as `namespace[Symbol.toStringTag]`. Calling `Object.getPrototypeOf()` or `Object.isExtensible()` does not trigger evaluation either. However, `Object.isSealed()` and `Object.isFrozen()` enumerate keys and therefore do trigger evaluation.

Top-level await

Reading a namespace property is synchronous, so it cannot wait for asynchronous module evaluation. Modules that contain top-level `await` are evaluated eagerly, along with the dependencies required to evaluate them. This includes modules reached through further deferred imports. The importing module waits for this asynchronous evaluation before running its own body.

If the directly imported module contains top-level `await`, its evaluation is not deferred. If only some of its dependencies contain top-level `await`, those dependencies are evaluated eagerly, but the synchronous parts of the graph that are not required for their evaluation can remain deferred. See Deferring a module with an asynchronous dependency.

Errors

Loading, parsing, and linking errors are not deferred. For example, a missing module, a syntax error in a dependency, or an unresolved named import prevents the importing module from running, even if the deferred namespace is never accessed. Errors from eagerly evaluated asynchronous dependencies also prevent the importing module from running.

Errors thrown during deferred evaluation are thrown synchronously by the operation that triggers evaluation. You can catch them with `try...catch` around that operation. The error is cached: subsequent operations that trigger evaluation throw the same error instead of retrying the module's code. This also applies if another import previously caused the module's evaluation to fail.

An operation that triggers evaluation throws a `TypeError` if the module or its dependencies are not ready for synchronous evaluation. This can happen with cyclic imports, when an access would require a module that is still being evaluated. An `import defer` declaration does not make every cyclic dependency safe to access during initialization. A readiness failure itself does not mark the requested module as having failed evaluation: a later access can succeed once its dependencies are ready.

Examples

Evaluating a module on first use

The following module initializes a lookup table when its top-level code runs:

```js
// -- squares.js --
console.log("Initializing squares");
const squares = Array.from({ length: 10000 }, (_, index) => index ** 2);

export function getSquare(index) {
  return squares[index];
}
```

The importing module can expose a synchronous function without initializing the table until it is needed:

```js
// -- main.js --
import defer * as squares from "./squares.js";

console.log("Ready");

export function showSquare(index) {
  console.log(squares.getSquare(index));
}

showSquare(3); // Logs "Initializing squares", then 9
showSquare(4); // Logs 16; initialization is not repeated
```

`"Ready"` is logged before `"Initializing squares"`. If `showSquare()` is never called and no other import causes `squares.js` to be evaluated, the lookup table is never initialized.

Deferring a module with an asynchronous dependency

In this example, `report.js` (which itself is synchronous) depends on both an asynchronous configuration module and a synchronous formatting module.

```js
// -- config.js --
export const locale = await Promise.resolve("en-US");
console.log("Configuration ready");
```

```js
// -- format.js --
console.log("Formatting module evaluated");

export function format(value, locale) {
  return new Intl.NumberFormat(locale).format(value);
}
```

```js
// -- report.js --
import { locale } from "./config.js";
import { format } from "./format.js";

console.log("Report module evaluated");

export function createReport(value) {
  return format(value, locale);
}
```

```js
// -- main.js --
import defer * as report from "./report.js";

console.log("Main module evaluated");
console.log(report.createReport(1000));
```

The output is:

```
Configuration ready
Main module evaluated
Formatting module evaluated
Report module evaluated
1,000
```

`config.js` is evaluated before `main.js` because it contains top-level `await`. Neither `format.js` nor `report.js` needs to run to evaluate `config.js`, so their evaluation is deferred until `report.createReport` is accessed.

Catching evaluation errors

```js
// -- broken.js --
export const value = 1;
throw new Error("Initialization failed");
```

```js
// -- main.js --
import defer * as broken from "./broken.js";

for (let attempt = 0; attempt < 2; attempt++) {
  try {
    console.log(broken.value);
  } catch (error) {
    console.log(error.message); // "Initialization failed" on both attempts
  }
}
```

Even though `value` was initialized before the exception was thrown, accessing it through the deferred namespace throws the cached evaluation error.

Exporting a deferred namespace

There is no `export defer` syntax (see export for more information). You can import a deferred namespace and then export its binding without triggering evaluation:

```js
// -- features.js --
import defer * as report from "./report.js";

export { report };
```

```js
// -- main.js --
import { report } from "./features.js";

// Accessing an export through report triggers its deferred evaluation.
console.log(report.createReport(1000));
```

Specifications

Specification
Deferred Imports Evaluation `https://tc39.es/proposal-defer-import-eval/#sec-imports`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import defer` Experimental	**No**	**No**	Prev.[1]	**No**	Prev.	**No**	**No**	**No**	**No**	**No**	**No**	**No**	?	?	?

1. Disabled by default; the user must enable the preference `javascript.options.experimental.defer_import_eval` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.import.defer`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript modules guide
- `import`
- `import.defer()`

import source

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import/source · MDN page last modified 2026-11-05

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`import source`** declaration is similar to the regular `import` declaration, except that it results in an object that represents the module's compiled source code. The module is fetched and compiled, but its dependencies are not loaded and it is not linked or evaluated. It can be imperatively evaluated later, such as by using dynamic import or `WebAssembly.instantiate()`.

To use `import source`, the target module must be of a kind that supports source phase imports. Currently, only WebAssembly modules support source phase imports, and result in `WebAssembly.Module` objects. JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

Syntax

```js
import source x from "module-name";
```

`x`

Name that will refer to the module source object. Must be a valid JavaScript identifier.

`module-name`

The module to import from. Handled the same way as the `module-name` in regular `import` declarations.

Import attributes are also supported, using a `with` clause after the module specifier.

`source` is not a reserved word. For example, `import source from "./module.js"` is still a regular default import whose local binding is named `source`.

Exceptions

`SyntaxError`

Thrown if the target module does not support source phase imports.

Description

By default, the `import` declaration performs many tasks at once: resolving the module specifier, fetching the module source code, parsing (potentially discovering transitive dependencies), linking, and evaluating it. This form of eager evaluation is not always desirable, especially when the module source code is intended to be evaluated in some other context, such as a worker thread.

The *import phase modifier* allows the module import process to stop at a particular phase. By adding `source` after `import`, the source code is parsed and compiled but remains unlinked and unevaluated. The source import does not load the module's transitive dependencies.

Traditionally, the only way to run a part of the pipeline was to manually implement the entirety of it. For example, with WebAssembly, you first use `fetch()` to retrieve the module's binary bytes, and later

compile them using `WebAssembly.compile()`, or compile the response stream using `WebAssembly.compileStreaming()`, into a `WebAssembly.Module` object.

```js
const myModuleSource = await WebAssembly.compileStreaming(
  fetch("./my-module.wasm"),
);
```

This kind of manual workflow is problematic for several reasons:

- It involves too much low-level manipulation, which may be inconsistent with the default module loader's behavior, such as caching, HTTP header settings, and module specifier resolution.
- It is imperative, making it difficult to reason about the module's dependencies and their evaluation order, especially for bundlers or other analysis tools.
- Userland evaluation of the raw source code is incompatible with strict CSP settings.

Using source phase imports avoids these problems.

```js
import source myModuleSource from "./my-module.wasm";
```

The object obtained from this import is an instance of a subclass of `AbstractModuleSource`. Each module type that supports source phase imports defines its own subclass. For WebAssembly, the subclass is `WebAssembly.Module`. This object is cached for the specific module so that later source imports resolving to the same module return the exact same object.

Unlike `import defer`, using source import also defers linking. Loading dependencies and linking up front catches missing dependencies or invalid imports before the module is used. Source-phase imports instead leave dependencies unloaded and allow you to control how the module is instantiated. For example, you can instantiate a source-imported WebAssembly module multiple times with different imports, or send it to a worker for instantiation there.

Unlike `import()`, the source-imported module is fetched and compiled up front without evaluating it. `import source` also enjoys most benefits of a static declaration, such as better static analysis.

Note that only the "default import" syntax is supported. You cannot use `import source { property } from "./my-module.wasm"`, etc.

Importing WebAssembly source

For WebAssembly, you supply the imported values when instantiating the module:

```js
import source myModuleSource from "./my-module.wasm";

const instance = await WebAssembly.instantiate(myModuleSource, {
  env: { log: console.log },
});
const { exports } = instance;
```

Unlike an ordinary WebAssembly module import, this creates a new instance with the imports you provide, rather than using the cached instance linked by the module loader. Its `exports` object is not a JavaScript module namespace object. For example, WebAssembly globals are exposed as `WebAssembly.Global` objects through `instance.exports`, whereas the ESM integration exposes their values to JavaScript importers.

> **Note:** Support for source phase imports of WebAssembly does not imply support for ordinary WebAssembly module imports. A host may support `import source mod from "./mod.wasm"` while rejecting `import * as ns from "./mod.wasm"`.

Source imports use the host's module loader, including its compilation settings. They are not necessarily equivalent to calling `WebAssembly.compileStreaming()` with default options. For example, Node.js enables JavaScript string builtins for Wasm imports and rejects reserved import and export names. Use the direct WebAssembly compilation APIs when you need custom compilation options.

Importing JavaScript source

For JavaScript (assuming the proposed `import(moduleSource)` support, also in ECMAScript Module Phase Imports):

```js
import * as namespace from "./my-module.js";

// Is equivalent to:

import source myModuleSource from "./my-module.js";

const namespace = await import(myModuleSource);
```

Caching semantics

The modifier applies to an import, not to the module itself. If another part of the application imports the same module without `source`, the module is evaluated as usual. Both forms share the same module state, and the module's code executes at most once. Changing the import phase does not create a separate module in the cache:

```js
import * as x from "foo";
import source xSource from "foo";

console.log((await import(xSource)) === x); // true
```

> **Note:** This cached instance is reused when you use `import()`, but not when you manually instantiate WebAssembly. Each call to `WebAssembly.instantiate(modSource, imports)` or `new WebAssembly.Instance(modSource, imports)` creates a new instance and runs its initialization, including any start function. In contrast, `WebAssembly.compileStreaming()` only compiles; it does not execute the module or use the ESM module cache.

In contrast, import attributes can affect module identity. For example, in a host that supports text modules, these two declarations request different module types:

```js
import * as mod from "./module.js";
import text from "./module.js" with { type: "text" };
```

The two imports are considered to be from different modules that happen to share the same string specifier (on the web, they will be requested with different HTTP headers). The supported attributes and their effects on loading and module identity are defined by the host.

Examples

Instantiating a module with different imports

Suppose `counter.wasm` imports an immutable `i32` global named `initial` from `env`, initializes an internal counter from it, and exports a function named `increment`. You can reuse its compiled source with different imports and independent internal counters:

```js
import source counterSource from "./counter.wasm";

const first = await WebAssembly.instantiate(counterSource, {
  env: { initial: 0 },
});
const second = await WebAssembly.instantiate(counterSource, {
  env: { initial: 100 },
});

console.log(first.exports.increment()); // 1
console.log(first.exports.increment()); // 2
console.log(second.exports.increment()); // 101
```

Each instance has its own internal state, although explicitly imported memories, tables, or other mutable objects can be shared between instances. You can also send the compiled source to a worker for instantiation there.

Exporting a module source

There is no `export source` syntax (see `export` for more information). You can import a module source and then export its binding without instantiating or evaluating the module:

```js
// -- sources.js --
import source counterSource from "./counter.wasm";

export { counterSource };
```

```js
// -- main.js --
import { counterSource } from "./sources.js";

const instance = new WebAssembly.Instance(counterSource, {
  env: { initial: 0 },
});
```

Specifications

Specification
Source Phase Imports https://tc39.es/proposal-source-phase-imports/#sec-imports

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`import source` Experimental	**No**	**No**	153	**No**	**No**	**No**	153	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**
Source phase import of WASM modules Experimental	**No**	**No**	Prev.[1]	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**	**No**

1. Disabled by default; the user must enable the preference `javascript.options.experimental.wasm_esm_integration` (set to `true`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.import.import_source`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript modules guide
- `import`
- `import.source()`
- `AbstractModuleSource`

JavaScript › Reference › Statements and declarations

Labeled statement

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/label · MDN page last modified 2026-09-04

Baseline: Widely available — available across major browsers since July 2015.

A **labeled statement** is any statement that is prefixed with an identifier. You can jump to this label using a `break` or `continue` statement nested within the labeled statement.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Labeled statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let i, j;

loop1: for (i = 0; i < 3; i++) {
  loop2: for (j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Expected output:
// "i = 0, j = 0"
// "i = 0, j = 1"
// "i = 0, j = 2"
// "i = 1, j = 0"
```

Syntax

```js
label:
  statement
```

`label`

Any JavaScript identifier that is not a reserved word.

`statement`

A JavaScript statement. `break` can be used within any labeled statement, and `continue` can be used within labeled looping statements.

Description

You can use a label to identify a statement, and later refer to it using a `break` or `continue` statement. Note that JavaScript has *no* `goto` statement; you can only use labels with `break` or `continue`.

Any `break` or `continue` that references `label` must be contained within the `statement` that's labeled by `label`. Think about `label` as a variable that's only available in the scope of `statement`.

If a `break label;` statement is encountered when executing `statement`, execution of `statement` terminates, and execution continues at the statement immediately following the labeled statement.

`continue label;` can only be used if `statement` is one of the looping statements. If a `continue label;` statement is encountered when executing `statement`, execution of `statement` continues at the next iteration of the loop. `continue;` without a label can only continue the innermost loop, while `continue label;` allows continuing any given loop even when the statement is nested within other loops.

A statement can have multiple labels. In this case, the labels are all functionally equivalent.

Examples

Using a labeled continue with for loops

```js
// The first for statement is labeled "loop1"
loop1: for (let i = 0; i < 3; i++) {
  // The second for statement is labeled "loop2"
  loop2: for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      continue loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Logs:
// i = 0, j = 0
// i = 0, j = 1
// i = 0, j = 2
// i = 1, j = 0
// i = 2, j = 0
// i = 2, j = 1
// i = 2, j = 2
```

Notice how it skips both "i = 1, j = 1" and "i = 1, j = 2".

Using a labeled break with for loops

```js
let i, j;

// The first for statement is labeled "loop1"
loop1: for (i = 0; i < 3; i++) {
  // The second for statement is labeled "loop2"
  loop2: for (j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break loop1;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}

// Logs:
// i = 0, j = 0
// i = 0, j = 1
// i = 0, j = 2
// i = 1, j = 0
```

Notice the difference with the previous `continue` example: when `break loop1` is encountered, the execution of the outer loop is terminated, so there are no further logs beyond "i = 1, j = 0"; when `continue loop1` is encountered, the execution of the outer loop continues at the next iteration, so only "i = 1, j = 1" and "i = 1, j = 2" are skipped.

Using a labeled continue statement

Given an array of items and an array of tests, this example counts the number of items that pass all the tests.

```js
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let itemsPassed = 0;

itemIteration: for (const item of items) {
  for (const test of tests) {
    if (!test.pass(item)) {
      continue itemIteration;
    }
  }

  itemsPassed++;
}
```

Note how the `continue itemIteration;` statement skips the rest of the tests for the current item as well as the statement that updates the `itemsPassed` counter, and continues with the next item. If you don't use a label, you would need to use a boolean flag instead.

```js
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let itemsPassed = 0;

for (const item of items) {
  let passed = true;
  for (const test of tests) {
    if (!test.pass(item)) {
      passed = false;
      break;
    }
  }
  if (passed) {
    itemsPassed++;
  }
}
```

Using a labeled break statement

Given an array of items and an array of tests, this example determines whether all items pass all tests.

```js
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let allPass = true;

itemIteration: for (const item of items) {
  for (const test of tests) {
    if (!test.pass(item)) {
      allPass = false;
      break itemIteration;
    }
  }
}
```

Again, if you don't use a label, you would need to use a boolean flag instead.

```js
// Numbers from 1 to 100
const items = Array.from({ length: 100 }, (_, i) => i + 1);
const tests = [
  { pass: (item) => item % 2 === 0 },
  { pass: (item) => item % 3 === 0 },
  { pass: (item) => item % 5 === 0 },
];
let allPass = true;
```

(code continues on the next page)

(code continued from the previous page)

```
for (const item of items) {
  let passed = true;
  for (const test of tests) {
    if (!test.pass(item)) {
      passed = false;
      break;
    }
  }
  if (!passed) {
    allPass = false;
    break;
  }
}
```

Using a labeled block with break

You can label statements other than loops, such as simple blocks, but only `break` statements can reference non-loop labels.

js

```
foo: {
  console.log("face");
  break foo;
  console.log("this will not be executed");
}
console.log("swap");

// Logs:
// "face"
// "swap"
```

Labeled function declarations

Labels can only be applied to statements, not declarations. There is a legacy grammar that allows function declarations to be labeled in non-strict code:

js

```
L: function F() {}
```

In strict mode code, however, this will throw a `SyntaxError`:

js · ✗ bad example

```
"use strict";
L: function F() {}
// SyntaxError: functions cannot be labelled
```

Non-plain functions, such as generator functions and async functions can neither be labeled in strict code, nor in non-strict code:

js · ✗ bad example

```
L: function* F() {}
// SyntaxError: generator functions cannot be labelled
```

The labeled function declaration syntax is deprecated and you should not use it, even in non-strict code. You cannot actually jump to this label within the function body.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-labelled-statements`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`label`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.label`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `break`
- `continue`

JavaScript › Reference › Statements and declarations

let

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let · MDN page last modified 2026-09-04

Baseline: Widely available — available across major browsers since September 2016.

The **`let`** declaration declares re-assignable, block-scoped local variables, optionally initializing each to a value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: let declaration". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let x = 1;

if (x === 1) {
  let x = 2;

  console.log(x);
  // Expected output: 2
}

console.log(x);
// Expected output: 1
```

Syntax

```js
let name1;
let name1 = value1;
let name1 = value1, name2 = value2;
let name1, name2 = value2;
let name1 = value1, name2, /* …, */ nameN = valueN;
```

Parameters

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN` Optional

Initial value of the variable. It can be any legal expression. Default value is `undefined`.

Description

The scope of a variable declared with `let` is one of the following curly-brace-enclosed syntaxes that most closely contains the `let` declaration:

- Block statement
- `switch` statement
- `try...catch` statement
- Body of one of the `for` statements, if the `let` is in the header of the statement
- Function body
- Static initialization block

Or if none of the above applies:

- The current module, for code running in module mode
- The global scope, for code running in script mode.

Compared with `var`, `let` declarations have the following differences:

- `let` declarations are scoped to blocks as well as functions.
- `let` declarations can only be accessed after the place of declaration is reached (see temporal dead zone). For this reason, `let` declarations are commonly regarded as non-hoisted.
- `let` declarations do not create properties on `globalThis` when declared at the top level of a script.
- `let` declarations cannot be redeclared by any other declaration in the same scope.
- `let` begins *declarations*, not *statements*. That means you cannot use a lone `let` declaration as the body of a block (which makes sense, since there's no way to access the variable).

  js · ✗ bad example

  ```
  if (true) let a = 1; // SyntaxError: Lexical declaration cannot appear in a single-
  statement context
  ```

Note that `let` is allowed as an identifier name when declared with `var` or `function` in non-strict mode, but you should avoid using `let` as an identifier name to prevent unexpected syntax ambiguities.

Many style guides (including MDN's) recommend using `const` over `let` whenever a variable is not reassigned in its scope. This makes the intent clear that a variable's type (or value, in the case of a primitive) can never change. Others may prefer `let` for non-primitives that are mutated.

The list that follows the `let` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list.

Temporal dead zone (TDZ)

A variable declared with `let`, `const`, or `class` is said to be in a "temporal dead zone" (TDZ) from the start of the block until code execution reaches the place where the variable is declared and initialized.

While inside the TDZ, the variable has not been initialized with a value, and any attempt to access it will result in a `ReferenceError`. The variable is initialized with a value when execution reaches the place in the code where it was declared. If no initial value was specified with the variable declaration, it will be initialized with a value of `undefined`.

This differs from `var` variables, which will return a value of `undefined` if they are accessed before they are declared. The code below demonstrates the different result when `let` and `var` are accessed in code before the place where they are declared.

js · ✗ bad example

```
{
  // TDZ starts at beginning of scope
  console.log(bar); // "undefined"
  console.log(foo); // ReferenceError: Cannot access 'foo' before initialization
  var bar = 1;
  let foo = 2; // End of TDZ (for foo)
}
```

The term "temporal" is used because the zone depends on the order of execution (time) rather than the order in which the code is written (position). For example, the code below works because, even though the function that uses the `let` variable appears before the variable is declared, the function is *called* outside the TDZ.

js

```
{
  // TDZ starts at beginning of scope
  const func = () => console.log(letVar); // OK

  // Within the TDZ letVar access throws `ReferenceError`

  let letVar = 3; // End of TDZ (for letVar)
  func(); // Called outside TDZ!
}
```

Using the `typeof` operator for a variable in its TDZ will throw a `ReferenceError`:

js · ✗ bad example

```
{
  typeof i; // ReferenceError: Cannot access 'i' before initialization
  let i = 10;
}
```

This differs from using `typeof` for undeclared variables, and variables that hold a value of `undefined`:

```js
console.log(typeof undeclaredVariable); // "undefined"
```

> **Note:** `let` and `const` declarations are only processed when the current script gets processed. If you have two `<script>` elements running in script mode within one HTML, the first script is not subject to the TDZ restrictions for top-level `let` or `const` variables declared in the second script, although if you declare a `let` or `const` variable in the first script, declaring it again in the second script will cause a redeclaration error.

Redeclarations

`let` declarations cannot be in the same scope as any other declaration, including `let`, `const`, `class`, `function`, `var`, and `import` declaration.

js · ✗ bad example

```
{
  let foo;
  let foo; // SyntaxError: Identifier 'foo' has already been declared
}
```

A `let` declaration within a function's body cannot have the same name as a parameter. A `let` declaration within a `catch` block cannot have the same name as the `catch`-bound identifier.

js · ✗ bad example

```
function foo(a) {
  let a = 1; // SyntaxError: Identifier 'a' has already been declared
}
try {
} catch (e) {
  let e; // SyntaxError: Identifier 'e' has already been declared
}
```

If you're experimenting in a REPL, such as the Firefox web console (**Tools** > **Web Developer** > **Web Console**), and you run two `let` declarations with the same name in two separate inputs, you may get the same re-declaration error. See further discussion of this issue in Firefox bug 1580891. The Chrome console allows `let` re-declarations between different REPL inputs.

You may encounter errors in `switch` statements because there is only one block.

js · ✗ bad example

```
let x = 1;

switch (x) {
  case 0:
    let foo;
    break;
  case 1:
    let foo; // SyntaxError: Identifier 'foo' has already been declared
    break;
}
```

To avoid the error, wrap each `case` in a new block statement.

```js
let x = 1;

switch (x) {
  case 0: {
    let foo;
    break;
  }
  case 1: {
    let foo;
    break;
  }
}
```

Examples

Scoping rules

Variables declared by `let` have their scope in the block for which they are declared, as well as in any contained sub-blocks. In this way, `let` works very much like `var`. The main difference is that the scope of a `var` variable is the entire enclosing function:

```js
function varTest() {
  var x = 1;
  {
    var x = 2; // same variable!
    console.log(x); // 2
  }
  console.log(x); // 2
}

function letTest() {
  let x = 1;
  {
    let x = 2; // different variable
    console.log(x); // 2
  }
  console.log(x); // 1
}
```

At the top level of programs and functions, `let`, unlike `var`, does not create a property on the global object. For example:

```js
var x = "global";
let y = "global";
console.log(this.x); // "global"
console.log(this.y); // undefined
```

TDZ combined with lexical scoping

The following code results in a `ReferenceError` at the line shown:

js · ✗ bad example

```
function test() {
  var foo = 33;
  if (foo) {
    let foo = foo + 55; // ReferenceError
  }
}
test();
```

The `if` block is evaluated because the outer `var foo` has a value. However due to lexical scoping this value is not available inside the block: the identifier `foo` *inside* the `if` block is the `let foo`. The expression `foo + 55` throws a `ReferenceError` because initialization of `let foo` has not completed — it is still in the temporal dead zone.

This phenomenon can be confusing in a situation like the following. The instruction `let n of n.a` is already inside the scope of the `for...of` loop's block. So, the identifier `n.a` is resolved to the property `a` of the `n` object located in the first part of the instruction itself (`let n`). This is still in the temporal dead zone as its declaration statement has not been reached and terminated.

js · ✗ bad example

```
function go(n) {
  // n here is defined!
  console.log(n); // { a: [1, 2, 3] }

  for (let n of n.a) {
    //          ^ ReferenceError
    console.log(n);
  }
}

go({ a: [1, 2, 3] });
```

Other situations

When used inside a block, `let` limits the variable's scope to that block. Note the difference between `var`, whose scope is inside the function where it is declared.

js

```
var a = 1;
var b = 2;

{
  var a = 11; // the scope is global
  let b = 22; // the scope is inside the block

  console.log(a); // 11
  console.log(b); // 22
}

console.log(a); // 11
console.log(b); // 2
```

However, this combination of `var` and `let` declarations below is a `SyntaxError` because `var` not being block-scoped, leading to them being in the same scope. This results in an implicit re-declaration of the variable.

js · ✗ bad example

```
let x = 1;

{
  var x = 2; // SyntaxError for re-declaration
}
```

Declaration with destructuring

The left-hand side of each = can also be a binding pattern. This allows creating multiple variables at once.

js

```
const result = /(a+)(b+)(c+)/.exec("aaabcc");
let [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-let-and-const-declarations

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
let	49[1,2]	14[3,4]	44[5,6,7]	17	10	49[1,2]	44[8,9,10]	18	10	5.0[11,12]	49[1,2]	10	1.0.0	1.0	6.0.0

1. Also from version 41–49: partial support.
2. Also from version 41–49: Strict mode is required.
3. Also from version 12–14: partial support.
4. Also from version 12–14: In Edge 12 and 13, `let` within a `for` loop initializer does not create a separate variable for each loop iteration as defined by ES2015. Instead, it behaves as though the loop were wrapped in a scoping block with the `let` immediately before the loop.
5. Before Firefox 44, `let` is only available to code blocks in HTML wrapped in a `<script type="application/javascript;version=1.7">` block (or higher version) and has different semantics (e.g. no temporal dead zone).
6. Before Firefox 46, a `TypeError` is thrown on redeclaration instead of a `SyntaxError`.
7. Firefox 54 adds support of `let` in workers.
8. Before Firefox for Android 44, `let` is only available to code blocks in HTML wrapped in a `<script type="application/javascript;version=1.7">` block (or higher version) and has different semantics (e.g. no temporal dead zone).
9. Before Firefox for Android 46, a `TypeError` is thrown on redeclaration instead of a `SyntaxError`.
10. Firefox for Android 54 adds support of `let` in workers.
11. Also from version 4.0–5.0: partial support.
12. Also from version 4.0–5.0: Strict mode is required.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.let`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- var
- const
- Hoisting
- ES6 In Depth: let and const on hacks.mozilla.org (2015)
- Breaking changes in let and const in Firefox 44 on blog.mozilla.org (2015)
- You Don't Know JS: Scope & Closures, Ch.3: Function vs. Block Scope by Kyle Simpson
- What is the Temporal Dead Zone? on Stack Overflow
- What is the difference between using let and var? on Stack Overflow
- Why was the name 'let' chosen for block-scoped variable declarations in JavaScript? on Stack Overflow

---

JavaScript › Reference › Statements and declarations

return

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/return · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **return** statement ends function execution and specifies a value to be returned to the function caller.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: return statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getRectArea(width, height) {
  if (width > 0 && height > 0) {
    return width * height;
  }
  return 0;
}

console.log(getRectArea(3, 4));
// Expected output: 12

console.log(getRectArea(-3, 4));
// Expected output: 0
```

Syntax

js

```
return;
return expression;
```

expression Optional

The expression whose value is to be returned. If omitted, undefined is returned.

Description

The `return` statement can only be used within function bodies. When a `return` statement is used in a function body, the execution of the function is stopped. The `return` statement has different effects when placed in different functions:

- In a plain function, the call to that function evaluates to the return value.
- In an async function, the produced promise is resolved with the returned value.
- In a generator function, the produced generator object's `next()` method returns `{ done: true, value: returnedValue }`.
- In an async generator function, the produced async generator object's `next()` method returns a promise fulfilled with `{ done: true, value: returnedValue }`.

If a `return` statement is executed within a `try` block, its `finally` block, if present, is first executed, before the value is actually returned.

Automatic semicolon insertion

The syntax forbids line terminators between the `return` keyword and the expression to be returned.

js · ✗ bad example

```
return
a + b;
```

The code above is transformed by automatic semicolon insertion (ASI) into:

js

```
return;
a + b;
```

This makes the function return `undefined` and the `a + b` expression is never evaluated. This may generate a warning in the console.

To avoid this problem (to prevent ASI), you could use parentheses:

js

```
return (
  a + b
);
```

Examples

Interrupt a function

A function immediately stops at the point where `return` is called.

js

```
function counter() {
  // Infinite loop
  for (let count = 1; ; count++) {
    console.log(`${count}A`); // Until 5
    if (count === 5) {
      return;
    }
    console.log(`${count}B`); // Until 4
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  console.log(`${count}C`); // Never appears
}

counter();

// Logs:
// 1A
// 1B
// 2A
// 2B
// 3A
// 3B
// 4A
// 4B
// 5A
```

Returning a function

See also the article about Closures.

js

```
function magic() {
  return function calc(x) {
    return x * 42;
  };
}

const answer = magic();
answer(1337); // 56154
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-return-statement

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
return	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.return`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Functions
- Closures

JavaScript › Reference › Statements and declarations

switch

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/switch · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`switch`** statement evaluates an expression, matching the expression's value against a series of `case` clauses, and executes statements after the first `case` clause with a matching value, until a `break` statement is encountered. The `default` clause of a `switch` statement will be jumped to if no `case` matches the expression's value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: switch statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const expr = "Papayas";
switch (expr) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Mangoes":
  case "Papayas":
    console.log("Mangoes and papayas are $2.79 a pound.");
    // Expected output: "Mangoes and papayas are $2.79 a pound."
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}
```

Syntax

js

```
switch (expression) {
  case caseExpression1:
    statements
  case caseExpression2:
    statements
  // …
  case caseExpressionN:
    statements
  default:
    statements
}
```

`expression`

An expression whose result is matched against each `case` clause.

`caseExpressionN` Optional

A `case` clause used to match against `expression`. If the value of `expression` matches the value of any `caseExpressionN`, execution starts from the first statement after that `case` clause until either the end of the `switch` statement or the first encountered `break`.

`default` Optional

A `default` clause; if provided, this clause is executed if the value of `expression` doesn't match any of the `case` clauses. A `switch` statement can only have one `default` clause.

Description

A `switch` statement first evaluates its expression. It then looks for the first `case` clause whose expression evaluates to the same value as the result of the input expression (using the strict equality comparison) and transfers control to that clause, executing all statements following that clause.

The clause expressions are only evaluated when necessary — if a match is already found, subsequent `case` clause expressions will not be evaluated, even when they will be visited by fall-through.

```js
switch (undefined) {
  case console.log(1):
  case console.log(2):
}
// Only logs 1
```

If no matching `case` clause is found, the program looks for the optional `default` clause, and if found, transfers control to that clause, executing statements following that clause. If no `default` clause is found, the program continues execution at the statement following the end of `switch`. By convention, the `default` clause is the last clause, but it does not need to be so. A `switch` statement may only have one `default` clause; multiple `default` clauses will result in a `SyntaxError`.

Breaking and fall-through

You can use the `break` statement within a `switch` statement's body to break out early, often when all statements between two `case` clauses have been executed. Execution will continue at the first statement following `switch`.

If `break` is omitted, execution will proceed to the next `case` clause, even to the `default` clause, regardless of whether the value of that clause's expression matches. This behavior is called "fall-through".

```js
const foo = 0;
switch (foo) {
  case -1:
    console.log("negative 1");
    break;
  case 0: // Value of foo matches this criteria; execution starts from here
    console.log(0);
  // Forgotten break! Execution falls through
  case 1: // no break statement in 'case 0:' so this case will run as well
    console.log(1);
    break; // Break encountered; will not continue into 'case 2:'
  case 2:
    console.log(2);
    break;
  default:
    console.log("default");
}
// Logs 0 and 1
```

In the appropriate context, other control-flow statements also have the effect of breaking out of the `switch` statement. For example, if the `switch` statement is contained in a function, then a `return` statement terminates the execution of the function body and therefore the `switch` statement. If the `switch` statement is contained in a loop, then a `continue` statement stops the `switch` statement and jumps to the next iteration of the loop.

Lexical scoping

The `case` and `default` clauses are like labels: they indicate possible places that control flow may jump to. However, they don't create lexical scopes themselves (neither do they automatically break out — as demonstrated above). For example:

js · ✗ bad example

```
const action = "say_hello";
switch (action) {
  case "say_hello":
    const message = "hello";
    console.log(message);
    break;
  case "say_hi":
    const message = "hi";
    console.log(message);
    break;
  default:
    console.log("Empty action received.");
}
```

This example will output the error "Uncaught SyntaxError: Identifier 'message' has already been declared", because the first `const message = 'hello';` conflicts with the second `const message = 'hi';` declaration, even when they're within their own separate case clauses. Ultimately, this is due to both `const` declarations being within the same block scope created by the `switch` body.

To fix this, whenever you need to use `let` or `const` declarations in a `case` clause, wrap it in a block.

js

```
const action = "say_hello";
switch (action) {
  case "say_hello": {
    const message = "hello";
    console.log(message);
    break;
  }
  case "say_hi": {
    const message = "hi";
    console.log(message);
    break;
  }
  default: {
    console.log("Empty action received.");
  }
}
```

This code will now output `hello` in the console as it should, without any errors.

Examples

Using switch

In the following example, if `expr` evaluates to `Bananas`, the program matches the value with case `case 'Bananas'` and executes the associated statement. When `break` is encountered, the program breaks out of `switch` and executes the statement following `switch`. If `break` were omitted, the statement for the `case 'Cherries'` would also be executed.

```js
switch (expr) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Apples":
    console.log("Apples are $0.32 a pound.");
    break;
  case "Bananas":
    console.log("Bananas are $0.48 a pound.");
    break;
  case "Cherries":
    console.log("Cherries are $3.00 a pound.");
    break;
  case "Mangoes":
  case "Papayas":
    console.log("Mangoes and papayas are $2.79 a pound.");
    break;
  default:
    console.log(`Sorry, we are out of ${expr}.`);
}

console.log("Is there anything else you'd like?");
```

Putting the default clause between two case clauses

If no match is found, execution will start from the `default` clause, and execute all statements after that.

```js
const foo = 5;
switch (foo) {
  case 2:
    console.log(2);
    break; // it encounters this break so will not continue into 'default:'
  default:
    console.log("default");
  // fall-through
  case 1:
    console.log("1");
}
```

It also works when you put `default` before all other `case` clauses.

Taking advantage of fall-through

This method takes advantage of the fact that if there is no `break` below a `case` clause, execution will continue to the next `case` clause regardless if that `case` meets the criteria.

The following is an example of a single operation sequential `case` statement, where four different values perform exactly the same.

```js
const Animal = "Giraffe";
switch (Animal) {
  case "Cow":
  case "Giraffe":
  case "Dog":
  case "Pig":
    console.log("This animal is not extinct.");
    break;
  case "Dinosaur":
  default:
    console.log("This animal is extinct.");
}
```

The following is an example of a multiple-operation sequential `case` clause, where, depending on the provided integer, you can receive different output. This shows you that it will traverse in the order that you put the `case` clauses, and it does not have to be numerically sequential. In JavaScript, you can even mix in definitions of strings into these `case` statements as well.

```js
const foo = 1;
let output = "Output: ";
switch (foo) {
  case 0:
    output += "So ";
  case 1:
    output += "What ";
    output += "Is ";
  case 2:
    output += "Your ";
  case 3:
    output += "Name";
  case 4:
    output += "?";
    console.log(output);
    break;
  case 5:
    output += "!";
    console.log(output);
    break;
  default:
    console.log("Please pick a number from 0 to 5!");
}
```

The output from this example:

Value	Log text
`foo` is `NaN` or not `1`, `2`, `3`, `4`, `5`, or `0`	Please pick a number from 0 to 5!
`0`	Output: So What Is Your Name?
`1`	Output: What Is Your Name?
`2`	Output: Your Name?
`3`	Output: Name?
`4`	Output: ?

Value	Log text
5	Output: !

An alternative to if...else chains

You may often find yourself doing a series of `if...else` matches.

```js
if ("fetch" in globalThis) {
  // Fetch a resource with fetch
} else if ("XMLHttpRequest" in globalThis) {
  // Fetch a resource with XMLHttpRequest
} else {
  // Fetch a resource with some custom AJAX logic
}
```

This pattern is not doing a sequence of `===` comparisons, but you can still convert it to a `switch` construct.

```js
switch (true) {
  case "fetch" in globalThis:
    // Fetch a resource with fetch
    break;
  case "XMLHttpRequest" in globalThis:
    // Fetch a resource with XMLHttpRequest
    break;
  default:
    // Fetch a resource with some custom AJAX logic
    break;
}
```

The `switch (true)` pattern as an alternative to `if...else` is especially useful if you want to utilize the fall-through behavior.

```js
switch (true) {
  case isSquare(shape):
    console.log("This shape is a square.");
  // Fall-through, since a square is a rectangle as well!
  case isRectangle(shape):
    console.log("This shape is a rectangle.");
  case isQuadrilateral(shape):
    console.log("This shape is a quadrilateral.");
    break;
  case isCircle(shape):
    console.log("This shape is a circle.");
    break;
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-switch-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`switch`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.switch`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `if...else`

JavaScript › Reference › Statements and declarations

throw

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/throw · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`throw`** statement throws a user-defined exception. Execution of the current function will stop (the statements after `throw` won't be executed), and control will be passed to the first `catch` block in the call stack. If no `catch` block exists among caller functions, the program will terminate.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: throw statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function getRectArea(width, height) {
  if (isNaN(width) || isNaN(height)) {
    throw new Error("Parameter is not a number!");
  }
}

try {
  getRectArea(3, "A");
} catch (e) {
  console.error(e);
  // Expected output: Error: Parameter is not a number!
}
```

Syntax

js

```
throw expression;
```

`expression`

The expression to throw.

Description

The `throw` statement is valid in all contexts where statements can be used. Its execution generates an exception that penetrates through the call stack. For more information on error bubbling and handling, see Control flow and error handling.

The `throw` keyword can be followed by any kind of expression, for example:

```js
throw error; // Throws a previously defined value (e.g. within a catch block)
throw new Error("Required"); // Throws a new Error object
```

In practice, the exception you throw should *always* be an `Error` object or an instance of an `Error` subclass, such as `RangeError`. This is because code that catches the error may expect certain properties, such as `message`, to be present on the caught value. For example, web APIs typically throw `DOMException` instances, which inherit from `Error.prototype`.

Automatic semicolon insertion

The syntax forbids line terminators between the `throw` keyword and the expression to be thrown.

js · ✗ bad example

```
throw
new Error();
```

The code above is transformed by automatic semicolon insertion (ASI) into:

```js
throw;
new Error();
```

This is invalid code, because unlike `return`, `throw` must be followed by an expression.

To avoid this problem (to prevent ASI), you could use parentheses:

```js
throw (
  new Error()
);
```

Examples

Throwing a user-defined error

This example defines a function that throws a `TypeError` if the input is not of the expected type.

```js
function isNumeric(x) {
  return ["number", "bigint"].includes(typeof x);
}

function sum(...values) {
  if (!values.every(isNumeric)) {
    throw new TypeError("Can only add numbers");
  }
  return values.reduce((a, b) => a + b);
}
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(sum(1, 2, 3)); // 6
try {
  sum("1", "2");
} catch (e) {
  console.error(e); // TypeError: Can only add numbers
}
```

Throwing an existing object

This example calls a callback-based async function, and throws an error if the callback receives an error.

js

```
readFile("foo.txt", (err, data) => {
  if (err) {
    throw err;
  }
  console.log(data);
});
```

Errors thrown this way are not catchable by the caller and will cause the program to crash unless (a) the `readFile` function itself catches the error, or (b) the program is running in a context that catches top-level errors. You can handle errors more naturally by using the `Promise()` constructor.

js

```
function readFilePromise(path) {
  return new Promise((resolve, reject) => {
    readFile(path, (err, data) => {
      if (err) {
        reject(err);
      }
      resolve(data);
    });
  });
}

try {
  const data = await readFilePromise("foo.txt");
  console.log(data);
} catch (err) {
  console.error(err);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-throw-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`throw`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.throw`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `try...catch`
- `Error`

JavaScript › Reference › Statements and declarations

try...catch

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch · MDN page last modified 2026-03-09

Baseline: Widely available — available across major browsers since July 2015.

The **`try...catch`** statement is comprised of a `try` block and either a `catch` block, a `finally` block, or both. The code in the `try` block is executed first, and if it throws an exception, the code in the `catch` block will be executed. The code in the `finally` block will always be executed before control flow exits the entire construct.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: try...catch statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
try {
  nonExistentFunction();
} catch (error) {
  console.error(error);
  // Expected output: ReferenceError: nonExistentFunction is not defined
  // (Note: the exact output may be browser-dependent)
}
```

Syntax

js

```
try {
  tryStatements
} catch (exceptionVar) {
  catchStatements
} finally {
  finallyStatements
}
```

`tryStatements`

The statements to be executed.

`catchStatements`

Statement that is executed if an exception is thrown in the `try` block.

`exceptionVar` Optional

An optional identifier or pattern to hold the caught exception for the associated `catch` block. If the `catch` block does not use the exception's value, you can omit the `exceptionVar` and its surrounding parentheses.

`finallyStatements`

Statements that are executed before control flow exits the `try...catch...finally` construct. These statements execute regardless of whether an exception was thrown or caught.

Description

The `try` statement always starts with a `try` block. Then, a `catch` block or a `finally` block must be present. It's also possible to have both `catch` and `finally` blocks. This gives us three forms for the `try` statement:

- `try...catch`
- `try...finally`
- `try...catch...finally`

Unlike other constructs such as `if` or `for`, the `try`, `catch`, and `finally` blocks must be *blocks*, instead of single statements.

js · ✗ bad example

```
try doSomething(); // SyntaxError
catch (e) console.log(e);
```

A `catch` block contains statements that specify what to do if an exception is thrown in the `try` block. If any statement within the `try` block (or in a function called from within the `try` block) throws an exception, control is immediately shifted to the `catch` block. If no exception is thrown in the `try` block, the `catch` block is skipped.

The `finally` block will always execute before control flow exits the `try...catch...finally` construct. It always executes, regardless of whether an exception was thrown or caught.

You can nest one or more `try` statements. If an inner `try` statement does not have a `catch` block, the enclosing `try` statement's `catch` block is used instead.

You can also use the `try` statement to handle JavaScript exceptions. See the JavaScript Guide for more information on JavaScript exceptions.

Catch binding

When an exception is thrown in the `try` block, `exceptionVar` (i.e., the `e` in `catch (e)`) holds the exception value. You can use this binding to get information about the exception that was thrown. This binding is only available in the `catch` block's scope.

It doesn't need to be a single identifier. You can use a destructuring pattern to assign multiple identifiers at once.

```js
try {
  throw new TypeError("oops");
} catch ({ name, message }) {
  console.log(name); // "TypeError"
  console.log(message); // "oops"
}
```

The bindings created by the `catch` clause live in the same scope as the `catch` block, so any variables declared in the `catch` block cannot have the same name as the bindings created by the `catch` clause. (There's one exception to this rule, but it's a deprecated syntax.)

js · ✗ bad example

```js
try {
  throw new TypeError("oops");
} catch ({ name, message }) {
  var name; // SyntaxError: Identifier 'name' has already been declared
  let message; // SyntaxError: Identifier 'message' has already been declared
}
```

The exception binding is writable. For example, you may want to normalize the exception value to make sure it's an `Error` object.

```js
try {
  throw "Oops; this is not an Error object";
} catch (e) {
  if (!(e instanceof Error)) {
    e = new Error(e);
  }
  console.error(e.message);
}
```

If you don't need the exception value, you can omit it along with the enclosing parentheses.

```js
function isValidJSON(text) {
  try {
    JSON.parse(text);
    return true;
  } catch {
    return false;
  }
}
```

The finally block

The `finally` block contains statements to execute after the `try` block and `catch` block(s) execute, but before the statements following the `try...catch...finally` block. Control flow will always enter the `finally` block, which can proceed in one of the following ways:

- Immediately after the control flow exits the `try` block in a `try...finally` construct (either after the last statement or a `throw`, `return`, `break`, or `continue` statement);
- Immediately after the control flow exits the `catch` block in a `try...catch...finally` construct;
- Immediately after the control flow exits the `try` block in a `try...catch...finally` construct, unless it exits via a `throw` statement (in which case control flow enters the `catch` block first).

If the `finally` block is entered after a control flow statement (`return`, `throw`, `break`, `continue`) in the `try` or `catch` block, the effect of this statement is deferred until after the last statement executed in the `finally` block. For example, if an exception is thrown from the `try` block, even when there's no `catch` block to handle the exception, the `finally` block still executes, and the exception is thrown immediately after the `finally` block finishes executing.

However, there is one exception to this rule: if the last statement executed in the `finally` block is itself a control flow statement, that statement will override the effect of the previous one (no deferral); see returning from a `finally` block for examples. It is generally a bad idea to use control flow statements (`return`, `throw`, `break`, `continue`) in the `finally` block because they can override the effect of previously executed control flow statements, which is rarely intended. Most of the time, the `finally` block should be reserved for cleanup code that does not modify the main logic.

Examples

Unconditional catch block

When a `catch` block is used, the `catch` block is executed when any exception is thrown from within the `try` block. For example, when the exception occurs in the following code, control transfers to the `catch` block.

```js
try {
  throw new Error("My exception"); // generates an exception
} catch (e) {
  // statements to handle any exceptions
  logMyErrors(e); // pass exception object to error handler
}
```

The `catch` block specifies an identifier (`e` in the example above) that holds the value of the exception; this value is only available in the scope of the `catch` block.

Conditional catch blocks

You can create "Conditional `catch` blocks" by combining `try...catch` blocks with `if...else if...else` structures, like this:

```js
try {
  myRoutine(); // may throw three types of exceptions
} catch (e) {
  if (e instanceof TypeError) {
    // statements to handle TypeError exceptions
  } else if (e instanceof RangeError) {
    // statements to handle RangeError exceptions
  } else if (e instanceof EvalError) {
    // statements to handle EvalError exceptions
  } else {
    // statements to handle any unspecified exceptions
    logMyErrors(e); // pass exception object to error handler
  }
}
```

A common use case for this is to only catch (and silence) a small subset of expected errors, and then re-throw the error in other cases:

```js
try {
  myRoutine();
} catch (e) {
  if (e instanceof RangeError) {
    // statements to handle this very common expected error
  } else {
    throw e; // re-throw the error unchanged
  }
}
```

This may mimic the syntax from other languages, like Java:

```java
try {
  myRoutine();
} catch (RangeError e) {
  // statements to handle this very common expected error
}
// Other errors are implicitly re-thrown
```

Nested try blocks

First, let's see what happens with this:

```js
try {
  try {
    throw new Error("oops");
  } finally {
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "finally"
// "outer" "oops"
```

Now, if we already caught the exception in the inner `try` block by adding a `catch` block:

```js
try {
  try {
    throw new Error("oops");
  } catch (ex) {
    console.error("inner", ex.message);
  } finally {
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "inner" "oops"
// "finally"
```

And now, let's rethrow the error.

```js
try {
  try {
    throw new Error("oops");
  } catch (ex) {
    console.error("inner", ex.message);
    throw ex;
  } finally {
    console.log("finally");
  }
} catch (ex) {
  console.error("outer", ex.message);
}

// Logs:
// "inner" "oops"
// "finally"
// "outer" "oops"
```

Any given exception will be caught only once by the nearest enclosing `catch` block unless it is rethrown. Of course, any new exceptions raised in the "inner" block (because the code in `catch` block may do something that throws), will be caught by the "outer" block.

Resource cleanup using finally

The following example shows one use case for the `finally` block. The code opens a file and then executes statements that use the file; the `finally` block makes sure the file always closes after it is used even if an exception was thrown.

```js
openMyFile();
try {
  // tie up a resource
  writeMyFile(theData);
} finally {
  closeMyFile(); // always close the resource
  // any uncaught exception is deferred here
}
```

In the same way, the effect of any `return` statement in the `try` block is deferred at the end of the `finally` block, although the return value expression is evaluated before entering the `finally` block.

```js
function safeWriteMyFile() {
  openMyFile();
  try {
    return writeMyFile(theData); // function call is evaluated
  } finally {
    closeMyFile(); // always close the resource
    // return is deferred here
  }
}
```

Returning from a finally block

The following example illustrates how control flow statements in the `finally` block behave. When control flow exits the `try` block via the first `return` statement, the return value expression (`order.sort()`) is evaluated before entering the `finally` block, and the function is planned to return that value after the `finally` block finishes executing. However, the `return` statement in the `finally` block overrides the effect of the previous `return` statement, including its return value.

```js
function doIt() {
  const order = ["z"];
  try {
    order.push("try");
    return order.sort(); // "z" is now after "try"
  } finally {
    order.push("finally");
    return order;
  }
}
doIt();
// returns ["try", "z", "finally"], not ["finally", "try", "z"] or ["try", "z"]
```

The same logic applies to other control flow statements. Here, the function is first planned to throw the value `"catch"`, but instead returns the value `"finally"`.

```js
function doIt() {
  try {
    throw "try"; // makes control flow enter the `catch` block
  } catch {
    throw "catch"; // makes control flow enter the `finally` block
  } finally {
    return "finally"; // returns "finally" instead of throwing "catch"
  }
}
doIt(); // returns "finally"
```

Again, control flow statements are discouraged in the `finally` block because this effect is likely not intended.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-try-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`try...catch`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Optional catch binding	66	79	58	53	11.1	66	58	47	11.3	9.0	66	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.try_catch`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`
- `throw`

---

using

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/using · MDN page last modified 2026-01-28

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`using`** declaration declares block-scoped local variables that are *synchronously disposed*. Like `const`, variables declared with `using` must be initialized and cannot be reassigned. The variable's value must be either `null`, `undefined`, or an object with a `[Symbol.dispose]()` method. When the variable goes out of scope, the `[Symbol.dispose]()` method of the object is called, to ensure that resources are freed.

Syntax

```js
using name1 = value1;
using name1 = value1, name2 = value2;
using name1 = value1, name2 = value2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier and *not* a destructuring binding pattern.

`valueN`

Initial value of the variable. It can be any legal expression but its value must be either `null`, `undefined`, or an object with a `[Symbol.dispose]()` method.

Description

This declaration can be used:

- Inside a block
- Inside any function body or class static initialization block
- At the top level of a module
- In the initializer of a `for`, `for...of`, or `for await...of` loop

Most notably, it cannot be used:

- At the top level of a script, because script scopes are persistent.

- At the top level of a `switch` statement.
- In the initializer of a `for...in` loop. Because the loop variable can only be a string or symbol, this doesn't make sense.

A `using` declares a disposable resource that's tied to the lifetime of the variable's scope (block, function, module, etc.). When the scope exits, the resource is disposed of synchronously. The variable is allowed to have value `null` or `undefined`, so the resource can be optionally present.

When the variable is first declared and its value is non-nullish, a *disposer* is retrieved from the object. If the `[Symbol.dispose]` property doesn't contain a function, a `TypeError` is thrown. This disposer is saved to the scope.

When the variable goes out of scope, the disposer is called. If the scope contains multiple `using` or `await using` declarations, all disposers are run in the reverse order of declaration, regardless of the type of declaration. All disposers are guaranteed to run (much like the `finally` block in `try...catch...finally`). All errors thrown during disposal, including the initial error that caused the scope exit (if applicable), are all aggregated inside one `SuppressedError`, with each earlier exception as the `suppressed` property and the later exception as the `error` property. This `SuppressedError` is thrown after disposal is complete.

`using` ties resource management to lexical scopes, which is both convenient and sometimes confusing. There are many ways to preserve the variable's value when the variable itself is out of scope, so you may hold a reference to an already-disposed resource. See below for some examples where it may not behave how you expect. If you want to hand-manage resource disposal, while maintaining the same error handling guarantees, you can use `DisposableStack` instead.

Examples

In the following examples, we assume a simple `Resource` class that has a `getValue` method and a `[Symbol.dispose]()` method:

```js
class Resource {
  value = Math.random();
  #isDisposed = false;

  getValue() {
    if (this.#isDisposed) {
      throw new Error("Resource is disposed");
    }
    return this.value;
  }

  [Symbol.dispose]() {
    this.#isDisposed = true;
    console.log("Resource disposed");
  }
}
```

`using` in a block

The resource declared with `using` is disposed when exiting the block.

```js
{
  using resource = new Resource();
  console.log(resource.getValue());
  // resource disposed here
}
```

`using` in a function

You can use `using` in a function body. In this case, the resource is disposed when the function finishes executing, immediately before the function returns.

```js
function example() {
  using resource = new Resource();
  return resource.getValue();
}
```

Here, `resource[Symbol.dispose]()` will be called after `getValue()`, before the `return` statement executes.

The resource may outlive the declaration, in case it's captured by a closure:

```js
function example() {
  using resource = new Resource();
  return () => resource.getValue();
}
```

In this case, if you call `example()()`, you will always execute `getValue` on a resource that's already disposed, because the resource was disposed when `example` returns. In case you want to dispose the resource immediately after the callback has been called once, consider this pattern:

```js
function example() {
  const resource = new Resource();
  return () => {
    using resource2 = resource;
    return resource2.getValue();
  };
}
```

Here, we *alias* a `const`-declared resource to a `using`-declared resource, so that the resource is only disposed after the callback is called; note that if it is never called then the resource will never be cleaned up.

`using` in a module

You can use `using` at the top level of a module. In this case, the resource is disposed when the module finishes executing.

```js
using resource = new Resource();
export const value = resource.getValue();
// resource disposed here
```

`export using` is invalid syntax, but you can `export` a variable declared elsewhere using `using`:

```js
using resource = new Resource();
export { resource };
```

This is still discouraged, because the importer will always receive a disposed resource. Similar to the closure problem, this causes the value of resource to live longer than the variable.

`using` with `for...of`

You can use `using` in the initializer of a `for...of` loop. In this case, the resource is disposed on every loop iteration.

```js
const resources = [new Resource(), new Resource(), new Resource()];
for (using resource of resources) {
  console.log(resource.getValue());
  // resource disposed here
}
```

Multiple `using`

The following are two equivalent ways to declare multiple disposable resources:

```js
using resource1 = new Resource(),
  resource2 = new Resource();

// OR

using resource1 = new Resource();
using resource2 = new Resource();
```

In both cases, when the scope exits, `resource2` is disposed before `resource1`. This is because `resource2` may have a dependency on `resource1`, so it's disposed first to ensure that `resource1` is still available when `resource2` is disposed.

Optional `using`

`using` allows the variable to have a value of `null` or `undefined`, so the resource can be optionally present. This means that, for example, if you are checking the availability of a resource of some kind:

```js
function acquireResource() {
  // Imagine some real-world relevant condition here,
  // such as whether there's space to allocate for this resource
  if (Math.random() < 0.5) {
    return null;
  }
  return new Resource();
}
```

You don't have to do this:

js · ✗ bad example

```
const maybeResource = acquireResource();

if (maybeResource) {
  using resource = maybeResource;
  console.log(resource.getValue());
} else {
  console.log(undefined);
}
```

And can instead do this:

js · ✓ good example

```
using resource = acquireResource();
console.log(resource?.getValue());
```

`using` declaration without using the variable

You can achieve automatic resource disposing using `using`, without even using the variable. This is very useful for setting up a context within a block, such as creating a lock:

js

```
{
  using _ = new Lock();
  // Perform concurrent operations here
  // Lock disposed (released) here
}
```

Note that _ is a normal identifier, but it's a convention to use it as a "throwaway" variable. To create multiple unused variables, you need to use distinct names, for example by using a variable name prefixed with _.

Initialization and temporal dead zones

`using` variables are subject to the same temporal dead zone restriction as `let` and `const` variables. This means that you can't access the variable before the initialization—the valid lifetime of the resource is strictly from its initialization to the end of its scope. This is enables RAII-style resource management.

js

```
let useResource;
{
  useResource = () => resource.getValue();
  useResource(); // Error: Cannot access 'resource' before initialization
  using resource = new Resource();
  useResource(); // Valid
}
useResource(); // Error: Resource is disposed
```

Error handling

The `using` declaration is the most useful for managing resource disposal in the presence of errors. If you are not careful, some resources may leak because the error prevents code afterwards from executing.

```js
function handleResource(resource) {
  if (resource.getValue() > 0.5) {
    throw new Error("Resource value too high");
  }
}

try {
  using resource = new Resource();
  handleResource(resource);
} catch (e) {
  console.error(e);
}
```

This will successfully catch the error thrown by `handleResource` and log it, and no matter if `handleResource` throws an error or not, the resource is disposed before exiting the `try` block.

Here, if you don't use `using`, you may do something like:

js · ✗ bad example

```js
try {
  const resource = new Resource();
  handleResource(resource);
  resource[Symbol.dispose]();
} catch (e) {
  console.error(e);
}
```

But, if `handleResource()` throws an error, then control never reaches `resource[Symbol.dispose]()`, and the resource is leaked. Furthermore, if you have two resources, then errors thrown in earlier disposals may prevent later disposals from running, leading to more leaks.

Consider a more complicated case where the disposer itself throws an error:

```js
class CantDisposeMe {
  #name;
  constructor(name) {
    this.#name = name;
  }
  [Symbol.dispose]() {
    throw new Error(`Can't dispose ${this.#name}`);
  }
}

let error;

try {
  using resource1 = new CantDisposeMe("resource1");
  using resource2 = new CantDisposeMe("resource2");
  throw new Error("Error in main block");
} catch (e) {
  error = e;
}
```

You can inspect the error thrown in your browser's console. It has the following structure:

```
SuppressedError: An error was suppressed during disposal
  suppressed: SuppressedError: An error was suppressed during disposal
    suppressed: Error: Can't dispose resource1
    error: Error: Error in main block
  error: Error: Can't dispose resource2
```

As you can see, `error` contains all the errors that were thrown during disposal, as a `SuppressedError`. Each additional error is added as the `error` property, and the original error is added as the `suppressed` property.

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-let-and-const-declarations

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`using` declaration	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.0.23	1.37	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.using`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript resource management
- `const`
- `let`
- `await using`
- `Symbol.dispose`
- `DisposableStack`

JavaScript › Reference › Statements and declarations

var

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/var · MDN page last modified 2026-08-27

Baseline: Widely available — available across major browsers since July 2015.

The **var** statement declares function-scoped or globally-scoped variables, optionally initializing each to a value.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: var statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
var x = 1;

if (x === 1) {
  var x = 2;

  console.log(x);
  // Expected output: 2
}

console.log(x);
// Expected output: 2
```

Syntax

js

```
var name1;
var name1 = value1;
var name1 = value1, name2 = value2;
var name1, name2 = value2;
var name1 = value1, name2, /* …, */ nameN = valueN;
```

`nameN`

The name of the variable to declare. Each must be a legal JavaScript identifier or a destructuring binding pattern.

`valueN` Optional

Initial value of the variable. It can be any legal expression. Default value is `undefined`.

Description

The scope of a variable declared with `var` is one of the following curly-brace-enclosed syntaxes that most closely contains the `var` statement:

- Function body
- Static initialization block

Or if none of the above applies:

- The current module, for code running in module mode
- The global scope, for code running in script mode.

```js
function foo() {
  var x = 1;
  function bar() {
    var y = 2;
    console.log(x); // 1 (function `bar` closes over `x`)
    console.log(y); // 2 (`y` is in scope)
  }
  bar();
  console.log(x); // 1 (`x` is in scope)
  console.log(y); // ReferenceError, `y` is scoped to `bar`
}

foo();
```

Importantly, other block constructs, including block statements, `try...catch`, `switch`, headers of one of the `for` statements, do not create scopes for `var`, and variables declared with `var` inside such a block can continue to be referenced outside the block.

```js
for (var a of [1, 2, 3]);
console.log(a); // 3
```

In a script, a variable declared using `var` is added as a non-configurable property of the global object. This means its property descriptor cannot be changed and it cannot be deleted using `delete`. JavaScript has automatic memory management, and it would make no sense to be able to use the `delete` operator on a global variable.

```
js · ✗ bad example
"use strict";
var x = 1;
Object.hasOwn(globalThis, "x"); // true
delete globalThis.x; // TypeError in strict mode. Fails silently otherwise.
delete x; // SyntaxError in strict mode. Fails silently otherwise.
```

In both Node.js CommonJS modules and native ECMAScript modules, top-level variable declarations are scoped to the module, and are not added as properties to the global object.

The list that follows the `var` keyword is called a *binding list* and is separated by commas, where the commas are *not* comma operators and the `=` signs are *not* assignment operators. Initializers of later variables can refer to earlier variables in the list and get the initialized value.

Hoisting

`var` declarations, wherever they occur in a script, are processed before any code within the script is executed. Declaring a variable anywhere in the code is equivalent to declaring it at the top. This also means that a variable can appear to be used before it's declared. This behavior is called *hoisting*, as it appears that the variable declaration is moved to the top of the function, static initialization block, or script source in which it occurs.

> **Note:** `var` declarations are only hoisted to the top of the current script. If you have two `<script>` elements within one HTML, the first script cannot access variables declared by the second before the second script has been processed and executed.

```js
bla = 2;
var bla;
```

This is implicitly understood as:

```js
var bla;
bla = 2;
```

For that reason, it is recommended to always declare variables at the top of their scope (the top of global code and the top of function code) so it's clear which variables are scoped to the current function.

Only a variable's declaration is hoisted, not its initialization. The initialization happens only when the assignment statement is reached. Until then the variable remains `undefined` (but declared):

```js
function doSomething() {
  console.log(bar); // undefined
  var bar = 111;
  console.log(bar); // 111
}
```

This is implicitly understood as:

```js
function doSomething() {
  var bar;
  console.log(bar); // undefined
  bar = 111;
  console.log(bar); // 111
}
```

Redeclarations

Duplicate variable declarations using `var` will not trigger an error, even in strict mode, and the variable will not lose its value, unless the declaration has an initializer.

```js
var a = 1;
var a = 2;
console.log(a); // 2
var a;
console.log(a); // 2; not undefined
```

`var` declarations can also be in the same scope as a `function` declaration. Both the function declaration and the `var` declaration are hoisted to the top, so the function value is only accessible from the start of its scope until the variable's initializer or first assignment, regardless of the two declarations' relative positions in the source code.

```js
console.log(typeof a); // "function"
function a() {}
var a;
console.log(typeof a); // "function"
```

```js
var a = 1;
function a() {}
console.log(a); // 1
```

`var` declarations cannot be in the same scope as a `let`, `const`, `class`, or `import` declaration.

js · ✗ bad example

```
var a = 1;
let a = 2; // SyntaxError: Identifier 'a' has already been declared
```

Because `var` declarations are not scoped to blocks, this also applies to the following case:

js · ✗ bad example

```
let a = 1;
{
  var a = 1; // SyntaxError: Identifier 'a' has already been declared
}
```

It does not apply to the following case, where `let` is in a child scope of `var`, not the same scope:

js · ✓ good example

```
var a = 1;
{
  let a = 2;
}
```

A `var` declaration within a function's body can have the same name as a parameter.

```js
function foo(a) {
  var a = 1;
  console.log(a);
}

foo(2); // Logs 1
```

A `var` declaration within a `catch` block can have the same name as the `catch`-bound identifier, but only if the `catch` binding is a simple identifier, not a destructuring pattern. This is a deprecated syntax and you should not rely on it. In this case, the declaration is hoisted to outside the `catch` block, but any value assigned within the `catch` block is not visible outside.

js · ✗ bad example

```
try {
  throw new Error();
} catch (e) {
  var e = 2; // Works
}
console.log(e); // undefined
```

Examples

Declaring and initializing two variables

```js
var a = 0,
  b = 0;
```

Assigning two variables with single string value

```js
var a = "A";
var b = a;
```

This is equivalent to:

```js
var a, b = a = "A";
```

Be mindful of the order:

```js
var x = y,
  y = "A";
console.log(x, y); // undefined A
```

Here, x and y are declared before any code is executed, but the assignments occur later. At the time `x = y` is evaluated, y exists so no `ReferenceError` is thrown and its value is `undefined`. So, x is assigned the undefined value. Then, y is assigned the value `"A"`.

Initialization of several variables

Be careful of the `var x = y = 1` syntax — y is not actually declared as a variable, so `y = 1` is an unqualified identifier assignment, which creates a global variable in non-strict mode.

```js
var x = 0;
function f() {
  var x = y = 1; // Declares x locally; declares y globally.
}
f();

console.log(x, y); // 0 1

// In non-strict mode:
// x is the global one as expected;
// y is leaked outside of the function, though!
```

The same example as above but with a strict mode:

```js
"use strict";

var x = 0;
function f() {
  var x = y = 1; // ReferenceError: y is not defined
}
f();

console.log(x, y);
```

Implicit globals and outer function scope

Variables that appear to be implicit globals may be references to variables in an outer function scope:

```js
var x = 0; // Declares x within file scope, then assigns it a value of 0.

console.log(typeof z); // "undefined", since z doesn't exist yet

function a() {
  var y = 2; // Declares y within scope of function a, then assigns it a value of 2.

  console.log(x, y); // 0 2

  function b() {
    x = 3; // Assigns 3 to existing file scoped x.
    y = 4; // Assigns 4 to existing outer y.
    z = 5; // Creates a new global variable z, and assigns it a value of 5.
    // (Throws a ReferenceError in strict mode.)
  }

  b(); // Creates z as a global variable.
  console.log(x, y, z); // 3 4 5
}

a(); // Also calls b.
console.log(x, z); // 3 5
console.log(typeof y); // "undefined", as y is local to function a
```

Declaration with destructuring

The left-hand side of each = can also be a binding pattern. This allows creating multiple variables at once.

```js
const result = /(a+)(b+)(c+)/.exec("aaabcc");
var [, a, b, c] = result;
console.log(a, b, c); // "aaa" "b" "cc"
```

For more information, see Destructuring.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-variable-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
var	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.var`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- let
- const

JavaScript › Reference › Statements and declarations

while

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/while · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **while** statement creates a loop that executes a specified statement as long as the test condition evaluates to true. The condition is evaluated before executing the statement.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: while statement". It runs live in the browser there; its source code follows.*

js · interactive example source

```
let n = 0;

while (n < 3) {
  n++;
}

console.log(n);
// Expected output: 3
```

Syntax

js

```
while (condition)
  statement
```

condition

An expression evaluated *before* each pass through the loop. If this condition evaluates to true, `statement` is executed. When condition evaluates to false, execution continues with the statement after the `while` loop.

statement

A statement that is executed as long as the condition evaluates to true. You can use a block statement to execute multiple statements.

Description

Like other looping statements, you can use control flow statements inside `statement`:

- break stops `statement` execution and goes to the first statement after the loop.
- continue stops `statement` execution and re-evaluates `condition`.

Examples

Using while

The following `while` loop iterates as long as `n` is less than three.

js

```
let n = 0;
let x = 0;

while (n < 3) {
  n++;
  x += n;
}
```

Each iteration, the loop increments `n` and adds it to `x`. Therefore, `x` and `n` take on the following values:

- After the first pass: `n` = 1 and `x` = 1
- After the second pass: `n` = 2 and `x` = 3
- After the third pass: `n` = 3 and `x` = 6

After completing the third pass, the condition `n < 3` is no longer true, so the loop terminates.

Using an assignment as a condition

In some cases, it can make sense to use an assignment as a condition. This comes with readability tradeoffs, so there are certain stylistic recommendations that would make the pattern more obvious for everyone.

Consider the following example, which iterates over a document's comments, logging them to the console.

js · ✗ bad example

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
let currentNode;
while (currentNode = iterator.nextNode()) {
  console.log(currentNode.textContent.trim());
}
```

That's not completely a good-practice example, due to the following line specifically:

js · ✗ bad example

```
while (currentNode = iterator.nextNode()) {
```

The *effect* of that line is fine — in that, each time a comment node is found:

1. `iterator.nextNode()` returns that comment node, which gets assigned to `currentNode`.
2. The value of `currentNode = iterator.nextNode()` is therefore truthy.
3. So the `console.log()` call executes and the loop continues.

…and then, when there are no more comment nodes in the document:

1. `iterator.nextNode()` returns `null`.
2. The value of `currentNode = iterator.nextNode()` is therefore also `null`, which is falsy.
3. So the loop ends.

The problem with this line is: conditions typically use comparison operators such as `===`, but the `=` in that line isn't a comparison operator — instead, it's an assignment operator. So that `=` *looks like* it's a typo for `===` — even though it's *not* actually a typo.

Therefore, in cases like that one, some code-linting tools such as ESLint's `no-cond-assign` rule — in order to help you catch a possible typo so that you can fix it — will report a warning such as the following:

> Expected a conditional expression and instead saw an assignment.

Many style guides recommend more explicitly indicating the intention for the condition to be an assignment. You can do that minimally by putting additional parentheses as a grouping operator around the assignment:

js · ✓ good example

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
let currentNode;
while ((currentNode = iterator.nextNode())) {
  console.log(currentNode.textContent.trim());
}
```

In fact, this is the style enforced by ESLint's `no-cond-assign`'s default configuration, as well as Prettier, so you'll likely see this pattern a lot in the wild.

Some people may further recommend adding a comparison operator to turn the condition into an explicit comparison:

js · ✓ good example

```
while ((currentNode = iterator.nextNode()) !== null) {
```

There are other ways to write this pattern, such as:

js · ✓ good example

```
while ((currentNode = iterator.nextNode()) && currentNode) {
```

Or, forgoing the idea of using a `while` loop altogether:

js · ✓ good example

```
const iterator = document.createNodeIterator(document, NodeFilter.SHOW_COMMENT);
for (
  let currentNode = iterator.nextNode();
  currentNode;
  currentNode = iterator.nextNode()
) {
  console.log(currentNode.textContent.trim());
}
```

If the reader is sufficiently familiar with the assignment as condition pattern, all these variations should have equivalent readability. Otherwise, the last form is probably the most readable, albeit the most verbose.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-while-statement`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`while`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.while`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `do...while`
- `for`
- `break`
- `continue`

JavaScript › Reference › Statements and declarations

with

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/with · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** Use of the `with` statement is not recommended, as it may be the source of confusing bugs and compatibility issues, makes optimization impossible, and is forbidden in strict mode. The recommended alternative is to assign the object whose properties you want to access to a temporary variable.

The **`with`** statement extends the scope chain for a statement.

Syntax

```js
with (expression)
  statement
```

`expression`

Adds the given expression to the scope chain used when evaluating the statement. The parentheses around the expression are required.

`statement`

Any statement. To execute multiple statements, use a block statement (`{ ... }`) to group those statements.

Description

There are two types of identifiers: a *qualified* identifier and an *unqualified* identifier. An unqualified identifier is one that does not indicate where it comes from.

```js
foo; // unqualified identifier
foo.bar; // bar is a qualified identifier
```

Normally, an unqualified identifier is resolved by searching the scope chain for a variable with that name, while a qualified identifier is resolved by searching the prototype chain of an object for a property with that name.

```js
const foo = { bar: 1 };
console.log(foo.bar);
// foo is found in the scope chain as a variable;
// bar is found in foo as a property
```

One exception to this is the global object, which sits on top of the scope chain, and whose properties automatically become global variables that can be referred to without qualifiers.

```js
console.log(globalThis.Math === Math); // true
```

The `with` statement adds the given object to the head of this scope chain during the evaluation of its statement body. Every unqualified name would first be searched within the object (through an `in` check) before searching in the upper scope chain.

Note that if the unqualified reference refers to a method of the object, the method is called with the object as its `this` value.

```js
with ([1, 2, 3]) {
  console.log(toString()); // 1,2,3
}
```

The object may have a `[Symbol.unscopables]` property, which defines a list of properties that should not be added to the scope chain (for backward compatibility). See the `Symbol.unscopables` documentation for more information.

The reasons to use a `with` statement include saving one temporary variable and reducing file size by avoiding repeating a lengthy object reference. However, there are far more reasons why `with` statements are not desirable:

- Performance: The `with` statement forces the specified object to be searched first for all name lookups. Therefore, all identifiers that aren't members of the specified object will be found more slowly in a `with` block. Moreover, the optimizer cannot make any assumptions about what each unqualified identifier refers to, so it must repeat the same property lookup every time the identifier is used.
- Readability: The `with` statement makes it hard for a human reader or JavaScript compiler to decide whether an unqualified name will be found along the scope chain, and if so, in which object. For example:

  ```js
  function f(x, o) {
    with (o) {
      console.log(x);
    }
  }
  ```

If you look just at the definition of `f`, it's impossible to tell what the `x` in the `with` body refers to. Only when `f` is called can `x` be determined to be `o.x` or `f`'s first formal parameter. If you forget to define `x` in the object you pass as the second parameter, you won't get an error — instead you'll just get unexpected results. It's also unclear what the actual intent of such code would be.

- Forward compatibility: Code using `with` may not be forward compatible, especially when used with something other than a plain object, which may gain more properties in the future. Consider this example:

```js
function f(foo, values) {
  with (foo) {
    console.log(values);
  }
}
```

If you call `f([1, 2, 3], obj)` in an ECMAScript 5 environment, the `values` reference inside the `with` statement will resolve to `obj`. However, ECMAScript 2015 introduces a `values` property on `Array.prototype` (so it will be available on every array). So, after upgrading the environment, the `values` reference inside the `with` statement resolves to `[1, 2, 3].values` instead, and is likely to cause bugs.

In this particular example, `values` is defined as unscopable through `Array.prototype[Symbol.unscopables]`, so it still correctly resolves to the `values` parameter. If it were not defined as unscopable, one can see how this would be a difficult issue to debug.

Examples

Using the with statement

The following `with` statement specifies that the `Math` object is the default object. The statements following the `with` statement refer to the `PI` property and the `cos` and `sin` methods, without specifying an object. JavaScript assumes the `Math` object for these references.

```js
let a, x, y;
const r = 10;

with (Math) {
  a = PI * r * r;
  x = r * cos(PI);
  y = r * sin(PI / 2);
}
```

Avoiding the with statement by destructuring properties into the current scope

You can usually avoid using `with` through property destructuring. Here we create an extra block to mimic the behavior of `with` creating an extra scope — but in actual usage, this block can usually be omitted.

```js
let a, x, y;
const r = 10;

{
  const { PI, cos, sin } = Math;
  a = PI * r * r;
  x = r * cos(PI);
  y = r * sin(PI / 2);
}
```

Avoiding the with statement by using an IIFE

If you're producing an expression that must reuse a long-named reference multiple times, and your goal is to eliminate that lengthy name within your expression, you can wrap the expression in an IIFE and provide the long name as an argument.

```js
const objectHavingAnEspeciallyLengthyName = { foo: true, bar: false };

if (((o) => o.foo && !o.bar)(objectHavingAnEspeciallyLengthyName)) {
  // This branch runs.
}
```

Creating dynamic namespaces using the with statement and a proxy

`with` will transform every variable lookup to a property lookup, while Proxies allow trapping every property lookup call. You can create a dynamic namespace by combining them.

```js
const namespace = new Proxy(
  {},
  {
    has(target, key) {
      // Avoid trapping global properties like `console`
      if (key in globalThis) {
        return false;
      }
      // Trap all property lookups
      return true;
    },
    get(target, key) {
      return key;
    },
  },
);

with (namespace) {
  console.log(a, b, c); // "a" "b" "c"
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/ecmascript-language-statements-and-declarations.html#sec-with-statement

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with` Deprecated	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.13	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.statements.with`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- block
- Strict mode
- `Symbol.unscopables`
- `Array.prototype[Symbol.unscopables]`